INSTITUTO FEDERAL DE SERGIPE – IFS

COORDENAÇÃO DO CURSO DE CIÊNCIA DA COMPUTAÇÃO – CCOMP

CAMPUS ITABAIANA - SERGIPE

JOÃO FERNANDES SANTOS FILHO

IMPLEMENTAÇÃO DE *CLUSTERS* DE BAIXO CUSTO E ALTO DESEMPENHO UTILIZANDO A *RASPBERRY PI* E O *APACHE HADOOP*

Itabaiana (SE)

2025

JOÃO FERNANDES SANTOS FILHO

IMPLEMENTAÇÃO DE *CLUSTERS* DE BAIXO CUSTO E ALTO DESEMPENHO UTILIZANDO A *RASPBERRY PI* E O *APACHE HADOOP*

Trabalho de Conclusão de Curso apresentado a Coordenação do Curso de Ciência da Computação - CCOMP, como requisito para a obtenção do título de Bacharel em Ciência da Computação.

Orientador: Dr. José Aprígio Carneiro Neto

Itabaiana (SE)

2025

Santos Filho, João Fernandes.
S237 Implementação de *clusters* de baixo custo e alto desempenho utilizando a *Raspberry Pi* e o *Apache Hadoop*. / João Fernandes Santos Filho. – Itabaiana, 2025.
220f.; il.

Trabalho de conclusão de curso (Bacharelado em Ciência da Computação) – Instituto Federal de Educação, Ciência e Tecnologia de Sergipe – IFS, 2025.
Orientador: Prof. Dr. José Aprígio Carneiro Neto.

1. Paralelismo. 2. *Apache Hadoop*. 3. *Raspberry Pi*. 4. *Big Data*. 5. *Cluster*. I. Instituto Federal de Educação Ciência e Tecnologia de Sergipe - IFS. II. Carneiro Neto, José Aprígio III. Título.

CDU: 004.3

Ficha catalográfica elaborada pela bibliotecária Jeane Gomes dos Santos – CRB 5/1293

JOÃO FERNANDES SANTOS FILHO

IMPLEMENTAÇÃO DE *CLUSTERS* DE BAIXO CUSTO E ALTO DESEMPENHO UTILIZANDO A *RASPBERRY PI* E O *APACHE HADOOP*

Trabalho de Conclusão de Curso apresentado a Coordenação do Curso de Ciência da Computação do Instituto Federal de Sergipe – IFS, Campus Itabaiana, como parte dos requisitos para a obtenção do Título de Bacharel em Ciência da Computação. Este TCC foi julgado e aprovado pela comissão abaixo assinada em **19** de **fevereiro** de **2025**.

Itabaiana, Sergipe, Brasil.

_____

Prof., Dr. José Aprígio Carneiro Neto
Instituto Federal de Sergipe (IFS) – Orientador

_____

Prof(a)., Ma. Mayka de Souza Lima – Membro Interno
Instituto Federal de Sergipe (IFS)

_____

Prof., Esp. Eduardo Henrique do Lago Silva – Membro Interno
Instituto Federal de Sergipe (IFS)

DEDICATÓRIA

Dedico este trabalho aos meus pais, João Fernandes e Tatiana, que sempre me incentivaram a seguir meus sonhos e me apoiaram com muito amor e carinho ao longo dessa jornada.

AGRADECIMENTOS

Agradeço aos meus pais pelo esforço, dedicação e ensinamentos que me permitiram realizar esse sonho.

Ao meu orientador Dr. José Aprígio e aos professores pela paciência, sabedoria, confiança e contribuições que enriqueceram o meu estudo.

E a todos aqueles que me ajudaram na conquista dos meus objetivos e na realização deste trabalho.

“A ciência nunca resolve um problema sem criar pelo menos outros dez”.

(George Bernard Shaw)

RESUMO

Com o crescimento acelerado do volume de dados, oriundos dos avanços tecnológicos, é essencial utilizar tecnologias capazes de realizar um processamento rápido e eficiente dessas informações, o que tem levado a indústria de computadores ao desenvolvimento de novas arquiteturas de computadores, mais velozes e robustas, denominadas de arquiteturas paralelas, compostas por computadores com múltiplos processadores, que colaboram entre si na solução de problemas complexos, explorando diferentes níveis de paralelismo. Nesse contexto, os supercomputadores e os *clusters* são amplamente adotados para essa finalidade, pois são capazes de otimizar o tempo de execução das tarefas, através do processamento paralelo. Entretanto, uma alternativa economicamente viável, utilizada para atender a elevada demanda de processamento de dados, bem como os custos com gastos de energia e os problemas causados pelo custo de aquisição e manutenção de supercomputadores, é a utilização de *clusters*, formados por aglomerados de computadores, homogêneos e/ou heterogêneos, que trabalham de forma distribuída e colaborativa, efetuando o processamento de diversas tarefas paralelas simultâneas, se comportando como um único sistema. Porém, para tirar o máximo de eficiência de um *cluster* é necessário o uso de uma biblioteca de paralelismo, que seja responsável pela comunicação e troca de mensagens entre os equipamentos desse sistema, trabalhando de forma simultânea e distribuída na execução de tarefas complexas, que seja, acima de tudo, eficiente, confiável, escalável e tolerante a falhas. Além disso, é imprescindível a utilização de uma plataforma de *hardware* com elevado poder de processamento, que seja de baixo custo de aquisição e possua uma excelente eficiência energética, como é o caso da *Raspberry Pi*, uma plataforma *SBC* (*Single Board Computer*) que possui todos os componentes integrados em uma única placa. Diante desse cenário, esse trabalho teve por objetivo implementar um *cluster* de baixo custo e alto desempenho, utilizando a plataforma *Raspberry Pi 4* e a biblioteca de paralelismo *Apache Hadoop*, tendo como objetivo principal, analisar o seu desempenho na execução de algoritmos paralelos de *Big Data*. A metodologia utilizada neste trabalho teve um caráter quantitativo, descritivo e exploratório, visando o aprofundamento dos temas pesquisados, contando inicialmente com um mapeamento sistemático da literatura, que serviu de norte para os estudos. Os testes realizados indicaram que o aumento do número de nós do *cluster* resultou em uma melhora significativa no desempenho do sistema, devido à maior disponibilidade de recursos computacionais. Entretanto, em algumas situações, esse não foi o comportamento apresentado, pois a eficiência do paralelismo foi limitada pela baixa granulosidade das tarefas executadas, comprometendo o ganho de desempenho. Com base nos testes realizados, os resultados evidenciaram o baixo consumo energético da *Raspberry Pi*, visto que a adição de mais nós não gerou impactos significativos no consumo de energia do sistema. Por fim, concluiu-se que a versão do *Apache Hadoop* utilizada, apresentou dificuldades para realizar um balanceamento eficiente da carga de trabalho entre os nós do *cluster*. No entanto, a plataforma *Raspberry Pi* mostrou um bom desempenho, sendo considerada uma alternativa viável, do ponto de vista de custos e de energia, para o desenvolvimento de *clusters*, com capacidade para realizar o processamento distribuído de grandes volumes de dados.

Palavras-chave: Paralelismo; *Apache Hadoop*; *Raspberry Pi*; *Big Data*; *Cluster*.

ABSTRACT

With the rapid growth in data volume resulting from technological advances, it is essential to use technologies capable of processing this information quickly and efficiently. This has led the computer industry to develop new, faster and more robust computer architectures, known as parallel architectures, consisting of computers with multiple processors that collaborate to solve complex problems by exploring different levels of parallelism. In this context, supercomputers and clusters are widely adopted for this purpose, as they can optimize task execution time through parallel processing. However, an economically viable alternative used to meet the high demand for data processing, as well as energy costs and the problems caused by the cost of acquiring and maintaining supercomputers, is the use of clusters, formed by groups of homogeneous and/or heterogeneous computers that work in a distributed and collaborative manner, processing several simultaneous parallel tasks, behaving as a single system. However, to get the most out of a cluster, it is necessary to use a parallelism library, which is responsible for communication and message exchange between the devices in this system, working simultaneously and in a distributed manner in the execution of complex tasks, and which is, above all, efficient, reliable, scalable and fault tolerant. In addition, it is essential to use a hardware platform with high processing power, which is low-cost to acquire and has excellent energy efficiency, such as the Raspberry Pi, an SBC (Single Board Computer) platform that has all components integrated into a single board. Given this scenario, this work aimed to implement a low-cost, high-performance cluster, using the Raspberry Pi 4 platform and the Apache Hadoop parallelism library, with the main objective of analyzing its performance in the execution of parallel Big Data algorithms. The methodology used in this work was quantitative, descriptive and exploratory, aiming at the deepening of the researched themes, initially counting on a systematic mapping of the literature, which served as a guide for the studies. The tests performed indicated that the increase in the number of nodes in the cluster resulted in a significant improvement in the system performance, due to the greater availability of computational resources. However, in some situations, this was not the behavior presented, since the efficiency of parallelism was limited by the low granularity of the executed tasks, compromising the performance gain. Based on the tests performed, the results evidenced the low energy consumption of the Raspberry Pi, since the addition of more nodes did not generate significant impacts on the system's energy consumption. Finally, it was concluded that the version of Apache Hadoop used presented difficulties in efficiently balancing the workload between the cluster nodes. However, the Raspberry Pi platform showed good performance, being considered a viable alternative, from the point of view of costs and energy, for the development of clusters, with the capacity to perform the distributed processing of large volumes of data.

Keywords: Parallelism; Apache Hadoop; Raspberry Pi; Big Data; Cluster.

LISTA DE FIGURAS

Figura 01 – Tipos de granulosidade 37
Figura 02 – Estrutura de um programa *MPI* 39
Figura 03 – Modelo computacional utilizado pela biblioteca *PVM* 41
Figura 04 – Arquitetura do sistema de arquivos do *Apache Hadoop* 43
Figura 05 – Ecossistema da biblioteca *Apache Hadoop* 45
Figura 06 – Etapas da seleção das publicações para a pesquisa 61
Figura 07 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – *Pi (π)* 81
Figura 08 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – *Pi (π)* 81
Figura 09 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – *Pi (π)* 81
Figura 10 – Métricas do *Cluster* com 2 *Nodes* – *Pi (π)* 82
Figura 11 – Métricas do *Cluster* com 4 *Nodes* – *Pi (π)* 82
Figura 12 – Métricas do *Cluster* com 8 *Nodes* – *Pi (π)* 83
Figura 13 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – "*Tera*" – 500 *MBytes* 109
Figura 14 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – "*Tera*" – 1000 *MBytes* 109
Figura 15 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – "*Tera*" – 2000 *MBytes* 110
Figura 16 – Métricas do *Cluster* com 2 *Nodes* – "*Tera*" – 500 *MBytes* 110
Figura 17 – Métricas do *Cluster* com 2 *Nodes* – "*Tera*" – 1000 *MBytes* 111
Figura 18 – Métricas do *Cluster* com 2 *Nodes* – "*Tera*" – 2000 *MBytes* 111
Figura 19 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – "*Tera*" – 500 *MBytes* 111
Figura 20 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – "*Tera*" – 1000 *MBytes* 112
Figura 21 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – "*Tera*" – 2000 *MBytes* 112
Figura 22 – Métricas do *Cluster* com 4 *Nodes* – "*Tera*" – 500 *MBytes* 113
Figura 23 – Métricas do *Cluster* com 4 *Nodes* – "*Tera*" – 1000 *MBytes* 113
Figura 24 – Métricas do *Cluster* com 4 *Nodes* – "*Tera*" – 2000 *MBytes* 113
Figura 25 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – "*Tera*" – 500 *MBytes* 114
Figura 26 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – "*Tera*" – 1000 *MBytes* 114
Figura 27 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – "*Tera*" – 2000 *MBytes* 115
Figura 28 – Métricas do *Cluster* com 8 *Nodes* – "*Tera*" – 500 *MBytes* 116

Figura 29 – Métricas do *Cluster* com 8 *Nodes* – "*Tera*" – 1000 *MBytes* 116
Figura 30 – Métricas do *Cluster* com 8 *Nodes* – "*Tera*" – 2000 *MBytes* 116
Figura 31 – Métricas de desempenho (*Hadoop* e Consumo Energético) – *DFSIO Write* 117
Figura 32 – Métricas de desempenho (*Hadoop* e Consumo Energético) – *DFSIO Read* 125
Figura 33 – Espaço em disco – 2 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes* 134
Figura 34 – Espaço em disco – 4 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes* 134
Figura 35 – Espaço em disco – 8 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes* 134
Figura 36 – Espaço em disco – 2 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes* 135
Figura 37 – Espaço em disco – 4 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes* 135
Figura 38 – Espaço em disco – 8 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes* 135
Figura 39 – Espaço em disco – 2 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes* 136
Figura 40 – Espaço em disco – 4 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes* 136
Figura 41 – Espaço em disco – 8 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes* 136
Figura 42 – Métricas do *Cluster* – 2 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes* 137
Figura 43 – Métricas do *Cluster* – 4 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes* 137
Figura 44 – Métricas do *Cluster* – 8 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes* 138
Figura 45 – Métricas do *Cluster* – 2 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes* 138
Figura 46 – Métricas do *Cluster* – 4 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes* 139
Figura 47 – Métricas do *Cluster* – 8 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes* 139
Figura 48 – Métricas do *Cluster* – 2 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes* 140
Figura 49 – Métricas do *Cluster* – 4 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes* 140
Figura 50 – Métricas do *Cluster* – 8 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes* 140
Figura 51 – Utilização de *CPU* – *Pi (π)* – 2 *nodes* 156
Figura 52 – Utilização de memória *RAM* – *Pi (π)* – 2 *nodes* 156
Figura 53 – Quantidade de processos – *Pi (π)* – 2 *nodes* 156
Figura 54 – Tráfego de rede – *Pi (π)* – 2 *nodes* 156
Figura 55 – Temperatura – *Pi (π)* – 2 *nodes* 157
Figura 56 – Utilização de *CPU* – *Pi (π)* – 4 *nodes* 157
Figura 57 – Utilização de memória *RAM* – *Pi (π)* – 4 *nodes* 157
Figura 58 – Quantidade de processos – *Pi (π)* – 4 *nodes* 157
Figura 59 – Tráfego de rede – *Pi (π)* – 4 *nodes* 158
Figura 60 – Temperatura – *Pi (π)* – 4 *nodes* 158
Figura 61 – Utilização de *CPU* – *Pi (π)* – 8 *nodes* 158

Figura 62 – Utilização de memória *RAM* – *Pi (π)* – 8 *nodes* 159
Figura 63 – Quantidade de processos – *Pi (π)* – 8 *nodes* 159
Figura 64 – Tráfego de rede – *Pi (π)* – 8 *nodes* 159
Figura 65 – Temperatura – *Pi (π)* – 8 *nodes* 160
Figura 66 – Utilização de *CPU* – *TeraGen* – 500 *Mbytes* – 2 *nodes* 160
Figura 67 – Utilização de memória *RAM* – *TeraGen* – 500 *Mbytes* – 2 *nodes* 160
Figura 68 – Quantidade de processos – *TeraGen* – 500 *Mbytes* – 2 *nodes* 160
Figura 69 – Tráfego de rede – *TeraGen* – 500 *Mbytes* – 2 *nodes* 161
Figura 70 – Temperatura – *TeraGen* – 500 *Mbytes* – 2 *nodes* 161
Figura 71 – Utilização de *CPU* – *TeraGen* – 500 *Mbytes* – 4 *nodes* 161
Figura 72 – Utilização de memória *RAM* – *TeraGen* – 500 *Mbytes* – 4 *nodes* 161
Figura 73 – Quantidade de processos – *TeraGen* – 500 *Mbytes* – 4 *nodes* 162
Figura 74 – Tráfego de rede – *TeraGen* – 500 *Mbytes* – 4 *nodes* 162
Figura 75 – Temperatura – *TeraGen* – 500 *Mbytes* – 4 *nodes* 162
Figura 76 – Utilização de *CPU* – *TeraGen* – 500 *Mbytes* – 8 *nodes* 163
Figura 77 – Utilização de memória *RAM* – *TeraGen* – 500 *Mbytes* – 8 *nodes* 163
Figura 78 – Quantidade de processos – *TeraGen* – 500 *Mbytes* – 8 *nodes* 163
Figura 79 – Tráfego de rede – *TeraGen* – 500 *Mbytes* – 8 *nodes* 164
Figura 80 – Temperatura – *TeraGen* – 500 *Mbytes* – 8 *nodes* 164
Figura 81 – Utilização de *CPU* – *TeraGen* – 1000 *Mbytes* – 2 *nodes* 164
Figura 82 – Utilização de memória *RAM* – *TeraGen* – 1000 *Mbytes* – 2 *nodes* 165
Figura 83 – Quantidade de processos – *TeraGen* – 1000 *Mbytes* – 2 *nodes* 165
Figura 84 – Tráfego de rede – *TeraGen* – 1000 *Mbytes* – 2 *nodes* 165
Figura 85 – Temperatura – *TeraGen* – 1000 *Mbytes* – 2 *nodes* 165
Figura 86 – Utilização de *CPU* – *TeraGen* – 1000 *Mbytes* – 4 *nodes* 165
Figura 87 – Utilização de memória *RAM* – *TeraGen* – 1000 *Mbytes* – 4 *nodes* 166
Figura 88 – Quantidade de processos – *TeraGen* – 1000 *Mbytes* – 4 *nodes* 166
Figura 89 – Tráfego de rede – *TeraGen* – 1000 *Mbytes* – 4 *nodes* 166
Figura 90 – Temperatura – *TeraGen* – 1000 *Mbytes* – 4 *nodes* 166
Figura 91 – Utilização de *CPU* – *TeraGen* – 1000 *Mbytes* – 8 *nodes* 167
Figura 92 – Utilização de memória *RAM* – *TeraGen* – 1000 *Mbytes* – 8 *nodes* 167
Figura 93 – Quantidade de processos – *TeraGen* – 1000 *Mbytes* – 8 *nodes* 167
Figura 94 – Tráfego de rede – *TeraGen* – 1000 *Mbytes* – 8 *nodes* 168

Figura 95 – Temperatura – *TeraGen* – 1000 *Mbytes* – 8 *nodes* 168
Figura 96 – Utilização de *CPU* – *TeraGen* – 2000 *Mbytes* – 2 *nodes* 168
Figura 97 – Utilização de memória *RAM* – *TeraGen* – 2000 *Mbytes* – 2 *nodes* 168
Figura 98 – Quantidade de processos – *TeraGen* – 2000 *Mbytes* – 2 *nodes* 169
Figura 99 – Tráfego de rede – *TeraGen* – 2000 *Mbytes* – 2 *nodes* 169
Figura 100 – Temperatura – *TeraGen* – 2000 *Mbytes* – 2 *nodes* 169
Figura 101 – Utilização de *CPU* – *TeraGen* – 2000 *Mbytes* – 4 *nodes* 169
Figura 102 – Utilização de memória *RAM* – *TeraGen* – 2000 *Mbytes* – 4 *nodes* 169
Figura 103 – Quantidade de processos – *TeraGen* – 2000 *Mbytes* – 4 *nodes* 170
Figura 104 – Tráfego de rede – *TeraGen* – 2000 *Mbytes* – 4 *nodes* 170
Figura 105 – Temperatura – *TeraGen* – 2000 *Mbytes* – 4 *nodes* 170
Figura 106 – Utilização de *CPU* – *TeraGen* – 2000 *Mbytes* – 8 *nodes* 171
Figura 107 – Utilização de memória *RAM* – *TeraGen* – 2000 *Mbytes* – 8 *nodes* 171
Figura 108 – Quantidade de processos – *TeraGen* – 2000 *Mbytes* – 8 *nodes* 171
Figura 109 – Tráfego de rede – *TeraGen* – 2000 *Mbytes* – 8 *nodes* 172
Figura 110 – Temperatura – *TeraGen* – 2000 *Mbytes* – 8 *nodes* 172
Figura 111 – Utilização de *CPU* – *TeraSort* – 500 *Mbytes* – 2 *nodes* 172
Figura 112 – Utilização de memória *RAM* – *TeraSort* – 500 *Mbytes* – 2 *nodes* 173
Figura 113 – Quantidade de processos – *TeraSort* – 500 *Mbytes* – 2 *nodes* 173
Figura 114 – Tráfego de rede – *TeraSort* – 500 *Mbytes* – 2 *nodes* 173
Figura 115 – Temperatura – *TeraSort* – 500 *Mbytes* – 2 *nodes* 173
Figura 116 – Utilização de *CPU* – *TeraSort* – 500 *Mbytes* – 4 *nodes* 173
Figura 117 – Utilização de memória *RAM* – *TeraSort* – 500 *Mbytes* – 4 *nodes* 174
Figura 118 – Quantidade de processos – *TeraSort* – 500 *Mbytes* – 4 *nodes* 174
Figura 119 – Tráfego de rede – *TeraSort* – 500 *Mbytes* – 4 *nodes* 174
Figura 120 – Temperatura – *TeraSort* – 500 *Mbytes* – 4 *nodes* 174
Figura 121 – Utilização de *CPU* – *TeraSort* – 500 *Mbytes* – 8 *nodes* 175
Figura 122 – Utilização de memória *RAM* – *TeraSort* – 500 *Mbytes* – 8 *nodes* 175
Figura 123 – Quantidade de processos – *TeraSort* – 500 *Mbytes* – 8 *nodes* 175
Figura 124 – Tráfego de rede – *TeraSort* – 500 *Mbytes* – 8 *nodes* 176
Figura 125 – Temperatura – *TeraSort* – 500 *Mbytes* – 8 *nodes* 176
Figura 126 – Utilização de *CPU* – *TeraSort* – 1000 *Mbytes* – 2 *nodes* 176
Figura 127 – Utilização de memória *RAM* – *TeraSort* – 1000 *Mbytes* – 2 *nodes* 177

Figura 128 – Quantidade de processos – *TeraSort* – 1000 *Mbytes* – 2 *nodes* 177
Figura 129 – Tráfego de rede – *TeraSort* – 1000 *Mbytes* – 2 *nodes* 177
Figura 130 – Temperatura – *TeraSort* – 1000 *Mbytes* – 2 *nodes* 177
Figura 131 – Utilização de *CPU* – *TeraSort* – 1000 *Mbytes* – 4 *nodes* 177
Figura 132 – Utilização de memória *RAM* – *TeraSort* – 1000 *Mbytes* – 4 *nodes* 178
Figura 133 – Quantidade de processos – *TeraSort* – 1000 *Mbytes* – 4 *nodes* 178
Figura 134 – Tráfego de rede – *TeraSort* – 1000 *Mbytes* – 4 *nodes* 178
Figura 135 – Temperatura – *TeraSort* – 1000 *Mbytes* – 4 *nodes* 178
Figura 136 – Utilização de *CPU* – *TeraSort* – 1000 *Mbytes* – 8 *nodes* 179
Figura 137 – Utilização de memória *RAM* – *TeraSort* – 1000 *Mbytes* – 8 *nodes* 179
Figura 138 – Quantidade de processos – *TeraSort* – 1000 *Mbytes* – 8 *nodes* 179
Figura 139 – Tráfego de rede – *TeraSort* – 1000 *Mbytes* – 8 *nodes* 180
Figura 140 – Temperatura – *TeraSort* – 1000 *Mbytes* – 8 *nodes* 180
Figura 141 – Utilização de *CPU* – *TeraSort* – 2000 *Mbytes* – 2 *nodes* 180
Figura 142 – Utilização de memória *RAM* – *TeraSort* – 2000 *Mbytes* – 2 *nodes* 181
Figura 143 – Quantidade de processos – *TeraSort* – 2000 *Mbytes* – 2 *nodes* 181
Figura 144 – Tráfego de rede – *TeraSort* – 2000 *Mbytes* – 2 *nodes* 181
Figura 145 – Temperatura – *TeraSort* – 2000 *Mbytes* – 2 *nodes* 181
Figura 146 – Utilização de *CPU* – *TeraSort* – 2000 *Mbytes* – 4 *nodes* 181
Figura 147 – Utilização de memória *RAM* – *TeraSort* – 2000 *Mbytes* – 4 *nodes* 182
Figura 148 – Quantidade de processos – *TeraSort* – 2000 *Mbytes* – 4 *nodes* 182
Figura 149 – Tráfego de rede – *TeraSort* – 2000 *Mbytes* – 4 *nodes* 182
Figura 150 – Temperatura – *TeraSort* – 2000 *Mbytes* – 4 *nodes* 182
Figura 151 – Utilização de *CPU* – *TeraSort* – 2000 *Mbytes* – 8 *nodes* 183
Figura 152 – Utilização de memória *RAM* – *TeraSort* – 2000 *Mbytes* – 8 *nodes* 183
Figura 153 – Quantidade de processos – *TeraSort* – 2000 *Mbytes* – 8 *nodes* 183
Figura 154 – Tráfego de rede – *TeraSort* – 2000 *Mbytes* – 8 *nodes* 184
Figura 155 – Temperatura – *TeraSort* – 2000 *Mbytes* – 8 *nodes* 184
Figura 156 – Utilização de *CPU* – *TeraValidate* – 500 *Mbytes* – 2 *nodes* 184
Figura 157 – Utilização de memória *RAM* – *TeraValidate* – 500 *Mbytes* – 2 *nodes* 185
Figura 158 – Quantidade de processos – *TeraValidate* – 500 *Mbytes* – 2 *nodes* 185
Figura 159 – Tráfego de rede – *TeraValidate* – 500 *Mbytes* – 2 *nodes* 185
Figura 160 – Temperatura – *TeraValidate* – 500 *Mbytes* – 2 *nodes* 185

Figura 161 – Utilização de *CPU* – *TeraValidate* – 500 *Mbytes* – 4 *nodes*..... 185
Figura 162 – Utilização de memória *RAM* – *TeraValidate* – 500 *Mbytes* – 4 *nodes* 186
Figura 163 – Quantidade de processos – *TeraValidate* – 500 *Mbytes* – 4 *nodes* 186
Figura 164 – Tráfego de rede – *TeraValidate* – 500 *Mbytes* – 4 *nodes* 186
Figura 165 – Temperatura – *TeraValidate* – 500 *Mbytes* – 4 *nodes* 186
Figura 166 – Utilização de *CPU* – *TeraValidate* – 500 *Mbytes* – 8 *nodes*..... 187
Figura 167 – Utilização de memória *RAM* – *TeraValidate* – 500 *Mbytes* – 8 *nodes* 187
Figura 168 – Quantidade de processos – *TeraValidate* – 500 *Mbytes* – 8 *nodes* 187
Figura 169 – Tráfego de rede – *TeraValidate* – 500 *Mbytes* – 8 *nodes* 188
Figura 170 – Temperatura – *TeraValidate* – 500 *Mbytes* – 8 *nodes* 188
Figura 171 – Utilização de *CPU* – *TeraValidate* – 1000 *Mbytes* – 2 *nodes*..... 188
Figura 172 – Utilização de memória *RAM* – *TeraValidate* – 1000 *Mbytes* – 2 *nodes* 189
Figura 173 – Quantidade de processos – *TeraValidate* – 1000 *Mbytes* – 2 *nodes* 189
Figura 174 – Tráfego de rede – *TeraValidate* – 1000 *Mbytes* – 2 *nodes* 189
Figura 175 – Temperatura – *TeraValidate* – 1000 *Mbytes* – 2 *nodes* 189
Figura 176 – Utilização de *CPU* – *TeraValidate* – 1000 *Mbytes* – 4 *nodes*..... 189
Figura 177 – Utilização de memória *RAM* – *TeraValidate* – 1000 *Mbytes* – 4 *nodes* 190
Figura 178 – Quantidade de processos – *TeraValidate* – 1000 *Mbytes* – 4 *nodes* 190
Figura 179 – Tráfego de rede – *TeraValidate* – 1000 *Mbytes* – 4 *nodes* 190
Figura 180 – Temperatura – *TeraValidate* – 1000 *Mbytes* – 4 *nodes* 190
Figura 181 – Utilização de *CPU* – *TeraValidate* – 1000 *Mbytes* – 8 *nodes*..... 191
Figura 182 – Utilização de memória *RAM* – *TeraValidate* – 1000 *Mbytes* – 8 *nodes* 191
Figura 183 – Quantidade de processos – *TeraValidate* – 1000 *Mbytes* – 8 *nodes* 191
Figura 184 – Tráfego de rede – *TeraValidate* – 1000 *Mbytes* – 8 *nodes* 192
Figura 185 – Temperatura – *TeraValidate* – 1000 *Mbytes* – 8 *nodes* 192
Figura 186 – Utilização de *CPU* – *TeraValidate* – 2000 *Mbytes* – 2 *nodes*..... 192
Figura 187 – Utilização de memória *RAM* – *TeraValidate* – 2000 *Mbytes* – 2 *nodes* 193
Figura 188 – Quantidade de processos – *TeraValidate* – 2000 *Mbytes* – 2 *nodes* 193
Figura 189 – Tráfego de rede – *TeraValidate* – 2000 *Mbytes* – 2 *nodes* 193
Figura 190 – Temperatura – *TeraValidate* – 2000 *Mbytes* – 2 *nodes* 193
Figura 191 – Utilização de *CPU* – *TeraValidate* – 2000 *Mbytes* – 4 *nodes*..... 193
Figura 192 – Utilização de memória *RAM* – *TeraValidate* – 2000 *Mbytes* – 4 *nodes* 194
Figura 193 – Quantidade de processos – *TeraValidate* – 2000 *Mbytes* – 4 *nodes* 194

Figura 194 – Tráfego de rede – *TeraValidate* – 2000 *Mbytes* – 4 *nodes* 194
Figura 195 – Temperatura – *TeraValidate* – 2000 *Mbytes* – 4 *nodes* 194
Figura 196 – Utilização de *CPU* – *TeraValidate* – 2000 *Mbytes* – 8 *nodes*..... 195
Figura 197 – Utilização de memória *RAM* – *TeraValidate* – 2000 *Mbytes* – 8 *nodes* 195
Figura 198 – Quantidade de processos – *TeraValidate* – 2000 *Mbytes* – 8 *nodes* 195
Figura 199 – Tráfego de rede – *TeraValidate* – 2000 *Mbytes* – 8 *nodes* 196
Figura 200 – Temperatura – *TeraValidate* – 2000 *Mbytes* – 8 *nodes* 196
Figura 201 – Utilização de *CPU* – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes*..... 196
Figura 202 – Utilização de memória *RAM* – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes* 197
Figura 203 – Quantidade de processos – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes* 197
Figura 204 – Tráfego de rede – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes* 197
Figura 205 – Temperatura – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes* 197
Figura 206 – Utilização de *CPU* – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes*..... 197
Figura 207 – Utilização de memória *RAM* – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes* 198
Figura 208 – Quantidade de processos – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes* 198
Figura 209 – Tráfego de rede – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes* 198
Figura 210 – Temperatura – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes* 198
Figura 211 – Utilização de *CPU* – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes*..... 199
Figura 212 – Utilização de memória *RAM* – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes* 199
Figura 213 – Quantidade de processos – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes* 199
Figura 214 – Tráfego de rede – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes* 200
Figura 215 – Temperatura – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes* 200
Figura 216 – Utilização de *CPU* – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes*..... 200
Figura 217 – Utilização de memória *RAM* – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes* ... 201
Figura 218 – Quantidade de processos – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes* 201
Figura 219 – Tráfego de rede – *TestDFSIO Write e* – 1000 *Mbytes* – 2 *nodes*..... 201
Figura 220 – Temperatura – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes* 201
Figura 221 – Utilização de *CPU* – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes*..... 201
Figura 222 – Utilização de memória *RAM* – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes* ... 202
Figura 223 – Quantidade de processos – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes* 202
Figura 224 – Tráfego de rede – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes* 202
Figura 225 – Temperatura – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes* 202
Figura 226 – Utilização de *CPU* – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes*..... 203

Figura 227 – Utilização de memória *RAM* – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes* ... 203
Figura 228 – Quantidade de processos – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes* 203
Figura 229 – Tráfego de rede – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes* 204
Figura 230 – Temperatura – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes* 204
Figura 231 – Utilização de *CPU* – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes*..... 204
Figura 232 – Utilização de memória *RAM* – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes* ... 205
Figura 233 – Quantidade de processos – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes* 205
Figura 234 – Tráfego de rede – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes* 205
Figura 235 – Temperatura – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes* 205
Figura 236 – Utilização de *CPU* – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes*..... 205
Figura 237 – Utilização de memória *RAM* – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes* ... 206
Figura 238 – Quantidade de processos – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes* 206
Figura 239 – Tráfego de rede – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes* 206
Figura 240 – Temperatura – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes* 206
Figura 241 – Utilização de *CPU* – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes*..... 207
Figura 242 – Utilização de memória *RAM* – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes* ... 207
Figura 243 – Quantidade de processos – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes* 207
Figura 244 – Tráfego de rede – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes* 208
Figura 245 – Temperatura – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes* 208
Figura 246 – Utilização de *CPU* – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes* 208
Figura 247 – Utilização de memória *RAM* – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*..... 209
Figura 248 – Quantidade de processos – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*..... 209
Figura 249 – Tráfego de rede – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes* 209
Figura 250 – Temperatura – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes* 209
Figura 251 – Utilização de *CPU* – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes* 209
Figura 252 – Utilização de memória *RAM* – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*..... 210
Figura 253 – Quantidade de processos – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*..... 210
Figura 254 – Tráfego de rede – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes* 210
Figura 255 – Temperatura – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes* 210
Figura 256 – Utilização de *CPU* – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes* 211
Figura 257 – Utilização de memória *RAM* – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*..... 211
Figura 258 – Quantidade de processos – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*..... 211
Figura 259 – Tráfego de rede – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes* 212

Figura 260 – Temperatura – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes* 212
Figura 261 – Utilização de *CPU* – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes* 212
Figura 262 – Utilização de memória *RAM* – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes* 213
Figura 263 – Quantidade de processos – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes* 213
Figura 264 – Tráfego de rede – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes* 213
Figura 265 – Temperatura – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes* 213
Figura 266 – Utilização de *CPU* – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes* 213
Figura 267 – Utilização de memória *RAM* – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes* 214
Figura 268 – Quantidade de processos – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes* 214
Figura 269 – Tráfego de rede – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes* 214
Figura 270 – Temperatura – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes* 214
Figura 271 – Utilização de *CPU* – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes* 215
Figura 272 – Utilização de memória *RAM* – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes* 215
Figura 273 – Quantidade de processos – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes* 215
Figura 274 – Tráfego de rede – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes* 216
Figura 275 – Temperatura – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes* 216
Figura 276 – Utilização de *CPU* – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes* 216
Figura 277 – Utilização de memória *RAM* – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes* 217
Figura 278 – Quantidade de processos – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes* 217
Figura 279 – Tráfego de rede – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes* 217
Figura 280 – Temperatura – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes* 217
Figura 281 – Utilização de *CPU* – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes* 217
Figura 282 – Utilização de memória *RAM* – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes* 218
Figura 283 – Quantidade de processos – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes* 218
Figura 284 – Tráfego de rede – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes* 218
Figura 285 – Temperatura – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes* 218
Figura 286 – Utilização de *CPU* – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes* 219
Figura 287 – Utilização de memória *RAM* – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes* 219
Figura 288 – Quantidade de processos – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes* 219
Figura 289 – Tráfego de rede – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes* 220
Figura 290 – Temperatura – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes* 220

LISTA DE TABELAS

Tabela 01 – Resumo dos trabalhos correlatos 58
Tabela 02 – Algoritmos de *Benchmark* 64
Tabela 03 – Modelos de *Raspberry Pi* utilizadas na implementação dos *clusters* 65
Tabela 04 – Versões do *Apache Hadoop* utilizadas na implementação dos *clusters* 67
Tabela 05 – Ferramentas de monitoramento e gerenciamento dos *clusters* 68
Tabela 06 – Métricas de desempenho 69
Tabela 07 – Quantidade de *nodes* do *cluster* 70
Tabela 08 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *Pi (π)* 73
Tabela 09 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *Pi (π)* 76
Tabela 10 – Resultados das métricas de tráfego de rede do *cluster* – 2 *Nodes* – *Pi (π)* 77
Tabela 11 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *Pi (π)* 78
Tabela 12 – Resultados das métricas de tráfego de rede do *cluster* – 4 *Nodes* – *Pi (π)* 78
Tabela 13 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *Pi (π)* 79
Tabela 14 – Resultados das métricas de tráfego de rede do *cluster* – 8 *Nodes* – *Pi (π)* 80
Tabela 15 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *TeraGen* 83
Tabela 16 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *TeraGen* 85
Tabela 17 – Resultados das métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TeraGen* 86
Tabela 18 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *TeraGen* 87
Tabela 19 – Resultados das métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TeraGen* 88
Tabela 20 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *TeraGen* 89
Tabela 21 – Resultados das métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TeraGen* 90
Tabela 22 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *TeraSort* 91
Tabela 23 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *TeraSort* 93
Tabela 24 – Resultados das métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TeraSort* 94
Tabela 25 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *TeraSort* 95
Tabela 26 – Resultados das métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TeraSort* 96
Tabela 27 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *TeraSort* 98
Tabela 28 – Resultados das métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TeraSort* 99

Tabela 29 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *TeraValidate* 100
Tabela 30 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *TeraValidate* 103
Tabela 31 – Métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TeraValidate* 103
Tabela 32 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *TeraValidate* 105
Tabela 33 – Métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TeraValidate* 105
Tabela 34 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *TeraValidate* 107
Tabela 35 – Métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TeraValidate* 108
Tabela 36 – Métricas de *hardware* do *cluster* – 2 *Nodes* – *TestDFSIO Write* 119
Tabela 37 – Métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TestDFSIO Write* 120
Tabela 38 – Métricas de *hardware* do *cluster* – 4 *Nodes* – *TestDFSIO Write* 121
Tabela 39 – Métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TestDFSIO Write* 122
Tabela 40 – Métricas de *hardware* do *cluster* – 8 *Nodes* – *TestDFSIO Write* 123
Tabela 41 – Métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TestDFSIO Write* 124
Tabela 42 – Métricas de *hardware* do *cluster* – 2 *Nodes* – *TestDFSIO Read* 127
Tabela 43 – Métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TestDFSIO Read* 128
Tabela 44 – Métricas de *hardware* do *cluster* – 4 *Nodes* – *TestDFSIO Read* 129
Tabela 45 – Métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TestDFSIO Read* 130
Tabela 46 – Métricas de *hardware* do *cluster* – 8 *Nodes* – *TestDFSIO Read* 132
Tabela 47 – Métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TestDFSIO Read* 133
Tabela 48 – Valores dos equipamentos utilizados na implementação do *cluster* 152
Tabela 49 – Capacidade computacional nominal do *cluster* 152
Tabela 50 – Valores dos equipamentos de um *cluster beowulf* 155

LISTA DE ABREVIATURA OU SIGLAS

AArch32	–	*ARM Architecture family 32*
AArch64	–	*ARM Architecture family 64*
API	–	*Application Programming Interfaces*
ARM	–	*Advanced RISC Machine*
CI	–	Critério de Inclusão
CE	–	Critério de Exclusão
CPU	–	*Central Processing Unit*
DFSIO	–	*Data File Systems Input Output*
GB	–	*Gigabyte*
GHz	–	*Gigahertz*
GPU	–	*Graphics Processing Units*
HDFS	–	*Hadoop Distributed File System*
HIPI	–	*Hadoop Image Processing Interface*
HPC	–	*High-Performance Computing*
IoT	–	*Internet of Things*
LAN	–	*Local Area Network*
MB	–	*Megabyte*
Mbps	–	*Megabit* por segundo
MHz	–	*Megahertz*
MIMD	–	*Multiple Instruction, Multiple Data*
MISD	–	*Multiple Instruction, Single Data*
ML	–	*Machine Learning*
MPI	–	*Message Passing Interface*
MIPI	–	*Mobile Industry Processor Interface*
NODES	–	Máquinas de Rede – Nós de Rede
NUMA	–	*Non-Uniform Memory Access*
OpenMP	–	*Open Multi-Processing*
PVM	–	*Parallel Virual Machine*

QP – *Questões de Pesquisa*

RAM – *Random Access Memory*

RISC – *Reduced Instruction Set Computer*

RPi – *Raspberry Pi*

SBC – *Single Board Computer*

SIMD – *Single Instruction, Multiple Data*

SISD – *Single Instruction, Single Data*

SMP – *Symmetric Multi-Processor*

SNMP – *Simple Network Management Protocol*

SQL – *Structured Query Language*

TB – *Terabyte*

UMA – *Uniform Memory Access*

SUMÁRIO

1. INTRODUÇÃO 26
1.1 Objetivos 29
1.1.1 Objetivo Geral 29
1.1.2 Objetivos Específicos 29
1.2 Justificativa e Relevância 30
1.3 Delimitações do Estudo 30
1.4 Estrutura do Trabalho 30
2. FUNDAMENTAÇÃO TEÓRICA 32
2.1 Arquiteturas Paralelas 32
2.2 Arquitetura em *Cluster* 35
2.3 Bibliotecas de Paralelismo 38
2.4 *Apache Hadoop* 41
2.5 Programação Paralela 48
2.6 *Benchmarks* 49
2.7 *Big Data* 51
2.8 Ferramentas Utilizadas no Monitoramento de Desempenho de um *Cluster* 52
3. TRABALHOS CORRELATOS 54
3.1 *An Efficient Implementation of Mobile Raspberry Pi Hadoop Clusters for Robust and Augmented Computing Performance* 54
3.2 *A Raspberry Pi Cluster for Teaching Big-Data Analytics* 54
3.3 *Benchmarking Raspberry Pi 2 Hadoop Cluster* 55
3.4 *On Energy Efficiency and Performance Evaluation of Single Board Computer Based Clusters A Hadoop Case Study* 55
3.5 *Optimizing Hadoop Scheduling in Single-Board-Computer-Based Heterogeneous Clusters* 56
3.6 *Performance of a Low Cost Hadoop Cluster for Image Analysis in Cloud Robotics Environment* 57
4. METODOLOGIA 59

5. RESULTADOS E DISCUSSÃO..... 64
5.1 Mapeamento Sistemático..... 64
5.1.1 Algoritmos de *Benchmark* 64
5.1.2 Modelos de *Raspberry Pi* 65
5.1.3 Versão do *Apache Hadoop* 66
5.1.4 *Software* de Monitoramento e Gerenciamento..... 67
5.1.5 Métricas de Desempenho 68
5.1.6 Quantidade de Nós (*Nodes*) dos *Clusters* 69
5.1.7 Resumo dos Trabalhos Analisados 70
5.2 Testes de Desempenho do *Cluster*..... 73
5.2.1 Testes envolvendo o algoritmo *Pi (π) - quasi-Monte Carlo (qMC)*..... 73
5.2.1.1 Métricas de desempenho do *Hadoop* e consumo energético do *cluster – Pi (π)*..... 73
5.2.1.2 Métricas de desempenho do *cluster* (*hardware* e tráfego de rede) – *Pi (π)* 75
5.2.1.2.1 *Cluster* formado por 2 *nodes – Pi (π)* 75
5.2.1.2.2 *Cluster* formado por 4 *nodes – Pi (π)* 77
5.2.1.2.3 *Cluster* formado por 8 *nodes – Pi (π)* 78
5.2.1.3 Capacidade de armazenamento dos *Clusters – Algoritmo Pi (π)* 80
5.2.2 Testes envolvendo os algoritmos "*Tera*" (*TeraGen - TeraSort - TeraValidate*) ... 83
5.2.2.1 Desempenho do *Hadoop* e consumo energético do *cluster – TeraGen* 83
5.2.2.2 Métricas de desempenho do *cluster* (*hardware* e tráfego de rede) – *TeraGen* 85
5.2.2.2.1 *Cluster* formado por 2 *nodes – TeraGen* 85
5.2.2.2.2 *Cluster* formado por 4 *nodes – TeraGen* 86
5.2.2.2.3 *Cluster* formado por 8 *nodes – TeraGen* 88
5.2.2.3 Desempenho do *Hadoop* e consumo energético do *cluster – TeraSort* 90
5.2.2.4 Métricas de desempenho do *cluster* (*hardware* e tráfego de rede) – *TeraSort* 92
5.2.2.4.1 *Cluster* formado por 2 *nodes – TeraSort* 92
5.2.2.4.2 *Cluster* formado por 4 *nodes – TeraSort* 94
5.2.2.4.3 *Cluster* formado por 8 *nodes – TeraSort* 96
5.2.2.5 Desempenho do *Hadoop* e consumo energético do *cluster – TeraValidate*..... 100
5.2.2.6 Desempenho do *cluster* (*hardware* e tráfego de rede) – *TeraValidate* 102
5.2.2.6.1 *Cluster* formado por 2 *nodes – TeraValidate* 102
5.2.2.6.2 *Cluster* formado por 4 *nodes – TeraValidate* 103
5.2.2.6.3 *Cluster* formado por 8 *nodes – TeraValidate* 106

5.2.2.7 Capacidade de armazenamento dos *Clusters* – Algoritmos "*Tera*" 109
5.2.3 Testes envolvendo o algoritmo *TestDFSIO* 117
5.2.3.1 Desempenho do *Hadoop* e consumo energético do *cluster* – *DFSIO Write* 117
5.2.3.2 Desempenho do *cluster* (*hardware* e tráfego de rede) – *DFSIO Write* 118
5.2.3.2.1 *Cluster* formado por 2 *nodes* – *DFSIO Write* 118
5.2.3.2.2 *Cluster* formado por 4 *nodes* – *DFSIO Write* 120
5.2.3.2.3 *Cluster* formado por 8 *nodes* – *DFSIO Write* 122
5.2.3.3 Desempenho do *Hadoop* e consumo energético do *cluster* – *DFSIO Read* 125
5.2.3.4 Desempenho do *cluster* (*hardware* e tráfego de rede) – *DFSIO Read* 126
5.2.3.4.1 *Cluster* formado por 2 *nodes* – *DFSIO Read* 126
5.2.3.4.2 *Cluster* formado por 4 *nodes* – *DFSIO Read* 128
5.2.3.4.3 *Cluster* formado por 8 *nodes* – *DFSIO Read* 130
5.2.3.5 Capacidade de armazenamento dos *Clusters* – *TestDFSIO Write*/*Read* 133
5.2.3.6 Métricas dos *Clusters* – Algoritmo *TestDFSIO* (*Write/Read*)..... 137
6. CONSIDERAÇÕES FINAIS 141
REFERÊNCIAS..... 144
APÊNDICE A 151
APÊNDICE B..... 156

1. INTRODUÇÃO

Os avanços tecnológicos têm influenciado o crescimento do volume de dados, além da complexidade das tarefas realizadas pelos computadores, fazendo com que o poder de processamento desses equipamentos aumentassem consideravelmente no decorrer dos anos. Para atender à toda essa demanda computacional, a indústria de componentes eletrônicos vem investindo no desenvolvimento de processadores mais robustos e velozes, com alto poder de processamento (LIMA *et al.*, 2016).

Porém, essa busca acelerada e constante tem deixado em alerta a indústria desse setor, devido às limitações impostas pelo aumento da frequência do *clock* dos processadores, que eleva o consumo de energia e a produção de calor dos componentes eletrônicos (SANTOS, 2015), tornando-se um desafio manter a Lei proposta por Gordon Moore, em 1965 (MOORE, 1965), que propõe um aumento contínuo no poder de processamento dos computadores ao longo dos anos.

Para contornar todos esses problemas, causados pela limitação dos processadores, em função do aumento no quantitativo de transistores embutidos no núcleo desses componentes eletrônicos, a indústria passou a desenvolver Circuitos Integrados (CI) capazes de processar dados em paralelo, ao invés de continuarem a aumentar o poder de processamento em um único núcleo do processador, motivando assim o surgimento de diversos tipos de arquiteturas paralelas (LIMA *et al.*, 2016).

As arquiteturas paralelas são estruturas de *hardware* que utilizam múltiplos processadores, em geral, com cada um desses processadores com vários núcleos, que trabalham de forma colaborativa na resolução de algoritmos e problemas complexos, explorando os diferentes níveis de paralelismo do processador. No mercado, existem diferentes tipos de arquiteturas paralelas, entre elas, destacam-se: as máquinas *multicore*, os multiprocessadores, os multicomputadores (*clusters*) e o sistema de *grid* (SCHEPKE, 2009).

De acordo com Bacellar (2009), uma alternativa economicamente viável para suprir a demanda elevada de processamento de grandes volumes de dados (*Big Data*), os custos com gastos de energia e os problemas causados pelo custo de aquisição e manutenção de supercomputadores é a utilização de uma arquitetura em *cluster*.

Segundo Pitanga (2004), os *clusters* são aglomerados de computadores, homogêneos e/ou heterogêneos, que trabalham de forma distribuída e colaborativa, efetuando o processamento de diversas tarefas paralelas, se comportando como um único sistema.

Entretanto, a sua implementação deverá estar condicionada ao tipo de aplicação que será executada, tendo em vista que o seu funcionamento poderá variar conforme os problemas apresentados, muitas vezes não sendo tão eficientes quanto se espera. Nesses casos, é indispensável o uso de um supercomputador.

Para montar um *cluster* é necessário ter um *hardware* com vários núcleos no seu processador, um sistema operacional robusto e uma biblioteca de comunicação que faça a comunicação entre os nós (*nodes*) do *cluster*, para efetuar o paralelismo das tarefas.

Durante a montagem de um *cluster* é possível escolher várias combinações de *hardware*, mas, é essencial utilizar componentes que possuam o desempenho adequado para as tarefas que serão realizadas. Dentre os componentes utilizados na implementação de um *cluster*, alguns possuem um destaque maior, pois influenciam diretamente no seu desempenho, são eles: a memória de armazenamento (memória secundária), que precisa ter altas velocidades de leitura e escrita, além de uma boa capacidade de armazenamento, visto que, armazenará grandes volumes de dados; a memória *RAM* (*Random-Access Memory*), que precisa ter frequências elevadas para não prejudicar o desempenho do processador e, consequentemente do *cluster*, tendo em vista que serão responsáveis pelo armazenamento temporário dos processos durante a execução dos algoritmos de *benchmark*; a placa-mãe (*motherboard*), que precisa suportar a frequência máxima que as memórias *RAM* trabalham; e por fim, o processador, que precisa possuir, obrigatoriamente, vários núcleos, além de uma boa frequência de processamento.

Com base em estudos recentes, observa-se que os processadores baseados na arquitetura *ARM* (*Advanced RISC Machine*) ganharam destaque na implementação de *clusters* de alto desempenho (*High-Performance Computing – HPC*) e em *benchmarks* na área computacional, motivado pelo alto poder de processamento desses processadores, pela sua arquitetura ser baseada em *RISC* (*Reduced Instruction Set Computer*), pelo baixo custo de aquisição, além de sua eficiência energética (KALAPOTHAS *et al*., 2023).

Os processadores *ARM* possuem uma arquitetura do tipo *RISC* (*Reduced Instruction Set Computer* - Conjunto Reduzido de Instruções) de 64 *bits*, uma *pipeline* em três estágios e registradores do tipo *load/store*, fazendo com que o processador não fique ocioso e esteja sempre executando diferentes estágios de instruções. Além disso, esses processadores possuem sete modos diferentes de execução, que são utilizados para o tratamento de exceções e de interrupções, entre outras funcionalidades (JUNIOR, 2023).

Esses tipos de processadores possuem ainda: registradores de uso geral de 64 *bits*, *SP* (*stack pointer*) e *PC* (*program counter*); processamento de dados de 64 *bits*; endereçamento

virtual estendido; e estados de execução que suportam três conjuntos de instruções principais: *AArch32* (conjunto de instruções de tamanho fixo de 32 *bits*); *T32*: (*Thumb* e *Thumb-2*); e *AArch64* (conjunto de instruções de comprimento fixo de 64 *bits*) (JUNIOR, 2023).

Por serem baseados na arquitetura *RISC*, os processadores *ARM* têm um conjunto de instruções mais simplificada (reduzido), tornando-os processadores menos complexos e com um excelente desempenho energético (baixo consumo de energia), ideal para a criação de soluções computacionais embarcadas e de alto desempenho (*HPC - High-Performance Computing*) (ZOMAYA; LEE, 2012).

Um *hardware* que se destaca na utilização desse tipo de processador é a *Raspberry Pi*, uma plataforma *SBC* (*Single Board Computer*), ou seja, um tipo de computador que possui todos os componentes integrados em uma única placa. Esse tipo de computador possui algumas características importantes, tais como: portabilidade, baixo custo de aquisição, baixo consumo de energia e um poder de processamento elevado, tornando-o interessante e acessível para a implementação de *clusters* de alto desempenho (*HPC*) e de baixo custo (ARIZA; BAEZ, 2022; BASFORD *et al*., 2020).

Para tirar o máximo de eficiência do poder de processamento dos nós (*nodes*) que compõem um *cluster*, torna-se indispensável o uso de uma biblioteca da programação paralela (responsável pela comunicação e troca de mensagens entre as *nodes*), que trabalhe de forma simultânea e distribuída, na execução de tarefas complexas envolvendo grandes volumes de dados, atuando de forma eficiente no gerenciamento dos processos, além de oferecerem uma abstração para a criação e o gerenciamento de processos, sendo ainda confiáveis, escaláveis e tolerante a falhas, como é o caso da biblioteca *Apache Hadoop* (MA; ZHAO; ZHAO, 2023).

Portanto, esse trabalho teve por objetivo, na parte teórica, realizar um mapeamento sistemático da literatura sobre os estudos relacionados ao desenvolvimento de *clusters* de baixo custo e alto desempenho, identificando as principais arquiteturas, bibliotecas de paralelismo e as plataformas de *hardware* utilizadas nas suas respectivas implementações. Já na parte prática, foi implementado um *cluster* de baixo custo e alto desempenho, utilizando como plataforma de *hardware* a *Raspberry Pi 4* (*RPi*), um *hardware SBC* (*Single Board Computer*) baseado na arquitetura *Quadcore ARM* e, como solução de *software*, foi utilizada a biblioteca de paralelismo *Apache Hadoop*, para fazer a comunicação e a troca de mensagens entre as *nodes* do *cluster*.

1.1 Objetivos

1.1.1 Objetivo Geral

Desenvolver um *cluster* de baixo custo e alto desempenho (*HPC*), utilizando como plataforma de *hardware* a *Raspberry Pi 4* e, de *software*, a biblioteca de paralelismo *Apache Hadoop*, com o objetivo de executar algoritmos computacionais paralelos de *Big Data*, a fim de medir e analisar o desempenho do *cluster*.

1.1.2 Objetivos Específicos

1. Realizar uma pesquisa bibliográfica sobre os temas: *cluster*, arquitetura paralela, *Apache Hadoop* e algoritmos computacionais paralelos de *Big Data*, utilizados para medir e analisar o desempenho de *clusters*;
2. Instalar e configurar um *cluster* de baixo custo e alto desempenho, utilizando como *hardware* a plataforma *SBC* (*Single Board Computer*) *Raspberry Pi 4* e, como solução de *software*, para fazer a comunicação entre os nós (*nodes*) do *cluster*, a biblioteca de paralelismo *Apache Hadoop*;
3. Instalar e configurar duas ferramentas de monitoramento e gerenciamento do *cluster*, o *Zabbix* e o *Grafana*;
4. Realizar testes de *Benchmark*, através da execução de diferentes tipos de algoritmos computacionais paralelos de *Big Data*, a fim de testar e medir o desempenho do *cluster*;
5. Medir, coletar e analisar as seguintes métricas de desempenho do *cluster*: tempo de execução das tarefas de *map* e *reduce*; quantidade de tarefas de *map* e *reduce* realizadas pelos nós (*nodes*) do *cluster* durante o processamento dos algoritmos paralelos de *Big Data*; quantidade de processos da executados pela *CPU*; percentual de utilização da *CPU*; percentual de uso da memória *RAM*; temperatura das *nodes*; taxa de utilização da rede; e consumo de energia do *cluster*.
6. Analisar o desempenho do *cluster* durante os testes de *Benchmark*;

1.2 Justificativa e Relevância

A evolução tecnológica tem provocado e impulsionado a necessidade do aumento da capacidade do poder de processamento dos computadores, a fim de que esses equipamentos sejam capazes de executar tarefas mais complexas e robustas, que envolvem grandes volumes de dados (*Big Data*) (COSTA, 2007). Mas, a produção de computadores dessa natureza, com elevado poder de processamento, é prejudicada pelos altos investimentos financeiros, necessários para a sua fabricação e manutenção, acompanhado pelo aumento do consumo de energia que eles possuem, necessário para que possam alcançar altas frequências de processamento.

Dessa forma, uma das alternativas viáveis, utilizadas para minimizar e/ou solucionar esses problemas, é a implementação de *clusters* de baixo custo financeiro e energético, utilizando para isso plataformas de *hardware* mais compactas (*SBC - Single Board Computer*), simples e baratas, de fácil programação, que sejam mais acessíveis para os usuários, academia e/ou empresas, para a realização de tarefas paralelas simples e/ou complexas, que envolvam a execução de grandes volumes de dados (*Big Data*).

Portanto, para tentar suprir as dificuldades e demandas apresentadas na execução de tarefas paralelas (paralelismo), que envolvem elevado poder de processamento dos equipamentos, esse trabalho teve por finalidade propor uma solução computacional de baixo custo financeiro e energético, através do uso da plataforma de *hardware Raspberry Pi 4* e da biblioteca de paralelismo *Apache Hadoop*.

1.3 Delimitações do Estudo

Este estudo trata do desenvolvimento de um *cluster* de baixo custo e alto desempenho, através da utilização de uma plataforma de *hardware SBC*, a *Raspberry Pi*, e do uso de uma biblioteca de paralelismo para a execução de algoritmos paralelos de *Big Data*, o *Apache Hadoop*.

1.4 Estrutura do Trabalho

Este trabalho encontra-se estruturado da seguinte forma:

1. **Primeiro Capítulo:** Faz uma explanação sobre os objetivos geral e específicos do

trabalho, trazendo a justificativa e a sua relevância, além das delimitações do estudo em questão;

2. **Segundo Capítulo:** Apresenta uma visão geral da literatura sobre os seguintes temas: arquiteturas paralelas, arquitetura em *cluster*, bibliotecas de paralelismo, *Apache Hadoop*, programação paralela, testes de *benchmarks*, *Big Data* e ferramentas utilizadas no gerenciamento e monitoramento das métricas de desempenho de um *cluster*;
3. **Terceiro Capítulo:** Aborda alguns trabalhos relacionados aos temas pesquisados neste trabalho, mostrando as principais infraestruturas de *hardware* utilizadas no desenvolvimento dos c*lusters*, as bibliotecas de paralelismo implementadas, as métricas usadas para medir e analisar o desempenho dos *clusters*, bem como os tipos de testes de *benchmark* executados;
4. **Quarto Capítulo:** Apresenta a metodologia a ser implementada neste trabalho, bem como as técnicas de coleta e análise dos dados utilizados no seu desenvolvimento, detalhando cada etapa necessária para a sua realização;
5. **Quinto Capítulo:** Mostra uma análise detalhada dos resultados obtidos referentes aos cenários de testes executados, bem como os seus respectivos desdobramentos;
6. **Sexto Capítulo:** Apresenta as considerações finais do trabalho, as principais descobertas sobre as investigações realizadas, além da sua contribuição para pesquisas futuras.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Arquiteturas Paralelas

As arquiteturas paralelas são estruturas que utilizam múltiplas unidades de processamento paralelo, que cooperam entre si na resolução de problemas complexos, desenvolvidas para obter o máximo de poder de processamento dos computadores, explorando diferentes níveis de paralelismo (SCHEPKE, 2009).

Porém, para obter o paralelismo é necessário que os processos ou fluxos de controle (*threads*) sejam executados de forma simultânea e distribuída, portanto, é essencial ter um *hardware*, ou um conjunto de *hardware* (*cluster*), que possa auxiliar na execução dessas tarefas (NAVAUX; ROSE; PILLA, 2011). Por esse motivo, a indústria dessa área vem projetando e desenvolvendo diferentes tipos de arquiteturas computacionais, visando atender à toda essa demanda computacional (SCHEPKE, 2009).

Entretanto, as arquiteturas paralelas podem ser melhor compreendidas através da classificação proposta por Michael J. Flynn (renomado cientista da área de computação da Universidade de *Stanford*, nos Estados Unidos), em 1966, denominada de Taxonomia de Flynn. Essa Taxonomia baseia-se na capacidade dos computadores em lidar com os fluxos de instruções que podem ser executados em um determinado instante e com os fluxos de dados que são processados em um mesmo instante (COELHO, 2012).

A classificação das arquiteturas de computadores proposta por Flynn, encontra-se organizada em quatro classes, são elas: *SISD* (*Single Instruction, Single Data*), *SIMD* (*Single Instruction, Multiple Data*), *MISD* (*Multiple Instruction, Single Data*) e *MIMD* (*Multiple Instruction, Multiple Data*) (HENNESY, 2012):

- ***Single Instruction, Single Data* (*SISD*):** Uma única unidade de controle (processador) é responsável pelo processamento de um único fluxo de instruções em um único fluxo de dados;
- ***Single Instruction, Multiple Data* (*SIMD*):** Uma única instrução é executada de forma simultânea em múltiplos fluxos de dados. Existe apenas uma unidade de controle, que é responsável por coordenar vários processadores para a execução paralela de apenas uma instrução. Para que todos os processadores consigam acessar a memória simultaneamente, a mesma deverá ser separada em vários módulos;

- ***Multiple Instruction, Single Data* (*MISD*):** Nessa classe, vários fluxos de instruções são processados sob um mesmo fluxo de dados, onde cada unidade de controle presente no sistema, possui um processador, sendo este responsável por executar diferentes instruções no mesmo fluxo de dados;
- ***Multiple Instruction, Multiple Data* (*MIMD*):** Vários fluxos de instruções são processados em vários fluxos de dados, onde cada unidade de controle possui um processador, que é responsável por executar diferentes instruções em diferentes fluxos de dados.

Com relação ao subsistema de memória, as arquiteturas paralelas podem ser classificadas em três categorias, a saber (PERISSATTO, 2007; COELHO, 2012):

- **Memória Distribuída:** Cada processador, pertencente a arquitetura, possui sua própria memória local, bem como um espaço de endereçamento local onde as tarefas serão executadas, não sendo visíveis aos outros processadores da arquitetura. As tarefas de comunicação entre os processos são realizadas por meio de troca de mensagens, ficando a cargo do programador o gerenciamento do trânsito desses dados;
- **Memória Compartilhada:** Todos os processadores envolvidos na arquitetura compartilham o mesmo espaço de endereçamento, sendo possível que dados manipulados durante a execução de uma tarefa, por um processador, sejam visíveis por outros processadores pertencentes a arquitetura. As rotinas de comunicação entre os processos são realizadas através do acesso a variáveis compartilhadas. Dessa forma, o programador não precisa se preocupar com a forma na qual os dados trafegam entre os processadores;
- **Memória Compartilhada Distribuída:** Cada processador envolvido na arquitetura possui uma memória local. Porém, cada processador consegue ter acesso aos espaços de endereçamento dos outros processadores pertencentes a arquitetura. As tarefas de movimentação e de acesso aos dados são implementadas pelo *hardware*, através da utilização de rotinas de envio e recebimento de mensagens. Esse tipo de categoria visa associar as melhores características das outras duas categorias mencionadas, adicionando a possibilidade de o programador fazer a distribuição dos dados nas memórias locais da forma que achar mais interessante.

As arquiteturas paralelas podem ser classificadas e organizadas de acordo com suas características e aplicações, a saber: máquinas *multicore*, multiprocessadores,

multicomputadores (*clusters*), vetorial e sistema de *grid* (formado por um conjunto computadores organizados em um sistema de grade, disponíveis em escala mundial na Internet).

Uma **arquitetura multiprocessada** consiste em um conjunto de processadores operando sobre um conjunto de memória, sendo que essas memórias podem ser acessadas de maneira uniforme, na mesma latência (*Uniform Memory Access - UMA*) ou não uniforme (*Non-Uniform Memory Access - NUMA*) (NAVAUX; ROSE; PILLA, 2011).

No acesso *UMA* (*Uniform Memory Access*), existe apenas uma única memória no sistema, compartilhada entre todos os processadores. Assim, o tempo necessário para acessar a memória é o mesmo para todos os processadores, mas não é possível realizar o acesso simultâneo (NAVAUX; ROSE; PILLA, 2011).

Já no acesso *NUMA* (*Non-Uniform Memory Access*), a memória é dividida em vários módulos, onde cada processador possui um deles. Nesse método de acesso, o tempo de acesso à memória pode variar, pois cada processador pode acessar a própria memória ou a memória dos outros processadores em tempos distintos. Um problema desse método de acesso é a dificuldade de realizar o gerenciamento das tarefas, quando se utiliza muitos processadores (NAVAUX; ROSE; PILLA, 2011).

É importante mencionar, que a escalabilidade de uma máquina multiprocessada é limitada, portanto, uma vez que a arquitetura seja criada, não será mais possível a adição de novos processadores. Além disso, quando se utiliza um número elevado de processadores, fica mais difícil fazer o gerenciamento desse tipo de arquitetura (SCHEPKE, 2009; NAVAUX; ROSE; PILLA, 2011).

A **arquitetura de multicomputadores (*clusters*)** pode ser vista como uma arquitetura constituída por um conjunto de computadores independentes, interconectados através de uma rede de computadores, onde cada computador possui sua própria unidade de processamento (*CPU - Central Processing Unit*) e memória *RAM*, sendo que a memória de um computador não poderá ser acessada diretamente por outro computador da arquitetura (ANDREWS, 2001; NAVAUX; ROSE; PILLA, 2011).

Neste tipo de arquitetura, cada computador, integrante do *cluster*, possui seu próprio espaço de endereçamento, permitindo o acesso somente aos dados disponíveis na sua memória. Através de uma rede de interconexão, é possível que os processadores dessa arquitetura troquem mensagens entre os processos. Essa troca de mensagens é determinada pelo código do programa que está sendo utilizado, no caso, um algoritmo paralelo (SCHEPKE, 2009; NAVAUX; ROSE; PILLA, 2011).

Uma **arquitetura vetorial** tem por objetivo prover instruções de alto-nível, através da utilização de vetores de dados, permitindo dessa forma a realização de operações aritméticas em um vetor ou entre vetores, sem a necessidade da implementação de um laço (*loop*) no programa, como ocorre nas arquiteturas não vetoriais (DONGARRA; DUFF; SORENSEN, 1991; NAVAUX; ROSE; PILLA, 2011).

Na arquitetura vetorial, a exploração do paralelismo ocorre a nível de dados, ao invés de tarefas. Esse tipo de arquitetura é ideal para cálculos científicos, mineração de dados e processamento de imagens, bem como para outros tipos de operações que exigem a execução de diversos cálculos sobre um conjunto de dados vetorizáveis. Os processadores vetoriais apresentam uma redução do número de instruções por programa, no custo de decodificação e nas falhas de memória *cache* (SCHEPKE, 2009; NAVAUX; ROSE; PILLA, 2011).

As **arquiteturas *multicore*** são formadas por processadores que possuem múltiplos núcleos de processamento. Dessa forma, múltiplos programas ou fluxo de execuções podem ser executados de forma simultânea e distribuída, conforme a quantidade de núcleos existentes na máquina. Portanto, é possível obter um nível maior de desempenho dos processadores durante a execução das tarefas. Além disso, os recursos da memória *cache* podem ser compartilhados entre os diferentes núcleos de processamento (GEPNER; KOWALIK, 2006; NAVAUX; ROSE; PILLA, 2011).

Já uma **arquitetura em *grid***, consiste em uma enorme infraestrutura de *hardware* e *software*, capaz de prover um elevado poder de processamento para a execução de aplicações que envolvem grandes volumes de tarefas e recursos computacionais (BERNHOLDT, 2007; NAVAUX; ROSE; PILLA, 2011). Porém, a construção desse tipo de arquitetura enfrenta vários desafios, entre eles: o gerenciamento dos recursos; a grande heterogeneidade de máquinas e de conexões de rede; a grande escalabilidade; o controle de acesso aos administradores e usuários; e as interfaces de programação (SCHEPKE, 2009), além da largura de banda utilizada na rede (Internet) e o tipo de infraestrutura de rede utilizada nesse tipo de arquitetura.

2.2 Arquitetura em *Cluster*

Uma das formas de explorar o paralelismo de *hardware* é através da construção de sistemas que realizam o processamento paralelo por meio da associação de dois ou mais computadores (*nodes*), conectados entre si por meio de uma rede local de computadores (*LAN - Local Area Network*), denominados de *clusters* (COSTA, 2007).

Os *clusters* são sistemas fracamente acoplados, onde cada computador possui sua própria memória individual, vistos como um sistema único e coerente, realizando o gerenciamento de memória necessário para manter a consistência das atividades computacionais a serem realizadas. Além disso, são uma forma de contornar o problema relacionado aos custo elevados de aquisição e manutenção de supercomputadores, tornando-se uma alternativa viável para o desenvolvimento de pesquisas, dentre outras atividades na área computacional (COSTA, 2007).

O objetivo de uma arquitetura em *cluster* é obter o máximo de poder de processamento dos computadores (*nodes*) que estão integrados ao sistema, necessário para a execução de tarefas complexas, que envolvem grandes de volumes da dados (*Big Data*). Em um *cluster*, existe um computador principal denominado de mestre (*master*), responsável por coordenar a execução das tarefas e controlar os outros computadores, denominados de nós (*nodes*), ou *slaves*, que cooperam entre si para a realização das tarefas. Para realizar a comunicação entre as *nodes* e a distribuição eficiente das tarefas, é necessário a utilização de um *software*, uma biblioteca de paralelismo, que seja capaz de fornecer a ideia de um sistema único para o usuário, totalmente transparente, responsável por permitir a comunicação entre todas as *nodes* que compõe o *cluster*, bem como a troca de mensagens entre eles, através de uma interconexão de rede (BACELLAR, 2009).

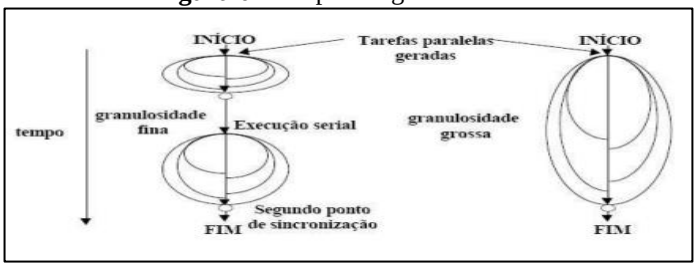
A implementação de um *cluster* deverá levar em consideração a aplicação a ser executada, tendo em vista que esse tipo de sistema não é tão eficiente quanto um supercomputador, no caso da resolução de problemas que envolvem uma granulosidade fina (divisão de um sistema em partes pequenas). No entanto, para problemas que envolvem uma granulosidade grossa (divisão de um sistema em parte maiores, com complexidade no conjunto de dados), os *clusters* têm se mostrado tão eficientes quanto os supercomputadores (BACELLAR, 2009).

O motivo dos *clusters* não terem se mostrado muito eficientes para tratar problemas de granulosidade fina, se deve ao fato da necessidade de uma troca mais intensa de informações entre a *node* mestre e as *nodes slaves*. Esse tipo de atividade provoca um aumento significativo no tráfego da rede, interferindo de forma direta no tempo de execução das tarefas, comprometendo o objetivo proposto pelo paralelismo, que é o ganho de tempo na execução das atividades. Já no caso de problemas que envolvem granulosidade grossa, onde é exigido um elevado poder de processamento de dados, os *clusters* têm se tornado uma solução viável, uma vez que as *nodes* recebem, de forma independente, uma carga elevada de dados para processar,

diminuindo a troca de informações na rede entre as *nodes* (BACELLAR, 2009). Além disso, em um *cluster*, todo o gerenciamento de memória compartilhada, necessário para manter o sistema computacional operando de forma consistente, é realizado de maneira confiável e segura (LIMA *et al.*, 2016).

Na Figura 01, observa-se os tipos de granulosidade existentes na execução de tarefas paralelas.

Figura 01 – Tipos de granulosidade



Fonte: Adaptada de Bacellar, 2009.

Em uma arquitetura em *cluster*, os maiores problemas estão relacionados no nível de divisão das tarefas e nas comunicações entre os computadores (*nodes*) que fazem parte do sistema. Os *clusters* são menos eficientes que os supercomputadores nas tarefas que envolvem granulosidade fina, devido ao aumento no número de troca de informações entre as *nodes* e por causa da redução da necessidade de processamento bruto. No caso da granulosidade grossa, os *clusters* se sobressaem em relação aos supercomputadores, pois as transações envolvem uma menor quantidade de troca de informações, bem como um aumento da necessidade de processamento bruto (LIMA *et al.*, 2016).

Na computação de alto desempenho (*HPC*), os *clusters* são amplamente utilizados por possuírem acesso restrito e dedicado, além de serem uma arquitetura confiável, escalável e flexível. A gestão de um *cluster* envolve diversos fatores, que vão desde a instalação do sistema operacional até a definição de ferramentas para sua configuração, manutenção, monitoramento e agendamento de tarefas (SCHEPKE, 2009). Além disso, os *clusters* possuem uma série de benefícios que contribuem para sua utilização em *HPC*, tais como: desempenho, expansibilidade, baixo custo, alta disponibilidade, balanceamento de carga e tolerância a falhas (BACELLAR, 2009; LIMA *et al.*, 2016).

Na literatura, são identificados diferentes tipos de *clusters*, onde cada modelo possui suas próprias características e situações em que podem ser aplicados, a saber:

- **Alto desempenho:** Consiste no agrupamento de computadores para a obtenção de um elevado poder de processamento, permitindo a execução de algoritmos mais complexos, que podem processar grandes volumes de dados em um curto espaço de tempo (NASCIMENTO; SOUZA, 2020);
- **Balanceamento de carga:** Utilizado para dividir as requisições de um sistema, onde as solicitações são divididas entre as *nodes* que fazem parte do *cluster*. Dessa forma, todos os recursos são utilizados igualmente, mantendo um baixo tempo de resposta das tarefas executadas (SHARMA, 2020);
- **Alta disponibilidade:** Utilizado para garantir que um serviço permaneça sempre ativo. Sendo assim, todas as *nodes* que fazem parte do *cluster* devem possuir as mesmas informações e, caso o computador principal (*master*) apresente uma falha, um outro computador será designado para manter a disponibilidade do sistema, até que o problema seja corrigido (SHARMA, 2020).

2.3 Bibliotecas de Paralelismo

Para resolver problemas computacionais em paralelo, os códigos (algoritmos) devem ser escritos de forma a explorar ao máximo o paralelismo de *hardware*, exigindo o domínio de técnicas de programação paralela, bem como o uso de bibliotecas que suportem o paralelismo, tais como: a *MPI* (*Message Passing Interface*), a *OpenMP* (*Open Multi-Processing*), a *PVM* (*Parallel Virual Machine*), o *Apache Hadoop*, entre outras.

Em um *cluster*, a biblioteca de paralelismo é responsável por gerenciar os recursos disponíveis no sistema, realizar a comunicação entre os nós (*nodes*) e coordenar a execução dos algoritmos paralelos. Cada biblioteca possui suas próprias características e peculiaridades, permitindo a obtenção do melhor aproveitamento dos recursos computacionais do *cluster*, sendo implementada e utilizada de forma distinta na resolução de cada problema, a fim de garantir que as suas funcionalidades sejam adequadas à necessidade da tarefa a ser executada (BARKER, 2015; SANTOS, 2013).

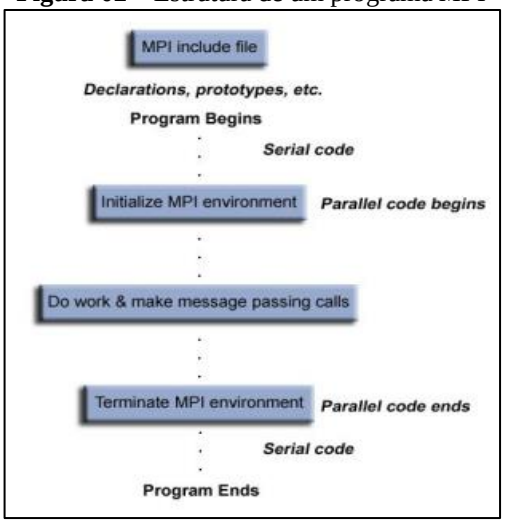
No mercado de *software* existem diversas tipos de bibliotecas de paralelismo disponíveis, sendo que cada uma delas possuem características específicas e, por isso, é necessário identificar o contexto em que elas serão utilizadas, para garantir que as funcionalidades disponibilizadas pelas mesmas sejam adequadas à necessidade do problema a ser resolvido (SANTOS, 2013).

A biblioteca de paralelismo *MPI* (*Message Passing Interface*) consiste em um modelo da programação paralela (conjunto de funções) para multiprocessamento baseado em *message-passing* (passagem-de-mensagem), que permite sistemas paralelos (multiprocessados), com memória distribuída, trocarem mensagens entre si, bem como compartilharem dados e instruções de controle, otimizando a comunicação e aumentando o desempenho das aplicações paralelas ou distribuídas (IGNÁCIO; FILHO, 2002).

As principais vantagens desta biblioteca são: escalabilidade, compatibilidade, disponibilidade e portabilidade, devido aos diferentes tipos de *hardware* que podem ser utilizados na sua implementação. Como desvantagem, a *MPI* apresenta dificuldade no gerenciamento das tarefas, já que toda a implementação e suas funções não são administradas pela própria biblioteca. Sendo assim, é necessário ter um conhecimento avançado de programação para tentar contornar essa situação. Além disso, a *MPI* não permite a paralelização incremental, possuindo uma curva de aprendizagem longa (RIBEIRO; FERNANDES, 2013; MPI, 2024).

A *MPI* pode ser implementada em qualquer tipo de *hardware* que possua memória distribuída e em plataformas de memória compartilhada, tais como: as arquiteturas *SMPs* (multiprocessadores simétricos) e *NUMA* (*Non-Uniform Memory Access*), tornando-se mais recomendada a sua utilização em plataformas híbridas, como *clusters* formados por máquinas *NUMA*. Na *MPI*, todo o paralelismo é explícito, de responsabilidade do programador e implementado através da utilização dos construtores da biblioteca no próprio código do programa que está sendo escrito (RIBEIRO; FERNANDES, 2013), como pode ser observado na Figura 02.

Figura 02 – Estrutura de um programa *MPI*



Fonte: Adaptada de Barney, 2021.

A *OpenMP* (*Open Multi-Processing*) é uma *API* (*Application Programming Interfaces*) utilizada para prover o processamento paralelo de algoritmos entre os núcleos de um processador com memória compartilhada, ou seja, várias *threads* são alocadas e executadas de forma simultânea através dos núcleos do processador. Esse processo consiste na utilização de diretivas de compilação, rotinas de biblioteca e de variáveis de ambiente, onde tem-se a paralelização do algoritmo de maneira que seu código sequencial seja modificado em sua maior granularidade e, pelo fato de utilizar memória compartilhada, suas *threads* vão se comunicar através de variáveis compartilhadas (RODRIGUES, 2012; OPENMP, 2024).

A utilização de variáveis compartilhadas pela *OpenMP* exige um maior controle de acesso, a fim de evitar problemas com inconsistência dos dados ou resultados imprecisos. Portanto, quanto menor for a utilização destas variáveis e a dependência entre as *threads*, melhor será o desempenho do algoritmo executado, diminuindo possíveis gargalos causados pelos acessos ou pela espera dos mesmos a estas variáveis (RODRIGUES, 2012).

Além disso, a *OpenMP* é uma *API* portátil, escalável e explícita, que oferece total controle ao programador, podendo ser utilizada em diferentes tipos de plataformas de arquitetura de memória compartilhada, desde computadores pessoais até supercomputadores (OPENMP, 2024).

Já a *PVM* (*Parallel Virtual Machine*), por sua vez, consiste em uma biblioteca de comunicação que disponibiliza uma coleção de ferramentas de *software*, utilizadas para emular uma máquina paralela em um conjunto de computadores, cuja função é emular computação concorrente heterogênea de propósito geral em máquinas interconectadas, utilizando para isso rotinas básicas, enviando e recebendo mensagens (BACELLAR, 2009).

Essa biblioteca possui suporte para as linguagens de programação *C*, *C++*, *Fortran*, entre outras, tendo como unidade básica, a tarefa, uma sequência independente de instruções que se comunica através da troca de mensagens com outras tarefas, sem a "necessidade de executar uma tarefa por processador" (MORETTI; BITTENCOURT, 1997).

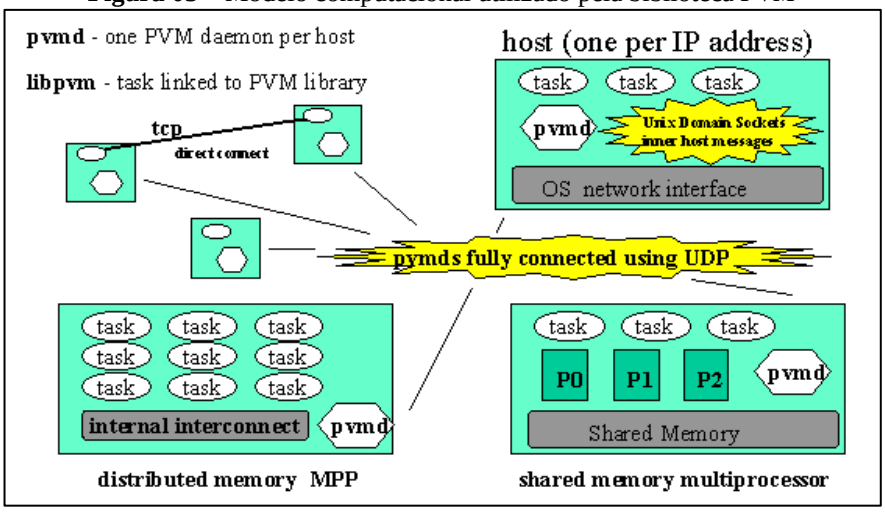
Para obter o paralelismo, a *PVM* adiciona uma camada de *software* sobre os protocolos de comunicação da rede, para gerenciar o envio dos processos para suas máquinas, sendo considerada uma arquitetura multicomputador (*cluster*) que possui alta disponibilidade e alta performance (TAMEHIRO *et al.*, 2012).

O modelo computacional utilizado pela *PVM* baseia-se no processo, permitindo a execução de múltiplas tarefas em um único processador. Em alguns casos, as implementações das tarefas podem não corresponder a um processo. Nesses casos, o programador terá a

responsabilidade de garantir a compatibilidade (MORETTI; BITTENCOURT, 1997).

Na Figura 03, observa-se o *design* do modelo computacional utilizado pela biblioteca de paralelismo *PVM*.

Figura 03 – Modelo computacional utilizado pela biblioteca *PVM*



Fonte: Adaptada de Geist *et.al.*, 1994.

Por fim, completando a lista das principais bibliotecas de paralelismo utilizadas no desenvolvimento de *clusters* de alto desempenho, tem-se o *Apache Hadoop*, uma estrutura de *software open-source* (*framework*), escrita em *JAVA* e desenvolvida especialmente para trabalhar com computação distribuída, em *cluster* formado por *hardwares* comuns.

Essa biblioteca disponibiliza uma série de funcionalidades para processar e armazenar grandes volumes de dados (*Big Data*), além de possuir características importantes, tais como: tolerância a falhas e escalabilidade, o que a torna uma solução popular em ambientes acadêmicos e empresariais (POLATO; RÉ; GOLDMAN; KON, 2014; HANNAN, 2016).

A estrutura da biblioteca *Apache Hadoop* possui alguns elementos essenciais para sua utilização, tais como: o *HDFS*, que é o sistema para armazenamento dos arquivos; o *MapReduce*, uma estrutura responsável por processar os dados; e alguns outros componentes, que auxiliam durante a execução dos algoritmos de processamento paralelo (BHOSALE; GADEKAR, 2014).

2.4 *Apache Hadoop*

O *Apache Hadoop* é uma biblioteca de paralelismo de código aberto, utilizada na execução de tarefas que necessitam de um elevado poder de processamento dos computadores,

para a manipulação de grandes volumes de dados (*Big Data*). Essa biblioteca é responsável por gerenciar os recursos de todos os computadores que compõem o sistema do *cluster*, bem como coordenar a execução dos algoritmos paralelos de processamento de dados (MENEZES; FREITAS; PARPINELLI, 2016).

As vantagens da utilização desta biblioteca são: o fato dela ser uma ferramenta de código aberto (*open-source*); a economia na sua implantação, por não ser necessário gastos com licenças e contratação de pessoal especializado; sua robustez, oferecendo estratégias de recuperação automática, no caso de situações de falhas dos equipamentos; escalabilidade, podendo aumentar a quantidade de máquinas (*nodes*) utilizadas no processamento dos dados, quando necessário, sem a necessidade de grandes alterações na codificação; e simplicidade, pelo fato de retirar do programador a responsabilidade de ter que gerenciar questões referentes à computação paralela, tais como: tolerância a falhas, escalonamento e balanceamento de carga, ficando essas tarefas a cargo da própria ferramenta (GOLDMAN *et al.*, 2012).

A biblioteca *Apache Hadoop* pode ser instalada em qualquer máquina que possua o sistema operacional *Linux*, com no mínimo 4 *GBytes* de memória *RAM* (*Random Access Memory*), podendo ser implementada em servidores ou em *clusters* (aglomeração de máquinas), caracterizando-se por ser uma ferramenta de código aberto, econômica, robusta e escalável (APACHE HADOOP, 2024).

Porém, o *Apache Hadoop* apresenta algumas desvantagens, dentre elas destacam-se: o fato da ferramenta estar em constante evolução, não conseguindo suportar todas as situações às quais está sendo submetida, devido a algumas de suas funcionalidades não estarem maduras o suficiente; apesar de poder utilizar várias máquinas como *nodes*, para o processamento e armazenamento dos dados, possui apenas um único *node* mestre, o *master*, criando dessa forma um ponto crítico suscetível a falha, vital para o funcionamento da aplicação; dificuldade no gerenciamento das *nodes* e na depuração de falhas, bem como na análise dos *logs*, que encontram-se distribuídos; problemas não paralelizáveis ou com alta dependência entre os dados, que possuem dificuldade de divisão de tarefas em tarefas menores, devido a impossibilidade de extração da porção paralelizável da aplicação, não sendo, portanto, bons candidatos para o *Apache Hadoop*; e o processamento de arquivos pequenos (GOLDMAN *et al.*, 2012).

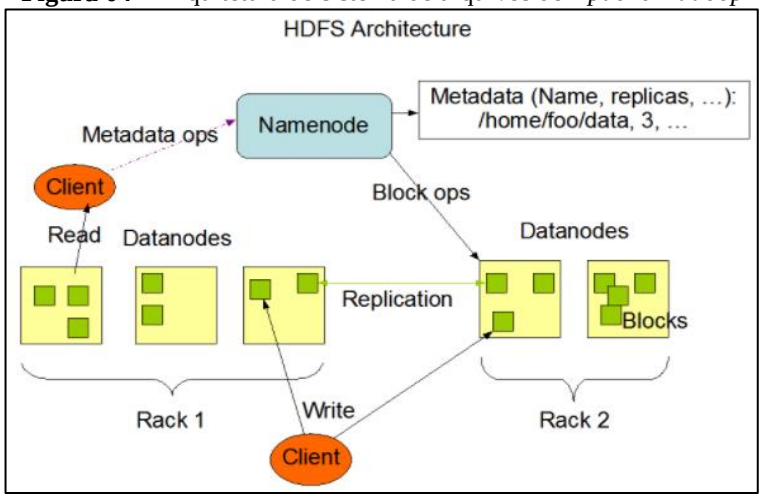
O *Apache Hadoop* possui semelhanças com os sistemas de arquivos distribuídos existentes, porém, é altamente tolerante a falhas e foi projetado para implementações em *hardware* de baixo custo, como a *Raspberry Pi*. Além disso, fornece uma taxa elevada de

transferência de acesso aos dados da aplicação, sendo adequado para trabalhar com aplicações que envolvem grandes volumes de dados, como é o caso de *Big Data* (APACHE HADOOP, 2024).

Sua estrutura possui alguns elementos essenciais, tais como: o *HDFS* (*Hadoop Distributed File System*), sistema responsável pelo armazenamento de arquivos; e o *MapReduce*, estrutura responsável por processar os dados. Além de alguns outros componentes que auxiliam durante a execução e o processamento dos algoritmos paralelos (GOLDMAN *et al*., 2012; BHOSALE; GADEKAR, 2014).

O *HDFS* (*Hadoop Distributed File System*) é o sistema de arquivos do *Apache Hadoop*, utilizado para armazenar e transmitir grandes volumes de dados. Nesse sistema, é utilizada a arquitetura cliente-servidor, em que o *NameNode*, localizado no computador *master* do *cluster*, é responsável por gerenciar a localização dos arquivos e realizar o controle de acesso. Já o *DataNode*, presente nos computadores *slaves* (escravos) do *cluster*, é responsável por armazenar os arquivos e buscá-los, quando for solicitado pelo *master* (MENEZES; FREITAS; PARPINELLI, 2016; MEHTA; MEHTA, 2016), como pode ser observado na Figura 04.

Figura 04 – Arquitetura do sistema de arquivos do *Apache Hadoop*



Fonte: Adaptado de *https://hadoop.apache.org/*, 2024.

O *NameNode* e o *DataNode* são *softwares* projetados para serem executados em máquinas convencionais, que utilizam um sistema operacional *Linux* (BORTHAKUR, 2007).

O *NameNode* é responsável por executar as operações de *namespace* do sistema de arquivos, tais como: abrir, fechar e renomear arquivos, e pastas, bem como determinar o mapeamento de blocos para os *DataNodes*. Em um sistema *HDFS*, a existência de apenas um único *NameNode* no *cluster*, torna sua arquitetura mais simples. Já o *DataNode*, é responsável

por atender às solicitações de leitura e gravação das *nodes* clientes do sistema de arquivos (APACHE HADOOP, 2024).

No sistema *HDFS*, um arquivo é dividido em um ou mais blocos, armazenados em um conjunto de *DataNodes*. Além disso, o sistema é projetado de tal forma que não permite os dados dos usuários fluírem através do *NameNode* (SHVACHKO, 2010).

Como o *HDFS* é desenvolvido em *JAVA*, tanto o *NameNode* quanto o *DataNode*, podem ser executados em qualquer máquina que possua suporte a essa linguagem de programação, ou seja, podem ser executados em uma ampla variedade de máquinas. Em uma implementação convencional do *HDFS*, temos uma máquina (*master*) dedicada que roda o *software* do *NameNode* e várias outras máquinas (*nodes*), a depender do tamanho do *cluster*, que rodam uma instância do *software* do *DataNode* (BORTHAKUR, 2007).

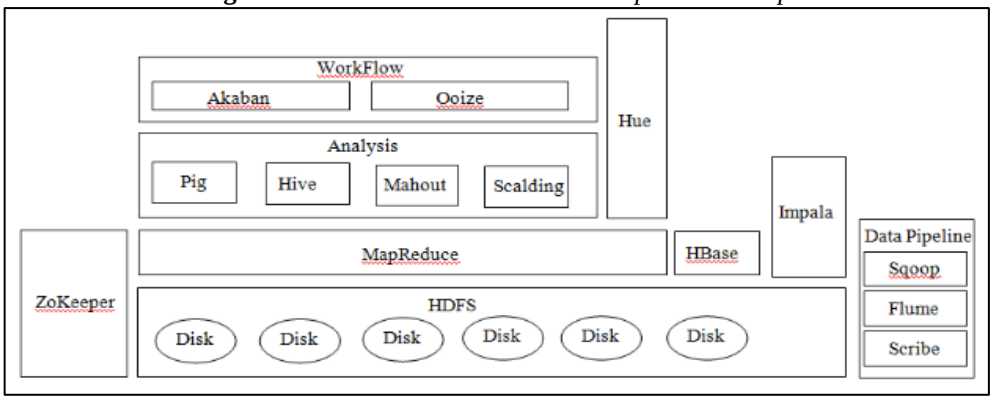
O *MapReduce* é um modelo de programação utilizado pela biblioteca de paralelismo *Apache Hadoop* para realizar o processamento paralelo de grandes volumes de dados, possuindo duas funções principais, o *Map* e o *Reduce*. Durante a execução dos algoritmos paralelos, o computador *master* envia os dados de entrada para os computadores *slaves* selecionados para a tarefa de *Map*, que transformam os dados para o formato chave-valor, para em seguida, utilizarem a função *Map* para a realização do processamento e geração de novos pares de dados, armazenando-os na memória. Em seguida, o *master* aciona os *slaves*, selecionados para a tarefa de *Reduce*, para buscarem os arquivos intermediários que foram gerados, e então, utilizarem a função *Reduce* para realizarem o processamento final dos dados, gerando um arquivo final, utilizado pelo *master* para retornar os resultados da tarefa (ANDRADE, 2012; MENEZES; FREITAS; PARPINELLI, 2016).

Durante a execução das aplicações no *Apache Hadoop* são utilizados cinco processos, são eles: o *NameNode*, o *DataNode*, o *JobTracker*, o *TaskTracker* e o *SecondaryNameNode*. O *NameNode* é responsável por gerenciar o *HDFS*, sendo algumas de suas atividades: gerenciar o sistema de arquivos, monitorar o comportamento dos *DataNodes* e a capacidade de armazenamento do *cluster*. O *DataNode* armazena os arquivos e envia continuamente informações dos dados armazenados para o *NameNode*. O *JobTracker* é responsável por monitorar as tarefas e gerenciar os recursos utilizados durante a execução das aplicações. O *TaskTracker*, presente em cada *slave*, é responsável por executar as tarefas *MapReduce* que foram atribuídas pelo *JobTracker*. O *SecondaryNameNode* é responsável por salvar o estado do *NameNode* em intervalos pré-definidos, permitindo a recuperação, caso ocorra uma falha (GOLDMAN *et al*., 2012; VASUJA; BHANDRALIA; CHUCHRA, 2018; MATHUR; JAIN,

2020).

Na estrutura do *Apache Hadoop* são disponibilizadas algumas ferramentas opcionais que fazem parte do seu ecossistema, a saber: *HBase*, *Hive*, *Pig*, *Mahout*, *Avro*, *Sqoop*, *Oozie*, *Chukwa*, *Flume* e *Zookeeper*, como pode ser observado na Figura 05.

Figura 05 – Ecossistema da biblioteca *Apache Hadoop*



Fonte: Adaptada de Uzunkaya; Ensari; Kavurucu, 2015.

Essas ferramentas fornecem novas funcionalidades à biblioteca, podendo ser úteis para projetos com necessidades específicas (GOLDMAN *et al*., 2012; MEHTA; MEHTA, 2016), a saber:

O *HBase* é um banco de dados não relacional, escalável, distribuído e orientado a colunas, criado sobre o *HDFS* e utilizado para armazenar grandes volumes de dados. Na sua estrutura são utilizadas tabelas com controle de alterações, para permitir a recuperação de informações (GOLDMAN *et al*., 2012). Além disso, esse banco de dados possui um acesso aleatório para a realização de leitura e gravação de dados em tempo real, disponibilizando um armazenamento otimizado, fornecendo uma consistência total, alta disponibilidade, arquitetura de escala horizontal e escala horizontal de dados de forma transparente, sendo eficiente entre máquinas no *cluster* (UZUNKAYA; ENSARI; KAVURUCU, 2015; MEHTA; MEHTA, 2016).

O *Hive* é um pacote de armazenamento de dados com suporte à linguagem de consulta *HiveQL* do *Apache Hadoop*. Essa linguagem permite que suas consultas sejam convertidas para *MapReduce*, otimizando a execução das tarefas, com funções semelhantes ao *SQL* (*Structured Query Language*) para dados no *HDFS*, com suporte as seguintes funções: *SELECT*, *JOIN*, *GROUP BY*, entre outras, facilitando as análises realizadas durante a execução das aplicações, além de permitir a integração com outras ferramentas de gerenciamento de dados (GOLDMAN *et al*., 2012; UZUNKAYA; ENSARI; KAVURUCU, 2015; MEHTA; MEHTA, 2016).

O *Pig* é uma plataforma utilizada para realizar análises de grandes volumes de dados, sendo uma abstração alternativa, que roda em cima da programação *MapReduce*, trabalhando com linguagens de *scripts* orientada a fluxo de dados especiais, tais como: o *Pig Latin*. Essa linguagem possui uma codificação simples, de fácil compreensão e sua estrutura permite que os códigos sejam transformados em programas *MapReduce*. Dessa forma, o sistema é capaz de otimizar a execução das tarefas automaticamente (GOLDMAN *et al*., 2012; MEHTA; MEHTA, 2016). Além disso, é recomendada para processamento em lote (*batch*) de grandes conjuntos de dados, não sendo recomendado para todo o tipo de processamento de dados (UZUNKAYA; ENSARI; KAVURUCU, 2015).

O *Mahout* é um projeto do *Apache Hadoop* desenvolvido para a construção de biblioteca *JAVA*, para a produção de algoritmos de aprendizado de máquina distribuídos e escalável, baseado em técnicas do *Hadoop* para várias implementações (UZUNKAYA; ENSARI; KAVURUCU, 2015).

Sua estrutura possibilita a implementação de algoritmos de mineração em grandes volumes de dados, utilizando o modelo da linguagem de programação *MapReduce*, garantindo com isso um bom desempenho durante a execução das aplicações. O *Mahout* disponibiliza algoritmos que são utilizados para as seguintes situações: classificação de filtragem coletiva, categorização, agrupamento de dados, mineração frequente de conjuntos de itens e a mineração paralela de padrões frequentes (MEHTA; MEHTA, 2016).

O *Avro* é um sistema utilizado para a serialização de dados, que possui uma estrutura baseada em esquemas no formato *JSON*, facilitando a implementação. Os dados são serializados em um formato binário compacto, tornando-os eficientes para tarefas *MapReduce*. O formato utilizado em sua estrutura permite que os dados sejam processados por várias linguagens de programação, são elas: *C*, *C#*, *C++*, *Python*, *JAVA* e *Ruby* (GOLDMAN *et al*., 2012; MEHTA; MEHTA, 2016).

O *Sqoop* é uma ferramenta de código aberto projetada para a transferência de dados entre o *Apache Hadoop*, os *Datastores* estruturados, outras estruturas de armazenamento e o *HDFS*, atuando como um banco de dados relacional, podendo importar dados de tabelas em um banco de dados relacional para o *HDFS* e vice-versa, para o preenchimento de tabelas do *Hive* ou do *HBase* (UZUNKAYA; ENSARI; KAVURUCU, 2015).

No *Sqoop*, a transferência de dados ocorre de forma paralela, permitindo uma utilização eficiente dos recursos disponíveis no sistema e um bom desempenho na execução das tarefas. O *Sqoop* disponibiliza ainda algumas ferramentas utilizadas para análise de dados

(ARAVINTH *et al*., 2015; MEHTA; MEHTA, 2016).

O *Oozie* é um sistema utilizado para o processamento de fluxos de trabalho, útil para gerenciar tarefas complexas de análise de dados. Sua estrutura é composta por dois componentes. O primeiro, chamado de fluxo de trabalho, armazena as ações em um Gráfico Acíclico Direcionado, para definir a sequência de execução das ações. O segundo componente, chamado de coordenador, é responsável por utilizar critérios pré-definidos para permitir a execução de um fluxo de trabalho (BANDI; ANITHA, 2018; YADAV; CHANDRA, 2019; MEHTA; MEHTA, 2016).

O *Chukwa* é um sistema utilizado para o monitoramento de ambientes distribuídos. Ele disponibiliza ferramentas úteis para coleta, análise e visualização de dados. Em sua estrutura são encontrados quatro componentes (agentes, coletores, *mapreduce* e a interface *web*). Os agentes são responsáveis por enviar os dados para os coletores, que por sua vez, recebem os dados e os armazenam. Para realizar a análise e o processamento dos dados são utilizadas tarefas *MapReduce*, garantindo a otimização na execução das tarefas. O último componente é uma interface *web*, utilizada para exibir as informações obtidas (GOLDMAN *et al*., 2012; MEHTA; MEHTA, 2016).

O *Flume* é um projeto desenvolvido pela *Cloudera*, com o objetivo de oferecer suporte à coleta de dados de *streaming* configurável e distribuída, altamente confiável e com serviços disponíveis para a coleta, a agregação e a movimentação de grandes volumes de dados de *streaming* para o *Hadoop Distributed File System* (*HDFS*) (UZUNKAYA; ENSARI; KAVURUCU, 2015). Sua estrutura é de simples implementação, com tolerância a falhas, permitindo que sejam utilizados vários tipos de fontes para a realização de coleta de dados. É muito utilizado para coletar e analisar arquivos de *log* em tempo real, que são gerados durante a execução das tarefas do *cluster* (RATHEE, 2013; MEHTA; MEHTA, 2016).

O *Zookeeper* é um serviço utilizado para realizar a sincronização das aplicações executadas no *cluster*. Sua estrutura permite que os serviços sejam implementados de forma redundante. Além disso, o *Zookeeper* fornece ferramentas úteis para auxiliar no gerenciamento das tarefas e na configuração das *nodes* do *cluster* (GOLDMAN *et al*., 2012; MEHTA; MEHTA, 2016).

Em resumo, a biblioteca *Apache Hadoop* é um *software* que fornece confiabilidade para o armazenamento de dados, possuindo uma estrutura de gerenciamento simplificada, pois a própria ferramenta é responsável por administrar a comunicação entre as *nodes* do *cluster* e a distribuição durante o processamento dos dados, além de possuir escalabilidade, pois não são

necessárias grandes modificações nas configurações para adicionar mais *nodes* (computadores) ao *cluster*.

Porém, é possível encontrar alguns problemas ao utilizá-la, tais como: dificuldade para encontrar falhas, pois durante a execução das tarefas são criados vários arquivos de *logs*, que ficam espalhados em diferentes pastas do sistema; o processamento de arquivos pequenos, pois a biblioteca não é bem adaptada para esse tipo de situação; e a existência de apenas um nó mestre (*master*), que cria um ponto crítico de falha para o funcionamento do *cluster* (GOLDMAN *et al*., 2012).

2.5 Programação Paralela

A programação paralela é uma técnica utilizada para otimizar a execução de tarefas, através da divisão de tarefas complexas em várias tarefas que são distribuídas para vários processadores, e executadas de forma simultânea. Dessa forma, o tempo necessário para concluir a execução das tarefas é reduzido, e todos os recursos disponíveis no *hardware* são utilizados de forma eficiente. Durante o processamento, são utilizadas variáveis armazenadas na memória *RAM*, para permitir o compartilhamento de informações (LORENZON, 2014).

Para utilizar esse modelo de programação é necessário ter um conhecimento avançado de arquitetura de computadores, de linguagens de programação e de bibliotecas de paralelismo, pois é preciso realizar a integração das tecnologias e saber como utilizar os recursos disponibilizados para extrair o máximo de desempenho dos computadores, para a execução das tarefas. Portanto, as máquinas (computadores) utilizadas nesse tipo de programação devem possuir uma arquitetura paralela (*multicore*) (SOARES; NOBRE; FREITAS, 2019), onde os processadores e/ou núcleos possam se comunicar entre si, para que haja uma sincronização na execução das tarefas (REBONATTO, 2004).

A programação paralela permite que a execução das tarefas seja otimizada, ao permitir a execução simultânea entre os vários processadores, mas, a limitação dos recursos disponíveis ao utilizar apenas um computador, torna-se um problema para conseguir um desempenho melhor de processamento. Para contornar essa situação é utilizada a programação distribuída, que permite a execução das tarefas de forma distribuída entre os vários computadores conectados em uma mesma rede. A comunicação entre esse conjunto de computadores (*cluster*) ocorre através da troca de mensagens (biblioteca de paralelismo), fazendo com que eles possam compartilhar as informações necessárias para a execução das tarefas (ANDREWS, 1999).

Esse modelo de programação fornece uma capacidade de processamento elevada, permitindo que novos computadores sejam adicionados ao conjunto (*cluster*), de acordo com a necessidade, de forma simples, disponibilizando mais recursos para a execução das tarefas. Porém, é necessário um nível maior de atenção ao utilizá-la, por causa dos problemas de interconexão de rede que podem comprometer a comunicação entre os computadores durante o processamento das tarefas (ANDREWS, 1999).

A programação paralela busca explorar o potencial concorrente existente no código de um programa, utilizando bibliotecas de paralelismo que trabalham com a execução explícita de processos ou *threads* simultaneamente (SOARES; NOBRE; FREITAS, 2019).

Considerando que as aplicações paralelas envolvem um grande poder computacional e possuem um tempo de resposta elevado, o processamento paralelo visa reduzir esse tempo, dividindo a carga computacional entre os processadores e núcleos do sistema, permitindo assim uma aceleração na execução das tarefas e, consequentemente, uma redução no tempo de resposta das tarefas (REBONATTO, 2004).

Portanto, os principais objetivos da programação paralela são: aumentar o desempenho computacional, através do uso mais eficaz dos recursos computacionais; redução do tempo de execução das tarefas; e resolver problemas computacionais complexos (REBONATTO, 2004).

2.6 *Benchmarks*

Os algoritmos de *benchmark* são utilizados no *cluster* para executar tarefas de processamento elevado, com isso é possível observar o comportamento dos equipamentos durante os testes, verificar o desempenho do *cluster* e identificar possíveis problemas que ocorram durante ou após a instalação do *cluster* (NETO, 2023).

Ao executar os testes de *benchmark* é possível extrair várias métricas, que permitem comparar a solução utilizada com outras soluções e analisar o desempenho dos equipamentos utilizados. A biblioteca *Apache Hadoop* disponibiliza um conjunto de aplicações (algoritmos) que podem ser utilizadas para testes de *benchmark*, tais como: *TeraGen*, *TeraSort*, *TeraValidate*, *TestDFSIO* e o cálculo do *Pi* (π) (VASCONCELOS, 2015).

Os algoritmos "*Tera*" consistem em um conjunto que possui três aplicações: *Teragen*, *Terasort* e o *Teravalidate*. O *TeraGen* é um programa que gera dados de entrada aleatório para o *cluster*. Este programa executa uma tarefa *MapReduce* (*Job MapReduce*) que gera dados binários de forma aleatória, com chaves e valores com diferentes tamanhos (O'MALLEY,

2008). Como parâmetro de entrada para a geração dos dados, o *TeraGen* utiliza linhas de 100 *Bytes*. Em cada linha de 100 *Bytes*, os dados gerados são formatados da seguinte forma: 10 *Bytes* para a chave, 10 *Bytes* para a identificação do dado gerado e 78 *Bytes* de caracteres, que são gerados através de 7 sequências de 10 caracteres de "A" a "Z" (WHITE, 2015). Em outras palavras, o *TeraGen* divide o número de linhas desejadas pelo número de tarefas, atribuído intervalos de linhas para cada *Mapper*. Na sequência, o *Mapper* modifica o gerador de números aleatórios para o valor correto da primeira linha e, em seguida, gera as próximas linhas (ALVES, 2015).

O *TeraSort* é um programa que utiliza uma amostra de dados gerada aleatoriamente pelo *TeraGen*, e através de um programa *Map/Reduce*, realiza a ordenação dessa amostra de dados. Para realizar essa tarefa, o *TeraSort* cria uma lista classificada de N-1 chaves, sendo N o número de *Tasks Reduces* que definem o intervalo de chave para cada *Task Reduce*. Dessa forma, as chaves geradas pelo *TeraGen* seguem uma lista *[i-1]<=chave<lista[i]*, que são enviadas para a execução do *Reduce i*, ordenando em cada *Reduce* as chaves recebidas. Em seguida, as saídas de cada *Reduce i* são reunidas seguindo a ordem *i* (O'MALLEY, 2008). Além disso, o tempo que o *TeraSort* leva para a organização dos dados de entrada gerados pelo *TeraGen*, serve de métrica para a avaliação de desempenho do *cluster*, bem como para a comparação de performance com outros *clusters*, servindo para verificar se as configurações dos parâmetros definidas no *Hadoop* foram satisfatórias para um bom desempenho do *cluster* (EADLINE, 2015).

Por fim, para fechar o pacote de algoritmos "*Tera*", tem-se o *TeraValidate*, que consiste em um programa *Map/Reduce* que tem por objetivo verificar se os dados de saída, gerados pelo *TeraSort*, foram ordenados corretamente. Para executar essa validação, o *TeraValidate* cria um *Mapper* por arquivo no diretório de saída, onde cada *Mapper* assegura que cada chave é inferior ou igual a chave anterior. Além disso, o *Mapper* cria registros com as primeiras e últimas chaves do arquivo, onde a função *Reduce* garante que a primeira chave do arquivo *i* é maior que a última chave de arquivo *i – 1*. Caso ocorra algum problema durante a validação das saídas do *Reduce*, pelo fato dessas chaves estarem fora de ordem, o programa irá reportar o erro (NETO, 2023; VASCONCELOS, 2015; ALVES, 2015).

O *TestDFSIO* é uma aplicação utilizada para testar o desempenho de leitura e escrita do *HDFS*. Esse teste é dividido em duas etapas. A primeira etapa é responsável por gerar arquivos (*write*), que serão utilizados posteriormente na segunda etapa para realizar um processo de leitura (*read*). Os parâmetros necessários para a execução são o número de arquivos que serão

gerados e o tamanho de cada arquivo (NETO, 2023; VASCONCELOS, 2015).

O processo de execução do algoritmo *TestDFSIO* funciona da seguinte forma: o programa de gravação inicia várias tarefas de *Map*, com cada tarefa sendo gravada em arquivos *HDFS* separados. Em seguida, o programa de leitura inicia várias tarefas de *Map*, fazendo a leitura dos arquivos gravados no *HDFS* pelo programa de gravação, com cada tarefa sendo lida de forma sequencial. Na sequência, são realizadas medições das tarefas realizadas (NETO, 2023).

O *TestDFSIO* utiliza uma única tarefa do *Reduce* para medir e calcular duas métricas de desempenho para cada tarefa do *Map*, são elas: a Taxa Média de *I/O* e a Taxa de Transferência. Dessa forma, gargalos e problemas de desempenho na configuração da rede, do *hardware*, do sistema operacional e do próprio *Apache Hadoop*, podem ser identificados e corrigidos (NETO, 2023).

Já o cálculo do *Pi* (π), é uma aplicação que utiliza o método estatístico *quasi-Monte Carlo* (*qMC*), para estimar o valor de *Pi*. Para executar esse teste, são utilizados dois parâmetros: o número de mapas, que são quadrados com círculos dentro e o número de pontos, que são as amostras inseridas de forma aleatória em cada mapa. Para calcular o valor de *Pi*, os pontos são distribuídos aleatoriamente em cada quadrado e, em seguida, é realizada uma contagem dos pontos que foram inseridos dentro dos círculos. Por fim, para estimar o valor, é calculada uma relação entre o número de pontos encontrados nos círculos e o número total de pontos que foram utilizados (VASCONCELOS, 2015; JUNIOR, 2023). O valor do *PI*, por referência, mostra o valor estimado com as 10 primeiras casas decimais (3,1415926535).

2.7 *Big Data*

Big Data é um termo utilizado para representar um grande conjunto de dados que são obtidos a partir de diversas fontes. Esse grande volume de dados torna as tarefas de armazenamento e processamento mais complexas, e dessa forma, os banco de dados que são normalmente utilizados, não são capazes de analisar e gerenciar esses dados (BHOSALE, GADEKAR, 2014; HANNAN, 2016; BEAKTA, 2015; ISHWARAPPA, ANURADHA, 2015).

Nesse contexto, os dados obtidos precisam ser processados rapidamente, para que se obtenha o máximo de informações possíveis. Mas, devido aos problemas encontrados no tratamento desse dados, é necessário utilizar técnicas e ferramentas que sejam adaptadas e otimizadas para realizar o processamento e armazenamento desse conjunto de dados.

De um modo geral, existem cinco características que podem ser utilizadas para definir *Big Data*, a saber (BHOSALE, GADEKAR, 2014; HANNAN, 2016; BEAKTA, 2015; ISHWARAPPA, ANURADHA, 2015):

- **Volume**: Trata-se de uma grande quantidade de dados que são gerados a partir de diversas fontes, e armazenados em extensos repositórios. Como o volume de dados gerados pelos dispositivos crescem a cada instante, é um desafio desenvolver ferramentas que sejam capazes de armazená-los e processá-los de forma eficiente;
- **Velocidade**: Representa a velocidade em que os dados são gerados e processados. Em diversos contextos, a velocidade da análise dos dados é muito importante, porque as informações que são obtidas podem ser utilizadas para auxiliar nas tomadas de decisões;
- **Valor**: Refere-se ao valor que os dados podem ter a partir das informações que podem ser obtidas ao analisá-los. Porém, a parte mais complexa é a análise, pois exige a utilização de sistemas e ferramentas complexas e avançadas, que sejam capazes de encontrar as relações existentes entre os dados, extraíndo o máximo de informações possíveis;
- **Veracidade**: Trata-se da confiabilidade das informações que são obtidas com a análise dos dados processados. Por isso, durante a fase de processamento, é importante realizar a limpeza e a reestruturação dos dados, aumentando a qualidade desses dados, efetuando as correções das inconsistências, quando necessário e permitindo a integração dos dados obtidos de diferentes fontes;
- **Variedade**: Refere-se à ampla diversidade dos tipos de dados que podem ser coletados. Como esses dados podem ser obtidos em vários formatos e de diversas fontes, é necessário utilizar ferramentas e técnicas adequadas para realizar à integração e a análise dos dados, para a obtenção de informações relevantes.

2.8 Ferramentas Utilizadas no Monitoramento das Métricas de Desempenho de um *Cluster*

As ferramentas de monitoramento são *softwares* utilizados para realizar a coleta e o acompanhamento de dados em tempo real. Algumas das principais funcionalidades oferecidas são: o monitoramento da utilização dos recursos de um sistema, a análise do desempenho e a verificação da disponibilidade de dispositivos, bem como a detecção de problemas de performance na rede (SILVA; SILVA, 2024).

A diversidade de características e funcionalidades específicas dessas ferramentas possibilita que elas sejam aplicadas em diversos tipos de cenários. Assim, elas podem ser utilizadas tanto em projetos mais simples quanto em redes mais complexas e avançadas, devido ao gerenciamento centralizado dos diversos dispositivos que as compõem (BELENTANI; MARCELLO; FLORIAN, 2018).

De acordo com Silva e Silva (2024), o *Zabbix* e o *Grafana* são algumas das ferramentas de monitoramento mais utilizadas atualmente. O destaque desses *softwares* é motivado por características como gratuidade, robustez e usabilidade simples.

O *Zabbix* é uma ferramenta utilizada para coletar dados em tempo real, com a finalidade de monitorar o desempenho e a disponibilidade de dispositivos instalados em uma rede. A sua estrutura é formada por dois componentes principais: o servidor e os dispositivos monitorados. O servidor, chamado de *Zabbix Server*, é responsável por coletar os dados que são enviados através da rede e processá-los, para produzir relatórios e executar algumas ações. Já os dispositivos monitorados, são responsáveis por disponibilizar os dados na rede, podendo ser configurados através do protocolo *SNMP* (*Simple Network Management Protocol*) ou o do *Zabbix Agent*, um módulo que pode ser instalado no equipamento, para facilitar a configuração e ampliar a quantidade de informações que podem ser coletadas. A interface *web* dessa ferramenta permite alterar suas configurações, visualizar todas as informações coletadas e definir ações que podem ser executadas automaticamente, em resposta a diversos tipos de eventos (SILVA; SILVA, 2024) (CHAVES; VIEIRA, 2020).

O *Grafana*, por sua vez, é uma ferramenta utilizada para visualização e monitoramento dos dados de um sistema. Ele se conecta a diversas fontes de dados, como o próprio *Zabbix*, para obter informações, concentrá-las em uma única plataforma e exibi-las de uma forma mais intuitiva. Através da sua interface *web*, podem ser criados *dashboards* utilizando diversos tipos de gráficos e personalizações, o que facilita a adaptação à uma grande variedade de dados que são obtidos (SILVA; SILVA, 2024) (CHAVES; VIEIRA, 2020) (BUENO; WEINGARTNER, 2022).

3. TRABALHOS CORRELATOS

3.1 *An Efficient Implementation of Mobile Raspberry Pi Hadoop Clusters for Robust and Augmented Computing Performance* (SRINIVASAN, *et al.*, 2018).

Na pesquisa, os autores desenvolveram um *pseudo-cluster* utilizando um computador equipado com um processador *Intel® Core™ i5-4440*, com 8 *GBytes* de memória *RAM* e outras seis combinações com a plataforma *Raspberry Pi 3*, associando entre quatro e nove nós (*nodes*) escravos (*slaves*) e um nó mestre (*master*) em todas as situações.

Nos *clusters* desenvolvidos foram realizados testes *benchmark* utilizando o *SURF*, um algoritmo aplicado em imagens para identificar pontos característicos, com o objetivo de comparar o desempenho entre o *cluster* formado por *Raspberry Pi* e o *cluster* constituído de computadores convencionais (comuns), bem como analisar os impactos do processamento ao acrescentar mais nós (*nodes*) aos respectivos *clusters*.

A partir dos resultados obtidos, os autores concluíram que os *clusters* utilizando *Raspberry Pi* possuem um desempenho superior em relação ao que utiliza computadores convencionais, e que a capacidade de processamento é aprimorada, ao aumentar o números de nós (*nodes*) na solução do *cluster*, ou seja, quando ela é escalonada.

3.2 *A Raspberry Pi Cluster for Teaching Big-Data Analytics* (GIGER; SRIKUGAN; PERSAUD, 2020).

O propósito do trabalho foi permitir que os alunos do curso de computação compreendessem a importância da paralelização de tarefas computacionais complexas, utilizando as principais funcionalidades disponibilizadas no *cluster*, com o objetivo de ajudar na escolha de *softwares* para o gerenciamento e execução de testes em ambientes educacionais.

O *cluster* utilizado neste trabalho era composto por 4 (quatro) *Raspberry Pi* 4, sendo 1 a mestre (*master*) da solução e as outras 3 os nós escravos (*slaves*). Para o gerenciamento do *cluster* foram utilizadas duas tecnologias de conteinerização, a *Docker Swarm* e o *Kubernetes*. Além disso, foi utilizada uma ferramenta de automação, o *Ansible*, para auxiliar na configuração de todas as máquinas do *cluster*.

Para a análise de desempenho do *cluster* foram utilizadas duas ferramentas, a *PySpark-Benchmark*, que disponibiliza vários *benchmarks*, sendo utilizada para gerar dados de testes

aleatórios, e a ferramenta *Movielens-Benchmark*, que disponibiliza um conjunto de dados utilizada em algoritmos de recomendação.

De acordo com os resultados dos testes, os autores concluíram que com um número maior de nós (*nodes*) é possível aumentar o desempenho do *cluster* e, que, o *Docker Swarm* e o *Kubernetes*, trabalhando em conjunto, faz com que o *cluster* fique menos sobrecarregado do que somente com o *YARN*, do *Apache Hadoop*, durante a execução das tarefas. Além disso, os autores sugerem e recomendam a utilização de *Kubernetes* ou *Docker Swarm* para o gerenciamento do *cluster*.

3.3 *Benchmarking Raspberry Pi 2 Hadoop Cluster* (PAPAKYRIAKOU, 2019).

No trabalho, os autores implementaram um *cluster* utilizando 15 (quinze) *Raspberry Pi 2B*, sendo uma como nó mestre (*master*) e outras 14 (quatorze) como nós escravos (*slaves*). Para obter mais espaço de armazenamento, foi conectado um *HD* externo de 320 *Gbytes* ao nó mestre. Além disso, foram utilizadas 4 (quatro) *fans*, para auxiliar no processo de resfriamento do *cluster* e 3 (três) medidores de tensão para monitorar o consumo energético do sistema.

Durante a realização dos testes foi utilizado o algoritmo *TestDFSIO*. A partir de sua execução, foi possível obter dados sobre a Taxa de Transferência e a Taxa Média de E/S (entrada/saída). Essas informações foram utilizadas para avaliar o desempenho de leitura e escrita do *HDFS*, além de identificar possíveis problemas de desempenho na rede e no sistema.

Dentre as métricas avaliadas, foi observado não apenas que o *HDFS* tem uma boa taxa de transferência e que a operação de leitura possui um desempenho superior à de escrita, como também que a largura de banda da rede influencia diretamente no desempenho do *cluster* durante a execução das tarefas. Em trabalhos futuros, os autores planejam combinar o *Apache Hadoop* com a linguagem de programação *R*. Dessa forma, será possível explorar os cálculos estatísticos e a análise dos dados no *cluster*.

3.4 *On Energy Efficiency and Performance Evaluation of Single Board Computer Based Clusters A Hadoop Case Study* (QURESHI; KOUBÂA, 2019).

Na pesquisa, os autores exploraram à viabilidade do desenvolvimento de *clusters* implementado com o *Apache Hadoop*, utilizando plataformas *SBC* em comparação com computadores pessoais. Na ocasião, foi observado o consumo energético, o tempo de

processamento e a velocidade de leitura/escrita durante a execução das tarefas, para realizar uma análise aprofundada do desempenho e da eficiência energética da *Raspberry Pi 2B* e da plataforma de *hardware Odroid Xu-4*.

Para executar os testes de *benchmark*, foram desenvolvidos três *clusters*. O primeiro, desenvolvido com *Raspberry Pi*, possuía 20 (vinte) nós (*nodes*) atuando como escravos. O segundo, também possuía 20 (vinte) nós (*nodes*) escravos, porém, a plataforma de *hardware* utilizada era o *Odroid Xu-4*. Já no terceiro *cluster*, foram utilizados 4 (quatro) nós (*nodes*) como escravos, sendo estes, configurados em máquinas virtuais executadas em computadores convencionais (comuns). O nó mestre (*master*) dos três *clusters* era uma máquina virtual criada em um computador com processador *Intel® Core™ i7* de 4ª geração, com 8 *GBytes* de memória *RAM*.

O primeiro teste executado foi o *Sysbench*, utilizado para analisar o desempenho individual das três plataformas utilizadas. Em seguida, os autores executaram os testes *Pi* (π), *Wordcount*, *TesteDFSIO* e *Terasort*, para avaliar o desempenho dos três *clusters* desenvolvidos. Para medir o consumo de energia durante a execução dos algoritmos, todas as fontes de alimentação dos equipamentos foram conectadas a um medidor de tensão.

De acordo com os resultados dos testes individuais, os autores concluíram que o computador pessoal, utilizado na solução do *cluster*, possui um tempo de execução mais rápido do que as máquinas *SBC*. Os experimentos realizados nos *clusters* indicaram que a *Raspberry Pi 2B* possui o pior desempenho e, que o *Odroid Xu-4*, possui um custo operacional menor em tarefas de baixa intensidade, porém, devido à limitação de recursos dos *SBC*, o computador possui um desempenho melhor para cargas maiores de trabalho, envolvendo uma quantidade maior de dados.

3.5 *Optimizing Hadoop Scheduling in Single-Board-Computer-Based Heterogeneous Clusters* (QURESHI, 2024).

No estudo, os autores desenvolveram um *cluster* envolvendo diferentes plataformas, para verificar o seu comportamento durante a execução das tarefas. Na implementação do *cluster* foi utilizada uma *Raspberry Pi 5*, como um nó (*node*) mestre (*master*) da solução, e para os nós (*nodes*) escravos (*slaves*), os autores utilizaram as plataformas de *hardware Rockpro64*, *Raspberry Pi 3B+*, *Odroid XU-4* e *Raspberry Pi 5*, combinando três unidades de cada uma.

Devido à diferença de recursos presente no *cluster* heterogêneo, o *Apache Hadoop* teve

dificuldade para otimizar o desempenho computacional, ao utilizar suas configurações padrões. Nesse sentido, para tentar solucionar o problema, foram propostas algumas alterações que foram realizadas no mecanismo de agendamento de tarefas.

Para isso, os autores utilizaram o índice de frugalidade, para categorizar os nós (*nodes*), de acordo com a capacidade de memória *RAM* e processamento de cada um deles, em conjunto com a política de configuração adaptável, responsável por alterar a distribuição de tarefas com base na carga de trabalho dos nós (*nodes*). Para avaliar as mudanças realizadas no mecanismo de agendamento de tarefas, os autores realizaram alguns testes utilizando os algoritmos *Wordcount* e *Terasort*.

Com base nos resultados, os autores concluíram que o mecanismo de agendamento de tarefas que foi desenvolvido, possui uma distribuição de tarefas mais eficiente do que a configuração nativa do *Apache Hadoop*, gerando melhorias significativas no desempenho e no consumo energético do *cluster*.

3.6 *Performance of a Low Cost Hadoop Cluster for Image Analysis in Cloud Robotics Environment* (QURESHI, *et al.*, 2016).

O objetivo da pesquisa foi explorar a utilização de uma estrutura remota para análise e processamento de imagens. De acordo com os autores, o principal objetivo era contornar o problema presente nos *VANTs* (Veículo Aéreo Não Tripulado), que devido à limitação de recursos computacionais, possui uma baixa capacidade de processamento. Com isso, o *VANT* enviaria os dados de vigilância coletados e o *cluster* seria responsável por realizar o processamento das imagens, reduzindo o tempo necessário para a conclusão da tarefa.

Para isso, os autores desenvolveram um *cluster* utilizando 20 (vinte) *Raspberry Pi 2B*, como nós (*nodes*) escravos (*slaves*), e 1 (uma) máquina virtual como nó mestre (*master*) do sistema. Para analisar o desempenho da *Raspberry Pi*, foi desenvolvido um *cluster* virtualizado, utilizando computadores pessoais, composto por 1 (um) nó (*node*) mestre (*master*) e 4 (quatro) nós (*nodes*) escravos (*slaves*). Além disso, foi utilizada a biblioteca *HIPI* (*Hadoop Image Processing Interface*) na configuração do *Apache Hadoop*, para melhorar a utilização dos recursos do *cluster*.

Durante os testes foram executados os algoritmos *Pi* (π) e *Terasort*, utilizando três cenários diferentes. No primeiro cenário, os autores limitaram-se a frequência do processador e a memória *RAM* disponível. No segundo, foram utilizados todos os recursos disponíveis na

Raspberry Pi 2B. Já na terceira configuração, foi utilizado o *cluster* virtualizado.

Os resultados mostraram que o primeiro cenário obteve um desempenho inferior durante a execução de tarefas mais simples, mas o desempenho foi similar em tarefas de alta carga de dados. Por fim, os autores concluíram que o *cluster* desenvolvido com *Raspberry Pi 2B* tem limitações de desempenho em comparação com o *cluster* formado por máquinas virtuais. Porém, as modificações realizadas ao utilizar a biblioteca *HIPI* geraram melhorias significativas na execução das tarefas no *cluster*.

Na Tabela 01, observa-se um resumo dos trabalhos correlatos pesquisados, contendo os tipos de plataformas utilizadas na implantação dos *clusters*, bem como os algoritmos de testes *benchmark* e capacidade de cada *cluster*, com as respectivas quantidades de nós (*nodes*).

Tabela 01 – Resumo dos trabalhos correlatos

Autores	**Plataformas**	**Algoritmos**	**Quantidade de Nós**
Srinivasan *et al*., 2018.	*Raspberry Pi 3* *Desktop*	*SURF*	1, 4, 5, 6 7, 8 e 9
Giger; Srikugan; Persaud, 2020.	*Raspberry Pi 4*	*PySpark-Bechmark* *Movielens-Benchmark*	3
Papakyriakou, 2019.	*Raspberry Pi 2*	*TestDFSIO*	14
Qureshi; Koubâa, 2019.	*Raspberry Pi 2B, Odroid Xu-4* Máquina virtual	*SysBench, Pi, Wordcount* *TestDFSIO e TeraSort*	4 e 20
Qureshi, 2024.	*Raspberry Pi 5, RockPro64* *Raspberry Pi 3B+ e Odroid Xu-4*	*Wordcount e TeraSort*	12
Qureshi *et al*., 2016.	*Raspberry Pi 2B* Máquina virtual	*Pi e TeraSort*	4 e 20

Fonte: Elaborada pelos Autores, 2025.

Neste trabalho de pesquisa, foi desenvolvido um *cluster* utilizando como plataforma de *hardware* a *SBC Raspberry Pi* 4 e como biblioteca de paralelismo o *Apache Hadoop*. Para analisar o seu desempenho foram utilizados algoritmos computacionais paralelos, tais como: Cálculo do *Pi* (π) (*quasi-Monte Carlo*), *TeraGen*, *TeraSort*, *TeraValidate* e *TestDFSIO* (*Write* e *Read*). Os cenários de testes contemplam a implementação de *clusters* configurados com 2, 4 e 8 *nodes slaves*, além de um nó *master* em cada um deles.

Para medir e analisar o desempenho dos *clusters*, foram coletadas, através do *software* de monitoramento de dados *Zabbix*, as seguintes métricas: utilização de *CPU*, utilização de memória *RAM*, temperatura, número de processos, tráfego de rede e consumo energético. Além disso, foi analisado o desempenho da biblioteca *Apache Hadoop*, através das seguintes métricas: tempo de execução das tarefas, *CPU Time*, *Map Time* e *Reduce Time*.

4. METODOLOGIA

A metodologia utilizada no desenvolvimento deste trabalho, do ponto de vista da natureza e abordagem, é classificada como uma pesquisa básica pura, focada na ampliação da base de conhecimento científico sobre um determinado tema de pesquisa (LOPES, 1991) e quantitativa, focada na análise de dados brutos, utilizando instrumentos padronizados para a coleta de dados, onde os resultados poderão ser quantificados através de técnicas estatísticas (DE FREITAS *et al*., 2019).

Quanto aos objetivos, está classificada como uma pesquisa descritiva, descrevendo as características de uma determinada população ou fenômeno, bem como o estabelecimento de relações entre as variáveis, utilizando técnicas padronizadas para a coleta de dados (GIL, 2008), podendo observar os fatos, registrá-los, analisá-los, classificá-los, compará-los e interpretá-los, sem a interferência do pesquisador (MARCONDES *et al*., 2022) e exploratória, visando o aprofundamento do tema pesquisado, tornando-o mais claro, levando a construção de questões importantes para a condução da pesquisa (RAUPP; BEUREN, 2006).

Com relação aos procedimentos técnicos, foi realizado inicialmente um mapeamento sistemático da literatura, identificando um conjunto de estudos finalizados sobre um determinado tema, avaliando os resultados desses estudos e evidenciando suas conclusões (HULLEY; CUMMINGS; GRADY, 2015), através de um levantamento do estado da arte, proporcionando uma visão panorâmica dos principais resultados das publicações (artigos científicos, teses, dissertações, livros, entre outros), utilizando-se das seguintes bases de dados acadêmicas, a saber: *Scopus* (*Elsevier*), *Web of Science* e *Science Direct* (*Elsevier*), cujo objetivo era compreender os principais conceitos relacionados aos temas abordados.

As bases de dados citadas foram escolhidas porque possuem uma abrangência internacional e concentram uma grande quantidade de publicações, de alta relevância para a comunidade científica em diversas áreas do conhecimento, possuindo ferramentas que auxiliam na coleta e análise estatística dos dados.

Os principais temas abordados neste trabalho de pesquisa foram: Arquiteturas Paralelas, *Cluster*, Bibliotecas de Paralelismo, *Apache Hadoop*, Programação Paralela, *Big Data* e Algoritmos Paralelos de *Big Data* para *Benchmarks* em *Clusters*.

Para conduzir e nortear o mapeamento sistemático da literatura foi utilizada a seguinte questão principal de pesquisa: Como desenvolver um cluster de baixo custo e alto desempenho utilizando a biblioteca paralela *Apache Hadoop* na plataforma computacional *Raspberry Pi* ?

Com base na questão principal desta pesquisa, foram definidas algumas questões secundárias, que contribuíram para responder à questão principal da pesquisa, a saber:

1. **QP1**: Quais algoritmos de *benchmark* foram utilizados nos testes dos *clusters* ?
2. **QP2**: Qual modelo da *Raspberry Pi* foi utilizada no desenvolvimento dos *clusters* ?
3. **QP3**: Qual a versão utilizada da biblioteca de paralelismo *Apache Hadoop* ?
4. **QP4**: Quais *softwares* foram utilizados no monitoramento dos *clusters* ?
5. **QP5**: Quais métricas de desempenho foram utilizadas para a análise dos *clusters* ?
6. **QP6**: Quantidade de nós utilizados no *cluster*?

Neste mapeamento foram utilizados os seguintes critérios para inclusão e exclusão das publicações identificadas nas buscas às base de dados científicas:

- Critérios de Inclusão (CI):

1. **I1:** Artigos publicados no período de 2019 a 2024;
2. **I2:** Artigos que possuíam a *string* principal de busca nos campus resumo, título ou palavras-chave, das respectivas bases de dados pesquisadas;
3. **I3:** Artigos escritos somente no idioma inglês e português;
4. **I4:** Artigos completos.

- Critérios de Exclusão (CE):

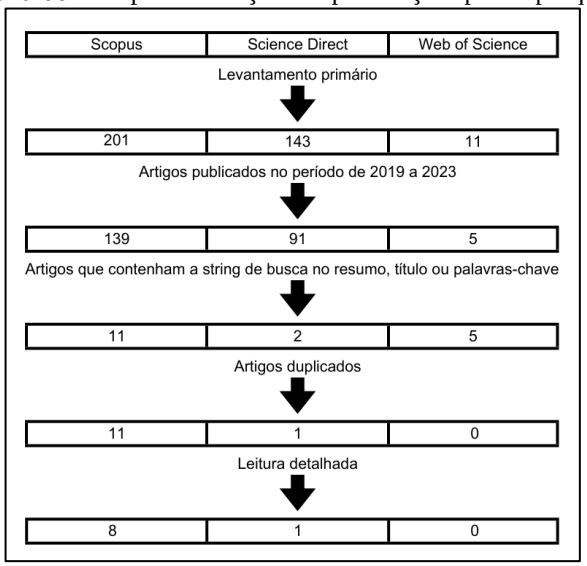
1. **E1:** Artigos duplicados;
2. **E2:** Artigos de literatura cinza;
3. **E3:** Artigos que disponibilizaram apenas conceitos.

As buscas nas bases de dados ocorreram no início de 2025, compreendendo o intervalo de publicações de 2019 a 2024, utilizando como critérios de busca, a inserção de palavras-chave referentes aos temas pesquisados nos respectivos campos de pesquisa das bases de dados: título (*article title*), resumo (*abstract*) e palavras-chave (*keywords*). Como critério de busca, foi utilizada a expressão (*string*): "*hadoop*" *AND "cluster" AND "raspberry*" (*TITLE-ABS-KEY ("hadoop" AND "cluster" AND "raspberry") AND PUBYEAR > 2018 AND PUBYEAR < 2025*). Essas palavras-chave foram utilizadas de forma isolada e/ou combinadas, através de operadores lógicos (*AND* e *OR*), que auxiliaram no refinamento das buscas, delimitando o escopo da pesquisa, garantindo a precisão e a relevância dos dados coletados.

Nas buscas realizadas nas bases de dados, no levantamento primário, foram identificadas 355 publicações que atenderiam inicialmente o escopo da pesquisa. Desse quantitativo, 201 foram encontradas na base de dados da *Scopus*, 143 na *Science Direct* e 11 na *Web of Science*.

Na sequência, foram aplicados alguns critérios para filtrar os trabalhos mais relevantes para este estudo. Dessa forma, foram excluídos 120 documentos por estarem fora do período de publicação proposto, 217 por não apresentarem as palavras-chave da expressão de busca nos campos citados e 6 por estarem duplicados. Por fim, foi realizada uma leitura detalhada de 12 publicações, restando 9 para serem analisadas por completo, das quais 8 pertenciam a base de dados da *Scopus* e 1 a base de dados da *Science Direct*, como pode ser observado na Figura 06.

Figura 06 – Etapas da seleção das publicações para a pesquisa



Fonte: Elaborada pelos Autores, 2025.

Após as buscas, os dados coletados foram transportados para uma planilha eletrônica, *Microsoft® Excel®* - 365 (Versão 2309 *Build* 16.0.16827.20166 - 64 *bits*), onde foram tabulados, analisados e armazenados, gerando dados estatísticos referentes aos temas pesquisados.

O mapeamento sistemático serviu para identificar e analisar o cenário mundial, apontado as tendências das pesquisas acadêmicas relacionadas aos temas em questão, servindo de base para a compreensão dos estudos realizados e de norte para o desenvolvimento deste trabalho.

A partir das informações coletadas no mapeamento sistemático, foi possível identificar algumas vantagens e desvantagens da utilização do *hardware Raspberry Pi* e da biblioteca de paralelismo *Apache Hadoop*, no desenvolvimento de *clusters* de alto desempenho e de baixo custo, bem como conhecer o funcionamento e o comportamento dos principais algoritmos paralelos de *benchmark* utilizados para medir e avaliar o desempenho dos *clusters* implementados.

Dando sequência ao trabalho, após a compreensão dos principais conceitos relacionados aos temas pesquisados neste trabalho, foi dado início a parte prática do trabalho, com a montagem de um *cluster*. O *cluster* foi desenvolvido utilizando como *hardware* a plataforma *SBC Raspberry Pi* e a biblioteca de paralelismo *Apache Hadoop*.

Na construção do *cluster* foram utilizadas 09 (nove) *Raspberry Pi* modelo *4 B*, com 4 *GByte* de *RAM*, Processador *Broadcom BCM2711*, 64-*bits*, com 04 núcleos *ARM Cortex-A72*, rodando a 1.5 *GHz*, sendo 1 (uma) utilizada como nó mestre (*master*) do *cluster* e servidor, para o sistema de gerenciamento e monitoramento do sistema, e as demais (08 *Raspberry Pi*), configuradas como máquinas escravos (*slaves*) do *cluster*. Como fonte de armazenamento secundário, em cada uma das *Raspberry Pi*, foram utilizados cartões de memória *SanDisk Ultra microSDXC UHS-I Card* com Adaptador, de 128 *GBytes – Speed up to* 120 *MB/s*.

O processo de desenvolvimento do *cluster* ocorreu da seguinte forma:

1. Inicialmente foi realizada a instalação física de todos os equipamentos utilizados na implementação do *cluster*, a saber: *Raspberry Pi* (cabo de alimentação, cabo *HDMI*, cabo de rede e cartão de memória), *switch*, monitor de vídeo, teclado, *mouse* e um *desktop*, utilizado para a configuração de todos os equipamentos do *cluster*;
2. Na sequência, foi realizada a configuração inicial de todas as *Raspberry Pi* utilizadas no projeto, com a instalação e configuração do sistema operacional nativo da plataforma, o *Raspberry Pi OS* (*Raspberry Pi OS with desktop*, *System: 64-bit*, *Kernel 6.6*, *Debian version 11*), nos cartões de memória de cada *Raspberry*;
3. Após a instalação do sistema operacional, foi realizada a instalação e configuração dos *softwares*, *JAVA* 11.0 e da biblioteca de paralelismo *Apache Hadoop* 3.4.0, em todas as *Raspberry Pi* do *cluster*;
4. Em seguida, foram instalados e configurados, na *Raspberry Pi master* (mestre) e servidor do *cluster*, os *softwares*, *Zabbix Server* (6.0-5) e *Grafana* (11.0.0), responsáveis por fazer o monitoramento e o gerenciamento de todas as máquinas do *cluster*;
5. Continuando o processo de instalação e configuração do *cluster*, foi instalada, em todas as *Raspberry Pi*, uma versão do *client* (agente) do *Zabbix*, responsável por efetuar a coleta dos dados e enviá-las para o sistema de gerenciamento;
6. Após a montagem e configuração do *cluster*, foram iniciados os testes de *benchmark*, com a utilização dos seguintes algoritmos paralelos: *TeraGen*, *TeraSort*, *TeraValidate*, *Pi* (*quasi-Monte Carlo*) e *TestDFSIO*;

7. Todos os testes foram realizados utilizando três combinações diferentes do *cluster*, com 2 (dois), 4 (quatro) e 8 (oito) nós (*nodes*). Além disso, nos testes envolvendo os algoritmos *TeraGen*, *TeraSort*, *TeraValidate* e *TestDFSIO*, foram utilizados diferentes tamanhos de arquivos com 500 *MBytes*, 1.000 *MBytes* (1 *GByte*) e 2.000 *MBytes* (2 *GByte*s);
8. Para os testes de *benchmark* envolvendo o algoritmo *PI (Quasi-Monte Carlo)*, foram utilizados cenários de *clusters* formados por 2, 4 e 8 *nodes*, onde os testes foram executados envolvendo 16 mapas (*Maps*), com 10.000.000 de amostras, para uma melhor obtenção do valor estimado do *Pi* (π).

Durante a execução dos algoritmos paralelos de *benchmark*, foram coletadas algumas métricas relacionadas ao desempenho do *cluster*, que possibilitaram avaliar e analisar o comportamento de todas as máquinas (*Raspberry Pi*) do *cluster*. As métricas coletadas foram:

1. Para os algoritmos *TeraGen*, *TeraSort*, *TeraValidate*: quantidade de processos executados pela *CPU*; tempo de execução das tarefas de *map* e *reduce*; quantidade de tarefas de *map* e *reduce*; frequência e uso do processador; frequência e uso das memórias; consumo de energia; e taxa de utilização da rede;
2. Para o algoritmo *TestDFSIO*: quantidade de processos executados pela *CPU*; tempo de execução das tarefas de *map* e *reduce*; quantidade de tarefas de *map* e *reduce*; frequência e uso do processador; frequência e uso das memórias; consumo de energia; taxa de utilização da rede; taxa média de *I/O*; e taxa de transferência;
3. Para o algoritmo do *Pi* (π), método estatístico *quasi-Monte Carlo* (*qMC*): quantidade de processos executados pela *CPU*; tempo de execução das tarefas de *map* e *reduce*; quantidade de tarefas de *map* e *reduce*; frequência e uso do processador; frequência e uso das memórias; consumo de energia; número de mapas (quadrados com círculos dentro); e número de pontos (as amostras inseridas de forma aleatória em cada mapa). Valor estimado do *Pi* com as 10 primeiras casas decimais (3,1415926535).

Após os testes de *benchmark*, as métricas coletadas foram transportadas para uma planilha eletrônica, *Microsoft® Excel®* - 365 (Versão 2309 *Build* 16.0.16827.20166 - 64 *bits*), onde foram tabuladas, analisadas e armazenadas, gerando dados estatísticos referentes ao desempenho do *cluster* formado pela plataforma de *hardware SBC Raspberry Pi*.

5. RESULTADOS E DISCUSSÃO

5.1 Mapeamento Sistemático

Nesta seção, serão apresentados e analisados os dados referentes ao mapeamento sistemático que respondem às questões de pesquisa norteadoras deste trabalho.

5.1.1 Algoritmos de *Benchmark*

Com base nas buscas realizadas nas bases de dados mencionadas na metodologia deste trabalho, foram identificados 9 tipos de *benchmarks* utilizados nos testes de desempenhos dos *clusters* implementados, sendo o *TestDFSIO* e o *TeraSort*, disponíveis no *Apache Hadoop*, os que mais apareceram nos trabalhos pesquisados, como mostra a Tabela 02.

Tabela 02 – Algoritmos de *Benchmark*

Algoritmos de *Benchmark*	**Referências**
TestDFSIO	(Neto; Neto; Moreno, 2022), (Lee; Oh; Park, 2021)
Terasort	(Neto; Neto; Moreno, 2022), (Lee; Oh; Park, 2021)
Wordcount	(Lee; Oh; Park, 2021)
Pi	(Lee; Oh; Park, 2021)
Sysbench	(Lee; Oh; Park, 2021)
Siege	(Nugroho; Widiyanto, 2020)
Regressão Linear	(Komninos *et al*., 2019)
Árvore de Decisão	(Komninos *et al*., 2019)
TPCx-HS	(Böther; Rabl, 2021)

Fonte: Elaborada pelos Autores, 2025.

O *Terasort* faz parte de um conjunto de algoritmos composto por três fases (*TeraGen*, *TeraSort* e *TeraValidate*), que permitem testar diferentes componentes do *cluster*. Nesse teste, o processador é intensamente utilizado durante as operações de criação e ordenação dos dados. A transferência de dados possibilita avaliar a capacidade da rede, sendo possível analisar as velocidades de leitura e gravação do dispositivo de armazenamento utilizado.

Já o algoritmo *TestDFSIO* é utilizado para avaliar, principalmente, a velocidade de transferência de dados da unidade de armazenamento do *cluster*. Durante sua execução, são gerados grandes volumes de dados, para monitorar o desempenho de leitura e escrita do sistema (*cluster*).

A aplicação de cálculo do *Pi* (π) é utilizada para estimar o valor do *Pi*, aplicando o método estatístico *quasi-Monte Carlo*. A execução desse algoritmo permite obter dados sobre a utilização e o desempenho do processador dos computadores utilizados no *cluster*.

O *Wordcount* é um algoritmo que permite analisar um arquivo de texto e indicar quantas vezes uma palavra foi encontrada nesse arquivo. Esse algoritmo é utilizado para aplicar uma carga de trabalho intensiva ao processador, permitindo analisar o seu desempenho durante a execução da tarefa.

O *Siege* consiste em uma ferramenta utilizada para estressar um servidor, ao simular um número de usuários e a quantidade de requisições realizadas por cada um. Ele permite registrar o número de requisições, bem como o total de *bytes* transferidos, o tempo de resposta e a duração de cada requisição. A partir disso, é possível analisar o desempenho da *CPU* e a capacidade de transferência da rede.

Um dos tipos de teste que merece um destaque é o *Sysbench*, aplicado em um dos trabalhos encontrados durante a pesquisa, que consiste em uma ferramenta utilizada para analisar o desempenho do sistema de forma individual, ou seja, de cada máquina e dos seus respectivos componentes, onde são disponibilizados diferentes testes para executar cargas de trabalho (teste de carga), que permitem analisar o desempenho dos componentes de cada equipamento pertencente ao *cluster*, como processador e memória *RAM*.

5.1.2 Modelos de *Raspberry Pi*

Nos trabalhos pesquisados, foram utilizados 4 modelos de *Raspberry Pi* para o desenvolvimento dos *clusters*. As versões mais utilizadas foram a *4B* e a *2B*, presentes em 4 (quatro) e 3 (três) trabalhos, respectivamente. Em dois estudos foram desenvolvidos dois *clusters* utilizando diferentes modelos de *Raspberry Pi*, a *3B* e a *3B+*, para analisar a evolução do desempenho das plataformas, como mostra a Tabela 03.

Tabela 03 – Modelos de *Raspberry Pi* utilizadas na implementação dos *clusters*

Modelos de *Raspberry Pi*	**Referências**
4B	(Neto; Neto; Moreno, 2022), (Böther; Rabl, 2021), (Lee; Oh; Park, 2021), (Komninos *et al*., 2019)
2B	(Penchalaiah *et al*., 2022), (Scolati *et al*., 2020), (Scolati *et al*., 2019)
3B+	(Böther; Rabl, 2021), (Nugroho; Widiyanto, 2020)
3B	(Lee; Oh; Park, 2021), (Nasir *et al*., 2019)

Fonte: Elaborada pelos Autores, 2025.

A *Raspberry Pi 2B*, lançada em 2015, possui um processador *quad-core* de 32 *bits*, com frequências de até 900 *MHz*. Essa versão possui 1 *GByte* de memória *RAM LPDDR2*, tem suporte a cartão de memória de no máximo 32 *GBytes* e a conexão de rede é *Fast Ethernet* (100 *Mbps*).

Já a *Raspberry 3B*, lançada em 2016, possui um processador *quad-core* 64 *bits*, que pode atingir até 1.2 *GHz* de frequência. Esse modelo também possui 1 *GByte* de memória *RAM* do tipo *SDRAM* e uma placa de rede com conexão *Fast Ethernet* (100 *Mbps*).

O modelo *3B+*, lançado em 2018, possui um processador *quad-core*, com uma arquitetura 64 *bits*, podendo atingir uma frequência de até *1.4 GHz*. Esse modelo é equipado com 1 *GByte* de memória *RAM* do tipo *LPDDR2* e uma placa de rede com suporte à conexão *Gigabit Ethernet* (1000 *Mbps*).

Por fim, a *Raspberry Pi 4B*, lançada em 2019, que também possui um processador *quad-core*, mas, sua arquitetura é de 64 *bits*, podendo atingir até 1.8 *GHz* de frequência. Sua memória *RAM* é do tipo *LPDDR4*, com 4 modelos, sendo eles de: 1, 2, 4 ou 8 *GBytes*. O tamanho máximo do cartão de memória que esse modelo suporta é de 1 *TByte* e a conexão de rede dessa *Raspberry Pi* é do tipo *Gigabit Ethernet* (1000 *Mbps*). Esse modelo de *Raspberry* foi a mais utilizada dentre os trabalhos pesquisados, pelo fato de possuir mais recursos disponíveis no seu *hardware* e, consequentemente, apresentar um melhor desempenho, tornando-se a opção ideal para o desenvolvimento de um *cluster*, como é a proposta apresentada neste trabalho.

Recentemente, a empresa fabricante do *hardware Raspberry*, a *Raspberry Pi Foundation*, lançou no mercado um modelo mais avançado dessa plataforma, a *Raspberry Pi 5*, lançada em setembro de 2023, com características de *hardware* superiores às identificadas neste trabalho e nos trabalhos pesquisados, porém, devido a questões de recursos e tempo hábil, não foi possível adquirir e efetuar os testes envolvendo este tipo de *hardware*. A título de informação, a nova *Raspberry* conta com um processador *ARM Cortex-A76*, *quad-core* de *64-bits*, *Broadcom BCM2712*, com frequência de 2.4*GHz*, *LPDDR4X-4267 SDRAM* (2*GB*, 4*GB* e 8*GB*), que permite um desempenho até três vezes mais rápido do que as versões anteriores.

5.1.3 Versão do *Apache Hadoop*

De acordo com os resultados dos trabalhos analisados, foi possível identificar a utilização de três versões da biblioteca de paralelismo *Apache Hadoop* na implementação dos *clusters*, sendo as versões 3.2.0 e 3.1.3 as mais utilizadas, presentes em dois trabalhos cada,

como pode ser evidenciado na Tabela 04.

Tabela 04 – Versões do *Apache Hadoop* utilizadas na implementação dos *clusters*

Versões do *Hadoop*	Referências
3.2.0	(Neto; Neto; Moreno, 2022), (Komninos *et al*., 2019)
3.1.3	(Böther; Rabl, 2021), (Lee; Oh; Park, 2021)
1.2.1	(Nugroho; Widiyanto, 2020)

Fonte: Elaborada pelos Autores, 2025.

Na versão 1.2.1 do *Apache Hadoop*, o *JobTracker* é o componente utilizado para gerenciar os recursos do sistema e a execução das tarefas, sendo um ponto crucial de falhas, pois, caso falhe, não será possível rastrear as tarefas. Essa versão foi lançada em 2013 e, infelizmente, não utiliza os recursos disponíveis no *hardware* com eficiência, além de possuir uma escalabilidade limitada e, por isso, indicada apenas para projetos mais simples.

Já a versão 3.1.3, lançada em 2019, apresenta algumas melhorias na utilização dos recursos do sistema, ao substituir o *JobTracker* pelo *YARN*. O *YARN* é um gerenciador de recursos criado com o objetivo de separar o mecanismo de processamento e a função de gerenciamento do *MapReduce* (modelo de programação que permite o processamento de dados massivos em um algoritmo paralelo e distribuído). Ele faz o monitoramento e o gerenciamento de cargas de trabalho dos recursos de alta disponibilidade do *Apache Hadoop*, além de implementar controles de segurança no sistema. Com essa mudança, a escalabilidade e a eficiência da biblioteca *Apache Hadoop* foram aprimoradas, permitindo o desenvolvimento de projetos mais avançados e complexos, para a execução de cargas de trabalho maiores, como é o caso de *Big Data*.

A versão 3.2.0, também lançada em 2019, trouxe melhorias significativas na utilização dos recursos do sistema, tais como: escalabilidade, segurança, tolerância a falhas, flexibilidade no armazenamento de dados e balanceamento de carga, proporcionando uma maior eficiência da biblioteca.

5.1.4 *Software* de Monitoramento e Gerenciamento

Com relação às ferramentas de monitoramento e gerenciamento de desempenho dos *clusters* implementados nos trabalhos pesquisados, foram identificados três tipos de ferramentas: *Zabbix*, *Grafana* e *Prometheus*, sendo as duas últimas as mais utilizadas pelos autores, como pode ser observado na Tabela 05.

Tabela 05 – Ferramentas de monitoramento e gerenciamento dos *clusters*

***Softwares* de monitoramento e gerenciamento**	**Referências**
Grafana	(Neto; Neto; Moreno, 2022), (Penchalaiah *et al*., 2022), (Scolati *et al*., 2020), (Scolati *et al*., 2019)
Prometheus	(Penchalaiah *et al*., 2022), (Scolati *et al*., 2020), (Scolati *et al*., 2019)
Zabbix	(Neto; Neto; Moreno, 2022)

Fonte: Elaborada pelos Autores, 2025.

O *Grafana* é uma ferramenta utilizada para monitorar e visualizar os dados de um sistema. A sua interface *web* permite criar *dashboards* com vários tipos de gráficos e personalizações, bem como gerenciar as configurações disponíveis no sistema.

O *Prometheus* é uma ferramenta utilizada para o monitoramento de dados de séries temporais. Em sua estrutura existem exportadores responsáveis por coletar métricas dos dispositivos e disponibilizá-las, para que sejam exibidas através de *dashboards*.

Já o *Zabbix* é uma ferramenta utilizada para monitorar o desempenho e a disponibilidade de dispositivos instalados em uma rede de computadores. Sua estrutura possui um servidor, chamado *Zabbix Server*, que é responsável por coletar os dados enviados através da rede e dos dispositivos monitorados, que podem ser configurados através do protocolo *SNMP* (*Simple Network Management Protocol*) ou utilizando o *Zabbix Agent*, que facilita a instalação nos equipamentos e amplia a quantidade de dados que podem ser coletados e enviados para o servidor.

Todas essas ferramentas são úteis para monitorar e gerenciar a utilização dos recursos de *hardware* de um *cluster* durante a execução das tarefas, permitindo a realização de análises com base em métricas coletadas que permitem compreender, analisar e melhorar o desempenho do sistema.

5.1.5 Métricas de Desempenho

A partir da análise dos artigos selecionados, foi possível identificar a utilização de diferentes tipos de métricas utilizadas para analisar o desempenho dos *clusters* implementados, com destaque para as seguintes métricas: utilização de *CPU*, utilização de *RAM*, *throughput*, tráfego de rede e *CPU time*, como pode ser observado na Tabela 06.

Tabela 06 – Métricas de desempenho

Métricas	Referências
Utilização de *CPU*	(Neto; Neto; Moreno, 2022), (Böther; Rabl, 2021), (Lee; Oh; Park, 2021), (Komninos *et al*., 2019), (Scolati *et al*., 2019), (Nasir *et al*., 2019)
Throughput (MB/s)	(Neto; Neto; Moreno, 2022), (Böther; Rabl, 2021), (Lee; Oh; Park, 2021), (Nugroho; Widiyanto, 2020), (Nasir *et al*., 2019)
Utilização de *RAM*	(Neto; Neto; Moreno, 2022), (Penchalaiah *et al*., 2022), (Scolati *et al*., 2020), (Scolati *et al*., 2019), (Nasir *et al*., 2019)
Tráfego de rede	(Neto; Neto; Moreno, 2022), (Böther; Rabl, 2021), (Lee; Oh; Park, 2021), (Nasir *et al*., 2019)
CPU time (ms)	(Neto; Neto; Moreno, 2022), (Penchalaiah *et al*., 2022), (Scolati *et al*., 2020), (Scolati *et al*., 2019)
Real Time (s)	(Neto; Neto; Moreno, 2022), (Böther; Rabl, 2021), (Lee; Oh; Park, 2021),
Temperatura	(Neto; Neto; Moreno, 2022), (Nasir *et al*., 2019)
Consumo de Energia	(Böther; Rabl, 2021), (Lee; Oh; Park, 2021)
Average I/O rate (MB/s)	(Neto; Neto; Moreno, 2022), (Lee; Oh; Park, 2021)
Map task time (ms)	(Neto; Neto; Moreno, 2022)
Reduce task time (ms)	(Neto; Neto; Moreno, 2022)
Average I/O (std deviation)	(Neto; Neto; Moreno, 2022)
Test Exec time (s)	(Neto; Neto; Moreno, 2022)
Transaction	(Nugroho; Widiyanto, 2020)
Response Time	(Nugroho; Widiyanto, 2020)
Transactions Rate	(Nugroho; Widiyanto, 2020)

Fonte: Elaborada pelos Autores, 2025.

A utilização dessas métricas permite analisar fatores que influenciam diretamente no desempenho do *cluster* durante a execução das tarefas, como processador, memória *RAM*, consumo de energia, temperatura e tráfego de rede. A partir das informações coletadas, é possível verificar a utilização dos principais recursos de *hardware* do *cluster* e realizar o gerenciamento do sistema, bem como fazer intervenções no sistema, caso seja necessário.

Dentre as métricas identificadas, as que apresentaram uma maior incidência nos trabalhos pesquisados foram: Utilização de *CPU, Throughput (MB/s),* Utilização de Memória *RAM*, Tráfego de Rede, *CPU time (ms)* e *Real Time (s).*

5.1.6 Quantidade de Nós (*Nodes*) dos *Clusters*

Como pode ser observado na Tabela 07, foram utilizadas diversas combinações de *hardware* para compor os nós escravos (*slaves*) dos *clusters* implementados nos trabalhos analisados. Dentre as opções de configuração de *clusters* identificadas, pode-se observar que as mais utilizadas foram formadas pelas seguintes quantidades de nós escravos (*nodes slaves*): 1,

4, 5 e 7 nós (*nodes*). Em quatro trabalhos analisados, foram aplicadas duas ou mais combinações de tamanho de *clusters* diferentes e, em três trabalhos, foram mencionados a implementação de um *cluster* formado por apenas uma única máquina, ou seja, um *cluster* de nó único, onde essa máquina se comportava, ao mesmo tempo, como nó *master* e *slave* da solução de *cluster*.

Tabela 07 – Quantidade de *nodes* do *cluster*

Quantidade de nós	Referências
1	(Lee; Oh; Park, 2021), (Nugroho; Widiyanto, 2020), (Nasir *et al*., 2019)
2	(Neto; Neto; Moreno, 2022), (Lee; Oh; Park, 2021)
3	(Lee; Oh; Park, 2021), (Nugroho; Widiyanto, 2020)
4	(Neto; Neto; Moreno, 2022), (Böther; Rabl, 2021), (Lee; Oh; Park, 2021)
5	(Lee; Oh; Park, 2021), (Nasir *et al*., 2019), (Komninos *et al*., 2019)
7	(Penchalaiah *et al*., 2022), (Scolati *et al*., 2020), (Scolati *et al*., 2019)
8	(Neto; Neto; Moreno, 2022)

Fonte: Elaborada pelos Autores, 2025.

A utilização de mais de uma combinação de máquinas em um *cluster* serve para demonstrar a evolução de desempenho do sistema, a escalabilidade, que pode ser obtida à medida que ocorre um aumento do número de máquinas na formação do *cluster*, tornando-o mais robusto e capaz de executar tarefas de forma mais rápida e eficiente.

5.1.7 Resumo dos Trabalhos Analisados

Neto, Neto e Moreno (2022) desenvolveram um guia para a instalação de um *cluster* de baixo custo utilizando o *Apache Hadoop* e a plataforma *Raspberry Pi 4B*. Para analisar o desempenho do sistema foram utilizados os algoritmos de *benchmark, TeraGen, TeraSort , TeraValidate* e *TestDFSIO*. Os testes foram realizados em *clusters* formados por 2, 4 e 8 nós (*nodes*) escravos, utilizando arquivos com tamanhos variando entre 250 *MBytes* e 1 *GByte*.

Para monitorar o consumo dos recursos de *hardware* do *cluster* durante os testes, os autores utilizaram as ferramentas *Zabbix* e *Grafana*, instaladas em uma *Raspberry Pi 3B+* (servidor). A partir dos resultados obtidos, os autores concluíram que, em razão do baixo custo de implantação do sistema e do bom desempenho da plataforma de *hardware*, a *Raspberry Pi* pode ser uma alternativa viável para o desenvolvimento de *clusters* para trabalhar com *Big Data*.

Penchalaiah *et al*. (2022) desenvolveram um *cluster* utilizando a plataforma *Raspberry Pi 2B*, em conjunto com as tecnologias *Apache Hadoop*, *Spark* e *Docker*. As ferramentas

selecionadas para monitorar o sistema e gerar as análises de desempenho do *cluster*, foram: o *Prometheus* e o *Grafana*. Para avaliar o desempenho do *cluster*, os autores executaram uma aplicação em *Nodejs*, para criar um conjunto de arquivos e uma outra aplicação em *Python*, para analisar os dados que foram gerados. O objetivo deste estudo foi explorar os benefícios que podem ser obtidos ao utilizar *containers* para o desenvolvimento de *clusters*, como por exemplo, a melhoria na tolerância a falhas e na manutenção do sistema.

Böther e Rabl (2021) estudaram a viabilidade na utilização de plataformas *ARM* para o desenvolvimento de sistemas capazes de processar grandes volumes de dados (*Big Data*). Com esse objetivo, foram desenvolvidos dois *clusters*, utilizando as plataformas *Raspberry Pi 3B+* e *4B*, ambos equipados com 4 (quatro) nós escravos. O desempenho dos *clusters* foi analisado com a execução do *benchmark TPCxHS*, baseado no algoritmo *TeraSort*, utilizando arquivos de 1 *GByte* e 10 *GBytes*. Com base nos dados coletados, os autores concluíram que a *Raspberry Pi 4B* apresentou bons resultados, considerando a relação custo/desempenho, e que os problemas encontrados no modelo *3B+*, foram resolvidos ao aumentar a quantidade de memória *RAM* disponível para o sistema.

Lee, Oh e Park (2021) estudaram os impactos causados pela velocidade do dispositivo de armazenamento no desempenho do *cluster*. Para isso, os autores desenvolveram três *clusters* com diferentes plataformas: *Desktop*, *Raspberry Pi 3B* e *4B*. Com o objetivo de analisar o desempenho do *cluster*, os autores utilizaram, para testes de *benchmark*, os seguintes algoritmos: *Wordcount*, *TeraGen*, *TestDFSIO* e *Pi*. Para obter mais informações sobre o desempenho do *cluster*, os autores também avaliaram o desempenho individual de cada plataforma, utilizando para isso a ferramenta *Sysbench*. Todos os testes foram realizados utilizando entre 1 (um) e 5 (cinco) nós escravos, associados a três tipos de armazenamento de memória secundária. Os resultados mostraram que o desempenho do *cluster* foi significativamente afetado pelos dispositivos de armazenamento utilizados.

Nugroho e Widiyanto (2020) avaliaram uma alternativa baseada em computação paralela para a implementação de servidores *IoT* (*Internet of Things*). Para isso, os autores desenvolveram um *cluster* utilizando a *Raspberry Pi 3B+* e o *Apache Hadoop*. Os testes foram executados com o *Siege*, uma ferramenta utilizada para realizar solicitações para um servidor. Para analisar o desempenho do sistema em diferentes situações, os autores fizeram 10, 50, 100 e 150 solicitações para o *cluster*, além de terem utilizado 1 (um) e 3 (três) nós escravos para a montagem dos *cluster*.

Scolati *et al*. (2020) investigaram os benefícios da utilização de *containers* para o

desenvolvimento de *clusters* para o processamento de dados. A partir desse objetivo, os autores desenvolveram um *cluster* composto por 7 (sete) nós escravos, utilizando a *Raspberry Pi 2B* como plataforma de *hardware* e os *softwares Spark*, *Docker* e *Apache Hadoop*. Além disso, os autores instalaram uma tecnologia *Blockchain*, utilizada para aumentar a confiabilidade do sistema, chamada *TOM Orchestrator*. O desempenho do *cluster* foi analisado com a execução de duas aplicações, uma delas responsável por gerar um conjunto de arquivos e a outra por analisá-los. Durante a execução dos testes, os autores utilizaram o *Grafana* e o *Prometheus* para monitorar o desempenho e o consumo dos recursos de *hardware* do *cluster*.

Komninos *et al*. (2019) desenvolveram um *cluster* composto por 5 (cinco) nós escravos, utilizando a *Raspberry Pi 4B* e os *softwares Spark* e o *Apache Hadoop*. O propósito foi avaliar a utilização dessa plataforma em aprendizado de máquina (*Machine Learning*), para executar previsões em aplicações turísticas. Para avaliar o desempenho do *cluster*, os autores executaram um algoritmo de regressão linear simples, utilizando um conjunto de dados de carros usados para prever os preços de venda e um algoritmo de classificação de árvore de decisão, utilizando dados de *check-ins* para recomendar locais de visita para um usuário. A partir dos resultados, os autores concluíram que o *cluster* de *Raspberry Pi* possui um bom desempenho para o treinamento e a execução de modelos de *Machine Learning* (*ML*).

Scolati *et al*. (2019) desenvolveram um *cluster* com 7 (sete) nós escravos, utilizando a *Raspberry Pi 2B* e os *softwares Docker*, *Apache Hadoop* e *Spark*. O objetivo foi avaliar uma alternativa capaz de coletar e processar dados localmente, apresentando as vantagens da utilização de *containers* no desenvolvimento de *clusters*. Os autores utilizaram as ferramentas *Prometheus* e *Grafana* para monitorar o sistema durante os testes de *benchmark*. Para analisar o desempenho do *cluster*, os autores utilizaram uma aplicação para gerar um conjunto de dados e outra para analisá-los.

Nasir *et al*. (2019) propuseram uma estrutura para processamento de vídeos de vigilância para cidades inteligentes, baseada na plataforma *Raspberry Pi*. O objetivo do sistema era gerar um resumo dos vídeos coletados a partir das câmeras, enviando-os para o armazenamento na nuvem, reduzindo o consumo de banda gerada pela transferência de grandes arquivos. Com a proposta, os autores desenvolveram um *cluster* utilizando a *Raspberry Pi 3B*, em conjunto com os *softwares Spark* e *Apache Hadoop*. Os testes foram realizados executando um algoritmo de sintetização de vídeo, utilizando arquivos com tamanhos variando entre 100 *MBytes* e 1 *GByte*. Com base nos resultados, os autores chegaram à conclusão de que o sistema se apresenta como uma solução viável para atender às necessidades de uma cidade inteligente.

5.2 Testes de Desempenho do *Cluster*

Para realizar uma análise de desempenho do *cluster*, desenvolvido com a plataforma *Raspberry Pi*, foram utilizados os seguintes algoritmos de *benchmark*: *Pi* (π) - *quasi-Monte Carlo* (*qMC*), *TeraGen*, *TeraSort*, *TeraValidate* e *TestDFSIO*. Todos os testes foram executados com 3 (três) diferentes combinações de tamanho do *cluster*, com 2, 4 e 8 nós escravos.

Durante a execução dos algoritmos, foram coletadas as seguintes métricas: uso do processador *CPU*) e da memória *RAM*; número de processos; tráfego de rede, *real time*, *cpu time*, *map time*, *reduce time*, temperatura e consumo de energia.

Para os testes envolvendo o algoritmo de cálculo do *Pi* (π) forma utilizados 16 mapas e 10.000.000 de amostras. Já para os testes envolvendo os algoritmos *TeraGen*, *TeraSort*, *TeraValidate* e *TestDFSIO*, foram utilizados arquivos com tamanhos variados, de 500 *MBytes*, 1 *GByte* e 2 *GBytes*, respectivamente.

5.2.1 Testes envolvendo o algoritmo *Pi* (π) - *quasi-Monte Carlo* (*qMC*)

Nesta seção, serão apresentados os resultados dos testes de *benchmark* envolvendo o algoritmo de cálculo do *Pi* (π) - *quasi-Monte Carlo* (*qMC*).

5.2.1.1 Métricas de desempenho do *Hadoop* e consumo energético do *cluster* – *Pi* (π)

Na Tabela 08, observa-se os resultados dos testes envolvendo o algoritmo de cálculo do *Pi* (π) com os 3 (três) cenários de *cluster*, evidenciando o comportamento da biblioteca *Apache Hadoop* e o consumo energético de cada *cluster*, durante a realização dos testes.

Tabela 08 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *Pi (π)*

Quantidade de Nodes	*Real Time (s)*	*CPU Time (ms)*	*Map Time (ms)*	*Reduce Time (ms)*	*Consumo Mínimo (W)*	*Consumo Máximo (W)*
2	2m12,942s	84.590	404.444	47.929	13,8	21
4	1m39,298s	88.210	519.710	11.813	22,5	33,9
8	1m44,437s	84.670	594.514	11.821	36,6	50,7

Fonte: Elaborada pelos Autores, 2025.

1. ***Real Time (s):*** Com relação ao tempo real de processamento das tarefas, observa-se que com o escalonamento dos nós (*nodes*) do *cluster*, ocorreu uma redução no tempo

de resposta das tarefas de aproximadamente 32%. O fato do *cluster* formado por 4 nós (*nodes*) ter ficado com o tempo de resposta um pouco abaixo do *cluster* de 8 nós (*nodes*), se deve a questão da intensa distribuição das tarefas para os vários nós (*nodes*) do sistema e ao tempo de resposta desses nós (*nodes*) na rede de interconexão, ocasionando uma pequena elevação no tempo de resposta final do processamento, de aproximadamente 3,6%;

2. ***CPU Time (ms):*** Os *clusters* formados por 2 e 8 *nodes*, gastaram, respectivamente, menos tempo de utilização da *CPU* durante à execução das tarefas, aproximadamente 4% a menos. O primeiro *cluster*, por ter apenas 2 *nodes*, não teve como realizar muitas tarefas de *map* (divisão das tarefas em blocos menores), portanto, o número de *mappers* (número de pedaços dos blocos) foi menor, em função da quantidade de nós. Já o segundo *cluster*, formado por 8 *nodes*, conseguiu realizar mais tarefas de *map*, aumentando o número de *mappers*, porém, como a quantidade de *nodes* no sistema era maior, foi realizada uma melhor distribuição dessas tarefas por parte do *Hadoop*, fazendo com que o tempo de utilização da *CPU* fosse menor;
3. ***Map Time (ms):*** O tempo de *map* (mapeamento), ou seja, o tempo de quebra dos pedaços de blocos em tamanhos menores, foi menor no *cluster* formado por apenas 2 *nodes*, pelo fato do *Hadoop* não conseguir realizar muitos *mappers*, devido à baixa quantidade de nós no sistema. Não se tornando, portanto, adequado e viável a utilização de uma biblioteca de paralelismo, como o *Apache Hadoop*, nesses casos. No *cluster* formado por 8 *nodes*, observa-se uma intensa atividade de *map*, com o aumento do tempo em 47%, devido ao escalonamento do número de nós participantes do *cluster*. Ou seja, quanto mais nós (*nodes*) tiver o sistema, maior será a quantidade de *mappers*, consequentemente, menor será o tamanho de cada tarefa a ser executada. As tarefas de *map* gastam mais tempo devido a filtragem e classificação dos dados;
4. ***Reduce Time (ms):*** O tempo das tarefas de *reduce*, ou seja, a agregação dos dados divididos, filtrados e classificados pelas tarefas de *map*, são mais reduzidos justamente por não existir, durante esse processo, a filtragem e a classificação dos dados. Durante os testes, observa-se que esse tempo de agregação dos dados é menor quando o tamanho do *cluster* é maior, com mais *nodes*, aproximadamente 75%. Quando a quantidade de *nodes* no *cluster* é menor, o paralelismo é mais limitado, e como a operação de redução, em geral, envolve a combinação de dados de várias partições, a falta de nós adicionais significa que o trabalho é dividido entre menos

máquinas, o que pode resultar em tempos de processamento mais longos.

Além dessa situação, outras três pode influenciar negativamente no aumento do tempo de *reduce*, são elas: 1) Carregamento e comunicação: Quando existe menos nós no *cluster*, a quantidade de dados que cada nó precisa processar aumenta, gerando sobrecarga na comunicação entre os nós (*nodes*), tendo em vista que o nó (*master*) central precisa reunir os resultados de todos os outros nós (*nodes*). Em resumo, menos nós no *cluster* significa que a comunicação entre eles pode ser mais lenta, especialmente se a rede de interconexão do *cluster* não for otimizada ou se a carga de dados for grande; 2) Desbalanceamento de carga: Se o número de nós do *cluster* for muito pequeno, a carga de trabalho pode não ser balanceada de forma eficiente, fazendo com que alguns nós fiquem sobrecarregados, enquanto outros ficam subutilizados. Esse desbalanceamento pode aumentar o tempo de execução do processo de redução (*reduce*), pois a operação não será distribuída de maneira adequada; 3) Capacidade de processamento reduzida: Os *clusters* formados por menos nós têm menos recursos de processamento, tais como *CPU* e memória *RAM*, o que pode limitar a capacidade de realizar reduções (*reduce*) complexas de maneira eficiente. Isso pode resultar em gargalos, especialmente se a quantidade de dados for grande ou a operação de redução for computacionalmente intensiva, mais complexa;

5. **Consumo de energia:** Durante os testes envolvendo o algoritmo de cálculo do *Pi*, observa-se que o consumo energético foi aumentando à medida em que o *cluster* era escalonado. O consumo máximo de energia do *cluster* chegou a 50,7 *W*, durante o processamento do algoritmo em um *cluster* formado por 8 (oito) *nodes*, um aumento de aproximadamente 141% em relação ao primeiro cenário de testes, com 2 (dois) *nodes*. Considerando que o número de máquinas do *cluster*, praticamente, quadruplicou, esse aumento não foi elevado e nem desproporcional, tendo em vista que o consumo médio de um computador *desktop*, em repouso (*standby*), segundo o *site* Canaltech (2021), é de aproximadamente 50 a 150 *Watts*, evidenciando o baixo consumo energético da plataforma *Raspberry Pi*.

5.2.1.2 Métricas de desempenho do *cluster* (*hardware* e tráfego de rede) – *Pi* (π)

5.2.1.2.1 *Cluster* formado por 2 *nodes* – *Pi* (π)

Os resultados das métricas de desempenho de *hardware*, mostram que o consumo de utilização de *CPU* foi maior no *Slave 1*, apresentando um leve desbalanceamento de carga, em relação ao node *Slave 2*. O nó *master* se manteve com o seu consumo de *CPU* baixo, em relação aos demais, evidenciando que com o paralelismo, o consumo de *CPU* é maior nos nós escravos da solução.

Com relação ao gasto de memória *RAM*, observa-se uma distribuição, praticamente, igual entre os nós do *cluster*, com uma leve diferença em relação ao consumo do nó *Slave 2*, que teve um consumo de aproximadamente 8% a mais.

Já a temperatura, se apresentou maior na *node Slave 1*, que teve um maior consumo de *CPU*, evidenciando que, com o trabalho intenso da *CPU* a temperatura do equipamento aumenta proporcionalmente. No que se refere a quantidade de processos, observa-se que o nó *master*, por ser o nó que efetua a distribuição das tarefas entres os demais nós do *cluster*, teve um volume maior de processos durante a realização do teste, como pode ser evidenciado na Tabela 09.

Tabela 09 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *Pi* (π)

Cluster – 2 Nodes								
Nodes	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
	Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
Master	3,78	10,3	36,8	37,3	31,2	36,5	224	228
Slave 1	1,51	57,3	14,5	35,8	43,8	53,1	155	169
Slave 2	1,54	46,5	14,6	40,3	32,6	44,8	154	171

Fonte: Elaborada pelos Autores, 2025.

Quanto ao tráfego da rede, observa-se que o *node Slave 1* foi o que mais enviou informações na rede. Esse volume tem um pouco de relação com a demanda de utilização da *CPU* desse nó, tendo em vista que foi o nó mais demandado em relação a esse recurso pelo nó *master*. Porém, com relação ao tráfego recebido, observa-se um volume muito equilibrado entre as *nodes* do *cluster*, com um pouco de demanda a mais pelo *Slave 2*, o que evidencia um balanceamento de carga das tarefas entre as *nodes* da solução, como pode ser observado na Tabela 10.

Tabela 10 – Resultados das métricas de tráfego de rede do *cluster* – 2 *Nodes* – *Pi* (π)

	***Cluster – 2 Nodes* (Tráfego de Rede)**			
Nodes	***Sent***		***Received***	
	Min.	***Máx.***	***Min.***	***Máx.***
Master	13,7 Kb/s	43,4 Kb/s	20,3 Kb/s	52,7 Kb/s
Slave 1	11,0 Kb/s	981 Kb/s	6,74 Kb/s	468 Kb/s
Slave 2	11,1 Kb/s	55,3 Kb/s	6,63 Kb/s	541 Kb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 51, 52,53, 54 e 55.

5.2.1.2.2 *Cluster* formado por 4 *nodes* – *Pi* (π)

Os resultados dos testes evidenciam um desbalanceamento de carga entre as *nodes* do *cluster*, com um consumo maior de utilização de *CPU* pelas *nodes Slave* 3 (sobrecarga) e *Slave 1*, respectivamente. O nó *master*, como participa apenas distribuindo as tarefas entre as *nodes slaves* do sistema, apresentou um consumo baixo de utilização de *CPU*, comprovando que no paralelismo o consumo de *CPU* é maior entre as *nodes slaves* do sistema.

Em relação ao gasto de memória *RAM*, observa-se um cenário de distribuição da carga mais equilibrado entre as *nodes* do sistema, com destaque apenas para o *node Slave 3*, que teve um consumo menor em relação aos demais, mesmo sendo uma das *nodes* que mais utilizou os recursos de *CPU*. Esse mesmo cenário é observado nos testes do *cluster* formado por 2 *nodes*, ou seja, o *node* que utiliza mais recurso de *CPU* tem sempre um gasto menor de consumo de memória *RAM*.

Durante os testes, verifica-se que a temperatura das *nodes* se manteve equilibrada, com um leve aumento na *node Slave 3*, que teve um consumo maior de utilização da *CPU*. Semelhante ao exemplo do cenário anterior, com 2 *nodes*, quanto mais intenso for o trabalho do *node*, maior será a temperatura do equipamento.

Quanto à quantidade de processos executados durante os testes, observa-se que o nó *master* apresentou um maior volume, justamente por ser o nó que efetua a distribuição das tarefas para os demais nós do *cluster*. Os demais *nodes*, mantiveram-se, praticamente com a quantidade de processos equilibrada, com uma leve diferença entre eles, como pode ser observado na Tabela 11.

Tabela 11 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *Pi* (π)

Cluster – 4 Nodes								
Nodes	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
	Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
Master	4,9	20,9	36,7	37,2	33,1	38,0	227	232
Slave 1	1,57	68,4	16,1	37,2	35,0	46,7	156	175
Slave 2	1,70	65,1	15,0	43,8	32,1	44,8	155	170
Slave 3	2,66	78,3	14,9	29,7	39,4	48,7	158	166
Slave 4	1,73	60,5	15,1	43,1	35,0	45,3	154	172

Fonte: Elaborada pelos Autores, 2025.

Com relação ao tráfego da rede, verifica-se que as *nodes Master* e *Slave 1* se destacaram com relação ao tráfego de envio (*sent*). O nó *master* é justificado pela quantidade de processos e tarefas que ele envia para os demais *nodes* do *cluster*, já o *node Slave 1*, observa-se que o *Hadoop* escolhe muito esse *node* para participar ativamente das tarefas, provavelmente por ser o primeiro *node* do sistema, e iniciar suas atividades por ele.

No que refere ao tráfego recebido, verifica-se um tráfego maior entre as *nodes Slave 1*, *3* e *4*, evidenciando a distribuição das tarefas no sistema. Entretanto, é possível perceber que esse balanceamento não é equilibrado entre todas as *nodes* do sistema, tendo em vista que o *node Slave 2* apresentou um tráfego menor. Isso pode ter ocorrido, por alguns fatores, dentre eles, uma dificuldade do *node* acessar a rede ou pelo fato do nó ser menos requisitado durante a execução das tarefas, como pode ser observado na Tabela 12.

Tabela 12 – Resultados das métricas de tráfego de rede do *cluster* – 4 *Nodes* – *Pi* (π)

***Cluster – 4 Nodes* (Tráfego de Rede)**				
Nodes	***Sent***		***Received***	
	Min.	***Máx.***	***Min.***	***Máx.***
Master	32,2 Kb/s	972 Kb/s	41,5 Kb/s	126 Kb/s
Slave 1	9,37 Kb/s	957 Kb/s	6,21 Kb/s	979 Kb/s
Slave 2	11,3 Kb/s	91,1Kb/s	5,60 Kb/s	543 Kb/s
Slave 3	9,66 Kb/s	537 Kb/s	9,86 Kb/s	937 Kb/s
Slave 4	10,5 Kb/s	531 Kb/s	7,07 Kb/s	940 Kb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 56, 57,58, 59 e 60.

5.2.1.2.3 *Cluster* formado por 8 *nodes* – *Pi* (π)

Apesar do escalonamento no número de *nodes* no sistema, os resultados dos testes

mostraram uma ineficiência do *Apache Hadoop* com relação ao balanceamento do uso dos recursos de *hardware* do *cluster*. Nesse cenário, com relação a utilização da *CPU*, observa-se uma participação maior de 4 *nodes* (*Slaves* 1, 2, 3 e 8) no processo, com os demais sendo subutilizados. Essa mesma situação pode ser evidenciada com relação ao consumo de memória *RAM*.

Com relação à temperatura, praticamente, houve um equilíbrio entre as *nodes*, com um leve destaque apenas para as *nodes* que participaram ativamente das tarefas, ou seja, os que tiveram um uso mais intenso da *CPU*.

Quanto à quantidade de processos executados durante os testes, observa-se novamente um volume maior na *node master*, pelo fato de ser o nó principal, que realiza a distribuição das tarefas para os demais *nodes* do sistema. As outras *nodes*, mantiveram um volume de processos muito próximo, com um pequeno destaque para as *nodes* que participaram mais ativamente da execução das tarefas, como pode ser evidenciado na Tabela 13.

Tabela 13 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *Pi* (π)

Cluster – 8 Nodes								
Nodes	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
	Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
Master	4,71	20,6	36,9	37,1	33,1	37,5	223	226
Slave 1	1,57	62,2	13,9	36,1	31,2	44,3	155	174
Slave 2	1,55	60,6	15,3	43,9	32,1	42,4	155	172
Slave 3	1,62	61,9	15,4	43,4	35,0	46,7	155	173
Slave 4	0,97	1,78	14,5	14,6	32,1	34,1	155	157
Slave 5	0,91	2,09	15,5	15,9	35,5	37,5	156	161
Slave 6	0,95	1,54	14,9	15,0	34,1	36,0	152	156
Slave 7	2,32	31,5	14,8	22,4	37,5	44,3	160	168
Slave 8	1,57	71,3	15,1	39,1	35,5	46,7	155	175

Fonte: Elaborada pelos Autores, 2025.

Nesse cenário de teste, verifica-se que o tráfego da rede, seja no envio ou no recebimento dos dados, não foi elevado, porém, está desequilibrado entre as *nodes* e desproporcional, com destaque no aumento do volume para as *nodes Slaves 3*,*5* e *8*. Um fato anormal, é que a *Slave* 5, apesar de não participar ativamente da execução das tarefas, teve um tráfego de envio de dados elevado, em relação aos demais. Isso pode ter ocorrido por algum problema na rede de interconexão ou na distribuição das tarefas.

Em relação ao tráfego recebido, observa-se novamente uma situação em que a *Slave 1* se sobressai em relação aos demais, evidenciando uma predileção do *Apache Hadoop* em escolher a *Slave 1* em suas transações. Quanto aos demais *nodes* do sistema, verifica-se um

equilíbrio nas taxas de recebimento dos dados, como pode ser observado na Tabela 14.

Tabela 14 – Resultados das métricas de tráfego de rede do *cluster* – 8 *Nodes* – *Pi* (π)

	***Cluster – 8 Nodes* (Tráfego de Rede)**			
Nodes	***Sent***		***Received***	
	Min.	***Máx.***	***Min.***	***Máx.***
Master	55,5 Kb/s	524 Kb/s	80,0 Kb/s	158 Kb/s
Slave 1	11,0 Kb/s	528 Kb/s	6,46 Kb/s	938 Kb/s
Slave 2	9,12 Kb/s	163 Kb/s	6,13 Kb/s	538 Kb/s
Slave 3	10,7 Kb/s	948 Kb/s	6,37 Kb/s	524 Kb/s
Slave 4	9,40 Kb/s	525 Kb/s	6,55 Kb/s	522 Kb/s
Slave 5	9,07 Kb/s	967 Kb/s	6,26 Kb/s	519 Kb/s
Slave 6	9,27 Kb/s	525 Kb/s	5,70 Kb/s	524 Kb/s
Slave 7	9,77 Kb/s	536 Kb/s	6,41 Kb/s	462 Kb/s
Slave 8	9,06 Kb/s	945 Kb/s	5,49 Kb/s	521 Kb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 61, 62,63, 64 e 65.

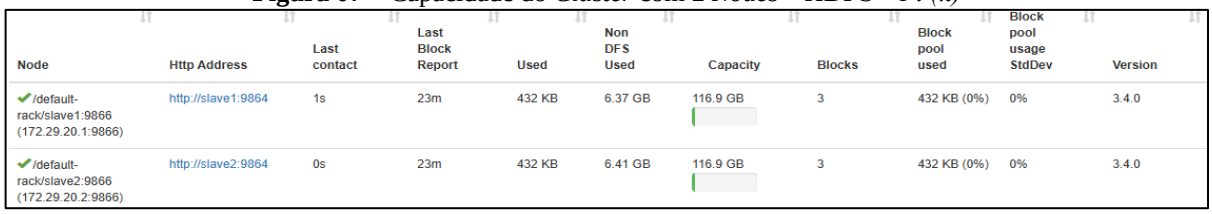
Em resumo, com base nos testes envolvendo o algoritmo do *Pi* (π), observa-se um desbalanceamento do *cluster*, em praticamente todas as métricas de *hardware*, à medida que o *cluster* vai sendo escalonado (aumentado), uma situação que deveria ser justamente oposta, tendo em vista que à medida que se aumenta a quantidade de nós, os recursos de *hardware* tendem a aumentar proporcionalmente. Isso demonstra as dificuldades do *Apache Hadoop* em distribuir e coordenar as tarefas de forma igualitária no *cluster*, não realizando, portanto, um balanceamento de carga adequado e eficiente, diminuindo dessa forma a otimização e a eficiência dos recursos de *hardware* disponíveis, subutilizando-os. Com relação ao consumo de espaço em disco, para o armazenamento dos dados processados, observa-se que o *Hadoop* apresentou um desempenho eficiente, distribuindo por igual essa demanda entre as *nodes* dos *cluster*, porém, cabe mencionar, que o algoritmo *Pi (π)* não precisa de muitos recursos de armazenamento para ser executado, por isso a baixa ocupação de espaço em disco identificada nos cenários de testes.

5.2.1.3 Capacidade de armazenamento dos *Clusters – Algoritmo Pi (π)*

Nas Figuras 07, 08 e 09, respectivamente, é possível verificar a utilização do espaço em disco de cada *node* dos *cluster*s implementados, após os cenários de testes envolvendo o algoritmo *Pi (π) – quasi-Monte Carlo* (*qMC*), com *clusters* de diferentes tamanhos, formados por 2, 4 e 8 *nodes slaves* e uma *master*, em cada configuração de *cluster*. Nelas é possível

identificar a baixa ocupação de espaço em disco, chegando a registrar menos de 1% (432 *KBytes*) do espaço total disponível nas unidades de armazenamento secundário (cartão de memória), em todos os cenários de testes.

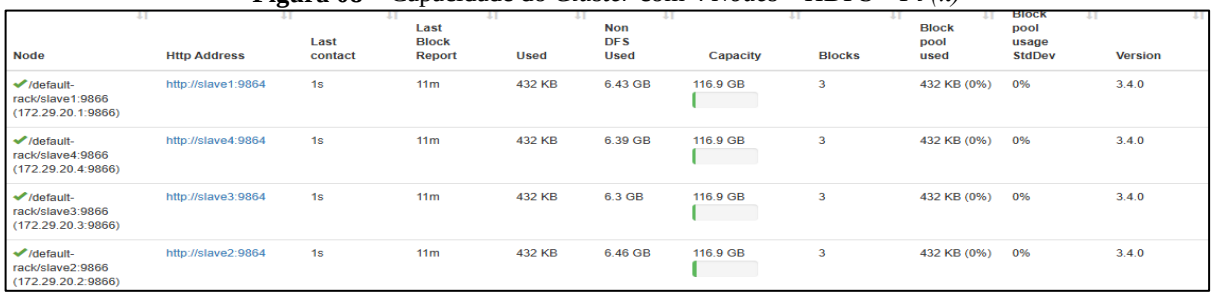
Figura 07 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – *Pi (π)*



Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	1s	23m	432 KB	6.37 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	0s	23m	432 KB	6.41 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

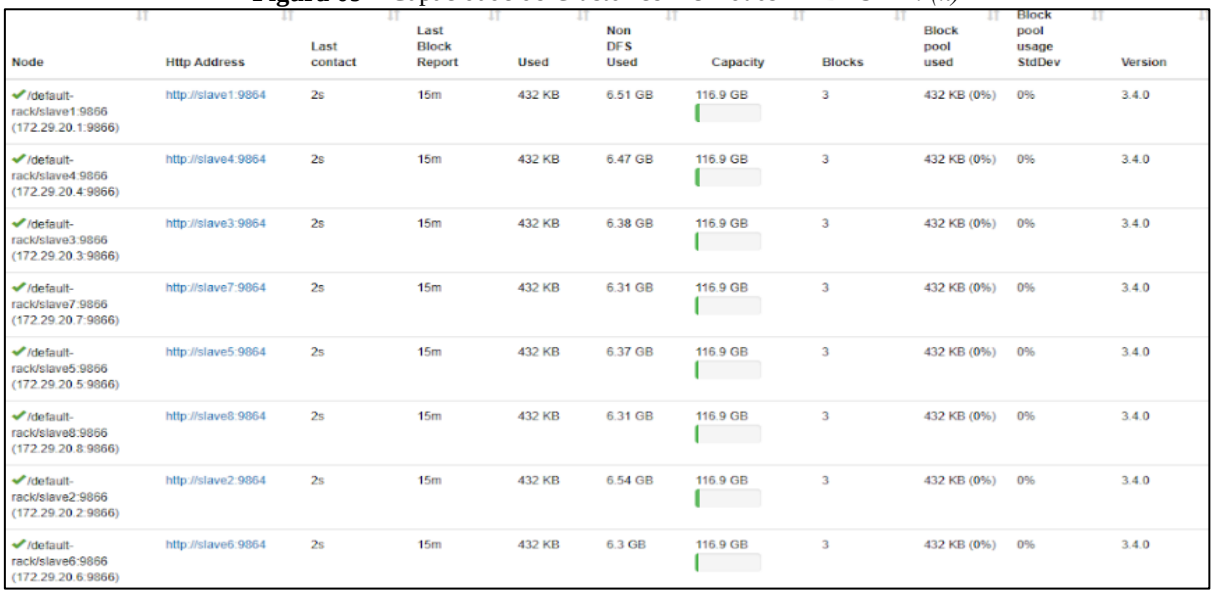
Figura 08 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – *Pi (π)*



Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	1s	11m	432 KB	6.43 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	1s	11m	432 KB	6.39 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	1s	11m	432 KB	6.3 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	11m	432 KB	6.46 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 09 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – *Pi (π)*



Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	15m	432 KB	6.51 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	2s	15m	432 KB	6.47 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	2s	15m	432 KB	6.38 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	2s	15m	432 KB	6.31 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	2s	15m	432 KB	6.37 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	2s	15m	432 KB	6.31 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	2s	15m	432 KB	6.54 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0
/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	2s	15m	432 KB	6.3 GB	116.9 GB	3	432 KB (0%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

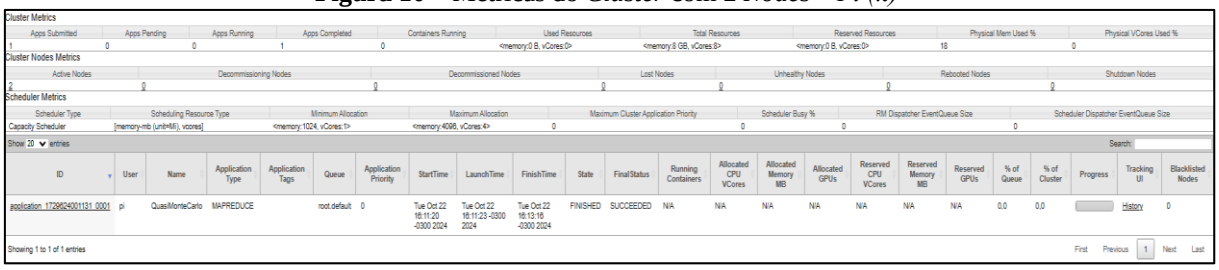
Como o algoritmo *Pi (π)* não trabalha com grandes volumes de arquivos, a demanda de ocupação de espaço em disco (memória secundária) é muito baixa, chegando a registrar, em todos os cenários executados, um consumo de apenas 432 *Kbytes*. Outro fato importante de ser

mencionado, é que a biblioteca *Hadoop* efetuou de forma eficiente e equilibrada, o balanceamento e a distribuição da carga dos arquivos processados em cada node do *cluster*, em todos os cenários de testes envolvendo esse algoritmo.

As Figuras 10, 11 e 12, respectivamente, mostram as telas de comprovação da execução dos testes, envolvendo o algoritmo *Pi (π)*, com todos os tamanhos de *clusters*, contendo informações sobre os recursos de *hardware* utilizados e disponíveis em cada *cluster*.

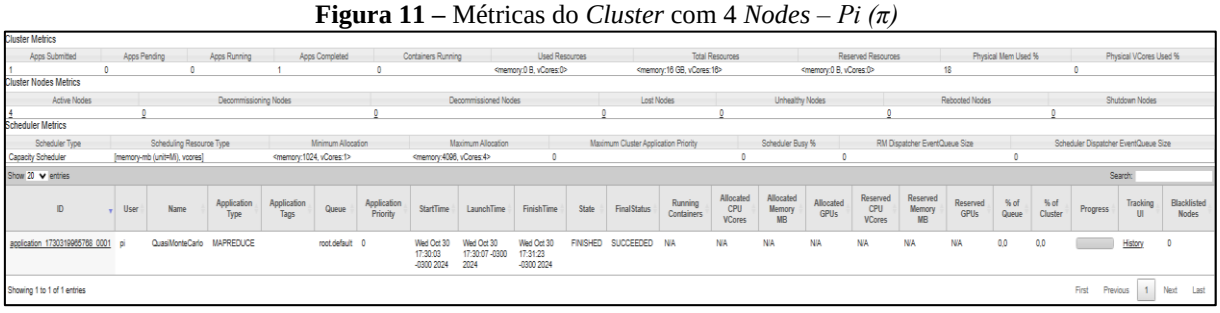
Nessas imagens é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (S*UCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 18%; a quantidade de nós (*nodes*) ativos na execução dos testes, no campo *Active Nodes*, neste caso com 2, 4, 8 *nodes slaves* e uma *node master*, em cada configuração, respectivamente. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*QuasiMonteCarlo*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources*, com 8 *GBytes*, 16 *GBytes* e 32 *GBytes*, de acordo com a capacidade dos *clusters* implementados, respectivamente.

Figura 10 – Métricas do *Cluster* com 2 *Nodes – Pi (π)*



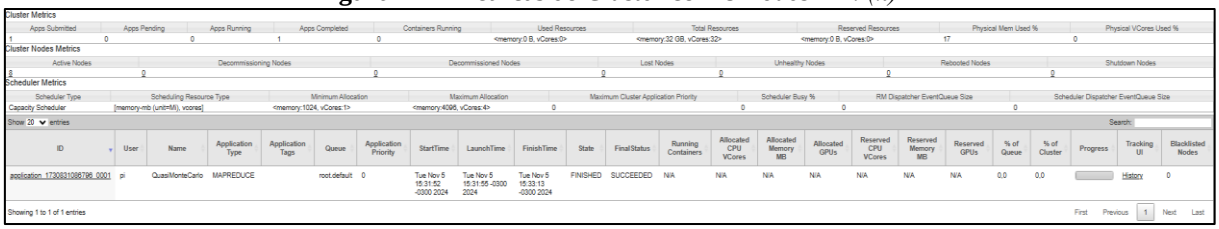
Fonte: Elaborada pelos Autores, 2025.

Figura 11 – Métricas do *Cluster* com 4 *Nodes – Pi (π)*



Fonte: Elaborada pelos Autores, 2025.

Figura 12 – Métricas do *Cluster* com 8 *Nodes – Pi (π)*



Fonte: Elaborada pelos Autores, 2025.

5.2.2 Testes envolvendo os algoritmos "*Tera*" (*TeraGen - TeraSort - TeraValidate*)

Nesta seção, serão apresentados os resultados dos testes de *benchmark* envolvendo os algoritmos *TeraGen*, *TeraSort* e *TeraValidate*.

5.2.2.1 Desempenho do *Hadoop* e consumo energético do *cluster* – *TeraGen*

O algoritmo *TeraGen* é responsável por gerar dados de entrada aleatório no *cluster*, executando uma tarefa *MapReduce* (*Job MapReduce*), que gera dados binários de forma aleatória, com chaves e valores com diferentes tamanhos.

A Tabela 15 apresenta os resultados obtidos durante a execução do algoritmo *TeraGen* em 3 (três) configurações de *cluster*, com 3 (três) tamanhos de arquivos. Esses dados ilustram o comportamento e o consumo energético do *cluster* durante a execução dos testes em diferentes cenários.

Tabela 15 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *TeraGen*

Quantidade de Nodes	***Arquivos (MBytes)***	***Real Time (s)***	***CPU Time (ms)***	***Map Time (ms)***	***Reduce Time (ms)***	***Consumo Mínimo (W)***	***Consumo Máximo (W)***
2	500	1m27,037s	37.090	77.582	-----	13,8	20,4
	1000	1m48,050s	58.360	118.065	-----	12,9	18,0
	2000	2m31,636s	99.830	185.884	-----	12,0	17,4
4	500	1m37,291s	39.420	97.480	-----	21,6	30,6
	1000	1m59,078s	66.460	139.613	-----	21,6	30,9
	2000	2m43,902s	98.370	229.145	-----	21,3	30,6
8	500	1m42,340s	37.790	104.008	-----	36,9	53,7
	1000	2m21,621s	61.030	185.518	-----	36,9	52,2
	2000	3m11,227s	97.950	284.456	-----	38,7	56,7

Fonte: Elaborada pelos Autores, 2025.

1. ***Real Time (s):*** Em relação ao tempo real de processamento das tarefas, observa-se que ao escalonar o *cluster* para 4 nós (*nodes*), houve um aumento de aproximadamente 9,6% na média do tempo de execução. Esse comportamento

também se repetiu ao expandir o *cluster* para 8 nós (*nodes*), resultando em um aumento de cerca de 14,3%. O aumento do tempo de resposta, mesmo com a adição de mais nós (*nodes*), mesmo mantendo o tamanho do arquivo em cada cenário, pode ser atribuído à replicação dos dados gerados entre todos os nós (*nodes*) que participaram da execução dos testes;

2. ***CPU Time (ms):*** A média do tempo de utilização da *CPU* no *cluster* composto por 2 nós (*nodes*) foi de 65.093 *ms*, durante a execução das tarefas. Ao ampliar o número de nós (*nodes*) do *cluster* para 4, foi observado um aumento de aproximadamente 4,6% nesse consumo. Esse comportamento pode ser causado pela adição de mais nós (*nodes*), permitindo uma melhor divisão das tarefas, elevando assim a carga de trabalho dos nós (*nodes*). No entanto, ao utilizar 8 nós (*nodes*) no *cluster*, para a execução do algoritmo, houve uma redução de cerca de 3,7% no tempo de utilização da *CPU*. Isso ocorre devido à distribuição mais eficiente das tarefas entre os nós (*nodes*), reduzindo a sobrecarga individual de cada node do *cluster*;
3. ***Map Time (ms):*** A média do tempo de *map* no *cluster* composto por apenas 2 nós (*nodes*) foi de 127.177 *ms*, o menor valor dentre as três configurações analisadas. Esse resultado mostra uma dificuldade do sistema em realizar muitas tarefas de mapeamento (*map*), devido ao baixo número de nós (*nodes*), resultando em blocos maiores e com poucos *mappers*. No *cluster* formado por 4 nós (*nodes*), o tempo de *map* apresentou um aumento de cerca de 22%, já no *cluster* com 8 nós (*nodes*), o aumento foi similar, aproximadamente 23%. Esse aumento está relacionado à maior quantidade de *mappers* que são gerados ao expandir (escalonar) o *cluster*, resultando em tarefas menores, mas que precisam de mais tempo para ser mapeadas, devido ao processo de classificação dos dados;
4. ***Reduce Time (ms):*** O algoritmo *TeraGen* não executa tarefas de *reduce*, pois ele é utilizado apenas para geração de dados aleatórios, não realizando nenhum tipo de processamento sobre esses dados, por esse motivo, a Tabela 14 não apresenta valores referentes a esse parâmetro;
5. **Consumo de Energia*:*** Durante a execução do algoritmo *TeraGen*, o consumo energético médio do *cluster* com 2 nós (*nodes*) foi de 18,6 *W*. Ao ampliar o tamanho do *cluster* para 4 nós (*nodes*), houve um aumento de 65% nesse consumo, enquanto, com um *cluster* de 8 nós (*nodes*), o acréscimo foi de aproximadamente 76,5%. Esses dados mostram que, o consumo de energia aumentou à medida em que mais nós

(*nodes*) foram adicionados ao *cluster*, porém, esse crescimento não foi proporcional, já que o número de nós (*nodes*) dobrou em ambas as situações. Isso evidencia uma eficiência energética da plataforma *Raspberry Pi*, mesmo ao ampliar a infraestrutura.

5.2.2.2 Métricas de desempenho do *cluster* (*hardware* e tráfego de rede) – *TeraGen*

5.2.2.2.1 *Cluster* formado por 2 *nodes* – *TeraGen*

Os resultados das métricas de desempenho de *hardware*, mostram um consumo equilibrado de utilização de *CPU* nos cenários que envolveram arquivos com tamanhos maiores, respectivamente, evidenciando uma melhor distribuição das tarefas entre as *nodes*, bem como uma utilização mais eficiente dos recursos de *hardware* disponíveis no *cluster*.

O gasto de memória *RAM* se mostrou, praticamente, equilibrado entre as *nodes slaves*, com um aumento do consumo, em todos os cenários, de aproximadamente 43%, nas *nodes master*. Esse comportamento se deve ao trabalho desses nós (*nodes master*) na distribuição das tarefas.

Com relação à temperatura das *nodes*, nos cenários de testes avaliados, verifica-se um aumento desse parâmetro nas *nodes* que tiveram um consumo maior de *CPU*, ou seja, dos que participaram mais ativamente do processamento das tarefas.

Já no que se refere à quantidade de processos, observa-se que, em todos os cenários, as *nodes master*, foram as que apresentaram um maior volume de processos, tendo em vista o seu papel de distribuição das tarefas entres os demais nós do *cluster*.

Os resultados de todos esses parâmetros podem ser evidenciados na Tabela 16.

Tabela 16 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *TeraGen*

Cluster – 2 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	4,69	17,4	37,3	37,5	31,2	36,0	226	228
	Slave 1	1,60	41,7	14,2	21,1	39,4	48,2	153	161
	Slave 2	1,52	68,6	14,4	27,6	35,5	42,4	155	165
1000	*Master*	4,00	21,2	36,3	36,6	33,6	39,4	224	226
	Slave 1	1,78	71,3	15,6	28,9	43,8	51,6	156	169
	Slave 2	1,15	70,1	14,7	20,7	34,6	42,8	154	163
2000	*Master*	3,32	18,6	35,1	36,5	33,1	39,9	226	231
	Slave 1	1,28	74,2	15,1	28,8	44,3	54,0	158	168
	Slave 2	1,25	70,2	15,2	21,2	37,0	43,8	155	161

Fonte: Elaborada pelos Autores, 2025.

O tráfego de envio dos dados teve um aumento significativo em todas as *nodes master* do *cluster*, em função das suas intensas atividades de distribuição e coordenação das tarefas para os demais *nodes* do sistema. Com relação ao tráfego recebido, é de se esperar que as *nodes slaves* apresentassem uma demanda maior, por serem muito requisitadas pelas *nodes master*. Entretanto, em todos os cenários de testes, o consumo do tráfego de rede não se mostrou muito elevado, como pode ser observado na Tabela 17.

Tabela 17 – Resultados das métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TeraGen*

***Cluster – 2 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	15,5 Kb/s	470 Kb/s	25,2 Kb/s	45,2 Kb/s
	Slave 1	8,42 Kb/s	133 Mb/s	6,98 Kb/s	261 Mb/s
	Slave 2	10,7 Kb/s	225 Mb/s	6,13 Kb/s	167 Mb/s
1000	*Master*	17,1 Kb/s	966 Kb/s	26,7 Kb/s	85,9 Kb/s
	Slave 1	12,3 Kb/s	259 Mb/s	9,18 Kb/s	177 Mb/s
	Slave 2	10,6 Kb/s	180 Mb/s	7,20 Kb/s	257 Mb/s
2000	*Master*	15,4 Kb/s	942 Kb/s	18,0 Kb/s	63,9 Kb/s
	Slave 1	11,8 Kb/s	313 Mb/s	6,04 Kb/s	288 Mb/s
	Slave 2	8,28 Kb/s	208 Mb/s	6,25 Kb/s	350 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 66, 67, 68, 69, 70, 81, 82, 83, 84, 85, 96, 97, 98, 99 e 100.

5.2.2.2.2 *Cluster* formado por 4 *nodes* – *TeraGen*

Com o escalonamento do *cluster* para 4 *nodes*, verifica-se uma melhor distribuição das tarefas entre as *nodes*, com uma redução do consumo de *CPU*. Porém, esse consumo é desbalanceado entre as *nodes*, com alguns *nodes* registrando um consumo mais elevado em relação as outras. Isso evidencia que a biblioteca *Hadoop* não consegue, nesta versão utilizada, efetuar corretamente o balanceamento de carga com relação a esse parâmetro, subutilizando os recursos de *hardware*.

No que se refere à utilização de memória *RAM*, observa-se um cenário equilibrado com relação à utilização desse recurso nos cenários avaliados, com um aumento do consumo registrado apenas nas *nodes master*, que obtiveram um incremento médio de aproximadamente 43%, na utilização da memória *RAM* em relação aos demais *nodes* do sistema, semelhante ao que foi identificado no cenário envolvendo 2 *nodes*. Isso se deve ao trabalho intenso desses nós

(*nodes master*) na distribuição e coordenação das tarefas.

Nos cenários avaliados, identifica-se uma temperatura média equilibrada entre as *nodes*, não evidenciando nenhum comportamento anormal com relação a esse parâmetro, apesar de alguns *nodes* apresentarem um consumo de *CPU* mais elevado do que outros, o que justificaria um aumento dessa temperatura, entretanto, isso não ocorreu.

Com relação à quantidade de processos, observa-se uma situação semelhante ao cenário envolvendo 2 *nodes*, de equilíbrio, com destaque no aumento desse volume para as *nodes master*, responsáveis pela distribuição e coordenação das tarefas de processamento do *cluster*. Os dados referentes aos parâmetros apresentados podem ser evidenciados na Tabela 18.

Tabela 18 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *TeraGen*

Cluster – 4 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	4,14	17,8	36,0	36,4	33,1	36,5	227	228
	Slave 1	1,54	48,3	14,2	20,4	35,0	42,8	158	162
	Slave 2	1,67	50,6	14,6	20,9	35,0	43,3	155	161
	Slave 3	1,60	26,8	14,3	15,0	36,5	41,9	158	160
	Slave 4	1,79	41,2	14,6	22,3	37,5	41,9	157	162
1000	*Master*	3,87	18,1	35,9	36,3	32,1	37,5	228	230
	Slave 1	1,60	60,0	14,2	20,4	34,1	43,8	157	167
	Slave 2	1,28	45,7	14,4	23,4	34,6	40,9	155	162
	Slave 3	1,16	65,8	15,4	21,7	36,5	45,8	157	165
	Slave 4	1,21	39,9	15,3	16,1	33,6	39,9	156	161
2000	*Master*	3,80	18,6	36,5	36,9	31,6	37,5	226	230
	Slave 1	1,17	69,3	15,2	22,2	33,6	41,9	155	167
	Slave 2	2,05	54,9	14,8	22,7	36,0	39,9	155	163
	Slave 3	1,21	59,2	14,5	20,6	36,0	43,8	155	166
	Slave 4	1,09	52,2	15,1	15,7	33,1	39,4	156	165

Fonte: Elaborada pelos Autores, 2025.

Já o tráfego de dados, seja no envio (*sent*) quanto no recebimento (*received*) dos dados, apresentou um aumento de volume em relação ao cenário envolvendo 2 *nodes*. Esse aumento significativo ocorreu principalmente nas *nodes* que participaram ativamente do processamento das tarefas), como pode ser observado na Tabela 19.

Tabela 19 – Resultados das métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TeraGen*

***Cluster – 4 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	29,1 Kb/s	959 Kb/s	37,8 Kb/s	127 Kb/s
	Slave 1	10,9 Kb/s	195 Mb/s	7,46 Kb/s	178 Mb/s
	Slave 2	8,66 Kb/s	354 Mb/s	8,04 Kb/s	140 Mb/s
	Slave 3	7,27 Kb/s	149 Mb/s	5,79 Kb/s	346 Mb/s
	Slave 4	12,2 Kb/s	353 Mb/s	8,55 Kb/s	326 Mb/s
1000	*Master*	34,1 Kb/s	962 Kb/s	40,5 Kb/s	81,7 Kb/s
	Slave 1	9,34 Kb/s	401 Mb/s	6,40 Kb/s	194 Mb/s
	Slave 2	12,3 Kb/s	348 Mb/s	11,9 Kb/s	447 Mb/s
	Slave 3	9,31 Kb/s	393 Mb/s	7,11 Kb/s	235 Mb/s
	Slave 4	8,91 Kb/s	335 Mb/s	5,86 Kb/s	442 Mb/s
2000	*Master*	34,3 Kb/s	485 Kb/s	46,3 Kb/s	71,4 Kb/s
	Slave 1	10,1 Kb/s	350 Mb/s	9,08 Kb/s	233 Mb/s
	Slave 2	9,80 Kb/s	359 Mb/s	7,69 Kb/s	362 Mb/s
	Slave 3	9,29 Kb/s	377 Mb/s	7,88 Kb/s	152 Mb/s
	Slave 4	9,32 Kb/s	386 Mb/s	6,86 Kb/s	365 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 71, 72, 73, 74, 75, 86, 87, 88, 89, 90, 101, 102, 103, 104 e 105.

Nesse cenário, o tráfego médio de rede teve um crescimento significativo, devido ao aumento do número de *mappers* distribuídos para as *nodes slaves* processarem, garantindo a replicação dos dados que foram gerados durante a execução dos testes.

Embora a replicação dos dados provoque um aumento no tempo de execução das tarefas e no tráfego de rede, devido à distribuição dos dados para todos os nós (*nodes*) que participam efetivamente da execução da tarefa, ela é necessária para garantir a segurança dos dados.

5.2.2.2.3 *Cluster* formado por 8 *nodes* – *TeraGen*

Nesse cenário, é possível identificar uma melhor utilização dos recursos de *hardware* pelo *cluster*, com uma redução significativa na utilização dos processadores (*CPU*) e no consumo de memória *RAM* pelos dispositivos que compõem o *cluster*. Apesar do escalonamento na quantidade de *nodes* do *cluster*, é possível observar que a biblioteca *Apache Hadoop*, na versão utilizada neste trabalho, não faz um balanceamento de carga eficiente como esperado, pois, alguns *nodes* se destacam dos demais em relação a um aumento de carga nos parâmetros de utilização de *CPU* e memória *RAM*.

Com relação ao consumo de memória *RAM*, verifica-se que as *nodes master*, em todos os testes, envolvendo diferentes tamanhos de arquivos, sempre consomem mais memória do

que os demais *nodes*. Esse consumo, como já mencionado, se deve às atividades intensas desses nós na distribuição e coordenação das tarefas de processamento dos dados.

Em todos os cenários de testes, a temperatura dos equipamentos se manteve equilibrada, com média de aproximadamente 39,6 ºC, não evidenciando nenhum comportamento anormal.

No que se refere à quantidade de processos, o comportamento do *cluster* é muito semelhante aos cenários anteriores (2 e 4 *nodes*), com um equilíbrio em relação a esse parâmetro entre as *nodes*, com destaque apenas para as *nodes master*, responsáveis pela distribuição e coordenação das tarefas de processamento do *cluster*, que por sua vez, em todos os cenários, registraram um pequeno aumento nesse volume. Os resultados dos testes, com relação aos parâmetros mencionados, podem ser observados na Tabela 20.

Tabela 20 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *TeraGen*

Cluster – 8 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	4,60	18,7	35,8	36,0	32,1	38,5	222	225
	Slave 1	1,37	25,1	14,6	15,1	31,6	36,5	155	162
	Slave 2	1,28	34,3	14,6	15,3	32,1	37,0	155	159
	Slave 3	1,39	50,6	14,5	20,8	35,3	43,3	157	165
	Slave 4	1,42	44,3	14,4	23,0	34,6	39,4	156	167
	Slave 5	1,34	34,3	14,9	16,0	35,0	41,4	155	158
	Slave 6	1,37	45,3	14,6	20,4	34,6	41,9	155	167
	Slave 7	1,34	26,1	14,9	15,5	35,3	42,4	155	159
	Slave 8	1,38	26,2	14,3	15,1	35,5	40,9	153	159
1000	*Master*	4,12	18,6	36,3	36,7	32,1	36,5	224	230
	Slave 1	1,53	52,0	15,0	21,4	31,6	39,9	157	165
	Slave 2	1,39	34,8	14,6	15,7	32,1	38,0	154	159
	Slave 3	1,32	41,4	15,3	16,4	34,6	40,9	156	165
	Slave 4	1,44	58,3	15,0	21,4	32,1	39,9	153	164
	Slave 5	1,49	36,2	14,0	14,6	35,0	39,9	154	159
	Slave 6	2,01	40,6	15,3	23,9	36,0	41,9	159	164
	Slave 7	1,42	42,0	15,0	15,2	35,5	41,4	154	160
	Slave 8	1,44	41,3	15,2	15,7	34,6	40,9	154	158
2000	*Master*	4,45	20,4	36,1	36,8	30,7	37,0	224	228
	Slave 1	1,44	42,7	14,6	15,9	30,7	37,5	156	161
	Slave 2	1,45	57,4	14,6	22,3	32,1	40,4	153	162
	Slave 3	1,54	45,6	14,6	15,4	33,6	40,9	157	165
	Slave 4	1,46	46,1	14,6	15,4	31,6	38,0	154	163
	Slave 5	1,50	43,5	15,4	16,3	35,0	42,4	154	162
	Slave 6	1,44	51,2	15,2	16,0	33,6	39,9	156	169
	Slave 7	1,39	53,9	14,5	20,6	35,5	43,8	154	165
	Slave 8	1,59	43,1	15,3	23,6	36,0	43,8	156	163

Fonte: Elaborada pelos Autores, 2025.

Outro parâmetro que se assemelha aos cenários anteriores é o de tráfego de rede. Nesse cenário, o envio (*sent*) e o recebimento (*received*) dos dados se mantiveram equilibrados, com

um leve aumento nas *nodes* que participam mais ativamente do processamento de dados. Esse aumento está relacionado com o aumento do número de replicação dos dados, referentes à distribuição das tarefas entre as *nodes*, como pode ser observado na Tabela 21.

Tabela 21 – Resultados das métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TeraGen*

***Cluster – 8 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	58,8 Kb/s	520 Kb/s	83,1 Kb/s	139 Kb/s
	Slave 1	9,18 Kb/s	157 Mb/s	6,06 Kb/s	312 Mb/s
	Slave 2	8,25 Kb/s	252 Mb/s	5,99 Kb/s	335 Mb/s
	Slave 3	8,46 Kb/s	300 Mb/s	6,18 Kb/s	130 Mb/s
	Slave 4	11,2 Kb/s	164 Mb/s	9,01 Kb/s	338 Mb/s
	Slave 5	8,72 Kb/s	314 Mb/s	6,87 Kb/s	250 Mb/s
	Slave 6	9,08 Kb/s	314 Mb/s	5,51 Kb/s	153 Mb/s
	Slave 7	9,12 Kb/s	249 Mb/s	7,41 Kb/s	339 Mb/s
	Slave 8	9,25 Kb/s	302 Mb/s	5,89 Kb/s	281 Mb/s
1000	*Master*	58,6 Kb/s	114 Kb/s	79,5 Kb/s	123 Kb/s
	Slave 1	7,97 Kb/s	327 Mb/s	6,14 Kb/s	174 Mb/s
	Slave 2	8,11 Kb/s	360 Mb/s	6,76 Kb/s	284 Mb/s
	Slave 3	8,69 Kb/s	363 Mb/s	6,06 Kb/s	326 Mb/s
	Slave 4	10,7 Kb/s	320 Mb/s	5,75 Kb/s	208 Mb/s
	Slave 5	8,24 Kb/s	242 Mb/s	6,74 Kb/s	357 Mb/s
	Slave 6	11,5 Kb/s	316 Mb/s	7,39 Kb/s	359 Mb/s
	Slave 7	9,06 Kb/s	308 Mb/s	5,14 Kb/s	283 Mb/s
	Slave 8	8,46 Kb/s	343 Mb/s	6,72 Kb/s	332 Mb/s
2000	*Master*	56,6 Kb/s	989 Kb/s	79,9 Kb/s	139 Kb/s
	Slave 1	8,48 Kb/s	529 Mb/s	5,89 Kb/s	505 Mb/s
	Slave 2	9,05 Kb/s	330 Mb/s	5,70 Kb/s	275 Mb/s
	Slave 3	7,54 Kb/s	362 Mb/s	6,20 Kb/s	519 Mb/s
	Slave 4	9,58 Kb/s	457 Mb/s	6,44 Kb/s	506 Mb/s
	Slave 5	8,98 Kb/s	499 Mb/s	6,30 Kb/s	452 Mb/s
	Slave 6	8,79 Kb/s	507 Mb/s	5,74 Kb/s	461 Mb/s
	Slave 7	9,32 Kb/s	411 Mb/s	5,56 Kb/s	259 Mb/s
	Slave 8	10,9 Kb/s	334 Mb/s	6,38 Kb/s	532 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 76, 77, 78, 79, 80, 91, 92, 93, 94, 95, 106, 107, 108, 109 e 110.

5.2.2.3 Desempenho do *Hadoop* e consumo energético do *cluster* – *TeraSort*

O algoritmo *TeraSort* utiliza uma amostra de dados geradas de forma aleatória pelo *TeraGen* e, através de um programa *Map/Reduce*, realiza a classificação e a ordenação dessa amostra. Além disso, esse algoritmo serve de métrica para testar e avaliar o funcionamento e o desempenho do *cluster*, bem como a performance da biblioteca *Apache Hadoop*.

Na Tabela 22, observa-se os resultados dos testes envolvendo a execução do algoritmo *TeraSort* nas 3 (três) configurações de *cluster*, com 3 (três) tamanhos de arquivos diferentes. Os resultados mostram o desempenho e o consumo energético do *cluster* nesse cenário de testes.

Tabela 22 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *TeraSort*

Quantidade de Nodes	***Arquivos (MBytes)***	***Real Time (s)***	***CPU Time (ms)***	***Map Time (ms)***	***Reduce Time (ms)***	***Consumo Mínimo (W)***	***Consumo Máximo (W)***
	500	2m42,117s	134.560	267.504	65.590	14,1	18,3
2	1000	4m8,027s	267.150	428.190	136.184	12,9	19,8
	2000	7m13,194s	528.970	1.082.910	307.379	11,7	18,6
	500	2m15,011s	135.770	157.237	58.877	21,6	27,3
4	1000	3m24,236s	283.400	422.584	127.171	21,3	31,2
	2000	6m11,316s	547.770	1.070.356	271.080	21,3	32,7
	500	2m11,626s	135.980	132.915	66.756	36,3	43,2
8	1000	2m54,806s	257.130	259.519	112.030	37,5	50,7
	2000	5m42,908s	536.040	772.707	278.731	38,7	56,7

Fonte: Elaborada pelos Autores, 2025.

1. ***Real Time (s):*** O tempo máximo de processamento real foi atingido no cenário envolvendo 2 *nodes*, com o tamanho de arquivo de 2 *Gbytes* (7m13,194s). À medida que os *cluster* foi sendo escalonado, esse tempo sofreu uma redução, em todos os cenários, 16% no *cluster* formado por 4 *nodes* e 8,9% no cenário envolvendo 8 *nodes*. Esses dados evidenciam a eficiência do escalonamento de *nodes* no *cluster* para a melhoria do desempenho do sistema;
2. ***CPU Time (ms):*** Nos *clusters* com 2 e 8 nós (*nodes*), a média do tempo de utilização da *CPU* foi aproximadamente 3,9% menor em comparação ao *cluster* formado por 4 nós (*nodes*). O aumento para 4 nós (*nodes*) permitiu uma divisão mais equilibrada das tarefas, resultando em uma maior carga de trabalho atribuída para cada nó (*node*). Porém, ao expandir o *cluster* para 8 nós (*nodes*), observou-se uma redução no tempo de utilização da *CPU*, causada pelo aumento da eficiência da distribuição das tarefas para os nós (*nodes*), reduzindo a sobrecarga do sistema durante a execução do teste;
3. ***Map Time (ms):*** A média do tempo de *map* apresentada pelo *cluster* formado por 2 nós (*nodes*) foi de 592.868 ms. No *cluster* com 4 nós (*nodes*), foi observada uma redução de aproximadamente 7,2%, enquanto no *cluster* com 8 nós (*nodes*), a redução foi mais significativa, cerca de 29,4%. O escalonamento do número de nós (*nodes*) permitiu ao sistema distribuir as tarefas de mapeamento de maneira mais eficiente, possibilitando a divisão da tarefa em blocos ainda menores, resultando na diminuição no tempo de *map*;

4. ***Reduce Time (ms):*** A média do tempo das tarefas de *reduce* no *cluster* formado por 2 nós (*nodes*) foi de 169.718 ms. Ao utilizar 4 nós (*nodes*) no *cluster*, o tempo de *reduce* foi reduzido em aproximadamente 10%, refletindo uma melhor distribuição das tarefas devido à maior disponibilidade de recursos no sistema. Entretanto, no *cluster* composto por 8 nós (*nodes*), ocorreu um pequeno aumento no tempo de *reduce*. Esse comportamento pode ser causado pelo nível de complexidade da tarefa se tornar muito baixo ao elevar o número de nós (*nodes*), limitando o impacto positivo do escalonamento. Além disso, pode ocorrer um aumento na troca de mensagens entre os nós (*nodes*), contribuindo para o acréscimo no tempo de *reduce*;
5. **Consumo de Energia*:*** Com relação ao consumo de energia, o valor médio registrado, ao executar o algoritmo *TeraSort*, utilizando 2 nós (*nodes*), foi de 18,9 *W*. Ao escalonar o *cluster* para 4 nós (*nodes*), o consumo sofreu um acréscimo de aproximadamente 61%, enquanto, ao utilizar 8 nós (*nodes*), o aumento foi similar, em torno de 65%. Esses resultados indicam que, embora o número de nós do *cluster* tenha sido duplicado nas duas situações, o consumo energético não apresentou um aumento proporcional, demonstrando a eficiência energética da plataforma *Raspberry Pi*. Os maiores consumos de energia foram registrados nos dois últimos cenários de testes, com respectivamente 50,7 e 56,7 *W*. Além disso, os aumentos do consumo energético foram ocorrendo gradativamente, à medida que o *cluster* era escalonado e os testes se tornavam mais complexos, com tamanho de arquivos maiores.

5.2.2.4 Métricas de desempenho do *cluster* (*hardware* e tráfego de rede) – *TeraSort*

5.2.2.4.1 *Cluster* formado por 2 *nodes* – *TeraSort*

Os resultados dos testes mostram um desbalanceamento de carga de *CPU* no cenário envolvendo tamanho de arquivos menores. Isso se deve ao fato da biblioteca *Hadoop* não trabalhar de forma eficiente a quebra de arquivos que possuem um tamanho menor, para efetuar o processamento, sobrecarregando as *nodes* envolvidas no processo. À medida que o tamanho dos arquivos vão aumentando, é possível observar um melhor desempenho da biblioteca com relação à questão do balanceamento de carga entre as *nodes*. Porém, observa-se que nos dois últimos testes, de tamanho de arquivos de 1 *GByte* e 2 *GBytes*, a carga de *CPU* de uma das

nodes slaves chegou a atingir, praticamente, 100% de utilização, evidenciando um desbalanceamento médio de carga, entre as *nodes slaves*, de aproximadamente 12%.

Com relação ao gasto de memória *RAM*, verifica-se que o *Hadoop* possui um desempenho melhor, com uma distribuição mais equilibrada entre as *nodes slaves*, com um destaque nesse consumo para as *nodes* que obtiveram cargas maiores de processamento.

Como as *nodes* foram muito exigidas durante o processamento das tarefas, é normal que as suas respectivas temperaturas sofressem uma elevação, principalmente as que se referem às *nodes slaves*, que participaram mais ativamente desse processo, chegando a atingir picos, em alguns cenários, acima de 50 ºC.

Já a quantidade de processos, se manteve, praticamente, equilibrada entre as *nodes slaves*, com um pequeno aumento desse volume nas *nodes master*, como já visto em cenários de testes anteriores, pelo fato desse *node* ser o responsável pela distribuição das tarefas entres as *nodes* do *cluster*. Todos os resultados dos testes, referentes a esse cenário, podem ser verificados na Tabela 23.

Tabela 23 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *TeraSort*

Cluster – 2 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	2,74	15,7	37,0	37,2	32,1	37,0	227	230
	Slave 1	2,01	21,6	18,2	23,4	41,9	46,7	156	162
	Slave 2	0,93	97,9	15,9	43,5	34,6	45,8	154	170
1000	*Master*	2,86	16,4	37,2	37,5	32,1	35,0	226	229
	Slave 1	1,37	86,1	17,0	50,3	42,8	53,1	154	170
	Slave 2	0,88	98,6	16,0	43,6	33,6	46,3	153	172
2000	*Master*	2,72	15,9	36,1	36,3	32,6	38,0	224	229
	Slave 1	2,04	87,2	18,7	53,8	45,8	55,5	155	173
	Slave 2	0,88	96,2	16,5	44,4	36,5	49,7	155	171

Fonte: Elaborada pelos Autores, 2025.

Com relação ao tráfego de dados na rede, observa-se que as *nodes slaves* tiveram uma demanda maior, tanto no envio quanto no recebimento de dados, justificada pela participação efetiva desses *nodes* durante o processamento das tarefas, como pode ser evidenciado na Tabela 24.

Tabela 24 – Resultados das métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TeraSort*

***Cluster – 2 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	17,0 Kb/s	960 Kb/s	23,0 Kb/s	47,6 Kb/s
	Slave 1	16,2 Kb/s	472 Kb/s	9,56 Kb/s	959 Kb/s
	Slave 2	8,55 Kb/s	31,2 Kb/s	6,01 Kb/s	931 Kb/s
1000	*Master*	16,6 Kb/s	967 Kb/s	23,2 Kb/s	51,4 Kb/s
	Slave 1	9,88 Kb/s	3,58 Mb/s	8,41 Kb/s	440 Mb/s
	Slave 2	8,22 Kb/s	457 Mb/s	5,64 Kb/s	3,86 Mb/s
2000	*Master*	15,8 Kb/s	963 Kb/s	26,2 Kb/s	47,6 Kb/s
	Slave 1	10,5 Kb/s	8,43 Mb/s	9,82 Kb/s	534 Mb/s
	Slave 2	8,20 Kb/s	512 Mb/s	5,76 Kb/s	6,78 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 111, 112, 113, 114, 115, 126, 127, 128, 129, 130, 141, 142, 143, 144 e 145.

5.2.2.4.2 *Cluster* formado por 4 *nodes* – *TeraSort*

Com o escalonamento do *cluster* para 4 *nodes*, observa-se um desempenho melhor da biblioteca *Hadoop* na utilização dos recursos de *hardware* nos cenários que envolveram arquivos com tamanho maiores, evidenciando a sua ineficiência quanto se trata do processamento de arquivos menores.

No cenário de testes envolvendo o arquivo de menor tamanho (500 *MBytes*), é possível verificar o comportamento inadequado do *Hadoop* na distribuição das tarefas e na utilização dos recursos de *CPU* das *nodes* disponíveis no *cluster*, com um total desbalanceamento de carga relacionado a esse parâmetro, com dois *nodes* (*Slaves 1 e 2*) sendo totalmente subutilizados. À medida que o tamanho do arquivo vai aumentando, o comportamento e o desempenho do *cluster* muda consideravelmente, pois já se observa uma melhor utilização e equilíbrio dos recursos de *hardware* (*CPU*), principalmente com o tamanho de arquivo de 1 *Gbyte*. Já no cenário de tamanho do arquivo de 2 *Gbyte*, a biblioteca deixa a desejar um pouco, elevando a carga de processamento entre 3 (três) *nodes slaves*, subutilizando os recursos de *CPU* da última *node*, a *Slave 4*.

Com relação ao consumo de memória *RAM*, é possível perceber, em todos os cenários, uma utilização significativa desse recurso pela *node master*, o que se justifica pelo sua atuação na distribuição e coordenação das tarefas entre as *nodes* do *cluster*, principalmente na fase inicial do processamento das tarefas. Já as demais *nodes*, as *slaves*, tiveram um consumo de

memória proporcional a utilização dos recursos de *CPU*, ou seja, à medida que as *nodes* são mais requisitadas durante o processamento, o seu consumo de memória *RAM* vai aumentando, não sendo identificado, portanto, nenhum desequilíbrio significativo em relação a esse parâmetro.

O cenário semelhante ao parâmetro anterior é identificado com relação a temperatura das *nodes* do *cluster*, ou seja, as suas respectivas temperaturas vão aumentando à medida que elas vão sendo requisitadas para o processamento das tarefas, não evidenciando nenhum comportamento anormal com relação a esses parâmetro.

No que se refere à quantidade de processos, verifica-se uma situação semelhante aos cenários anteriores, de equilíbrio da carga de processos, com uma demanda maior nesse volume nas *nodes master*, pelo seu papel de atuação na distribuição e coordenação das tarefas de processamento do *cluster*.

Os dados referentes aos parâmetros analisados podem ser comprovados através das informações constantes na Tabela 25.

Tabela 25 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *TeraSort*

Cluster – 4 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	3,16	16,0	37,6	37,8	33,1	36,5	227	229
	Slave 1	0,73	1,08	15,8	15,9	33,6	35,0	152	158
	Slave 2	0,73	1,01	16,3	16,4	32,6	36,0	154	156
	Slave 3	0,97	76,3	15,1	35,6	37,0	48,2	154	165
	Slave 4	4,55	73,6	18,2	36,2	36,5	44,8	157	169
1000	*Master*	3,10	16,3	37,2	37,4	31,6	34,6	227	230
	Slave 1	0,77	1,96	15,5	15,6	32,6	35,0	154	160
	Slave 2	1,83	84,9	16,2	46,6	36,0	44,8	155	169
	Slave 3	0,95	88,5	16,6	38,1	35,5	49,7	154	168
	Slave 4	0,97	88,8	16,4	38,5	33,6	46,3	158	169
2000	*Master*	3,07	17,6	38,3	38,7	31,2	35,0	226	231
	Slave 1	0,90	95,9	17,1	45,1	33,6	41,9	155	176
	Slave 2	1,10	95,0	16,0	44,1	33,1	44,8	153	170
	Slave 3	0,82	97,4	16,0	44,5	35,0	49,2	155	166
	Slave 4	1,54	79,1	16,5	37,3	34,6	45,3	156	165

Fonte: Elaborada pelos Autores, 2025.

O tráfego de rede, referente ao envio (*sent*) de dados, diminuiu com o escalonamento do *cluster*, já com relação ao recebimento (*received*), observa-se um aumento no volume, com destaque para algumas *nodes* em cada um dos cenários, como por exemplo: *Slave 3*, com arquivos de 500 *MBytes*; *Slave 2*, com 1 *GByte*; e *Slave 2*, com 2 *GBytes*. Esse aumento se justifica pela participação ativa dessas *nodes* durante o processamento das tarefas, como pode

ser observado na Tabela 26.

Tabela 26 – Resultados das métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TeraSort*

***Cluster – 4 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	31,3 Kb/s	65,9 Kb/s	46,0 Kb/s	67,9 Kb/s
	Slave 1	8,16 Kb/s	7,24 Mb/s	6,29 Kb/s	458 Kb/s
	Slave 2	8,40 Kb/s	942 Kb/s	5,98 Kb/s	12,0 Kb/s
	Slave 3	10,3 Kb/s	5,33 Mb/s	7,02 Kb/s	401 Mb/s
	Slave 4	9,70 Kb/s	452 Mb/s	9,94 Kb/s	2,77 Mb/s
1000	*Master*	33,0 Kb/s	1,05 Mb/s	45,7 Kb/s	82,0 Kb/s
	Slave 1	8,45 Kb/s	473 Kb/s	5,73 Kb/s	377 Kb/s
	Slave 2	9,51 Kb/s	9,79 Mb/s	8,90 Kb/s	684 Mb/s
	Slave 3	8,23 Kb/s	258 Mb/s	5,80 Kb/s	4,62 Mb/s
	Slave 4	8,92 Kb/s	294 Mb/s	6,38 Kb/s	4,88 Mb/s
2000	*Master*	30,2 Kb/s	492 Kb/s	41,2 Kb/s	96,3 Kb/s
	Slave 1	7,30 Kb/s	235 Mb/s	6,30 Kb/s	4,17 Mb/s
	Slave 2	8,10 Kb/s	12,8 Mb/s	6,02 Kb/s	702 Mb/s
	Slave 3	7,62 Kb/s	237 Mb/s	6,07 Kb/s	4,23 Mb/s
	Slave 4	11,6 Kb/s	231 Mb/s	12,5 Mb/s	4,31 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 116, 117, 118, 119, 120, 131, 132, 133, 134, 135, 146, 147, 148, 149 e 150.

5.2.2.4.3 *Cluster* formado por 8 *nodes* – *TeraSort*

Ao escalonar o *cluster* para 8 *nodes*, observa-se novamente um comportamento desequilibrado do sistema com relação à utilização dos recursos de *CPU*. No primeiro teste, envolvendo o tamanho de arquivo menor, de 500 *MBytes*, apenas 4 (quatro) *nodes* participaram do processamento das tarefas, os demais ficaram totalmente subutilizados. Mesmo nessa situação, com quatro *nodes* trabalhando, não ocorreu um balanceamento de carga entre eles, com relação a esse recurso de *hardware*. No cenário seguinte, envolvendo o arquivo com tamanho de 1 *GByte*, a situação é um pouco melhor, porém, duas *nodes* não participaram do processamento das tarefas, e as *nodes* que participaram do processamento, também não mantiveram um equilíbrio no balanceamento de carga de *CPU*. Já no último cenário, com o tamanho de arquivo maior, de 2 *GBytes*, verifica-se a participação de todas as *nodes* no processamento das tarefas, entretanto, com a utilização de *CPU* totalmente desbalanceada entre eles, com dois *nodes* sobrecarregados, com aproximadamente 92% de utilização dos recursos de *CPU*, enquanto os demais estavam com esse recurso subutilizado.

Esse cenário mostra que a biblioteca *Apache Hadoop*, não apresenta um desempenho satisfatório com relação a distribuição de carga entre as *CPUs* das *nodes* participantes do processamento das tarefas, principalmente quando o tamanho dos arquivos a serem processados for pequeno, menos complexo.

O consumo de memória *RAM*, por sua vez, foi maior em todas as *nodes master*, referentes aos testes executados. Como mencionado, esse aumento no consumo se deve às atividades que elas exercem na distribuição e coordenação das tarefas de processamento dos dados. Com relação ao desempenho das *nodes slaves*, observa-se que o aumento do consumo de memória foi evidenciado entre as *nodes* que participaram mais ativamente das tarefas de processamento, não chegando a atingir 45% de utilização, no pior cenário. O que mostra um desempenho melhor da biblioteca *Hadoop* com relação a utilização desse recurso de *hardware*, nos *clusters* avaliados, porém, sem um equilíbrio adequado no balanceamento de carga.

Em todos os cenários de testes analisados, a temperatura dos equipamentos que compõem o *cluster* se manteve, praticamente, equilibrada, com algumas leves alterações nas *nodes* que tiveram um consumo maior dos recursos de *CPU*, com uma taxa média de aproximadamente 40 ºC, não sendo identificada nenhuma anormalidade ou elevações significativas com relação a esse parâmetro entre as *nodes* dos sistemas avaliados.

Com relação a quantidade de processos, observa-se um comportamento semelhante em todos os cenários avaliados e entre os cenários anteriores, com diferentes tamanhos de *cluster*, de equilíbrio entre as *nodes*, com um pequeno aumento apenas nas *nodes master*, devido ao papel que exercem no processo de distribuição e coordenação das tarefas de processamento.

Todos os resultados dos cenários de testes avaliados, podem ser verificados com base nas informações da Tabela 27.

Tabela 27 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *TeraSort*

Cluster – 8 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	4,23	18,2	36,7	36,8	31,6	37,0	224	231
	Slave 1	1,21	27,1	15,0	23,1	32,1	38,9	156	162
	Slave 2	0,79	1,34	15,4	15,4	31,6	33,6	155	156
	Slave 3	0,89	2,32	16,3	16,4	34,6	36,5	157	165
	Slave 4	0,80	1,28	16,4	16,5	32,1	34,1	156	167
	Slave 5	0,77	1,26	16,1	16,2	35,0	36,6	155	158
	Slave 6	0,93	58,5	16,1	34,8	33,1	41,9	155	164
	Slave 7	2,36	44,7	15,9	29,4	37,5	45,3	154	165
	Slave 8	1,35	64,1	15,2	29,7	36,0	46,3	153	162
1000	*Master*	4,13	17,8	37,2	37,4	31,2	35,5	223	226
	Slave 1	1,71	49,7	18,2	31,0	32,6	39,4	155	163
	Slave 2	0,75	1,58	15,8	15,9	31,2	33,1	155	158
	Slave 3	1,38	66,1	16,3	42,2	34,6	43,8	155	168
	Slave 4	0,86	26,1	16,9	24,1	31,6	38,5	156	164
	Slave 5	0,74	26,4	14,9	22,8	35,0	42,4	153	160
	Slave 6	0,79	1,24	17,0	17,1	33,1	36,0	154	160
	Slave 7	1,26	63,2	15,2	30,8	35,0	47,2	153	167
	Slave 8	1,02	60,8	15,5	31,0	35,0	45,8	153	165
2000	*Master*	3,42	19,2	37,6	37,8	30,2	35,0	224	228
	Slave 1	0,98	78,8	16,0	30,4	30,7	40,4	156	161
	Slave 2	1,31	78,5	17,2	44,5	31,6	41,9	154	167
	Slave 3	0,84	92,6	14,6	15,4	33,6	46,7	155	170
	Slave 4	0,83	30,7	14,6	15,4	30,7	38,0	154	163
	Slave 5	0,82	61,8	15,4	16,3	34,1	45,8	154	162
	Slave 6	1,05	92,3	15,9	37,6	32,6	44,8	155	171
	Slave 7	1,43	47,2	17,7	29,9	35,0	44,8	157	168
	Slave 8	0,73	59,4	16,9	30,7	34,6	44,8	155	167

Fonte: Elaborada pelos Autores, 2025.

Com relação ao tráfego de rede, verifica-se que o escalonamento do *cluster* contribuiu para uma redução desse tráfego, porém, ele é mais acentuado nas *nodes* que participam efetivamente do processamento das tarefas. Entretanto, é possível identificar um tráfego elevado nos seguintes cenários: na *Slave 6*, tráfego recebido, 447*Mb/s* – arquivo de 500 *MBytes*; na *Slave 6*, tráfego enviado, 946*Mb/s* – arquivo de 1000 *MBytes*; na *Slave 3*, tráfego recebido, 602*Mb/s* – arquivo de 1000 *MBytes*; e na *Slave 2*, tráfego recebido, 812*Mb/s* – arquivo de 2000 *MBytes*. O aumento do tráfego nessas *nodes* se justifica pela intensa participação delas durante a execução das atividades de processamento das tarefas. Nas demais *nodes* do *cluster*, o tráfego permaneceu relativamente baixo, sem anormalidades.

Na Tabela 28, pode-se verificar os resultados referentes ao tráfego da rede nos cenários de testes executados.

Tabela 28 – Resultados das métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TeraSort*

***Cluster – 8 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	52,7 Kb/s	997 Kb/s	87,7 Kb/s	4,40 Kb/s
	Slave 1	8,67 Kb/s	214 Mb/s	6,27 Kb/s	4,24 Mb/s
	Slave 2	8,25 Kb/s	468 Kb/s	5,82 Kb/s	937 Kb/s
	Slave 3	8,36 Kb/s	478 Kb/s	5,95 Kb/s	944 Kb/s
	Slave 4	9,01 Kb/s	459 Kb/s	5,20 Kb/s	942 Kb/s
	Slave 5	9,05 Kb/s	939 Kb/s	5,95 Kb/s	452 Kb/s
	Slave 6	8,45 Kb/s	8,27 Mb/s	6,23 Kb/s	447 Mb/s
	Slave 7	15,4 Kb/s	218 Mb/s	9,91 Kb/s	2,24 Mb/s
	Slave 8	9,59 Kb/s	249 Mb/s	6,26 Kb/s	6,29 Mb/s
1000	*Master*	55,7 Kb/s	1,04 Mb/s	84,6 Kb/s	142 Kb/s
	Slave 1	10,6 Kb/s	230 Mb/s	12,4 Kb/s	3,24 Mb/s
	Slave 2	8,14 Kb/s	456 Kb/s	5,74 Kb/s	938 Kb/s
	Slave 3	9,77 Kb/s	14,7 Mb/s	6,51 Kb/s	602 Mb/s
	Slave 4	8,89 Kb/s	282 Mb/s	5,27 Kb/s	3,13 Mb/s
	Slave 5	8,63 Kb/s	235 Mb/s	6,58 Kb/s	1,92 Mb/s
	Slave 6	8,26 Kb/s	946 Mb/s	5,24 Kb/s	455 Kb/s
	Slave 7	8,14 Kb/s	335 Mb/s	5,77 Kb/s	3,72 Mb/s
	Slave 8	8,20 Kb/s	244 Mb/s	5,58 Kb/s	3,56 Mb/s
2000	*Master*	58,2 Kb/s	1,03 Mb/s	83,4 Kb/s	144 Kb/s
	Slave 1	8,38 Kb/s	228 Mb/s	5,89 Kb/s	4,38 Mb/s
	Slave 2	9,61 Kb/s	12,0 Mb/s	5,98 Kb/s	812 Mb/s
	Slave 3	6,96 Kb/s	152 Mb/s	5,87 Kb/s	5,13 Mb/s
	Slave 4	9,02 Kb/s	149 Mb/s	5,32 Kb/s	2,89 Mb/s
	Slave 5	9,05 Kb/s	149 Mb/s	5,77 Kb/s	3,26 Mb/s
	Slave 6	8,23 Kb/s	230 Mb/s	4,65 Kb/s	4,35 Mb/s
	Slave 7	13,7 Kb/s	135 Mb/s	8,46 Kb/s	2,95 Mb/s
	Slave 8	8,76 Kb/s	182 Mb/s	5,62 Kb/s	3,45 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 121, 122, 123, 124, 125, 136, 137, 138, 139, 140, 151, 152, 153, 154 e 155.

Durante a execução dos testes, envolvendo o algoritmo *TeraSort*, considerado um dos algoritmos mais importantes para medir e avaliar o desempenho de um *cluster*, tendo em vista que faz um uso intenso dos recursos de *hardware* disponíveis nos equipamentos, foi possível identificar as seguintes situações:

1. O escalonamento do *cluster* foi importante para diminuir a carga de trabalho individual dos equipamentos, bem com o consumo de memória *RAM* a temperatura do processador. A redução na temperatura permite que o processador possa atingir frequências mais altas, contribuindo para uma melhora no tempo de processamento, necessário para a conclusão da execução das tarefas;
2. A biblioteca *Apache Hadoop* não conseguiu realizar um balanceamento de carga eficiente, mesmo com um *cluster* escalonado, formado por mais *nodes*, resultando

em uma subutilização dos recursos de *hardware* disponíveis no *cluster*;

3. O *Apache Hadoop* trabalha melhor com arquivos grandes e complexos;
4. A replicação de dados, efetuada pela biblioteca *Apache Hadoop*, provoca um aumento do tempo de execução das tarefas, porém, é importante para garantir a segurança dos dados na rede;
5. O tráfego da rede aumenta à medida que ocorre um crescimento do número de *mappers* distribuídos para as *nodes* processarem as tarefas.

Portanto, é importante que seja realizado um equilíbrio entre a quantidade de nós (*nodes*) no *cluster* e o volume de dados a ser processado, garantindo dessa forma uma melhor utilização e eficiência dos recursos de *hardware* disponíveis no sistema, evitando a subutilização ou a sobrecarga dos nós durante a execução das tarefas.

5.2.2.5 Desempenho do *Hadoop* e consumo energético do *cluster* – *TeraValidate*

O algoritmo *TeraValidate* tem por objetivo verificar se os dados de saída, gerados pelo *TeraSort*, foram ordenados corretamente.

A Tabela 29 apresenta os resultados da execução do algoritmo *TeraValidate*, envolvendo a utilização de 3 (três) configurações de nós (*nodes*) para o *cluster* e diferentes tamanhos de arquivos. Esses dados fornecem uma análise detalhada do comportamento e desempenho do *cluster* ao longo da execução dos testes, além do consumo energético do sistema.

Tabela 29 – Desempenho do *cluster* (*Hadoop*/Consumo Energético) – *TeraValidate*

Quantidade de Nodes	***Arquivos (MBytes)***	***Real Time (s)***	***CPU Time (ms)***	***Map Time (ms)***	***Reduce Time (ms)***	***Consumo Mínimo (W)***	***Consumo Máximo (W)***
2	500	1m6,654s	16.070	18.531	10.191	14,4	17,1
	1000	1m24,588s	27.100	32.312	10.880	13,2	15,9
	2000	1m53,244s	46.940	62.673	10.630	12,9	15,9
4	500	1m10,481s	17.700	19.463	11.787	21,6	24,9
	1000	1m19,434s	26.120	27.288	10.916	21,6	25,2
	2000	1m52,192s	47.170	62.127	10.793	21,3	24,9
8	500	1m11,699s	17.300	18.967	11.152	36,9	40,2
	1000	1m16,915s	24.620	27.284	11.018	37,8	41,7
	2000	1m55,953s	47.100	62.912	13.247	37,2	40,2

Fonte: Elaborada pelos Autores, 2025.

1. ***Real Time (s):*** O tempo de execução do algoritmo *TeraValidate* aumenta à medida que o tamanho dos arquivos vão ficando maiores e mais complexos, mesmo com o *cluster* sendo escalonado. Isso ocorre devido a um crescimento do número de

replicações dos dados, efetuada pela biblioteca *Apache Hadoop*, como forma de garantir a segurança dos dados, impactando diretamente no desempenho do sistema e no tempo necessário para a conclusão das tarefas de processamento. Além disso, como o algoritmo *TeraValidate* trabalha apenas com a missão de checagem dos dados, o tempo real de processamento é menor quando comparado com os tempos de execução dos algoritmos *TeraGen* e *TereaSort*;

2. ***CPU Time (ms):*** O tempo de utilização da *CPU* segue o mesmo comportamento do tempo real de processamento, ou seja, vai aumentando à medida que o tamanho dos arquivos ficam maiores, apesar do escalonamento do *cluster*, motivado pelo aumento do número de replicações dos dados na rede, que contribui significativamente para uma elevação no tempo de execução das tarefas;
3. ***Map Time (ms):*** O mesmo cenário, identificado nos parâmetros anteriores, é verificado com o tempo de execução das tarefas de *map* (mapeamento). Ou seja, também vai aumentando com o tamanho dos arquivos e com o escalonamento do *cluster*, motivado pelas mesmas razões já mencionadas;
4. ***Reduce Time (ms):*** Com relação ao tempo de execução das tarefas do *reduce*, identifica-se um comportamento diferente e oposto. Como o *reduce* é responsável pela agregação dos dados distribuídos e mapeados, o cenário que teve o menor tamanho de arquivo, onde não foram realizadas muitas réplicas e quebra em pedaços menores, foi o que gastou menos tempo para ser agrupado, justamente pelos pedaços dos arquivos serem maiores. Porém, no cenário onde o *cluster* teve seu maior escalonamento, com o tamanho maior de arquivo a ser processado, é possível verificar um gasto de tempo maior para a agregação dos dados, pelo fato de ter mais réplicas, com pedaços de arquivos menores, espalhadas entre as *nodes* do sistema, o que contribuiu para o desempenho menor do *cluster* com relação a esse parâmetro;
5. **Consumo de energia*:*** O consumo médio de energia registrado durante a execução do algoritmo *TeraValidate* apresentou um aumento de aproximadamente 53,4% ao utilizar o *cluster* com 4 nós (*nodes*), enquanto, ao ampliar para 8 nós (*nodes*), o aumento no consumo foi de 62,8%. Esses dados mostram que os aumentos foram ocorrendo gradativamente, à medida que o *cluster* era escalonado e os testes se tornavam mais complexos, com tamanho de arquivos maiores, tendo em vista o aumento do consumo de *CPU* pelas *nodes* que participaram ativamente da realização das tarefas de processamento. O maior consumo de energia foi registrado no teste

envolvendo o *cluster* formado por 8 *nodes*, com o arquivo de 1000 *MBytes*, que foi de 41,7 ºC.

5.2.2.6 Desempenho do *cluster* (*hardware* e tráfego de rede) – *TeraValidate*

5.2.2.6.1 *Cluster* formado por 2 *nodes* – *TeraValidate*

Nesse cenário, observa-se um equilíbrio maior no balanceamento da carga de utilização dos recursos do processador com arquivos de tamanho menores, o contrário do que foi observado na execução dos algoritmos *TeraGen* e *TeraSort*. Isso pode ter sido motivado pelo fato desse algoritmo ter efetuado a quebra dos blocos de maneira mais adequada. Entretanto, à medida que o tamanho do arquivo vai aumentado, o desbalanceamento de carga de *CPU*, volta a aparecer, se tornando mais evidente, principalmente no último cenário de teste, com uma subutilização dos recursos do processador da *Slave 2*. Esses dados mostram, mais uma vez, que a biblioteca *Hadoop* não consegue efetuar um balanceamento de carga de *CPU* eficiente e adequado, apesar de ter a sua disposição mais recursos de *hardware* no *cluster*.

Já com relação à utilização de memória *RAM*, pode-se evidenciar um comportamento mais equilibrado das *nodes slaves* do *cluster* com relação a esse parâmetro, em todos os cenários de testes, onde se observa um desempenho melhor da biblioteca. No caso das *nodes master*, como evidenciado em testes anteriores, seu consumo é sempre um pouco mais elevado, por serem mais exigidas durante a fase de distribuição das tarefas de processamento.

A temperatura das *nodes*, por sua vez, acompanha o comportamento da carga do processador, ou seja, elas aumentam com a elevação da utilização da *CPU*, o que já é esperado nesses casos, principalmente nas *nodes slaves* que participam de forma mais intensa do processamento.

No que se refere à quantidade de processos, o desempenho das nodes é semelhante aos dos testes anteriores, envolvendo os algoritmos utilizados, praticamente, equilibrada entre as *nodes slaves*, com um aumento apenas nas *nodes master*, por motivos já explicados anteriormente.

Os resultados desse cenário de teste podem ser observados por meios dos dados disponibilizados na Tabela 30.

Tabela 30 – Resultados das métricas de *hardware* do *cluster* – 2 *Nodes* – *TeraValidate*

Cluster – 2 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	13,0	14,5	37,2	37,4	34,6	39,4	226	229
	Slave 1	1,51	22,3	16,4	23,3	43,8	49,2	156	160
	Slave 2	0,75	26,5	16,5	23,3	36,0	43,3	153	157
1000	*Master*	3,29	13,7	36,3	36,5	36,0	39,9	226	228
	Slave 1	1,42	37,3	17,6	24,1	42,4	50,6	158	164
	Slave 2	3,60	24,5	18,4	23,3	37,0	42,4	157	159
2000	*Master*	7,48	16,9	36,0	36,8	33,6	37,0	225	228
	Slave 1	1,06	48,9	17,8	24,7	42,8	51,6	158	167
	Slave 2	2,11	23,4	19,6	25,0	36,5	43,8	158	162

Fonte: Elaborada pelos Autores, 2025.

Em relação ao tráfego de dados na rede, verifica-se um volume relativamente baixo, com um aumento apenas nas *nodes* que foram mais requisitadas durante o processamento das tarefas, não sendo evidenciado, portanto, nenhum comportamento anormal ou grandes volumes de tráfego na rede, como pode ser observado na Tabela 31.

Tabela 31 – Métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TeraValidate*

***Cluster – 2 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	16,4 Kb/s	947 Kb/s	25,0 Kb/s	44,0 Kb/s
	Slave 1	8,33 Kb/s	954 Kb/s	8,26 Kb/s	942 Kb/s
	Slave 2	9,59 Kb/s	26,0 Kb/s	8,42 Kb/s	930 Kb/s
1000	*Master*	16,0 Kb/s	477 Kb/s	19,5 Kb/s	45,5 Kb/s
	Slave 1	8,17 Kb/s	18,8 Kb/s	6,26 Kb/s	935 Kb/s
	Slave 2	9,02 Kb/s	537 Kb/s	7,78 Kb/s	473 Kb/s
2000	*Master*	16,4 Kb/s	592 Kb/s	26,2 Kb/s	89,4 Kb/s
	Slave 1	9,54 Kb/s	20,4 Kb/s	6,33 Kb/s	938 Kb/s
	Slave 2	13,1 Kb/s	623 Kb/s	8,70 Kb/s	577 Kb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 156, 157, 158, 159, 160, 171, 172, 173, 174, 175, 186, 187, 188, 189 e 190.

5.2.2.6.2 *Cluster* formado por 4 *nodes* – *TeraValidate*

Com o escalonamento do *cluster* para 4 *nodes*, identifica-se, em todos os cenários de testes, uma utilização menor dos recursos de *CPU* pelas *nodes*, devido ao aumento na quantidade de processadores participando do processamento, porém, observa-se que algumas

nodes ficaram sem participar do processamento, por uma decisão aleatória do próprio *Hadoop*, sendo totalmente subutilizadas. Caso a biblioteca tivesse utilizado adequadamente todos os recursos de *hardware* dos equipamentos disponíveis, a carga de *CPU* teria ficado ainda menor e balanceada, o que comprova, mais uma vez, a ineficiência do *Hadoop* com relação ao balanceamento de carga *CPU* e à utilização adequada dos recursos disponíveis no *cluster*.

No que refere ao consumo de memória *RAM*, observa-se um consumo, praticamente, baixo desse recurso em todas as *nodes* do *cluster*, com destaque no aumento do consumo apenas para as *nodes master*, que participam mais ativamente do processo inicial de distribuição das tarefas entre as *nodes slaves* do *cluster* e para as *nodes* escolhidas pelo *Hadoop* para participarem do processamento das tarefas.

Com relação à temperatura, verifica-se um aumento em todas as *nodes* encarregadas do processamento das tarefas, o que já é esperado nessa situação, tendo em vista que a temperatura aumenta à medida que o processador é mais requisitado durante a execução das tarefas. No entanto, foi evidenciado duas situações anormais com relação ao aumento de temperatura, no cenário com o tamanho do arquivo de 1000 *MBytes*, envolvendo as *Raspberry Slaves 3* e *4*, em que foram registradas temperaturas de 49,7 ºC e 46,3 ºC. O motivo para essa situação não foi identificado, uma vez que elas não estavam participando do processamento das tarefas e não apresentaram nenhum problema técnico durante a execução das tarefas. Na ocasião, foi verificado, inclusive, o funcionamento das suas ventoinhas (*cooler*), que estavam normais.

Nesse cenário, observa-se que a quantidade de processos estava semelhante aos cenários anteriores, de equilíbrio na carga dos processos, com uma demanda um pouco mais acentuada no volume nas *nodes master*, pelo papel que elas exercem no *cluster*.

Todos os dados, referentes aos parâmetros analisados, podem ser verificados através das informações constantes na Tabela 32.

Tabela 32 – Resultados das métricas de *hardware* do *cluster* – 4 *Nodes* – *TeraValidate*

Cluster – 4 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	9,26	14,7	36,9	37,2	33,1	35,5	226	230
	Slave 1	0,93	19,1	15,8	22,9	34,1	41,4	157	164
	Slave 2	1,07	1,24	16,4	16,4	34,1	35,5	157	158
	Slave 3	3,95	26,3	18,4	23,4	28,9	43,8	161	166
	Slave 4	0,83	11,4	16,5	20,6	34,1	39,4	155	158
1000	*Master*	3,84	15,1	37,7	37,8	32,6	37,0	226	229
	Slave 1	0,79	21,7	15,6	22,7	33,6	40,4	157	161
	Slave 2	2,48	33,9	18,3	27,6	35,5	40,9	159	163
	Slave 3	0,81	1,03	17,2	17,2	35,5	49,7	155	157
	Slave 4	0,76	0,88	17,9	17,9	33,6	46,3	155	158
2000	*Master*	3,29	14,9	37,3	37,4	31,2	37,0	228	231
	Slave 1	0,71	0,90	18,3	18,3	32,1	34,6	156	159
	Slave 2	2,89	54,7	17,7	30,6	35,0	40,4	158	164
	Slave 3	0,72	0,97	17,0	17,1	34,6	37,0	155	156
	Slave 4	0,96	9,72	17,7	21,6	32,6	38,5	157	160

Fonte: Elaborada pelos Autores, 2025.

Com o escalonamento do *cluster*, o tráfego de rede se manteve relativamente baixo, registrando um aumento nos dois primeiros cenários de testes, onde as *nodes slaves* foram mais requisitadas durante o processamento das tarefas. No último cenário, com o tamanho de arquivo maior, o tráfego de envio e recebimento dos dados não passou dos kilobits por segundo (*Kb/s*), como pode ser observado na Tabela 33.

Tabela 33 – Métricas de tráfego de rede do *cluster* – 4 *Nodes* – *TeraValidate*

***Cluster – 4 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	33,4 Kb/s	958 Kb/s	44,6 Kb/s	65,0 Kb/s
	Slave 1	9,72 Kb/s	863 Mb/s	6,00 Kb/s	562 Mb/s
	Slave 2	8,32 Kb/s	944 Kb/s	5,72 Kb/s	939 Kb/s
	Slave 3	10,4 Kb/s	534 Mb/s	5,90 Kb/s	960 Kb/s
	Slave 4	10,0 Kb/s	942 Kb/s	6,39 Kb/s	934 Kb/s
1000	*Master*	34,5 Kb/s	505 Kb/s	44,0 Kb/s	72,2 Kb/s
	Slave 1	8,35 Kb/s	1,07 Mb/s	6,64 Kb/s	601 Mb/s
	Slave 2	13,4 Kb/s	616 Mb/s	10,1 Kb/s	1,00 Mb/s
	Slave 3	7,85 Kb/s	938 Kb/s	6,30 Kb/s	517 Kb/s
	Slave 4	8,48 Kb/s	529 Kb/s	6,22 Kb/s	524 Kb/s
2000	*Master*	25,3 Kb/s	970 Kb/s	44,9 Kb/s	63,9 Kb/s
	Slave 1	8,24 Kb/s	486 Kb/s	5,92 Kb/s	541 Kb/s
	Slave 2	9,46 Kb/s	553 Kb/s	8,51 Kb/s	946 Kb/s
	Slave 3	7,76 Kb/s	549 Kb/s	5,99 Kb/s	931 Kb/s
	Slave 4	8,92 Kb/s	542 Kb/s	6,22 Kb/s	934 Kb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 161, 162, 163, 164, 165, 176, 177, 178, 179, 180, 191, 192, 193, 194 e 195.

5.2.2.6.3 *Cluster* formado por 8 *nodes* – *TeraValidate*

Ao escalonar o *cluster* para 8 *nodes*, observa-se uma redução significativa de utilização dos recursos dos processadores das *nodes*, em praticamente todos os cenários de tamanho de arquivos diferentes, com exceção de uma única *node* (*Slave 2*), do cenário de teste com o tamanho do arquivo de 2 *GBytes*, onde foi registrado um consumo de aproximadamente 48% de *CPU*. Isso ocorreu pelo fato da biblioteca *Hadoop* não ter distribuído adequadamente a carga deste recurso entre as *nodes*, ficando várias nodes *subutilizadas*, sem participarem do processamento das tarefas, evidenciando, mais uma vez, a ineficiência do *Hadoop* com relação ao balanceamento de carga de *CPU* e a distribuição das tarefas entre os *nodes* do *cluster*.

A utilização dos recursos de memória *RAM* apresenta-se em todas as *nodes,* de todos os cenários de testes avaliados, com um aumento, já conhecido, em relação as *nodes master*, que devido ao papel que exercem, tem sempre seu consumo de memória mais evidente, do que as demais *nodes* do *cluster*. Com relação a esse recurso de *hardware*, identifica-se um desempenho melhor por parte do *Hadoop*, porém, ainda é evidente o desbalanceamento em todo o *cluster*.

Com relação à temperatura das *nodes*, em todos os cenários de testes analisados, não foram evidenciados picos elevados, porém, nas *nodes* que participaram mais ativamente do processamento é possível verificar um aumento nesse consumo, limitando-se em 43,3 ºC.

A quantidade de processos vem mantendo um comportamento semelhante em todos os cenários de testes, registrando um equilíbrio entre as *nodes slaves* e um pequeno aumento nas *nodes master*, devido às funções que elas exercem na distribuição das tarefas, mas nada tão elevado e significativo.

Os resultados contendo todos os cenários de testes executados, podem ser observados com base nas informações da Tabela 34.

Tabela 34 – Resultados das métricas de *hardware* do *cluster* – 8 *Nodes* – *TeraValidate*

Cluster – 8 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
	Master	7,60	13,8	36,7	36,9	32,1	36,5	224	228
	Slave 1	0,81	0,92	16,2	16,2	30,7	32,1	155	157
	Slave 2	0,86	0,93	15,6	15,6	31,6	33,1	154	158
	Slave 3	0,83	0,95	16,5	16,5	34,6	36,0	156	157
500	*Slave 4*	0,80	9,70	16,5	20,2	31,6	38,0	153	160
	Slave 5	2,30	24,3	16,4	23,7	37,0	42,4	155	164
	Slave 6	0,79	2,58	16,2	16,4	32,6	36,5	152	156
	Slave 7	1,04	17,8	16,6	23,5	35,5	42,4	152	158
	Slave 8	0,84	0,93	16,3	16,3	35,0	36,5	153	159
	Master	5,40	14,6	37,1	37,2	31,6	34,6	223	227
	Slave 1	0,75	0,92	17,1	17,1	30,7	32,6	156	161
	Slave 2	0,78	1,08	16,0	16,0	31,2	33,1	154	156
	Slave 3	0,99	3,67	17,5	17,7	33,6	37,5	155	157
1000	*Slave 4*	0,83	9,69	17,2	20,9	30,7	37,5	153	160
	Slave 5	0,80	0,89	16,0	16,0	34,6	36,0	154	157
	Slave 6	0,86	0,98	17,1	17,1	33,1	34,6	153	156
	Slave 7	0,91	23,1	16,7	23,5	37,0	43,3	156	159
	Slave 8	0,84	22,2	17,3	24,4	35,0	43,3	155	164
	Master	3,62	16,8	37,6	37,7	31,2	35,0	224	228
	Slave 1	0,75	1,10	18,0	18,1	29,7	32,1	154	156
	Slave 2	0,82	47,7	18,5	25,0	30,7	37,5	153	162
	Slave 3	0,71	1,04	17,1	17,1	33,6	35,5	155	161
2000	*Slave 4*	2,24	24,6	19,2	24,0	32,1	38,9	155	162
	Slave 5	0,70	0,92	16,2	31,1	33,1	36,0	153	156
	Slave 6	1,01	1,75	18,0	18,1	32,6	34,6	155	158
	Slave 7	0,70	0,83	16,4	16,4	34,1	35,5	154	158
	Slave 8	0,77	12,8	17,4	21,5	34,6	40,4	152	164

Fonte: Elaborada pelos Autores, 2025.

Nesse cenário, observa-se que o escalonamento do *cluster* para 8 *nodes*, contribuiu ainda mais para uma redução significativa no tráfego de rede, tendo em vista que mais *nodes* participaram da execução das tarefas. Entretanto, nos cenários onde o tamanho dos arquivos eram menores, foi identificado um aumento de tráfego mais acentuado em algumas *nodes*, motivado pela baixa complexidade das tarefas, que demandava uma necessidade maior de comunicação durante a sua execução, aumento significativamente o tráfego na rede.

Já no último cenário de teste, com o tamanho de arquivo maior, o tráfego de rede se manteve ainda mais baixo, sem nenhuma anormalidade e nem valores elevados, chegando no máximo a 1,19 *Mb/s*, no envio e *1,18 Mb/s* no recebimento dos dados, como pode ser identificado na Tabela 35.

Tabela 35 – Métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TeraValidate*

***Cluster – 8 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	65,2 Kb/s	1,19 Mb/s	88,0 Kb/s	153 Kb/s
	Slave 1	9,06 Kb/s	585 Mb/s	6,60 Kb/s	1,14 Mb/s
	Slave 2	8,18 Kb/s	535 Kb/s	6,64 Kb/s	1,11 Mb/s
	Slave 3	7,66 Kb/s	525 Kb/s	5,94 Kb/s	1,10 Mb/s
	Slave 4	9,08 Kb/s	537 Kb/s	5,91 Kb/s	1,12 Mb/s
	Slave 5	10,8 Kb/s	1,16 Mb/s	9,37 Kb/s	536 Kb/s
	Slave 6	9,50 Kb/s	463 Mb/s	6,94 Kb/s	1,14 Mb/s
	Slave 7	10,0 Kb/s	1,14 Mb/s	7,02 Kb/s	563 Mb/s
	Slave 8	8,75 Kb/s	560 Kb/s	6,86 Kb/s	1,10 Mb/s
1000	*Master*	62,6 Kb/s	103 Kb/s	87,3 Kb/s	173 Kb/s
	Slave 1	9,00 Kb/s	523 Kb/s	6,36 Kb/s	598 Kb/s
	Slave 2	9,36 Kb/s	943 Kb/s	6,15 Kb/s	520 Kb/s
	Slave 3	11,8 Kb/s	622 Mb/s	7,84 Kb/s	1,13 Mb/s
	Slave 4	8,98 Kb/s	495 Kb/s	6,58 Kb/s	511 Kb/s
	Slave 5	8,25 Kb/s	525 Kb/s	5,94 Kb/s	933 Kb/s
	Slave 6	8,79 Kb/s	600 Kb/s	6,23 Kb/s	944 Kb/s
	Slave 7	12,3 Kb/s	953 Kb/s	7,94 Kb/s	39,2 Kb/s
	Slave 8	10,7 Kb/s	1,20 Mb/s	7,54 Kb/s	591 Mb/s
2000	*Master*	62,5 Kb/s	545 Kb/s	83,3 Kb/s	122 Kb/s
	Slave 1	8,78 Kb/s	1,16 Mb/s	5,66 Kb/s	572 Kb/s
	Slave 2	9,10 Kb/s	630 Kb/s	6,16 Kb/s	540 Kb/s
	Slave 3	7,83 Kb/s	583 Kb/s	5,69 Kb/s	1,14 Mb/s
	Slave 4	11,3 Kb/s	1,19 Mb/s	9,01 Kb/s	485 Kb/s
	Slave 5	8,22 Kb/s	580 Kb/s	5,77 Kb/s	1,18 Mb/s
	Slave 6	8,78 Kb/s	579 Kb/s	5,39 Kb/s	1,18 Mb/s
	Slave 7	8,98 Kb/s	1,18 Mb/s	5,82 Kb/s	525 Kb/s
	Slave 8	9,03 Kb/s	584 Kb/s	5,98 Kb/s	1,15 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 166, 167, 168, 169, 170, 181, 182, 183, 184, 185, 196, 197, 198, 199 e 200.

Em resumo, como pode ser observado nos cenários analisados, o algoritmo *TeraValidate* não demanda muitos recursos de *hardware* do *cluster* durante a sua execução, por efetuar apenas uma checagem de dados. Porém, mesmo com o escalonamento do *cluster*, foi possível perceber a dificuldade encontrada pelo *Hadoop* para efetuar a distribuição das tarefas de forma eficiente e balanceada, apesar de ser evidenciado que o aumento do número de máquinas no *cluster*, contribuiu para uma redução considerável na carga de trabalho e no consumo de alguns recursos de *hardware*, mesmo que ainda de forma desequilibrada e com algumas *nodes* sendo subutilizadas durante a execução das tarefas. Essa situação de desequilíbrio de utilização dos recursos do *cluster*, pode impactar diretamente no desempenho do sistema, bem como no tempo necessário para a conclusão do processamento das tarefas.

Apesar dos problemas mencionados, o escalonamento do *cluster* permitiu uma redução

significativa no tráfego da rede, até mesmo nas situações que envolveram arquivos de tamanho menores, onde a dificuldade de quebra do arquivo e distribuição das tarefas é ainda maior, devido à necessidade das *nodes* se comunicarem de forma mais intensa durante a execução das atividades, aumentando a utilização da rede.

Por fim, por se tratar de um algoritmo relativamente leve, do ponto de vista de processamento, onde a carga de *CPU* não é muito elevada, não foram registradas temperaturas anormais nos cenários de testes realizados.

5.2.2.7 Capacidade de armazenamento dos *Clusters* – *Algoritmos "Tera"*

As Figuras 13, 14 e 15, respectivamente, mostram a utilização de espaço em disco de cada *node* do *cluster*, após os cenários de testes envolvendo os algoritmos *TeraGen*, *TeraSort* e *TeraValidate*, com tamanhos de arquivos diferentes, em um *cluster* composto por 2 *nodes slaves* e uma *master*. Nas figuras, é possível identificar a baixa ocupação de espaço em disco nas unidades de armazenamento secundário (cartão de memória), chegando a registrar um mínimo de 0,42% (504,98 *MBytes*) do espaço total disponível, no cenário com tamanho de arquivo de 500 *Mbytes* e de 4,03 *GBytes* (3,45%), com tamanho de arquivo de 2 *GBytes*,

Figura 13 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – "*Tera*" – 500 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	40m	504.98 MB	6.37 GB	116.9 GB	14	504.98 MB (0.42%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	0s	48m	1008.57 MB	6.41 GB	116.9 GB	15	1008.57 MB (0.84%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 14 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – "*Tera*" – 1000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	18m	2.02 GB	6.4 GB	116.9 GB	26	2.02 GB (1.73%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	48m	1.01 GB	6.43 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 15 – Capacidade do *Cluster* com 2 *Nodes* – *HDFS* – "*Tera*" – 2000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	30m	4.03 GB	6.41 GB	116.9 GB	42	4.03 GB (3.45%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	41m	2.02 GB	6.44 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0

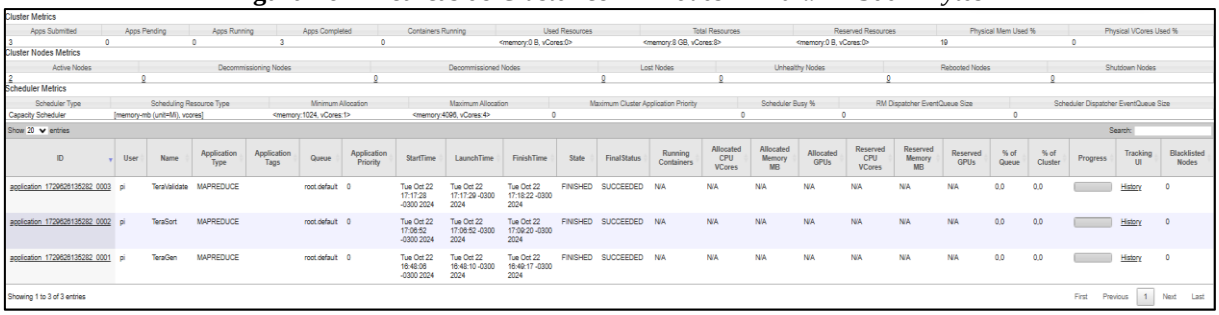
Fonte: Elaborada pelos Autores, 2025.

Observando os resultados dos cenários avaliados, é possível identificar que o volume de espaço em disco (memória secundária) entre as *nodes* do *cluster* se encontra desbalanceado e baixo (3,45%), evidenciando que a capacidade disponível em disco foi suficiente para a execução dos testes, bem como a dificuldade do *Hadoop* em equilibrar os recursos de *hardware* disponíveis.

Nas Figuras 16, 17 e 18, respectivamente, observa-se as telas de comprovação da execução dos testes envolvendo os algoritmos "*Tera*", com os tamanhos de arquivos diferentes, contendo informações sobre os recursos de *hardware* utilizados e disponíveis no *cluster*.

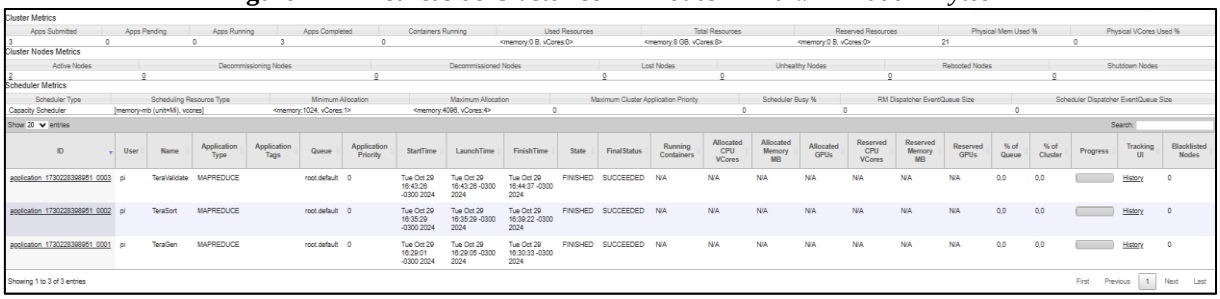
Nas imagens, é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (*SUCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 21%; a quantidade de nós (*nodes*) ativos na execução dos testes, no campo *Active Nodes*, neste caso com 2 *nodes slaves* e uma *node master*. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*TeraGen*, *TeraSort* e *TeraValidate*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources* (8 *GBytes*), de acordo com a capacidade do *cluster* implementado.

Figura 16 – Métricas do *Cluster* com 2 *Nodes* – *"Tera"* – 500 *MBytes*



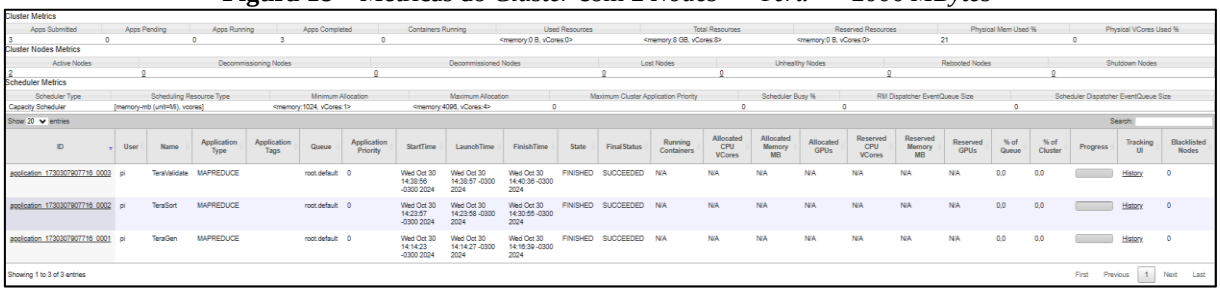
Fonte: Elaborada pelos Autores, 2025.

Figura 17 – Métricas do *Cluster* com 2 *Nodes* – *"Tera"* – 1000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Figura 18 – Métricas do *Cluster* com 2 *Nodes* – *"Tera"* – 2000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

As Figuras 19, 20 e 21, respectivamente, apresentam a utilização de espaço em disco de cada *node* do *cluster*, após os cenários de testes envolvendo os algoritmos *TeraGen*, *TeraSort* e *TeraValidate*, com tamanhos de arquivos diferentes, em um *cluster* composto por 4 *nodes slaves* e uma *master*. Nas figuras, é possível identificar a baixa ocupação de espaço em disco nas unidades de armazenamento secundário (cartão de memória), chegando a registrar um mínimo de 0,42% (504,96 *MBytes*) do espaço total disponível, no cenário com tamanho de arquivo de 500 *Mbytes* e de 4,03 *GBytes* (3,45%), com tamanho de arquivo de 2 *GBytes*,

Figura 19 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – "*Tera*" – 500 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	28m	504.62 MB	6.48 GB	116.9 GB	11	504.62 MB (0.42%)	0%	3.4.0
✔/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	0s	28m	504.96 MB	6.44 GB	116.9 GB	14	504.96 MB (0.42%)	0%	3.4.0
✔/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	0s	28m	1008.55 MB	6.35 GB	116.9 GB	15	1008.55 MB (0.84%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	0s	28m	504.62 MB	6.5 GB	116.9 GB	11	504.62 MB (0.42%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 20 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – "*Tera*" – 1000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	30m	1.01 GB	6.48 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	2s	30m	1.01 GB	6.44 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	2s	30m	1.01 GB	6.35 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	2s	30m	2.02 GB	6.5 GB	116.9 GB	26	2.02 GB (1.73%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 21 – Capacidade do *Cluster* com 4 *Nodes* – *HDFS* – "*Tera*" – 2000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	36m	2.02 GB	6.48 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	2s	36m	2.02 GB	6.44 GB	116.9 GB	26	2.02 GB (1.73%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	2s	36m	2.02 GB	6.35 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	0s	36m	4.03 GB	6.5 GB	116.9 GB	39	4.03 GB (3.45%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

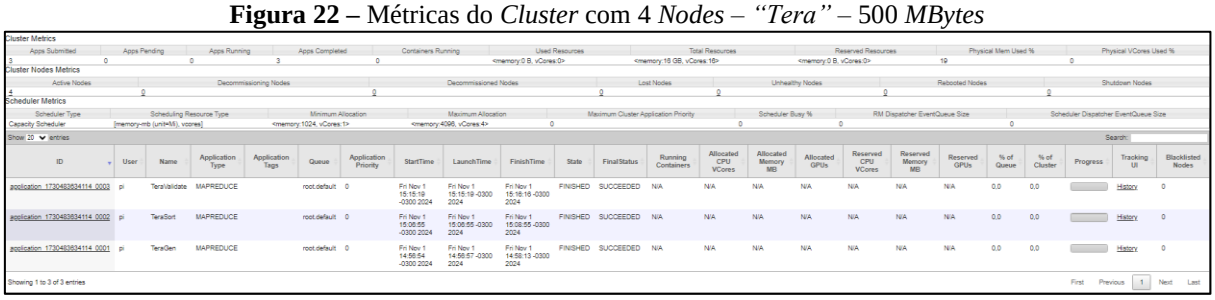
Com o escalonamento para 4 *nodes*, o volume de espaço em disco (memória secundária) entre as *nodes* do *cluster* continua apresentando um desequilíbrio, porém, distribuído entre todas elas, com um volume maior na *node Slave 2*, que registrou um consumo de 4,03 *GBytes*. Nas demais *nodes* do *cluster*, já se observa um comportamento equilibrado com relação ao consumo de espaço em disco, de 2,02 *GBytes*, mesmo assim, é possível evidenciar uma ineficiência da biblioteca *Hadoop* com relação ao balanceamento de carga entre as *nodes*. O maior consumo de espaço em disco registrado, nos cenários de testes executados, foi da *Slave 2*, com 3,45% de ocupação, um percentual relativamente baixo.

As Figuras 22, 23 e 24, respectivamente, mostram as telas de comprovação da execução dos testes envolvendo os algoritmos "*Tera*", com os tamanhos de arquivos diferentes, contendo informações sobre os recursos de *hardware* utilizados e disponíveis no *cluster*.

Nas imagens, é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (*SUCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 21%; a quantidade de nós (*nodes*) ativos na execução dos

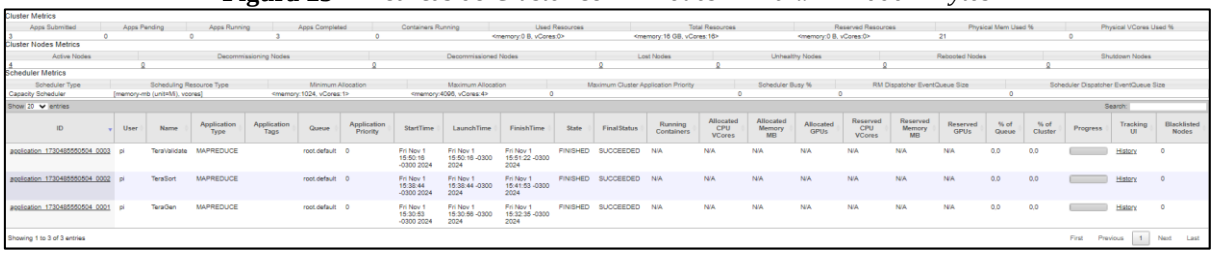
testes, no campo *Active Nodes*, neste caso com 4 *nodes slaves* e uma *node master*. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*TeraGen*, *TeraSort* e *TeraValidate*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources* (16 *GBytes*), de acordo com a capacidade do *cluster* implementado.

Figura 22 – Métricas do *Cluster* com 4 *Nodes* – *"Tera"* – 500 *MBytes*



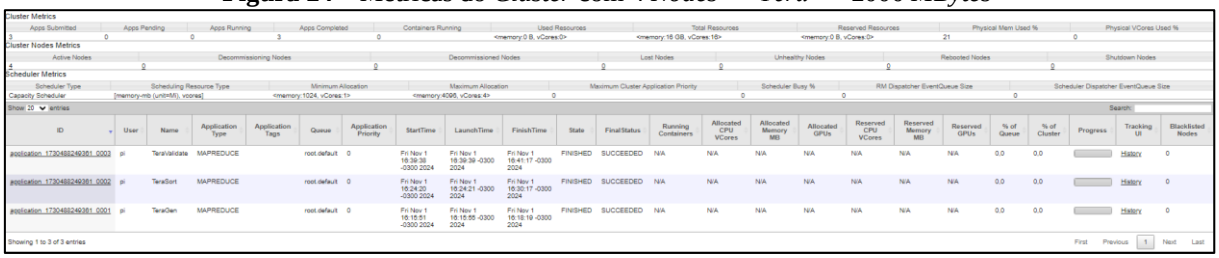
Fonte: Elaborada pelos Autores, 2025.

Figura 23 – Métricas do *Cluster* com 4 *Nodes* – *"Tera"* – 1000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Figura 24 – Métricas do *Cluster* com 4 *Nodes* – *"Tera"* – 2000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Nas Figuras 25, 26 e 27, respectivamente, observa-se o desempenho da utilização de espaço em disco de cada *node* do *cluster*, após os cenários de testes envolvendo os algoritmos *TeraGen*, *TeraSort* e *TeraValidate*, com tamanhos de arquivos diferentes, em um *cluster* composto por 8 *nodes slaves* e uma *master*. Nas figuras, é possível identificar a baixa ocupação

de espaço em disco nas unidades de armazenamento secundário (cartão de memória), chegando a registrar um mínimo de 0,42% (504,64 *MBytes*) do espaço total disponível, no cenário com tamanho de arquivo de 500 *Mbytes* e de 4,03 *GBytes* (3,45%), com tamanho de arquivo de 2 *GBytes*,

Figura 25 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – "*Tera*" – 500 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	36m	504.64 MB	6.51 GB	116.9 GB	11	504.64 MB (0.42%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	2s	36m	504.64 MB	6.47 GB	116.9 GB	11	504.64 MB (0.42%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	2s	36m	504.64 MB	6.38 GB	116.9 GB	11	504.64 MB (0.42%)	0%	3.4.0
/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	2s	36m	504.95 MB	6.31 GB	116.9 GB	14	504.95 MB (0.42%)	0%	3.4.0
/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	2s	36m	504.67 MB	6.37 GB	116.9 GB	11	504.67 MB (0.42%)	0%	3.4.0
/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	2s	34m	504.64 MB	6.31 GB	116.9 GB	11	504.64 MB (0.42%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	2s	36m	504.64 MB	6.54 GB	116.9 GB	11	504.64 MB (0.42%)	0%	3.4.0
/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	2s	36m	1008.57 MB	6.31 GB	116.9 GB	15	1008.57 MB (0.84%)	0%	3.4.0

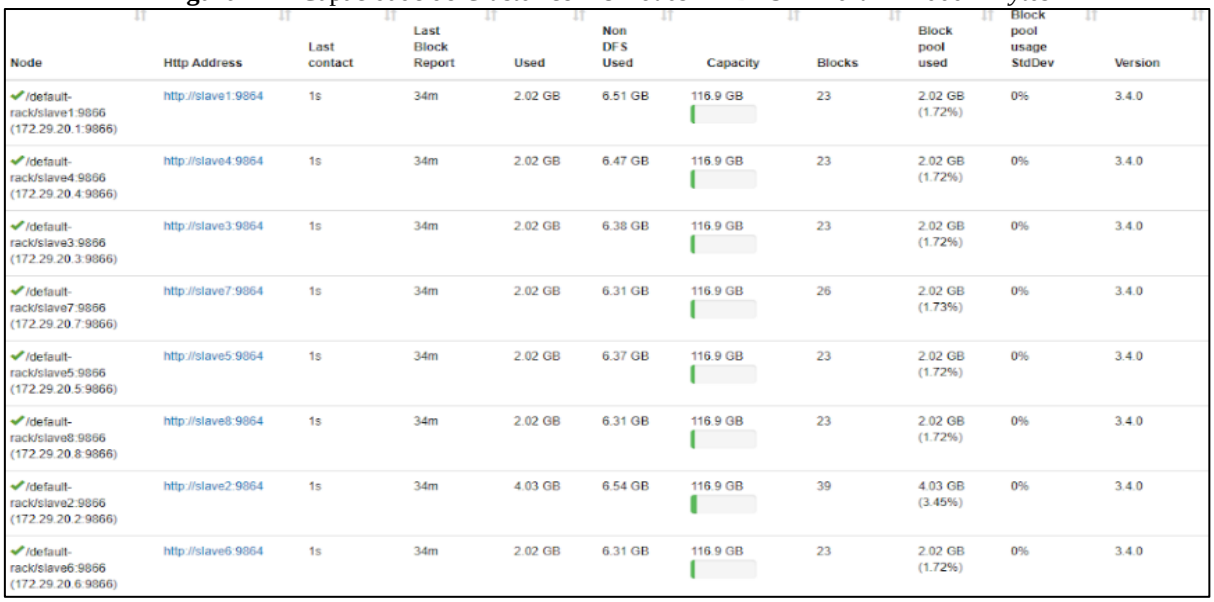
Fonte: Elaborada pelos Autores, 2025.

Figura 26 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – "*Tera*" – 1000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	30m	1.01 GB	6.51 GB	116.9 GB	18	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	0s	30m	1.01 GB	6.47 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	0s	30m	2.02 GB	6.38 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	0s	30m	1.01 GB	6.31 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	0s	30m	1.01 GB	6.37 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	0s	30m	1.01 GB	6.31 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	0s	30m	1.01 GB	6.54 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0
/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	0s	30m	1.01 GB	6.31 GB	116.9 GB	15	1.01 GB (0.86%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 27 – Capacidade do *Cluster* com 8 *Nodes* – *HDFS* – "*Tera*" – 2000 *MBytes*



Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	1s	34m	2.02 GB	6.51 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	1s	34m	2.02 GB	6.47 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	1s	34m	2.02 GB	6.38 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	1s	34m	2.02 GB	6.31 GB	116.9 GB	26	2.02 GB (1.73%)	0%	3.4.0
/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	1s	34m	2.02 GB	6.37 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	1s	34m	2.02 GB	6.31 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	34m	4.03 GB	6.54 GB	116.9 GB	39	4.03 GB (3.45%)	0%	3.4.0
/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	1s	34m	2.02 GB	6.31 GB	116.9 GB	23	2.02 GB (1.72%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Após o escalonamento para 8 *nodes*, verifica-se que o volume ocupado de espaço em disco (memória secundária) entre as *nodes* do *cluster* foi se elevando à medida que o tamanho do arquivo processado aumentava.

No cenário com o arquivo menor, de 500 *MBytes*, o espaço utilizado foi equilibrado entre as *nodes*, de menos de 0,5%. Porém, em um única *node*, na *Slave 6*, identifica-se um duplicação desse consumo. Esse cenário se repete em todos os testes, é como se a biblioteca *Hadoop* escolhesse uma *node*, aleatoriamente, para armazenar o restante do resultado dos arquivos processados. Entretanto, o consumo de espaço em disco não ultrapassou 1%, nesse cenário de teste.

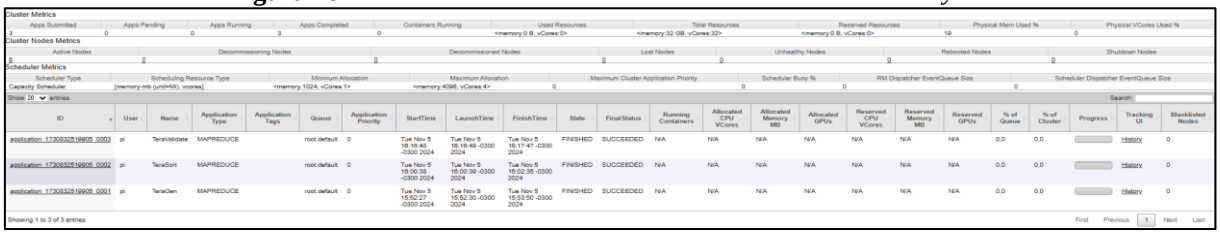
O cenário anterior, se repete no teste envolvendo o arquivo de 1000 *MBytes*, o comportamento de ocupação do espaço em disco das *nodes* se manteve equilibrado, em praticamente todas as *nodes*, abaixo de 1%, exceto com relação a *node Slave 2*, que registrou o dobro desse consumo, com 1,72%, um cenário igualmente semelhante ao observado no teste envolvendo o arquivo de 500 *MBytes*.

No teste envolvendo o arquivo de tamanho maior, de 2000 *MBytes*, a mesma situação se repete, com um consumo equilibrado e distribuído, de ocupação do espaço em disco, em quase todas as *nodes*, com exceção da *node Slave 2*, que registrou uma ocupação de 3,45% (taxa de ocupação baixa), o dobro das demais, evidenciando um comportamento singular do *Hadoop* com relação ao balanceamento e a distribuição da ocupação do espaço em disco entre as *nodes* do *cluster*, durante a execução dos testes realizados.

As Figuras 28, 29 e 30, respectivamente, mostram as telas de comprovação da execução dos testes envolvendo os algoritmos “*Tera*”, com os tamanhos de arquivos diferentes, contendo informações sobre os recursos de *hardware* utilizados e disponíveis no *cluster*.

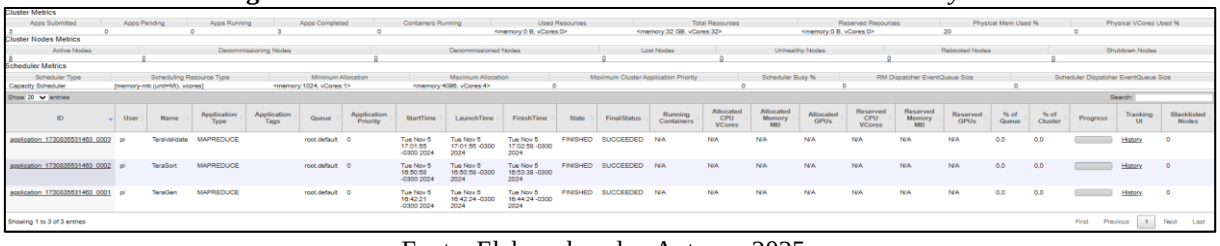
Nas figuras, é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (S*UCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 21%; a quantidade de nós (*nodes*) ativos na execução dos testes, no campo *Active Nodes*, neste caso com 8 *nodes slaves* e uma *node master*. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*TeraGen, TeraSort* e *TeraValidate*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources* (32 *GBytes*), de acordo com a capacidade do *cluster* implementado.

Figura 28 – Métricas do *Cluster* com 8 *Nodes* – *“Tera”* – 500 *MBytes*



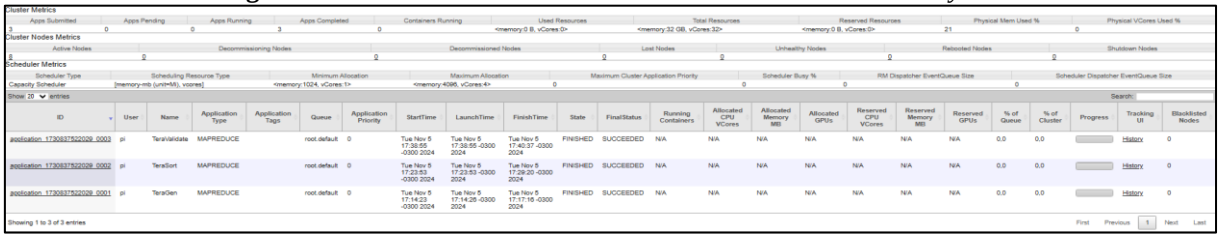
Fonte: Elaborada pelos Autores, 2025.

Figura 29 – Métricas do *Cluster* com 8 *Nodes* – *“Tera”* – 1000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Figura 30 – Métricas do *Cluster* com 8 *Nodes* – *“Tera”* – 2000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

5.2.3 Testes envolvendo o algoritmo *TestDFSIO*

Nesta seção, serão apresentados os resultados dos testes de *benchmark* envolvendo o algoritmo *TestDFSIO*, com tamanhos de *clusters* e de arquivos diferentes.

O *TestDFSIO* é um algoritmo que vem embutido na distribuição do *Hadoop*, sendo utilizado para medir a taxa agregada dos dados de *Input*/*Output* de um *cluster HDFS*, além do *throughput* (taxa de transferência dos dados), comparando o desempenho do *cluster* durante os processos de gravação (*write*) e leitura (*read*) dos dados.

5.2.3.1 Desempenho do *Hadoop* e consumo energético do *cluster* – *DFSIO Write*

A Figura 31 mostra os resultados obtidos durante a execução do algoritmo *TestDFSIO Write*, nas 3 (três) configurações de *cluster*, com 3 (três) tamanhos de arquivos diferentes. A partir dos dados obtidos é possível analisar o desempenho do *cluster* durante a execução dos testes.

Figura 31 – Métricas de desempenho (*Hadoop* e Consumo Energético) – *DFSIO Write*

		Tamanho arquivo (MB)	Real time (s)	CPU time (ms)	Map time (ms)	Reduce time (ms)	Taxa de transferência	Taxa média de I/O	Consumo mínimo (W)	Consumo máximo (W)
Nós	2	500	5m26,888s	247.750	1.233.060	103.897	7,59	9,37	12,9	19,2
		1000	8m45,601s	359.950	2.285.523	213.966	5,55	6,34	12,9	18,6
		2000	15m38,923s	552.690	4.266.428	391.631	5,55	7,23	12,3	18
	4	500	5m16,207s	267.000	2.319.224	59.379	2,6	2,62	21,6	30,9
		1000	8m41,477s	404.980	4.296.254	34.098	2,61	2,62	22,5	32,4
		2000	15m58,482s	605.260	8.632.319	63.929	2,44	2,44	21,6	31,2
	8	500	6m5,218s	272.370	2.766.806	67.715	2,13	2,14	34,8	53,7
		1000	10m4,618s	401.170	5.177.858	99.879	2,08	2,09	36	55,2
		2000	18m1,653s	622.430	9.780.984	107.805	2,13	2,14	36,3	55,2

Fonte: Elaborada pelos Autores, 2025.

1. ***Real Time (s):*** À medida que o tamanho dos arquivos vão aumentando, o tempo real de processamento também vai crescendo, mesmo com o tamanho do *cluster* escalonado para mais *nodes*. Esse aumento se dá em função do número de tarefas de *map* que vão sendo distribuídas entre os *nodes*, elevando o tempo de resposta do processamento;
2. ***CPU Time (ms):*** Assim como no parâmetro de tempo real, o tempo de utilização da *CPU* segue o mesmo comportamento, conforme o tamanho dos arquivos e do *cluster* vão aumentando, a demanda entre as *nodes* cresce na mesma proporção, motivada pela quantidade de tarefas que são enviadas para elas, que acaba gerando uma carga maior de processamento;
3. ***Map Time (ms):*** O escalonamento no número de máquinas no *cluster* possibilitou ao

sistema gerar uma quantidade maior de *mappers*, permitindo quebrar das tarefas em blocos ainda menores, gerado um processamento adicional para a classificação dos dados, o que culminou com uma elevação no tempo de *map* das tarefas;

4. ***Reduce Time (ms):*** Com relação ao tempo de *reduce* das tarefas, observa-se o mesmo comportamento do tempo de *map*, pois, à medida que o tamanho dos arquivos e do *cluster* vão ficando maiores, existe um esforço significativo do sistema para quebrar as tarefas em blocos menores, o que resulta em uma grande quantidade de informações para serem processadas, que por sua vez, durante a tarefa de *reduce*, precisam ser reagrupadas, elevando o tempo de resposta desse parâmetro;
5. **Taxa de Transferência (*mb/sec*):** A taxa de transferência de dados (*throughput*) vai diminuindo à medida que o *cluster* vai sendo escalonado e os arquivos se tornando maiores. Isso ocorre por causa da quantidade de pedaços de arquivos, que foram quebrados no processo de *map*, elevando a necessidade de comunicação entre as *nodes* do sistema, durante o processo de escrita dos arquivos em disco;
6. **Taxa Média de I/O (*mb/sec*):** Esse parâmetro acompanha o mesmo comportamento do *throughput* da rede, ou seja, a taxa média de *input/output* vai diminuindo conforme o tamanho dos arquivos e do *cluster* vão ficando maiores. Como os pedaços dos blocos a serem gravados (*write*) são menores e em grande quantidade, isso exige um tempo maior para ser executado, influenciando na referida taxa;
7. **Consumo de energia*:*** À medida que o tamanho dos arquivos e do *cluster* vão ficando maiores, como já reportado anteriormente, a quantidade de *mappers* vão crescendo de forma significativa, elevando a carga de processamento dessas tarefas, que resulta em um maior consumo de energia por parte das *nodes*, principalmente daquelas que participam mais intensamente desse processamento. Entretanto, como pode ser evidenciado na Figura 31, esse cenário ocorre de forma gradativa;

5.2.3.2 Desempenho do *cluster* (*hardware* e tráfego de rede) – *DFSIO Write*

5.2.3.2.1 *Cluster* formado por 2 *nodes* – *DFSIO Write*

Com o *cluster* formado por 2 *nodes*, observa-se um uso intenso da *CPU* por parte do sistema, em todos os cenários, com diferentes tamanhos de arquivos. Esse comportamento é causado pelo aumento da carga de trabalho atribuída aos nós, exigindo assim, uma maior utilização dos recursos computacionais disponíveis para concluir a execução das tarefas. Além

disso, pode ser verificado um desbalanceamento de carga entre os nós, de aproximadamente 17%, o que demonstra uma dificuldade do *Hadoop* em efetuar um balanceamento de carga de *CPU* eficiente no sistema, resultando em sobrecarga para alguns equipamentos.

No que refere à utilização de memória *RAM*, identifica-se um comportamento semelhante no *cluster*, quando comparado ao consumo de *CPU*, ou seja, uma elevação das taxas, principalmente, nas *nodes* que tiveram o consumo dos recursos de *CPU* maior. Esse desempenho pode ser observado em todos os cenários de testes, com todos os tamanhos de arquivos, inclusive, com um desbalanceamento desses recursos na mesma proporção da *CPU*, de aproximadamente 17%.

Cenário semelhante é observado com relação à temperatura das *nodes* do *cluster*, acompanhando o mesmo comportamento da carga do processador, ou seja, aumentando de acordo com o consumo dos recursos da *CPU*. Esse aumento é mais significativo nas *nodes* que demandam uma carga maior de processamento.

Com relação à quantidade de processos, verifica-se uma distribuição equilibrada entre as nodes *slaves*, com um aumento registrado apenas nas *nodes master*, pelo papel que exercem na distribuição e coordenação das tarefas.

Todos os resultados, referentes a execução desse cenário de teste, podem ser verificados na Tabela 36.

Tabela 36 – Métricas de *hardware* do *cluster* – 2 *Nodes* – *TestDFSIO Write*

Cluster – 2 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	3,01	21,5	36,8	37,3	32,6	38,9	226	229
	Slave 1	2,89	81,7	14,8	41,2	45,3	54,5	160	174
	Slave 2	1,62	95,0	14,7	47,7	36,0	46,7	158	173
1000	*Master*	2,86	20,6	36,0	36,9	31,6	38,5	226	232
	Slave 1	1,66	78,8	14,8	39,9	43,8	54,0	158	174
	Slave 2	1,65	95,4	14,9	48,4	35,5	47,2	158	173
2000	*Master*	2,81	19,7	36,1	37,5	36,0	42,4	225	230
	Slave 1	1,64	96,3	15,4	48,8	43,3	55,0	157	176
	Slave 2	1,82	81,8	15,3	41,9	39,4	46,7	157	170

Fonte: Elaborada pelos Autores, 2025.

Como o algoritmo *TestDFSIO – Write* envolve muita troca de informações na rede e acesso ao disco, em função das sua própria natureza, é possível identificar um aumento significativo desse volume de transações, em todos os cenários de teste, tanto no envio quanto no recebimento dos dados, mantendo um desbalanceamento entre as *nodes* de

aproximadamente 20%, no pior cenário, com um arquivo de 1000 *MBytes*. Porém, não foi registrado nenhum comportamento anormal desse tráfego durante a execução das tarefas, como pode ser observado na Tabela 37.

Tabela 37 – Métricas de tráfego de rede do *cluster* – 2 *Nodes* – *TestDFSIO Write*

***Cluster – 2 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	14,2 Kb/s	472 Kb/s	24,6 Kb/s	75,3 Kb/s
	Slave 1	9,17 Kb/s	432 Mb/s	9,26 Kb/s	380 Mb/s
	Slave 2	8,94 Kb/s	467 Mb/s	6,82 Kb/s	401 Mb/s
1000	*Master*	11,3 Kb/s	3,26 Mb/s	17,5 Kb/s	63,9 Kb/s
	Slave 1	12,5 Kb/s	347 Mb/s	6,05 Kb/s	426 Mb/s
	Slave 2	8,30 Kb/s	413 Mb/s	5,34 Kb/s	387 Mb/s
2000	*Master*	18,1 Kb/s	3,27 Mb/s	26,3 Kb/s	104 Kb/s
	Slave 1	8,61 Kb/s	439 Mb/s	6,46 Kb/s	396 Mb/s
	Slave 2	13,0 Kb/s	458 Mb/s	10,6 Kb/s	495 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 201, 202, 203, 204, 205, 216, 217, 218, 219, 220, 231, 232, 233, 234 e 235.

5.2.3.2.2 *Cluster* formado por 4 *nodes* – *DFSIO Write*

Com o escalonamento do *cluster* para 4 *nodes*, observa-se um esforço maior das *nodes* para trabalhar com arquivos de tamanhos menores, em relação a carga de utilização dos processadores, melhorando esse cenário à medida que os arquivos vão aumentando de tamanho, porém, ainda evidencia-se um desequilíbrio de carga, em todos os cenários, com uma situação um pouco mais equilibrada no último cenário envolvendo o arquivo de 2000 *MBytes*, mesmo assim, uma das *nodes* encontra-se mais subutilizada, em relação às demais.

A utilização de memória *RAM* está diretamente associada ao consumo de *CPU*, identificando-se um consumo mais elevado nas *nodes* que efetivamente tiveram um processamento maior. Porém, com um cenário de desequilíbrio de carga entre as *nodes* do *cluster*, ficando algumas delas subutilizadas com relação a esse recurso disponível, evidenciando, mais uma vez, a dificuldade da biblioteca em trabalhar com um balanceamento de carga eficiente.

Com o aumento da quantidade de máquinas no *cluster* foi possível distribuir as tarefas entre mais *nodes*, mesmo que ainda de forma desequilibrada, resultando em uma redução na

temperatura dos equipamentos, em relação ao cenário do *cluster* formado por 2 *nodes*. Entretanto, percebe-se que esse consumo é maior em todas as *nodes* que atuam mais ativamente durante o processamento das tarefas, mesmo assim, não foi registrado nenhum aumento anormal de temperatura no *cluster*, com a máxima girando em torno dos 48ºC, abaixo das registradas no cenário envolvendo um *cluster* menor.

Nesse cenário, observa-se que a quantidade de processos está semelhante à do cenário anterior, no *cluster* formado por 2 *nodes*, de equilíbrio na carga dos processos entre as *nodes slaves*, com uma demanda um pouco maior no volume nas *nodes master*, em função das atividades que elas exercem de distribuição e coordenação das tarefas durante o processamento.

Todos os resultados dos parâmetros avaliados nesse cenário de teste, podem ser evidenciados na Tabela 38.

Tabela 38 – Métricas de *hardware* do *cluster* – 4 *Nodes* – *TestDFSIO Write*

Cluster – 4 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	3,62	23,1	37,1	37,1	31,2	38,5	225	230
	Slave 1	1,69	87,2	14,4	48,5	33,1	43,3	158	180
	Slave 2	2,28	73,0	14,9	33,1	33,1	42,4	156	166
	Slave 3	1,83	92,3	14,8	49,1	36,5	47,2	157	175
	Slave 4	1,76	67,7	14,9	26,8	33,6	41,4	158	170
1000	*Master*	3,02	21,2	36,9	37,7	30,7	36,0	227	232
	Slave 1	1,62	95,4	14,4	48,5	33,1	43,3	158	177
	Slave 2	1,58	93,2	14,9	49,4	33,1	43,3	156	170
	Slave 3	2,11	73,8	14,7	19,8	36,0	44,3	157	164
	Slave 4	2,15	82,4	14,2	39,3	35,5	42,8	159	174
2000	*Master*	2,95	20,5	36,5	37,4	32,1	37,5	223	229
	Slave 1	3,42	69,3	14,8	24,8	34,6	41,9	156	167
	Slave 2	1,34	89,6	15,4	49,7	35,5	46,3	153	175
	Slave 3	1,36	87,4	15,3	49,7	35,5	47,7	156	178
	Slave 4	1,50	80,3	15,3	33,8	33,6	42,8	156	170

Fonte: Elaborada pelos Autores, 2025.

Apesar do escalonamento do *cluster*, verifica-se que o algoritmo *TestDFSIO – Write* realiza muita troca de informações na rede, bem como acesso ao disco, o que eleva o tráfego na rede. À medida que o tamanho dos arquivos vai aumentando, esse volume também aumenta, em função do número de *mappers* realizados no *cluster* e das quebras dos arquivos em pedaços menores, que são distribuídos para as *nodes* processarem. Como registrado anteriormente, em outros cenários de testes, existe um desbalanceamento desse tráfego, porém, sem nenhuma anormalidade durante a execução das tarefas, como pode ser observado na Tabela 39.

Tabela 39 – Métricas de tráfego de rede do *cluster* – 4 *Nodes – TestDFSIO Write*

***Cluster – 4 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
	Master	27,3 Kb/s	1,44 Mb/s	42,9 Kb/s	102 Kb/s
	Slave 1	9,04 Kb/s	327 Mb/s	5,86 Kb/s	261 Mb/s
500	*Slave 2*	8,20 Kb/s	400 Mb/s	6,01 Kb/s	366 Mb/s
	Slave 3	7,99 Kb/s	406 Mb/s	5,99 Kb/s	269 Mb/s
	Slave 4	11,8 Kb/s	341 Mb/s	7,50 Kb/s	382 Mb/s
	Master	30,8 Kb/s	3,27 Mb/s	41,8 Kb/s	90,7 Kb/s
	Slave 1	9,42 Kb/s	408 Mb/s	7,14 Kb/s	243 Mb/s
1000	*Slave 2*	7,62 Kb/s	392 Mb/s	5,44 Kb/s	278 Mb/s
	Slave 3	9,04 Kb/s	346 Mb/s	8,19 Kb/s	471 Mb/s
	Slave 4	14,0 Kb/s	435 Mb/s	6,74 Kb/s	380 Mb/s
	Master	30,5 Kb/s	3,28 Mb/s	41,8 Kb/s	120 Kb/s
	Slave 1	11,2 Kb/s	640 Mb/s	9,50 Kb/s	477 Mb/s
2000	*Slave 2*	8,94 Kb/s	484 Mb/s	6,88 Kb/s	340 Mb/s
	Slave 3	8,70 Kb/s	554 Mb/s	6,20 Kb/s	412 Mb/s
	Slave 4	10,2 Kb/s	482 Mb/s	5,38 Kb/s	481 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 206, 207, 208, 209, 210, 221, 222, 223, 224, 225, 236, 237, 238, 239 e 240.

5.2.3.2.3 *Cluster* formado por 8 *nodes – DFSIO Write*

No cenário de testes envolvendo o *cluster* com 8 *nodes*, verifica-se que o *Hadoop* conseguiu distribuir a carga de processamento para todas as *nodes*, o que não se observa nos testes envolvendo outros algoritmos, a exemplo dos algoritmos "*Tera*" e "*Pi*", porém, essa carga continua sendo mal distribuída entre as *nodes* do *cluster*, evidenciando-se uma subtilização dos recursos de *hardware*. Essa situação é um pouco melhor no último teste, com o arquivo de tamanho maior, o que demonstra um melhor desempenho da biblioteca com relação à arquivos dessa natureza.

A utilização de memória *RAM*, por sua vez, acompanha o consumo de *CPU* das *nodes*, evidenciando-se um consumo mais elevado nos nós que participam mais intensamente do processamento das tarefas, entretanto, como no parâmetro anterior, não existe um balanceamento adequado dessa carga, observando-se uma dificuldade da biblioteca em lidar com essa questão de forma eficiente.

Com o escalonamento do *cluster* e a distribuição das tarefas para todas as *nodes*, identifica-se um valor mais equilibrado com relação à temperatura das *nodes*, com um aumento mais significativo nos equipamentos que possuem um consumo maior de *CPU*, pois esses dois

parâmetros estão diretamente relacionados. Entretanto, não foi registrado nenhum aumento anormal de temperatura no *cluster*.

A quantidade de processos, a exemplo de outros cenários de testes, se concentra de forma mais evidente nas *nodes master*, devido a função que essas máquinas exercem no *cluster*. As demais apresentam uma situação, praticamente, de equilíbrio dos processos.

Os dados contendo os resultados dos parâmetros analisados, podem ser observados na Tabela 40.

Tabela 40 – Métricas de *hardware* do *cluster* – 8 *Nodes* – *TestDFSIO Write*

Cluster – 8 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
	Master	4,00	26,0	38,0	38,5	34,6	40,4	223	228
	Slave 1	1,52	80,4	14,3	47,5	34,1	46,3	156	176
	Slave 2	1,48	90,3	14,3	48,5	36,0	46,3	155	175
	Slave 3	1,44	69,1	14,7	32,1	37,5	46,7	155	169
500	*Slave 4*	1,52	58,6	15,7	25,1	36,5	43,8	153	167
	Slave 5	1,49	56,2	14,6	16,9	38,0	47,2	157	165
	Slave 6	1,51	63,2	14,0	15,8	37,0	43,8	156	167
	Slave 7	1,54	56,0	14,6	16,3	36,5	45,3	155	166
	Slave 8	1,39	54,2	15,0	17,1	38,5	45,8	154	161
	Master	3,66	21,2	36,3	37,4	33,6	39,9	224	228
	Slave 1	1,52	57,7	14,2	16,9	33,1	40,4	155	165
	Slave 2	1,57	91,8	14,5	49,9	35,0	45,8	158	172
	Slave 3	1,51	90,6	14,5	49,1	36,5	49,2	154	175
1000	*Slave 4*	1,43	59,5	14,5	32,7	34,6	43,3	155	170
	Slave 5	1,55	58,1	15,0	17,1	38,0	46,3	155	166
	Slave 6	1,56	58,5	15,3	25,2	38,5	43,3	156	167
	Slave 7	2,70	63,2	15,1	17,4	36,5	44,8	154	165
	Slave 8	1,74	58,7	14,8	16,5	38,0	46,3	154	164
	Master	3,20	21,1	37,5	38,7	32,1	39,9	222	231
	Slave 1	1,51	86,8	14,0	49,1	33,1	44,3	154	176
	Slave 2	1,40	62,7	14,2	16,4	33,6	41,9	154	162
	Slave 3	1,48	57,7	14,3	17,0	36,0	43,8	155	164
2000	*Slave 4*	1,84	62,8	15,1	17,3	33,6	40,9	155	166
	Slave 5	1,42	87,9	14,4	48,4	37,0	48,2	157	177
	Slave 6	1,51	66,2	14,6	32,7	35,0	44,3	156	172
	Slave 7	1,65	62,0	14,7	25,3	38,0	44,8	156	167
	Slave 8	1,88	62,1	15,1	17,7	37,5	45,8	155	166

Fonte: Elaborada pelos Autores, 2025.

Nesse cenário, observa-se que, com o escalonamento do *cluster*, uma quantidade maior de nós participaram da execução das tarefas, permitindo uma melhor distribuição do tráfego de dados na rede, mesmo que ainda de forma desbalanceada. Isso ocorreu devido ao aumento do número de *mappers* e de replicação dos dados no *cluster*, permitindo quebrar as tarefas em unidades ainda menores e menos complexas, para distribuí-las entre os nós, demandando uma necessidade maior de comunicação durante a execução das tarefas, aumentando dessa forma o

tráfego na rede. Esse situação tornou-se mais acentuada no último cenário de teste, com o arquivo de tamanho maior (2000 *MBytes*), como pode ser identificado na Tabela 41.

Tabela 41 – Métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TestDFSIO Write*

Arquivos MBytes	***Nodes***	***Cluster – 8 Nodes* (Tráfego de Rede)**			
		Sent		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	57,6 Kb/s	518 Mb/s	79,4 Kb/s	169 Kb/s
	Slave 1	9,94 Kb/s	459 Mb/s	7,02 Kb/s	403 Mb/s
	Slave 2	8,19 Kb/s	473 Mb/s	5,85 Kb/s	277 Mb/s
	Slave 3	7,93 Kb/s	500 Mb/s	7,10 Kb/s	458 Mb/s
	Slave 4	10,4 Kb/s	364 Mb/s	10,1 Kb/s	475 Mb/s
	Slave 5	9,01 Kb/s	374 Mb/s	5,95 Kb/s	500 Mb/s
	Slave 6	8,02 Kb/s	419 Mb/s	6,95 Kb/s	496 Mb/s
	Slave 7	10,2 Kb/s	456 Mb/s	6,83 Kb/s	497 Mb/s
	Slave 8	9,57 Kb/s	380 Kb/s	6,83 Kb/s	530 Mb/s
1000	*Master*	59,5 Kb/s	524 Kb/s	92,8 Kb/s	185 Kb/s
	Slave 1	9,65 Kb/s	463 Mb/s	5,82 Kb/s	517 Mb/s
	Slave 2	9,22 Kb/s	417 Mb/s	5,22 Kb/s	354 Mb/s
	Slave 3	11,8 Kb/s	415 Mb/s	6,43 Kb/s	335 Mb/s
	Slave 4	9,89 Kb/s	398 Mb/s	6,76 Kb/s	510 Mb/s
	Slave 5	8,26 Kb/s	446 Mb/s	6,06 Kb/s	470 Mb/s
	Slave 6	12,2 Kb/s	486 Mb/s	9,41 Kb/s	462 Mb/s
	Slave 7	8,79 Kb/s	476 Mb/s	5,48 Kb/s	575 Mb/s
	Slave 8	8,82 Kb/s	490 Mb/s	6,06 Kb/s	516 Mb/s
2000	*Master*	52,7 Kb/s	3,30 Mb/s	89,1 Kb/s	174 Kb/s
	Slave 1	8,46 Kb/s	511 Mb/s	6,13 Kb/s	345 Mb/s
	Slave 2	8,78 Kb/s	579 Mb/s	5,78 Kb/s	512 Mb/s
	Slave 3	8,26 Kb/s	447 Mb/s	6,04 Kb/s	514 Mb/s
	Slave 4	9,86 Kb/s	549 Mb/s	5,19 Kb/s	500 Mb/s
	Slave 5	9,50 Kb/s	507 Mb/s	6,02 Kb/s	352 Mb/s
	Slave 6	9,57 Kb/s	460 Mb/s	5,86 Kb/s	569 Mb/s
	Slave 7	11,0 Kb/s	567 Mb/s	11,6 Kb/s	489 Mb/s
	Slave 8	8,99 Kb/s	554 Mb/s	5,56 Kb/s	514 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 211, 212, 213, 214, 215, 226, 227, 228, 229, 230, 241, 242, 243, 244 e 245.

Em resumo, nos testes envolvendo o algoritmo *TestDFSIO – Write*, conclui-se que, o escalonamento do *cluster* permitiu uma redução na carga de utilização da *CPU*, contribuindo, consequentemente, para uma redução na temperatura das *nodes* do *cluster*, porém desequilibrada, desperdiçando recursos computacionais que poderiam ser aproveitados pelo sistema durante a execução das tarefas.

No entanto, esse escalonamento levou a um crescimento no tráfego da rede do sistema, devido a um aumento na replicação dos dados gerados durante os testes, pois o algoritmo envolve muita troca de informações e acesso ao disco, em função das sua própria natureza.

O consumo de energia, mesmo com o *cluster* escalonado para 8 *nodes*, não aumentou na mesma proporção, evidenciando-se uma eficiência energética do sistema.

5.2.3.3 Desempenho do *Hadoop* e consumo energético do *cluster – DFSIO Read*

Na Figura 32 observa-se os resultados obtidos durante a execução do algoritmo *TestDFSIO Read*, nas 3 (três) configurações de *cluster*, com 3 (três) tamanhos de arquivos diferentes. Com base nos dados é possível analisar o desempenho do *cluster* durante a execução dos cenários de testes.

Figura 32 – Métricas de desempenho (*Hadoop* e Consumo Energético) – *DFSIO Read*

		Tamanho arquivo (MB)	Real time (s)	CPU time (ms)	Map time (ms)	Reduce time (ms)	Taxa de transferência	Taxa média de I/O	Consumo mínimo (W)	Consumo máximo (W)
Nós	2	500	2m28,319s	107.570	492.810	41.144	18,19	21,14	12,9	18,6
		1000	3m40,676s	141.160	813.980	73.577	16,68	18,16	12,6	18,6
		2000	6m8,419s	194.710	1.514.025	122.048	15,54	17,08	12,9	18,6
	4	500	1m51,066s	109.440	614.265	32.734	14,4	15,3	22,8	31,2
		1000	2m38,843s	142.870	1.060.968	55.804	12,71	13,56	21,6	30
		2000	4m18,801s	209.900	1.936.861	105.074	12,07	12,91	21,6	30,3
	8	500	1m54,755s	111.600	640.133	30.365	14,58	15,29	36,3	44,7
		1000	2m40,198s	144.500	1.061.220	56.845	12,9	13,9	36	44,7
		2000	4m15,165s	206.160	1.910.390	97.642	12,13	13,01	36,3	44,7

Fonte: Elaborada pelos Autores, 2025.

1. ***Real Time (s):*** O tempo real de processamento do algoritmo *DFSIO Read* é muito inferior ao tempo do *DFSIO Write*, consumindo menos tempo para ser executado e, consequentemente, menos recursos de processamento durante a realização dos cenários de testes, com um desempenho melhor no cenário que envolve arquivos maiores, como o de 2000 *MBytes*. Isso ocorre, porque o processo de leitura em disco envolve um processamento menor;
2. ***CPU Time (ms):*** O tempo de utilização de *CPU*, segue o mesmo comportamento do parâmetro anterior, do *real time*, tendo em vista que, com menos esforço para a realização das tarefas, o consumo de recursos de utilização da *CPU* é menor. Em todos os cenários de tamanho de *clusters* diferentes, observa-se que à medida que os arquivos vão aumentando de tamanho o esforço de processamento se torna maior, em função da quantidade de replicações de dados distribuídos entre as *nodes*, porém, essa carga é significativamente menor do que a carga do algoritmo de escrita (*Write*);
3. ***Map Time (ms):*** Com relação ao tempo de mapeamento das tarefas, é possível identificar uma situação semelhante ao tempo gasto pela *CPU*, ou seja, o tempo vai aumentando em cada cenário de crescimento de tamanho dos arquivos, em função do esforço de processamento pela quantidade de replicações de dados distribuídos entre

as *nodes*, porém, com um tempo inferior ao realizado nas tarefas de *map* do algoritmo *DFSIO Write*, que precisam ser quebradas e classificadas, gerado um processamento adicional e, consequentemente, um aumento no tempo de *map* das tarefas;

4. ***Reduce Time (ms):*** Com o tempo de agregação das tarefas, o *reduce time*, verifica-se que ele se eleva, à medida que os arquivos vão aumentando de tamanho, em todos os cenários de tamanho de *clusters* diferentes, porém, no cenário em que o *cluster* é menor, com apenas duas *nodes*, esse aumento no tempo de execução das tarefas de *reduce* é maior. Isso ocorre devido à dificuldade que o *Hadoop* possui e, até mesmo do paralelismo em si, em trabalhar a quebra de arquivos para processamento, em *clusters* com capacidade de nós menores;
5. **Taxa de Transferência (*mb/sec*):** A taxa de transferência de dados (*throughput*) do *DFSIO Read* é superior à de transferência do *DFSIO Write*, principalmente, no *cluster* formado por poucas máquinas, como é o caso do de 2 *nodes*. Entretando, à medida que o *cluster* vai sendo escalonado e os arquivos se tornando maiores, essa taxa vai diminuindo. Isso ocorre devido à grande quantidade de pedaços de arquivos, que foram quebrados no processo de *map*, elevando a necessidade de comunicação entre as *nodes* do sistema, durante o processo de leitura dos arquivos em disco;
6. **Taxa Média de I/O (*mb/sec*):** A taxa média de *Input/Output* acompanha o mesmo comportamento ao observado com o *throughput*, ou seja, vai diminuindo de acordo com o aumento do tamanho dos arquivos e do escalonamento do *cluster*, devido aos mesmos motivos explicados no parâmetro anterior;
7. **Consumo de energia*:*** O consumo de energia vai aumentado à medida que o *cluster* vai sendo escalonado e o tamanho dos arquivos vão ficando maiores, como já observado em outros testes, de forma gradativa. Isso ocorre devido à elevação de carga dos processadores envolvidos nas tarefas, que resulta obviamente em um consumo maior de energia, por parte desses equipamentos. Porém, quando comparado ao algoritmo de escrita, esse consumo é bem menor, tendo em vista o baixo volume de processamento envolvido nas tarefas de leitura.

5.2.3.4 Desempenho do *cluster* (*hardware* e tráfego de rede) – *DFSIO Read*

5.2.3.4.1 *Cluster* formado por 2 *nodes* – *DFSIO Read*

No *cluster* formado por 2 *nodes*, identifica-se o uso intenso da *CPU* por parte do sistema, em todos os cenários, com diferentes tamanhos de arquivos. Isso ocorre devido a um aumento da carga de trabalho atribuída às *nodes slaves*, exigindo delas os recursos de processamento disponíveis para a conclusão das tarefas, evidenciando-se um desbalanceamento de carga entre as *nodes slaves*, o que mostra uma dificuldade do *Hadoop* em relação a essa questão, resultando em uma sobrecarga adicional para alguns equipamentos. Além disso, observa-se uma baixa utilização dos recursos de *CPU* pelas *nodes master*. Esse fato ocorre porque elas não participam ativamente do processamento das tarefas, atuando apenas na distribuição e coordenação delas.

A memória *RAM* de todas as *nodes* são requisitadas durante a execução dos testes, em todos os cenários, registrando um aumento maior nas taxas das *nodes slaves* que tiveram um consumo mais elevado dos recursos de *CPU*, porém, é possível observar um desbalanceamento dessas taxas nos cenários de testes analisados.

A temperatura das *nodes* do *cluster*, acompanha o comportamento de carga da *CPU* das *nodes*, ou seja, à medida que a carga de processamento delas se eleva, a temperatura sobe. Porém, essa elevação é mais significativa nas *nodes slaves* que demandam uma carga maior de processamento.

No que se refere à quantidade de processos, é possível identificar uma distribuição equilibrada entre as nodes *slaves*, com um aumento dessa quantidade observado apenas nas *nodes master*, pelo função que exercem na distribuição e coordenação das tarefas no *cluster*.

Os resultados dos cenários de testes apresentados, podem ser observados na Tabela 42.

Tabela 42 – Métricas de *hardware* do *cluster* – 2 *Nodes* – *TestDFSIO Read*

Cluster – 2 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	3,30	15,8	37,1	37,2	31,2	37,5	227	233
	Slave 1	1,05	89,6	16,8	45,1	43,3	55,0	156	172
	Slave 2	1,26	77,1	17,4	37,5	37,5	46,3	156	168
1000	*Master*	2,98	16,1	36,9	37,3	31,6	36,0	226	230
	Slave 1	0,93	88,6	17,2	45,7	42,8	54,0	155	175
	Slave 2	0,89	68,2	17,3	37,5	37,5	44,8	153	166
2000	*Master*	2,58	15,3	37,4	37,7	37,0	41,9	227	230
	Slave 1	0,98	88,6	18,2	46,3	44,3	57,0	155	174
	Slave 2	1,55	65,7	18,3	37,8	38,5	47,2	157	169

Fonte: Elaborada pelos Autores, 2025.

Nesse cenário, evidencia-se que o algoritmo *TestDFSIO – Read* contribui com um volume baixo de tráfego na rede durante a sua execução. Com relação ao tráfego de envio (*sent*),

observa-se que as *nodes master* se destacam, com um volume a mais de tráfego, em comparação com as *nodes slaves*. Isso ocorre devido ao envio das tarefas (*mappers*) para as *slaves* processarem, o que pode ser comprovado através do tráfego recebido pelas *nodes slaves*. Esse aumento de tráfego nas *nodes slaves* se dá em função do aumento do número de *mappers* e de replicação dos dados recebidos para o processamento. Essa situação ocorre em todos os cenários de testes, com tamanhos diferentes de arquivos, como pode ser identificado na Tabela 43.

Tabela 43 – Métricas de tráfego de rede do *cluster* – 2 *Nodes – TestDFSIO Read*

***Cluster – 2 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	17,0 Kb/s	3,29 Mb/s	24,5 Kb/s	104 Kb/s
	Slave 1	8,99 Kb/s	63,0 Kb/s	6,01 Kb/s	3,28 Mb/s
	Slave 2	13,0 Kb/s	3,31 Mb/s	10,4 Kb/s	3,31 Mb/s
1000	*Master*	13,0 Kb/s	3,28 Mb/s	20,4 Kb/s	51,2 Kb/s
	Slave 1	8,58 Kb/s	94,6 Kb/s	6,11 Kb/s	548 Kb/s
	Slave 2	12,3 Kb/s	3,27 Mb/s	7,61 Kb/s	3,33 Mb/s
2000	*Master*	13,1 Kb/s	3,28 Mb/s	24,1 Kb/s	107 Kb/s
	Slave 1	9,54 Kb/s	70,3 Kb/s	5,74 Kb/s	3,28 Mb/s
	Slave 2	14,1 Kb/s	700 Kb/s	6,63 Kb/s	3,33 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 246, 247, 248, 249, 250, 261, 262, 263, 264, 265, 276, 277, 278, 279 e 280.

5.2.3.4.2 *Cluster* formado por 4 *nodes – DFSIO Read*

Com o *cluster* escalonado para 4 *nodes*, identifica-se um situação de total desbalanceamento de carga entre as *nodes*, sendo mais acentuado nos dois primeiros testes, envolvendo arquivos com tamanhos menores, onde algumas *nodes* não participam do processamento das tarefas, ficando subutilizadas. À medida que o tamanho do arquivo fica maior, no cenário com o tamanho de 2000 *MBytes*, observa-se a atuação de todas as *nodes slaves* durante o processamento, porém, ainda com uma situação de desequilíbrio de carga entre elas. Esse cenário demonstra, uma dificuldade da biblioteca em trabalhar essa questão de balanceamento dos recursos de *hardware* dos nós envolvidos nas tarefas de processamento.

Já o consumo de memória *RAM*, acompanha o desempenho da *CPU*, ou seja, as *nodes* que consumiram mais recursos de memória foram justamente as que participaram ativamente do processamento das tarefas, entretanto, com um cenário de desequilíbrio entre elas em todos

os testes analisados.

Com relação à temperatura das *nodes* do *cluster*, durante o processamento das tarefas, observa-se uma diminuição e um pouco mais de equilíbrio à medida que os arquivos vão ficando maiores. Apesar da temperatura estar associada diretamente a utilização dos recursos de *CPU* das *nodes*, não foi evidenciada, nos cenários de testes realizados, nenhuma situação de aumento significativo entre os equipamentos que participaram mais intensamente das tarefas de processamento, com uma máxima registrada de 46,7 ºC, menor do que a registrada no cenário envolvendo um *cluster* menor, de duas *nodes*, demonstrando que o escalonamento do *cluster* contribuiu para uma diminuição do consumo de temperatura entre os equipamentos.

Com relação à quantidade processos, identifica-se, nos cenários de testes envolvendo arquivos com tamanho menores, uma situação de desbalanceamento na distribuição, entretanto, à medida que os arquivos aumentam de tamanho essa situação diminui, sendo registrado um volume maior nas *nodes slaves*, em razão da participação desses equipamentos na distribuição das tarefas entre as demais *nodes* do *cluster* durante a execução das atividades de processamento do sistema. Isso é observado em, praticamente, todos os cenários de testes realizados, inclusive, com outros algoritmos de *benchmark* utilizados neste trabalho.

Os resultados contendo todos os parâmetros analisados durante esse cenário de teste, podem ser identificados na Tabela 44.

Tabela 44 – Métricas de *hardware* do *cluster* – 4 *Nodes* – *TestDFSIO Read*

Cluster – 4 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
500	*Master*	3,56	16,7	37,7	37,8	31,6	36,0	225	229
	Slave 1	1,04	89,3	16,4	43,6	33,6	43,8	158	175
	Slave 2	0,80	1,07	17,7	17,8	31,6	34,6	153	156
	Slave 3	4,82	67,1	20,1	38,0	39,4	46,3	159	172
	Slave 4	1,11	88,9	16,5	43,9	33,1	43,8	158	170
1000	*Master*	3,44	16,4	38,6	38,7	31,2	35,5	227	230
	Slave 1	0,83	1,11	16,6	16,7	31,6	34,1	154	158
	Slave 2	0,99	89,8	17,4	46,1	32,1	43,3	154	170
	Slave 3	1,06	89,9	16,4	45,6	35,5	46,7	157	176
	Slave 4	1,24	65,8	16,5	36,5	33,6	42,4	157	168
2000	*Master*	2,85	16,0	37,1	37,3	32,6	36,0	224	228
	Slave 1	0,95	90,2	16,7	43,8	33,1	43,8	156	173
	Slave 2	0,89	65,1	17,9	32,3	35,0	43,8	153	165
	Slave 3	1,74	19,8	17,9	24,1	36,0	42,4	154	162
	Slave 4	0,93	89,2	17,2	45,2	33,1	42,8	153	173

Fonte: Elaborada pelos Autores, 2025.

Durante a execução desse cenário de testes, é possível identificar que o algoritmo

TestDFSIO – Read não consome muitos recursos da rede de interconexão, registrando um volume de tráfego baixo na rede, porém, no cenário envolvendo o arquivo com tamanho de 1000 *MBytes*, verifica-se uma situação de anormalidade nas *nodes Slaves 3* e *4*, com um aumento elevado desse tráfego no envio e recebimento dos dados, não sendo identificada uma causa precisa que levou a essa situação, entretanto, presume-se que foi alguma dificuldade ou *bug* do sistema durante a distribuição das tarefas entre esses equipamentos, ou até algum problema na rede de interconexão. Tal situação não foi registrada anteriormente em nenhum outro cenário de teste envolvendo o mesmo algoritmo ou outros algoritmos de *benchmark* diferentes, como pode ser evidenciado na Tabela 45.

Tabela 45 – Métricas de tráfego de rede do *cluster* – 4 *Nodes – TestDFSIO Read*

***Cluster – 4 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	29,4 Kb/s	3,43 Mb/s	40,0 Kb/s	140 Kb/s
	Slave 1	8,25 Kb/s	5,87 Mb/s	6,49 Kb/s	3,23 Mb/s
	Slave 2	8,07 Kb/s	494 Kb/s	6,04 Kb/s	3,43 Mb/s
	Slave 3	10,7 Kb/s	580 Kb/s	6,84 Kb/s	3,42 Mb/s
	Slave 4	11,3 Kb/s	533 Kb/s	4,83 Kb/s	3,40 Mb/s
1000	*Master*	32,3 Kb/s	3,30 Mb/s	49,4 Kb/s	148 Kb/s
	Slave 1	8,20 Kb/s	3,30 Mb/s	5,90 Kb/s	3,30 Mb/s
	Slave 2	11,1 Kb/s	538 Kb/s	5,90 Kb/s	3,30 Mb/s
	Slave 3	9,04 Kb/s	346 Mb/s	8,19 Kb/s	471 Mb/s
	Slave 4	14,0 Kb/s	435 Mb/s	6,74 Kb/s	380 Mb/s
2000	*Master*	33,9 Kb/s	496 Kb/s	46,4 Kb/s	89,0 Kb/s
	Slave 1	10,0 Kb/s	709 Kb/s	6,27 Kb/s	3,24 Mb/s
	Slave 2	11,2 Kb/s	2,87 Mb/s	5,97 Kb/s	546 Kb/s
	Slave 3	12,9 Kb/s	3,28 Mb/s	8,45 Kb/s	76,2 Kb/s
	Slave 4	9,78 Kb/s	3,29 Mb/s	6,54 Kb/s	548 Kb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 251, 252, 253, 254, 255, 266, 267, 268, 269, 270, 281, 282, 283, 284 e 285.

5.2.3.4.3 *Cluster* formado por 8 *nodes – DFSIO Read*

Com o escalonamento do *cluster* para 8 *nodes*, observa-se uma subutilização dos recursos de *CPU* (processamento) por parte das *nodes slaves*, em todos os cenários de testes, registrando uma perda de 50% do sistema no cenário em que o tamanho do arquivo é menor, de 500 *MBytes*, reduzindo para 25% nos demais cenários, o que demonstra uma má utilização dos recursos de *hardware* disponíveis no *cluster* pela biblioteca *Apache Hadoop*, no *benchmark*

envolvendo esse algoritmo. Essa situação também é observada em outros testes de *benchmark*, evidenciando, de fato, a ineficiência da biblioteca em relação à utilização dos recursos de *hardware* disponíveis no *cluster*, pelo menos na versão do *Hadoop* utilizada nesse trabalho, a versão 3.4.0.

O consumo de memória *RAM*, por sua vez, como observado nesse cenário, registra uma elevação nas *nodes master*, pela função que elas exercem na distribuição dos processos e das tarefas no *cluster*, bem como nas *nodes slaves* que participam de forma mais intensa do processamento das tarefas. Além disso, evidencia-se um desequilíbrio de carga na utilização desse recurso de *hardware* por parte do *cluster*, motivado pela dificuldade da biblioteca em gerenciar de forma eficiente esses recursos.

Já a temperatura das *nodes* do *cluster*, acompanha o consumo dos processadores, ou seja, à medida que eles são demandos, ocorre uma elevação desse consumo, sendo mais evidente e elevado, nas *nodes* mais requisitadas com relação ao processamento das tarefas.

No que se refere à quantidade de processos, verifica-se uma situação semelhante, à observada em outros cenários de testes, ou seja, de elevação dessa quantidade nas *nodes master*, devido ao seu papel no *cluster*, e de um pequeno desbalanceamento nas demais *nodes*, em todos os cenários de testes realizados, sem anormalidades e discrepâncias registradas. Essa situação é identificada em, praticamente, todos os outros algoritmos de *benchmark* utilizados nesse trabalho.

Todos as informações referentes aos cenários avaliados, podem ser verificadas com base nos dados contidos na Tabela 46.

Tabela 46 – Métricas de *hardware* do *cluster* – 8 *Nodes* – *TestDFSIO Read*

Cluster – 8 Nodes									
Arquivos MBytes	***Nodes***	***Utilização de CPU (%)***		***Utilização de Memória RAM (%)***		***Temperatura (ºC)***		***Número de Processos***	
		Min.	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***	***Min.***	***Máx.***
	Master	3,88	17,8	38,7	39,4	35,5	38,5	224	227
	Slave 1	0,96	89,5	16,8	44,5	35,5	45,3	152	175
	Slave 2	0,79	1,21	17,1	17,1	36,0	38,0	152	153
	Slave 3	0,95	1,08	16,8	16,9	37,5	39,9	155	169
500	*Slave 4*	1,10	21,4	17,2	27,4	36,5	42,4	153	167
	Slave 5	1,03	51,9	15,9	29,5	39,4	47,7	157	165
	Slave 6	0,80	1,09	15,0	15,0	37,0	38,9	152	156
	Slave 7	0,81	1,16	15,8	15,9	37,5	39,9	154	157
	Slave 8	1,07	89,7	15,9	43,7	39,9	50,6	159	176
	Master	3,60	17,3	37,4	37,5	33,1	38,0	224	227
	Slave 1	0,74	1,13	15,7	15,7	33,1	35,0	152	157
	Slave 2	2,64	24,8	20,0	24,9	35,0	43,3	157	162
	Slave 3	0,91	89,5	17,4	45,7	36,5	47,2	155	172
1000	*Slave 4*	0,73	10,1	16,5	20,4	33,6	39,4	154	161
	Slave 5	1,13	64,0	15,8	29,5	38,5	46,7	155	169
	Slave 6	0,75	1,08	17,0	17,0	34,6	37,0	153	160
	Slave 7	0,99	88,3	16,2	46,5	37,0	48,2	157	174
	Slave 8	0,76	1,11	15,9	16,0	37,5	39,9	154	157
	Master	3,12	17,2	39,0	39,1	31,6	36,0	225	230
	Slave 1	0,65	0,98	17,5	17,6	31,2	33,6	152	161
	Slave 2	0,70	1,14	15,2	15,3	32,6	36,0	152	157
	Slave 3	2,64	23,1	15,7	26,3	36,5	42,4	159	169
2000	*Slave 4*	0,96	65,4	15,7	30,0	32,6	42,4	151	168
	Slave 5	0,71	0,96	17,3	17,4	36,0	38,9	153	159
	Slave 6	0,90	88,6	16,8	45,8	35,0	45,3	158	176
	Slave 7	0,95	89,0	16,4	46,0	36,5	48,2	155	176
	Slave 8	0,69	1,01	16,3	16,4	36,5	38,9	154	163

Fonte: Elaborada pelos Autores, 2025.

Com o escalonamento do *cluster*, verifica-se uma distribuição do tráfego mais equilibrada na rede, sendo mais evidenciada nos cenários de testes com os arquivos de tamanhos maiores, não ultrapassando os 3,5 *Mb/s*, durante o processo de recebimento dos dados, ou seja, um consumo de tráfego relativamente baixo durante a fase de recebimento das tarefas para o processamento. Já no cenário de envio de dados, onde as *nodes* trocam informações na rede, observa-se um comportamento fora da curva na *Slave* 7, registrando um tráfego de 6,01 *Mb/s*, praticamente o dobro em relação às demais, no cenário envolvendo o tamanho de arquivo de 2000 *MBytes*. Essa anormalidade, pode ter ocorrido devido à diversos fatores: recebimento de um carga adicional de dados; dificuldade ou *bug* do sistema durante a execução das tarefas; dificuldade de comunicação entre os equipamentos; e por problemas ocorridos na rede de interconexão. Todas os dados referentes a essas métricas, podem ser observados na Tabela 47

Tabela 47 – Métricas de tráfego de rede do *cluster* – 8 *Nodes* – *TestDFSIO Read*

***Cluster – 8 Nodes* (Tráfego de Rede)**					
Arquivos MBytes	***Nodes***	***Sent***		***Received***	
		Min.	***Máx.***	***Min.***	***Máx.***
500	*Master*	54,0 Kb/s	3,34 Mb/s	84,0 Kb/s	141 Kb/s
	Slave 1	8,46 Kb/s	522 Kb/s	5,86 Kb/s	551 Kb/s
	Slave 2	9,06 Kb/s	3,27 Mb/s	5,80 Kb/s	3,30 Mb/s
	Slave 3	8,02 Kb/s	526 Kb/s	5,93 Kb/s	3,28 Mb/s
	Slave 4	11,6 Kb/s	3,24 Mb/s	9,52 Kb/s	3,30 Mb/s
	Slave 5	8,21 Kb/s	529 Kb/s	6,52 Kb/s	3,25 Mb/s
	Slave 6	8,66 Kb/s	529 Kb/s	5,66 Kb/s	3,24 Mb/s
	Slave 7	8,27 Kb/s	3,27 Mb/s	5,50 Kb/s	3,28 Mb/s
	Slave 8	8,17 Kb/s	529 Kb/s	5,97 Kb/s	3,24 Mb/s
1000	*Master*	56,9 Kb/s	3,33 Mb/s	87,9 Kb/s	150 Kb/s
	Slave 1	8,89 Kb/s	3,22 Mb/s	5,94 Kb/s	3,26 Mb/s
	Slave 2	16,3 Kb/s	587 Kb/s	10,1 Kb/s	3,28 Mb/s
	Slave 3	9,43 Kb/s	3,31 Mb/s	5,92 Kb/s	556 Kb/s
	Slave 4	8,70 Kb/s	524 Kb/s	4,50 Kb/s	3,25 Mb/s
	Slave 5	9,08 Kb/s	531 Kb/s	6,33 Kb/s	533 Kb/s
	Slave 6	8,30 Kb/s	3,26 Mb/s	4,97 Kb/s	521 Kb/s
	Slave 7	8,25 Kb/s	527 Kb/s	6,43 Kb/s	2,76 Mb/s
	Slave 8	8,68 Kb/s	3,29 Mb/s	5,68 Kb/s	520 Kb/s
2000	*Master*	57,9 Kb/s	3,34 Mb/s	82,6 Kb/s	183 Kb/s
	Slave 1	8,22 Kb/s	3,28 Mb/s	5,88 Kb/s	705 Kb/s
	Slave 2	8,22 Kb/s	80,7 Kb/s	5,68 Kb/s	3,24 Mb/s
	Slave 3	12,0 Kb/s	532 Kb/s	7,19 Kb/s	3,27 Mb/s
	Slave 4	8,77 Kb/s	3,25 Mb/s	5,26 Kb/s	3,31 Mb/s
	Slave 5	8,23 Kb/s	715 Kb/s	5,78 Kb/s	3,22 Mb/s
	Slave 6	9,43 Kb/s	670 Kb/s	5,85 Kb/s	3,27 Mb/s
	Slave 7	9,14 Kb/s	6,01 Mb/s	6,54 Kb/s	3,27 Mb/s
	Slave 8	8,79 Kb/s	530 Kb/s	5,60 Kb/s	3,28 Mb/s

Fonte: Elaborada pelos Autores, 2025.

Os dados referentes as tabelas mencionadas podem ser evidenciados no Apêndice B, através das Figuras 256, 257, 258, 259, 260, 271, 272, 273, 274, 275, 286, 287, 288, 289 e 290.

5.2.3.5 Capacidade de armazenamento dos *Clusters* – *TestDFSIO Write/Read*

Nas Figuras 33, 34 e 35, respectivamente, observa-se a utilização de espaço em disco de cada *node* do *cluster*, após os cenários de testes realizados envolvendo os algoritmos *TestDFSIO Write* e *Read*, com o tamanho de arquivo de 500 *MBytes*, em um *cluster* formado por 2 *nodes slaves* e uma *master*. Nesse cenário, é possível identificar um consumo do espaço em disco relativamente baixo e equilibrado entre as *nodes* do *cluster*, não chegando a atingir 5% da capacidade total de cada uma delas, em função, obviamente, do tamanho do arquivo utilizado durante a execução das tarefas.

Figura 33 – Espaço em disco – 2 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	22m	4.92 GB	6.41 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	22m	4.92 GB	6.44 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 34 – Espaço em disco – 4 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	24m	4.92 GB	6.48 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	2s	24m	4.92 GB	6.44 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	0s	24m	4.92 GB	6.35 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	0s	24m	4.92 GB	6.5 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 35 – Espaço em disco – 8 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✔/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	27m	4.92 GB	6.53 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	0s	27m	4.92 GB	6.49 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	1s	27m	4.92 GB	6.39 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	0s	27m	4.92 GB	6.33 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	0s	27m	4.92 GB	6.38 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	0s	21m	4.92 GB	6.32 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	2s	27m	4.92 GB	6.55 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0
✔/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	0s	27m	4.92 GB	6.32 GB	116.9 GB	58	4.92 GB (4.21%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Com base nas Figuras 36, 37 e 38, respectivamente, é possível identificar que, com o aumento do tamanho do arquivo a ser processado, o consumo de espaço em disco apresenta um volume maior, proporcional, praticamente o dobro da capacidade evidenciada no cenário de teste anterior, porém, totalmente distribuído entre as *nodes* do *cluster*, chegando a uma

ocupação individual de aproximadamente 8,5% da capacidade total de cada uma delas.

Figura 36 – Espaço em disco – 2 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	1s	36m	9.84 GB	6.41 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	36m	9.84 GB	6.44 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 37 – Espaço em disco – 4 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	19m	9.84 GB	6.48 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	1s	26m	9.84 GB	6.44 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	1s	26m	9.84 GB	6.35 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	26m	9.84 GB	6.5 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 38 – Espaço em disco – 8 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	1s	31m	9.84 GB	6.53 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	0s	31m	9.84 GB	6.49 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	1s	28m	9.84 GB	6.39 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	0s	31m	9.84 GB	6.33 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	1s	31m	9.84 GB	6.38 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	1s	31m	9.84 GB	6.32 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	31m	9.84 GB	6.55 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0
/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	1s	31m	9.84 GB	6.32 GB	116.9 GB	98	9.84 GB (8.42%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Nos cenários dos *clusters* com o tamanho do arquivo de 2000 *MBytes*, identifica-se um comportamento muito semelhante ao registrado nos cenário anteriores, com arquivos menores.

O volume de ocupação do espaço em disco, após os testes, aumentou, devido ao tamanho do arquivo utilizado, porém, mantendo a mesma distribuição, no consumo, entre todas as *nodes* do *cluster*, com uma ocupação média de aproximadamente 16,84% da capacidade total de cada uma delas, como pode ser evidenciado nas Figuras 39, 40 e 41, respectivamente.

Figura 39 – Espaço em disco – 2 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	39m	19.69 GB	6.41 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	39m	19.69 GB	6.44 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 40 – Espaço em disco – 4 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	0s	19m	19.69 GB	6.49 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	0s	44m	19.69 GB	6.46 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	2s	44m	19.69 GB	6.36 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	44m	19.69 GB	6.52 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0

Fonte: Elaborada pelos Autores, 2025.

Figura 41 – Espaço em disco – 8 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes*

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
/default-rack/slave1:9866 (172.29.20.1:9866)	http://slave1:9864	2s	39m	19.69 GB	6.53 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave4:9866 (172.29.20.4:9866)	http://slave4:9864	0s	39m	19.69 GB	6.49 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave3:9866 (172.29.20.3:9866)	http://slave3:9864	0s	39m	19.69 GB	6.39 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave7:9866 (172.29.20.7:9866)	http://slave7:9864	0s	39m	19.69 GB	6.33 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave5:9866 (172.29.20.5:9866)	http://slave5:9864	1s	39m	19.69 GB	6.38 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave8:9866 (172.29.20.8:9866)	http://slave8:9864	0s	39m	19.69 GB	6.32 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave2:9866 (172.29.20.2:9866)	http://slave2:9864	1s	39m	19.69 GB	6.55 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0
/default-rack/slave6:9866 (172.29.20.6:9866)	http://slave6:9864	1s	39m	19.69 GB	6.32 GB	116.9 GB	178	19.69 GB (16.84%)	0%	3.4.0

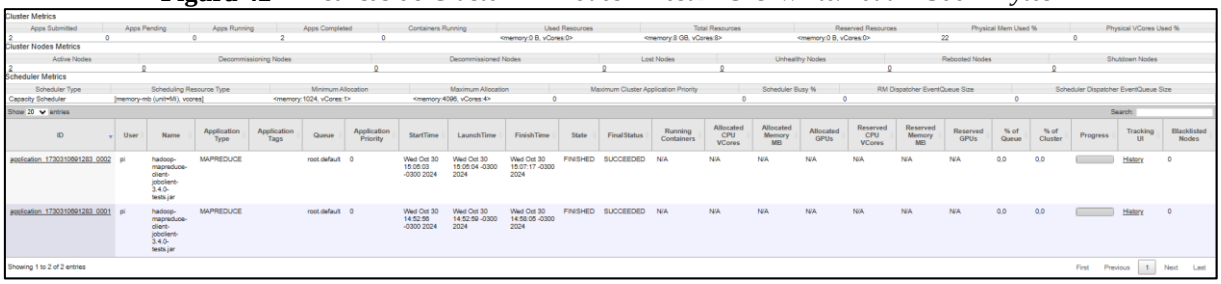
Fonte: Elaborada pelos Autores, 2025.

5.2.3.6 Métricas dos *Clusters* – *Algoritmo TestDFSIO* (*Write*/*Read*)

As Figuras 42, 43 e 44, respectivamente, mostram as telas de comprovação da execução dos testes envolvendo os algoritmos *TestDFSIO Write* e *Read*, com tamanhos de *clusters* diferentes, com arquivos de 500 *MBytes* de tamanho, contendo informações sobre os recursos de *hardware* utilizados e disponíveis em cada *cluster*.

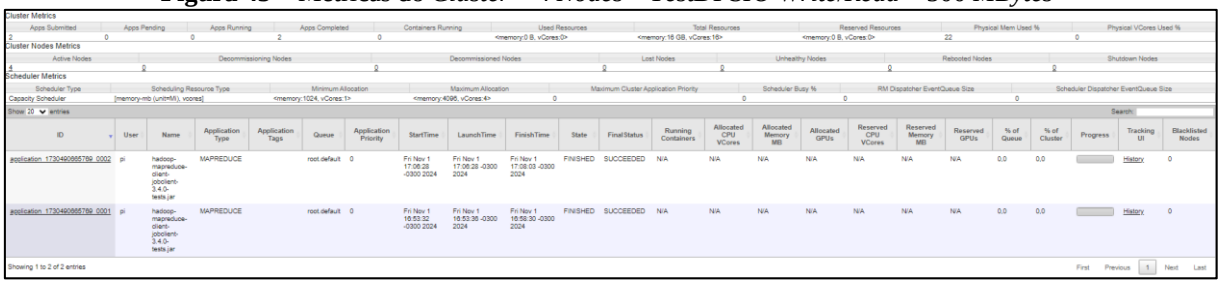
Nas figuras, é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (S*UCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 22%; a quantidade de nós (*nodes*) ativos na execução dos testes, no campo *Active Nodes*, neste caso com 2, 4, 8 *nodes slaves* e uma *master*, em cada configuração, respectivamente, com tamanhos de arquivos de 500 *MBytes*. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*TestDFSIO Write*/*Read*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources*, com 8 *GBytes*, 16 *GBytes* e 32 *GBytes*, de acordo com a capacidade dos *clusters* implementados, respectivamente.

Figura 42 – Métricas do *Cluster* – 2 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes*



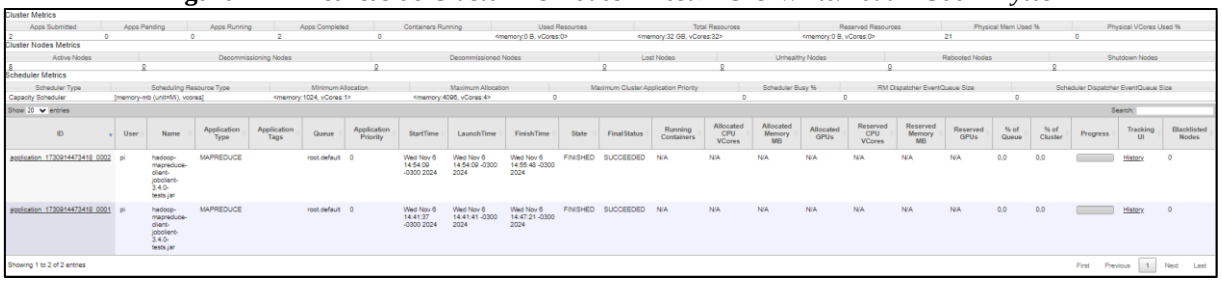
Fonte: Elaborada pelos Autores, 2025.

Figura 43 – Métricas do *Cluster* – 4 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Figura 44 – Métricas do *Cluster* – 8 *Nodes* – *TestDFSIO Write*/*Read* – 500 *MBytes*

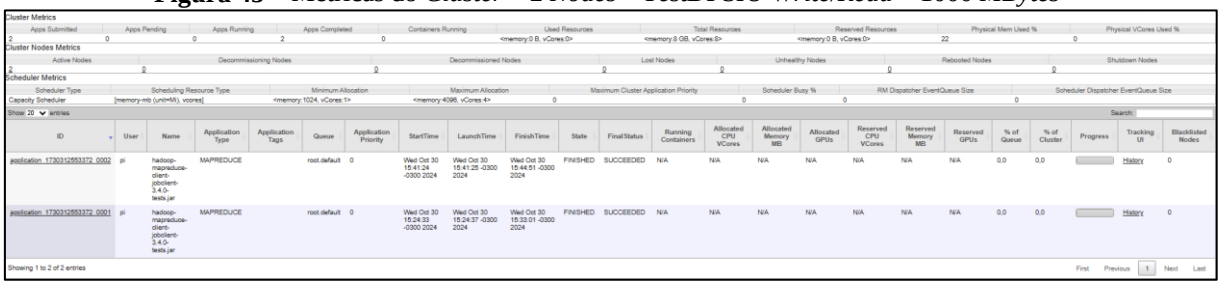


Fonte: Elaborada pelos Autores, 2025.

Nas Figuras 45, 46 e 47, respectivamente, evidencia-se as telas de comprovação da execução dos testes envolvendo os algoritmos *TestDFSIO Write* e *Read*, com tamanhos de *clusters* diferentes, com arquivos de 1000 *MBytes* de tamanho, contendo informações sobre os recursos de *hardware* utilizados e disponíveis em cada *cluster*.

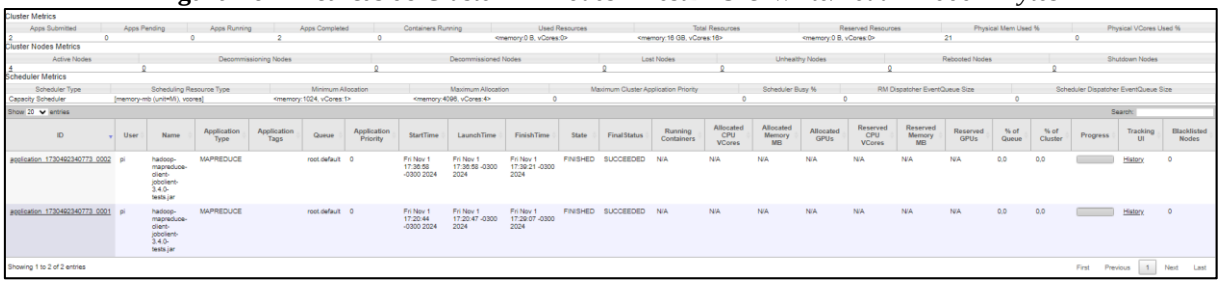
Nas figuras, é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (S*UCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 22%; a quantidade de nós (*nodes*) ativos na execução dos testes, no campo *Active Nodes*, neste caso com 2, 4, 8 *nodes slaves* e uma *master*, em cada configuração, respectivamente, com tamanhos de arquivos de 1000 *MBytes*. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*TestDFSIO Write*/*Read*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources*, com 8 *GBytes*, 16 *GBytes* e 32 *GBytes*, de acordo com a capacidade dos *clusters* implementados, respectivamente.

Figura 45 – Métricas do *Cluster* – 2 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes*



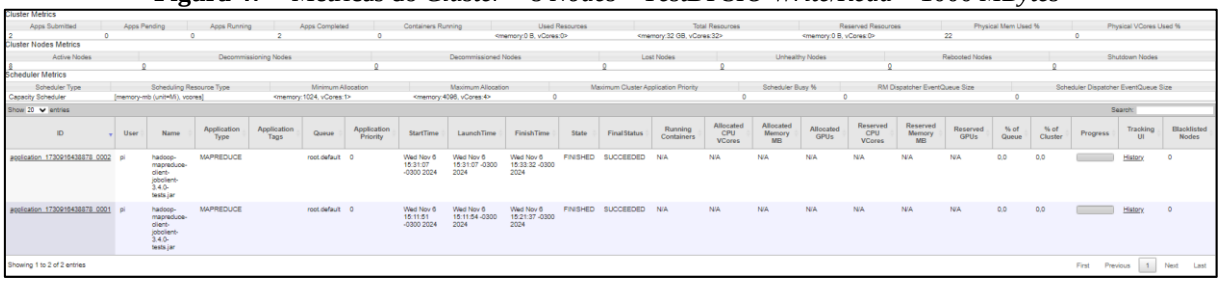
Fonte: Elaborada pelos Autores, 2025.

Figura 46 – Métricas do *Cluster* – 4 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Figura 47 – Métricas do *Cluster* – 8 *Nodes* – *TestDFSIO Write*/*Read* – 1000 *MBytes*

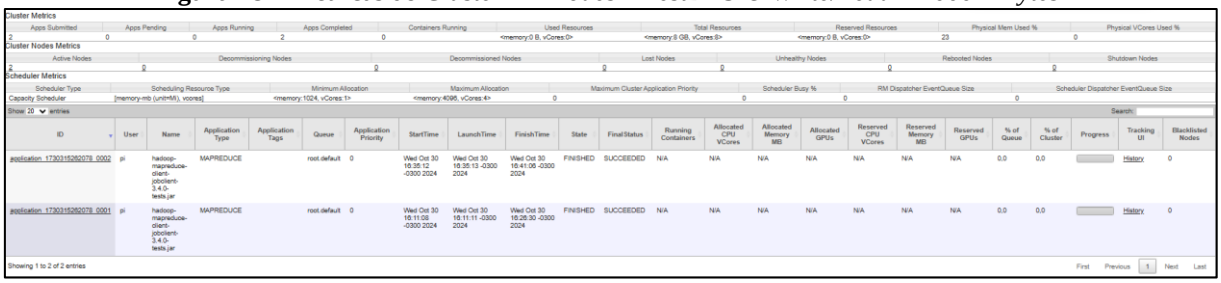


Fonte: Elaborada pelos Autores, 2025.

Já as Figuras 48, 49 e 50, respectivamente, apresentam as telas de comprovação da execução dos testes envolvendo os algoritmos *TestDFSIO Write* e *Read*, com tamanhos de *clusters* diferentes, com arquivos de 2000 *MBytes* de tamanho, contendo informações sobre os recursos de *hardware* utilizados e disponíveis em cada *cluster*.

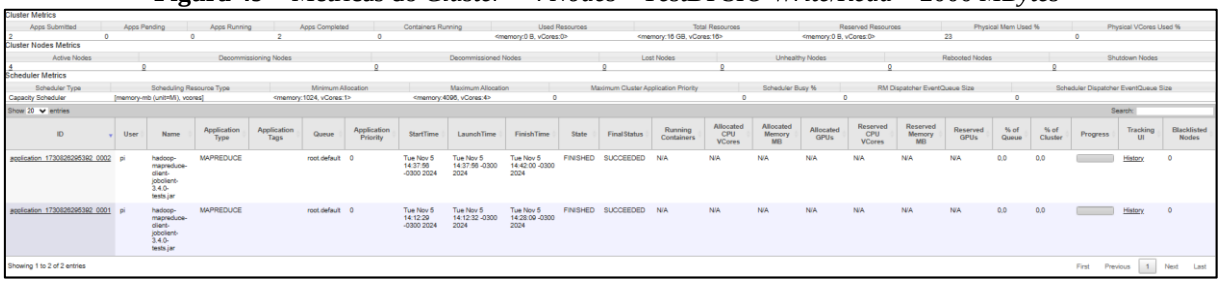
Nas figuras, é possível identificar: o sucesso da execução dos testes, no campo *FinalStatus* (S*UCCEEDED*), mostrando que não ocorreu nenhum erro na sua realização; o percentual de memória física (*RAM*) utilizada, no campo *Physical Mem Used (%)*, com um percentual máximo de utilização de 22%; a quantidade de nós (*nodes*) ativos na execução dos testes, no campo *Active Nodes*, neste caso com 2, 4, 8 *nodes slaves* e uma *master*, em cada configuração, respectivamente, com tamanhos de arquivos de 2000 *MBytes*. Além disso, observa-se ainda o nome do algoritmo utilizado no teste, no campo *Name* (*TestDFSIO Write*/*Read*); a data e hora em que os testes foram realizados, nos campos *StartTime* e *FinishTime*, bem como o tipo da aplicação do *Apache Hadoop* utilizada, no campo *Application Type* (*MAPREDUCE*) e a quantidade de recursos de memória disponível no *cluster*, em cada cenário, evidenciados no campo *Total Resources*, com 8 *GBytes*, 16 *GBytes* e 32 *GBytes*, de acordo com a capacidade dos *clusters* implementados, respectivamente.

Figura 48 – Métricas do *Cluster* – 2 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes*



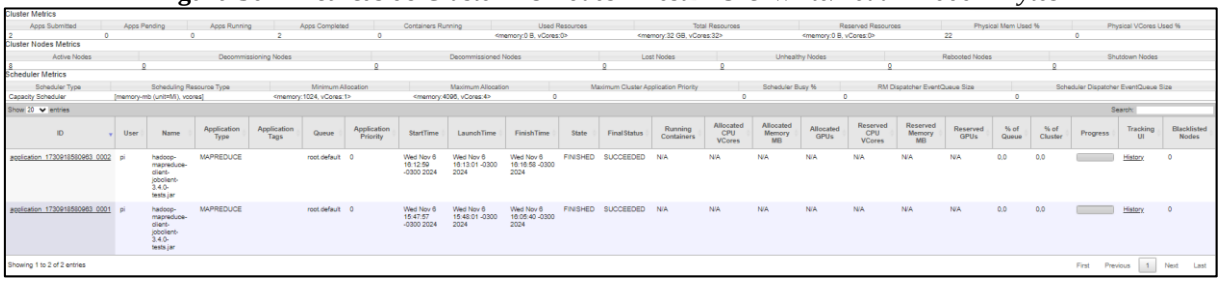
Fonte: Elaborada pelos Autores, 2025.

Figura 49 – Métricas do *Cluster* – 4 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Figura 50 – Métricas do *Cluster* – 8 *Nodes* – *TestDFSIO Write*/*Read* – 2000 *MBytes*



Fonte: Elaborada pelos Autores, 2025.

Em resumo, nos testes envolvendo o algoritmo *TestDFSIO – Read*, conclui-se que, à medida que a quantidade nós vai aumentado no *cluster*, com a disponibilização de mais recursos de *hardware*, ocorre uma redução significativa no tempo de execução das tarefas, porém, ainda com subutilização desses recursos, o que resulta em uma perda considerável de desempenho do *cluster* e de recursos computacionais, principalmente, à medida que os arquivos vão se tornando maiores e mais complexos. Com relação ao consumo de espaço em disco, fica evidente que o algoritmo de escrita consome mais do que o de leitura, mesmo assim, na execução desses algoritmos, não foi registrada nenhuma ocupação elevada na unidade de armazenamento, que chegasse a comprometer o desempenho dos testes realizados.

6. CONSIDERAÇÕES FINAIS

Os avanços tecnológicos têm provocado um aumento significativo do volume de dados gerados e da complexidade das tarefas para processá-los, sendo necessário para isso, desenvolver processadores mais rápidos e eficientes. No entanto, o aumento da frequência dos processadores resulta em um maior consumo de energia e no aquecimento dos componentes. Para contornar esse problema, são utilizadas arquiteturas paralelas, devido à capacidade da utilização de múltiplos processadores para executar tarefas de forma colaborativa, melhorando o desempenho do sistema.

Dentre os modelos de arquitetura paralelas, os *clusters* podem ser utilizados para obter mais desempenho, necessário para o processamento de grandes volumes de dados. Os *clusters* funcionam por meio da união de vários computadores, trabalhando de forma colaborativa para executar tarefas de forma paralela. Para formar o conjunto, podem ser utilizados *desktops* convencionais, como também as plataformas *SBC* (*Single Board Computer*), que vêm ganhando destaque devido ao bom desempenho oferecido, mantendo um baixo custo e um consumo energético eficiente.

Atualmente, o Brasil possui um supercomputador chamado Santos Dumont, que foi desenvolvido em 2015 e está localizado no Laboratório Nacional de Computação Científica (LNCC), no Rio de Janeiro. Esse equipamento custou aproximadamente 50 milhões de reais e possui cerca de 1.134 nós com diferentes configurações, totalizando 36.472 núcleos de processamento, 212.976 *GBytes* de memória *RAM* e 2.340 *TBytes* de armazenamento. O supercomputador brasileiro foi atualizado duas vezes, em 2019 e, mais recentemente em 2024, permitindo que ele voltasse para o TOP500, uma lista dos supercomputadores mais rápidos do mundo. O Santos Dumont é importante para o desenvolvimento de pesquisas científicas no país, pois anteriormente os pesquisadores precisavam da cooperação de instituições de outros países para a realização dessas tarefas, prejudicando o desenvolvimento científico e tecnológico, bem como a realização de estudos mais avançados (LNCC, 2024).

Entretanto, utilizar uma máquina desse porte dentro de uma instituição de ensino e pesquisa, para a realização de estudos e pesquisas acadêmicas, possui um custo elevado, tanto na sua aquisição quanto para a manutenção. Além disso, depende de profissionais altamente qualificados, para a realização de suas operações e configurações. Por esse motivo, este trabalho teve por finalidade apresentar uma solução mais viável economicamente, para a realização de pesquisas cientificas no ambiente acadêmico.

Nesse contexto, foi realizada neste trabalho a implementação de um *cluster* de baixo custo utilizando para isso a plataforma *SBC Raspberry Pi* e a biblioteca de paralelismo *Apache Hadoop*, além da realização de testes de *benchmark* para avaliar o desempenho do sistema e a utilização dos recursos de *hardware*.

Para o desenvolvimento do *cluster*, foram utilizadas 9 (nove) *Raspberry Pi 4B*, sendo uma a nó mestre (*master*) do sistema e as demais máquinas escravas (*slaves*). A solução implementada custou o equivalente a R$ 6.431,60 (US$ 1,236,84). Quando comparada a um *cluster Beowulf*, formado por máquinas convencionais, a redução foi de aproximadamente 134%. Isso indica que a solução desenvolvida neste trabalho é ainda mais barata do que utilizar computadores convencionais (*desktops*) para implementar um *cluster*, permitindo que mais instituições de ensino possam utilizar esse tipo de máquina para o desenvolvimento de pesquisas, indicando a viabilidade econômica da solução.

O desempenho do *cluster* foi avaliado por meio da execução de três *benchmarks*: Cálculo do *Pi*, algoritmos "*Tera*" (*TeraGen*, *TeraSort* e *TeraValidate*) e *TestDFSIO* (*Write* e *Read*). Para analisar o impacto do escalonamento dos recursos computacionais do sistema, os testes foram realizados com três combinações de quantidade de nós (2, 4, 8). Além disso, foram utilizados diferentes tamanhos de arquivos nos testes (500 *MBytes*, 1 *GBytes*, 2 *GBytes*), permitindo realizar uma análise sobre o impacto causado pelo aumento da carga de trabalho no desempenho do sistema. Para coletar as informações dos dispositivos que compõem o *cluster* e exibi-las através de gráficos (*dashboards*), foram utilizadas as ferramentas *Zabbix* e *Grafana*.

Os resultados obtidos durante a execução dos testes indicam que, ao processar arquivos maiores ou executar tarefas mais complexas, o escalonamento do número de nós do *cluster* proporciona uma melhora significativa no desempenho, devido ao aumento da quantidade de recursos disponíveis para execução das tarefas. No entanto, foi observado que em algumas situações, o aumento do número de nós do *cluster* não resultou em ganhos no desempenho. Esse comportamento ocorreu quando a tarefa tinha uma granulosidade baixa, limitando assim, a eficiência do paralelismo realizado pela biblioteca. Dessa forma, é fundamental analisar a complexidade da tarefa que será executada, para saber se a disponibilização de mais recursos computacionais contribuirá para melhorar o desempenho do sistema.

A realização dos testes permitiu demonstrar o baixo consumo energético do *Raspberry Pi*, uma vez que o aumento no consumo de energia não foi proporcional ao escalonamento da quantidade de nós utilizados no *cluster*. Dessa forma, foi possível disponibilizar mais recursos para o sistema, sem causar um impacto significativo no consumo de energia, reforçando a

eficiência da plataforma.

É importante destacar que a biblioteca de paralelismo utilizada neste trabalho, apresenta características importantes, como escalabilidade e tolerância a falhas. A escalabilidade é representada pelos poucos ajustes que são necessários ao adicionar mais nós, possibilitando escalonar o *cluster* de forma simples. Já a tolerância a falhas, deve-se à capacidade do sistema de permanecer ativo, mesmo que os nós escravos apresentem problemas, bem como a replicação dos dados que são armazenados, proporcionando maior confiabilidade para o sistema. Essas características, bem como outras ferramentas disponíveis na biblioteca *Apache Hadoop*, permitem criar um sistema robusto e que pode ser adaptado a diversos contextos. Entretanto, existe um aspecto crítico relacionado à existência de apenas um nó mestre, criando, dessa forma, um ponto que pode comprometer o funcionamento do *cluster*.

Durante a realização dos testes foi observado que a biblioteca de paralelismo *Apache Hadoop*, apresentou um desbalanceamento na carga de trabalho atribuída aos nós do *cluster*, durante a execução dos algoritmos, sendo esse comportamento mais evidente em testes que realizaram o processamento de um baixo volume de dados, com arquivos menos complexos. Esse comportamento ocorreu porque o *Apache Hadoop*, na versão utilizada neste trabalho, não conseguiu realizar um balanceamento de carga eficiente e adequado no sistema, causando a sobrecarga de alguns nós, além da subutilização dos recursos de *hardware* disponíveis no *cluster*, comprometendo o desempenho do sistema.

Com base nos resultados apresentados, conclui-se que a *Raspberry Pi* oferece um bom desempenho a partir de um preço acessível e que representa uma alternativa viável para o desenvolvimento de *clusters* utilizados para tarefas de processamento distribuído. O baixo custo dessa plataforma permite implementar sistemas capazes de realizar o processamento de grandes volumes de dados em ambientes acadêmicos, sem a necessidade dos investimentos necessários para um supercomputador. Essa alternativa viabiliza o aumento da quantidade de pesquisas e estudos desenvolvidos nessa área, contribuindo para a utilização dessa plataforma em grandes empresas e centros tecnológicos.

Por fim, pesquisas futuras podem estudar configurações e ajustes adicionais para desenvolver estratégias capazes de resolver os problemas que foram identificados durante a execução dos testes, permitindo aprimorar o desempenho do *cluster* e otimizar a utilização dos recursos.

REFERÊNCIAS

ANDRADE, T. P. C. MapReduce: Conceitos e aplicações. **Relatório da Disciplina Arquitetura de Computadores. UNICAMP. Campinas**, 2012.

ANDREWS, G. R. **Foundations of Multithreaded, Parallel and Distributed Programming**. Addison-Wesley Longman Publishing Co., Inc., 1999.

ANDREWS, G. R. **Foundations of Multithreaded, Parallel, and Distributed Programming**. Reading: Addison-Wesley, 2001. 664p.

ALVES, N. D. M. **Uma abordagem não intrusiva e automática para configuração do Hadoop**. 2015.

APACHE HADOOP. **Apache Hadoop**. 2024. Disponível em: https://hadoop.apache.org/. Acesso em: 26 abr. 2024.

ARAVINTH, S. S.; BEGAM, A. H.; SHANMUGAPRIYAA, S.; SOWMYA, S.; ARUN, E. An efficient HADOOP frameworks SQOOP and ambari for big data processing. **International Journal for Innovative Research in Science and Technology**, v. 1, n. 10, p. 252-255, 2015.

ARIZA, J. A.; BAEZ, H. Understanding the role of single-board computers in engineering and computer science education: A systematic literature review. **Computer Applications in Engineering Education**, v. 30, n. 1, p. 304-329, 2022.

BACELLAR, H. V. **Cluster: Computação de alto desempenho. Instituto de Computação**, Universidade Estadual de Campinas. Campinas, São Paulo, 2009.

BANDI, R.; ANITHA, G. Machine learning based oozie workflow for hive query schedule mechanism. In: **2018 International Conference on Smart Systems and Inventive Technology (ICSSIT)**. IEEE, 2018. p. 513-517.

BARKER, B. Message passing interface (mpi). In: **Workshop: High Performance Computing on Stampede**. 2015.

BARNEY, B. **Message passing interface (mpi)**. 2021. Disponível em: https://computing.llnl.gov/tutorials/mpi/. Acesso em: 23 abr. 2024.

BASFORD, P. J.; JOHNSTON, S. J.; PERKINS, C. S.; GARNOCK-JONES, T.; TSO, F. P.; PEZAROS, D.; COX, S. J. Performance analysis of single board computer clusters. **Future Generation Computer Systems**, v. 102, p. 278-291, 2020.

BEAKTA, RAHUL. Big Data and Hadoop: A Review Paper. **International Journal of Computer Science & Information**, 2015.

BELENTANI, L.; MARCELLO, J.; FLORIAN, F. A utilização de ferramentas de monitoramento para a otimização do gerenciamento da rede. **Revista Interface Tecnológica**, v. 15, n. 2, p. 99–110, 2018.

BERNHOLDT, D. E. Component architectures in the next generation of ultrascale scientific computing: challenges and opportunities. In: **Compframe '07: Proceedings of The 2007 Symposium on Component and Framework Technology in High-Performance and Scientific Computing**, 2007, New York, NY, USA. Anais. . . ACM, 2007. p.1–10.

BHOSALE, H. S.; GADEKAR, D. P. A review paper on big data and hadoop. **International Journal of Scientific and Research Publications**, v. 4, n. 10, p. 1-7, 2014.

BORTHAKUR, D. The hadoop distributed file system: Architecture and design. **Hadoop Project Website**, v. 11, n. 2007, p. 21, 2007.

BÖTHER, M.; RABL, T. Scale-down experiments on TPCx-HS. In: **Proceedings of the International Workshop on Big Data in Emergent Distributed Environments**. 2021. p. 1-6.

BUENO, L.; WEINGARTNER, T. **Monitoramento de dispositivos em rede utilizando Zabbix e Grafana**. Instituto Federal Farroupilha Campus Panambi. Panambi, 2022.

CANELTECH. 2021. **Descubra quanta energia seu PC consome e como reduzir o consumo**. Disponível em: https://canaltech.com.br/desktop/descubra-quanta-energia-seu-pc-consome-e-como-reduzir-isso//. Acesso em: 28 out. 2024.

CHAVES, A.; VIEIRA, R. **Implantação de uma solução open source na gestão de uma rede de computadores em uma média empresa com as ferramentas Zabbix e Grafana**. Universidade Federal de Alagoas. Maceió, 2020.

COELHO, S. A. Introdução a computação paralela com o open mpi. **Simpósio Brasileiro de Computação Paralela com o Open MPI**, p. 24-44, 2012.

COSTA, R. A. G. **Desempenho e Consumo de Energia de Algoritmos Criptográficos do MiBench em Sistemas Móveis**. Universidade Estadual do Amazonas - Amazonas. 2007.

DE FREITAS MUSSI, R. F.; MUSSI, L. M. P. T.; ASSUNÇÃO, E. T. C.; NUNES, C. P. Pesquisa Quantitativa e/ou Qualitativa: distanciamentos, aproximações e possibilidades. **Revista Sustinere**, v. 7, n. 2, p. 414-430, 2019.

DONGARRA, J. J.; DUFF, I. S.; SORENSEN, D. C.; VAN DER VORST, H. A. **Solving Linear Systems on Vector and Shared Memory Computers**. Philadelphia, PA: SIAM, 1991.

EADLINE, Douglas. **Hadoop 2 Quick-Start Guide: Learn the Essentials of Big Data Computing in the Apache Hadoop 2 Ecosystem**. Addison-Wesley Professional, 2015.

GEIST, A., BEGUELIN, A., DONGARRA, J., JIANG, W., MANCHEK, R., SUNDERMAN, V. **PVM: Parallel Virutal Machine - A Users' Guide and Tutorial for Networked Parallel Computing**. MIT Press, Cambridge. 1994.

GEPNER, P.; KOWALIK, M. F. Multi-core processors: New way to achieve high system performance. In: **International Symposium on Parallel Computing in Electrical Engineering (PARELEC'06)**. IEEE, 2006. p. 9-13.

GIGER, P.; SRIKUGAN, S.; PERSAUD, B. L. A raspberry pi cluster for teaching big-data analytics. **Master's thesis, Universitat Zurich**, 2020.

GOLDMAN, A.; KON, F.; JÚNIOR, F. P.; POLATO, I.; FÁTIMA, P. R. Apache Hadoop: conceitos teóricos e práticos, evolução e novas possibilidades. **XXXI Jornadas de atualizações em informática**, p. 88-136, 2012.

HANNAN, S. A. An overview on big data and hadoop. **International Journal of Computer Applications**, v. 154, n. 10, 2016.

HENNESY, JOHN L.; PATTERSON, D. A. **Computer Architecture**. Morgan Kaufamann, 5ª edição. 2012.

HULLEY, S. B.; CUMMINGS, S. R.; BROWNER, W. S.; GRADY, D. G.; NEWMAN, T. B. **Delineando a pesquisa clínica-4**. Artmed Editora. 2015.

IGNÁCIO, A. A. V.; FERREIRA, F.; VIRGÍLIO, J. M. MPI: uma ferramenta para implementação paralela. **Pesquisa Operacional**, v. 22, p. 105-116, 2002.

ISHWARAPPA; ANURADHA, J. A brief introduction on big data 5Vs characteristics and Hadoop technology. **Procedia computer science**, v. 48, p. 319–324, 2015.

JUNIOR, R. M. C. **Avaliação de desempenho de aplicações hadoop em nuvens computacionais**. 2023.

KALAPOTHAS, S.; GALETAKIS, M.; FLAMIS, G.; PLESSAS, F.; KITSOS, P. A survey on risc-v-based machine learning ecosystem. **Information**, v. 14, n. 2, p. 64, 2023.

KOMNINOS, A.; SIMOU, I.; GKORGKOLIS, N.; GAROFALAKIS, J. Performance of raspberry pi microclusters for edge machine learning in tourism. **Network (Mbps)**, v. 100, n. 100, p. 100, 2019.

LEE, E.; OH, H.; PARK, D. Big data processing on single board computer clusters: Exploring challenges and possibilities. **IEEE Access**, v. 9, p. 142551-142565, 2021.

LIMA, F. D. A; MORENO, E. D.; DIAS, W. R. A. Design and Performance of a Low Cost Cluster using ARM-based Platform. In: **Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA)**. The Steering Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp), p. 175. 2016.

LNCC. Laboratório Nacional de Computação Científica. **Atualização do Supercomputador Santos Dumont.** 2024. Disponível em: https://www.gov.br/lncc/pt-br/assuntos/noticias/ultimas-noticias-1/atualizacao-do-supercomputador-santos-dumont. Acesso em: 10 jan. 2025.

LOPES, O. U. Pesquisa básica versus pesquisa aplicada. **Estudos avançados**, v. 5, p. 219-221, 1991.

LORENZON, A. F. **Avaliação do desempenho e consumo energético de diferentes interfaces de programação paralela em sistemas embarcados e de propósito geral**. 2014.

MA, C.; ZHAO, M.; ZHAO, Y. An overview of Hadoop applications in transportation big data. **Journal of traffic and transportation engineering (English edition)**, 2023.

MARCONDES, M. M.; DE ARAÚJO, M. A. D.; DE SOUZA, W. J.; DA SILVA, M. G. K. Observatórios sociais e desigualdades no Brasil: Uma análise exploratória e descritiva. **Cadernos Gestão Pública e Cidadania**, v. 27, n. 86, p. 1-18, 2022.

MATHUR, N.; JAIN, M. **A Distribution of Nodes in Big Data using Hadoop Open Source System**, 2020.

MEHTA, S.; MEHTA, V. Hadoop ecosystem: An introduction. **International Journal of Science and Research (IJSR)**, v. 5, n. 6, p. 557-562, 2016.

MENEZES, S. L.; FREITAS, R. S.; PARPINELLI, R. S. Mineraçao em grandes massas de dados utilizando hadoop mapreduce e algoritmos bio-inspirados: Uma revisão sistemática. **Revista de Informática Teórica e Aplicada**, v. 23, n. 1, p. 69-101, 2016.

MOORE, G. E. Cramming more Components onto Integrated Circuits. In: **Reprinted from Electronics**, 38(8):114, April 1965.

MORETTI, C. O.; BITTENCOURT, T. N. **Aspectos Gerais da Computação Paralela e do Sistema PVM**. 1997.

MPI. **Open MPI: Open Source High Performance Computing**. 2024. Disponível em: http://www.openmpi.org/. Acesso em: 18 jun. 2024.

NASCIMENTO, E. B.; SOUZA, M. Silva. **Cluster e Balanceamento de Carga:** Sistemas essenciais de alta disponibilidade. 2020.

NAVAUX, P.; DE ROSE, C.; PILLA, L. Fundamentos das arquiteturas para processamento paralelo e distribuído. **XI Escola Regional de Alto Desempenho do Estado do Rio Grande do Sul-2011-Porto Alegre, RS**, p. 22-59, 2011.

NASIR, M.; MUHAMMAD, K.; LLORET, J.; SANGAIAH, A. K.; SAJJAD, M. Fog computing enabled cost-effective distributed summarization of surveillance videos for smart cities. **Journal of Parallel and Distributed Computing**, v. 126, p. 161-170, 2019.

NETO, Antônio José Alves. **Desenvolvimento e avaliação de desempenho de um cluster Raspberry Pi e Apache Hadoop em aplicações big data**. 2023.

NETO, A. J. A.; NETO, J. A. C.; MORENO, E. D. The development of a low-cost big data cluster using Apache Hadoop and Raspberry Pi. A complete guide. **Computers and Electrical Engineering**, v. 104, p. 108403, 2022.

NUGROHO, S.; WIDIYANTO, A. Designing parallel computing using raspberry pi clusters for IoT servers on apache Hadoop. In: **Journal of Physics: Conference Series**. IOP Publishing, 2020. p. 012070.

OPENMP. **The OpenMP API Specification for Parallel Programming**. 2024. Disponível em: http://openmp.org/. Acesso em: 17 jun. 2024.

O'MALLEY, O. **Terabyte sort on apache hadoop**. 2008.

PAPAKYRIAKOU, Dimitrios. Benchmarking Raspberry Pi 2 Hadoop Cluster. **International Journal of Computer Applications**, v. 975, p. 8887, 2019.

PENCHALAIAH, N.; AL-HUMAIMEEDY, A. S.; MAASHI, M.; BABU, J. C.; KHALAF, O. I.; ALDHYANI, T. H. Clustered Single-Board Devices with Docker Container Big Stream Processing Architecture. **Computers, Materials & Continua**, v. 73, n. 3, 2022.

PERISSATTO, M. G.; JÚNIOR, N. A. G.; GONÇALVES, R. A.; MARTINI, J. A. **Ferramenta para Simulação de Multiprocessadores Superescalares de Memória Compartilhada**. 2007.

PITANGA, M. **Computação em cluster: o estado da arte da computação**. Brasport, 2004.

POLATO, I.; RÉ, R.; GOLDMAN, A.; KON, F. A comprehensive view of Hadoop research — A systematic literature review. **Journal of Network and Computer Applications**, v. 46, p. 1-25, 2014.

QURESHI, B.; JAVED, Y.; KOUBÂA, A.; SRITI, M. F.; ALAJLAN, M. Performance of a low cost hadoop cluster for image analysis in cloud robotics environment. **Procedia Computer Science**, v. 82, p. 90-98, 2016.

QURESHI, B.; KOUBÂA, A. On energy efficiency and performance evaluation of single board computer based clusters: A hadoop case study. **Electronics**, v. 8, n. 2, p. 182, 2019.

QURESHI, B. Optimizing Hadoop Scheduling in Single-Board-Computer-Based Heterogeneous Clusters. **Computation**, v. 12, n. 5, p. 96, 2024.

RATHEE, S. Big data and Hadoop with components like Flume, Pig, Hive and Jaql. In: **International conference on cloud, big data and trust**, 2013.

RAUPP, F. M.; BEUREN, I. M. Metodologia da pesquisa aplicável às ciências. **Como elaborar trabalhos monográficos em contabilidade: teoria e prática**. São Paulo: Atlas, p. 76-97, 2006.

RASPBERRY PI FOUNDATION. **Raspberry Pi** Foundation. 2024. Disponível em: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/. Acesso em: 26 out. 2024.

REBONATTO, M. T. Introdução à programação paralela com MPI em agregados de computadores (clusters). In: **Congresso Brasileiro de Ciência da Computação**, Itajaí. 2004. p. 938-955.

RIBEIRO, N. S.; FERNANDES, L. G. L. **Programação Híbrida**: MPI e OpenMP. 2013.

RODRIGUES, L. M. **Proposta e Avaliação do Algoritmo K-Means Paralelo e Distribuído Para Predição do Sítio de Início de Tradução em RNA Mensageiro Para Identificação de Proteínas**. 94 p. Dissertação (Mestrado) — Pontifícia Universidade Católica de Minas Gerais

(PUC), Belo Horizonte, MG, 2012. Disponível em: http://homepages.dcc.ufmg.br/~laerte/public/docs/dissertacao.pdf. Acesso em: 20 jun. 2024.

SANTOS, A. P. C. Criar um Cluster de Processamento Paralelo MPI com Raspberrys. **Revista Programar**. 2015. Disponível em: https://www.revista-programar.info/artigos/criar-um-cluster-de-processamento-paralelo-mpi-com-raspberrys/. Acesso em: 17 mar. 2024.
SANTOS, F. H. **Revisão Bibliográfica Sobre Arquiteturas Para o Processamento Paralelo e Computação em Cluster**. 2013.

SCHEPKE, C. **Ambientes da programação paralela**. Trabalho Individual I, Universidade Federal do Rio Grande do Sul/PPGC. 2009.

SHARMA, N. **To Perform and Join Cluster High Availability Using Load Balancing Approach**. 2020.

SHVACHKO, K.; KUANG, H.; RADIA, S.; CHANSLER, R. (2010, May). The hadoop distributed file system. **In 2010 IEEE 26th symposium on mass storage systems and technologies** (MSST). IEEE, 2010. p. 1-10.

SCOLATI, R.; FRONZA, I.; EL IOINI, N.; SAMIR, A.; PAHL, C. A Containerized Big Data Streaming Architecture for Edge Cloud Computing on Clustered Single-board Devices. In: **Closer**. 2019. p. 68-80.

SCOLATI, R.; FRONZA, I.; EL IOINI, N.; SAMIR, A.; PAHL, C. A containerized edge cloud architecture for data stream processing. In: **Cloud Computing and Services Science: 9th International Conference, CLOSER 2019, Heraklion, Crete, Greece, May 2–4, 2019, Revised Selected Papers 9**. Springer International Publishing, 2020. p. 150-176.

SOARES, F. A. L.; NOBRE, C. N.; DE FREITAS, H. C. Parallel programming in computing undergraduate courses: a systematic mapping of the literature. In: **IEEE Latin America Transactions**, v. 17, n. 08, p. 1371-1381, 2019.

SILVA, W.; SILVA, R. Implementação de monitoramento de rede de dados com Zabbix e Grafana. **P2P E INOVAÇÃO**, v. 10, n. 2, p. e-6904, 2024.

SRINIVASAN, K.; CHANG, C. Y.; HUANG, C. H.; CHANG, M. H.; SHARMA, A.; ANKUR, A. An efficient implementation of mobile raspberry Pi hadoop clusters for robust and augmented computing performance. **Journal of Information Processing Systems**, v. 14, n. 4, p. 989-1009, 2018.

TAMEHIRO, L. K.; TAKAHASHI, M. S.; SANTOS, E. F. **Reaproveitamento de Comutadores através de Máquinas Virtuais Paralelas (PVM)**. São Paulo, 2012.

TPLINK. **Roteador TPLINK**. 2024. Disponível em: https://www.tp-link.com/br/business-networking/unmanaged-switch/tl-sg1016/. Acesso em: 25 mar. 2024.

UZUNKAYA, C.; ENSARI, T.; KAVURUCU, Y. Hadoop ecosystem and its analysis on tweets. **Procedia-Social and Behavioral Sciences**, v. 195, p. 1890-1897, 2015.

VASCONCELOS, P. R. M. **Avaliação de desempenho de virtualizações OpenVZ e KVM em nuvens computacionais usando Hadoop MapReduce**. 2015.

VASUJA, R.; BHANDRALIA, A.; CHUCHRA, K. Daemons of Hadoop: An Overview. **International Journal of Engineering Research & Technology (IJERT)**, v. 7, 2018.

WHITE, T. **Hadoop The Definitive Guide**. 4. ed. O'Reilly Media, 2015.

YADAV, D.; CHANDRA, U. Modern Technologies of Big Data Analytics: Case study on Hadoop Platform. **International Journal of Emerging Trends & Technology in Computer Science (IJETTCS)**, v. 6, p. 044-050, 2019.

ZOMAYA, A. Y.; LEE, Y. C. **Energy-efficient distributed computing systems**. 2012.

APÊNDICE A

1. ESPECIFICAÇÕES TÉCNICAS DO *CLUSTER*

Nesta seção será apresentada a infraestrutura de *hardware* utilizada na implementação do *cluster*, bem como todas as especificações técnicas relacionadas a essa infraestrutura.

1.1 *Hardware*

Para a solução de *cluster* implementada neste trabalho foram utilizados os seguintes equipamentos:

- 09 *Raspberry Pi 4 Model B de* 4 *GB* de memória *RAM*;
- 01 *Switch Gigabit Ethernet* de 16 Portas - *TL-SG1016D* - *TP Link* – 110/220V;
- 09 Fontes de alimentação com tensão de saída de 5,1V *DC* de 3,0*A* para as *Raspberry Pi* 4 *Model B*;
- 01 Teclado genérico com fio *USB* modelo ABNT2 (Associação Brasileira de Normas Técnicas);
- 01 Mouse com fio *USB* 12 V;
- 01 Cabo *HDMI* de 1,8 metros 4K (Alta Resolução);
- 01 Adaptador de Áudio e Vídeo *HDMI* Fêmea para Mini *HDMI* Macho;
- 10 Cabos *UTP CAT. 6* (padrão *Gigabit Ethernet*) de 2,5 metros (*Patch Cord*);
- 09 Cartões de memória *SanDisk Ultra microSDXC UHS-I Card* com Adaptador – 128 *GB – Speed up to* 120 *MB/s*;
- 03 *Nobreak UPS* Mini 600 Bivolt *Black*, com 06 tomadas (110/220V) cada, com comprimento do cabo de entrada de 1,5 metros.

Na Tabela 48, observa-se o valor total da solução de *cluster* implementada neste trabalho.

Tabela 48 – Valores dos equipamentos utilizados na implementação do *cluster*

Equipamentos	Quantidade	Unitário (R$)	Unitário (*US$*)	Total em (R$)	Total em (*US$*)
Raspberry Pi 4 Model B de 4 GB de memória RAM	09	286,00	55.00	2.574,00	495.00
Switch Gigabit Ethernet de 16 Portas - TL-SG1016D - TP Link – 110/220V	01	806,00	155.00	806,00	155.00
Fonte de alimentação com tensão de saída de 5,1V DC de 3,0A	09	104,00	20.00	936,00	180.00
Teclado genérico com fio USB modelo ABNT2	01	31,20	6.00	31,20	6.00
Mouse com fio USB 12 V	01	26,00	5.00	26,00	5.00
Cabo HDMI de 1,8 metros 4K	01	20,80	4.00	20,80	4.00
Adaptador de Áudio e Vídeo HDMI Fêmea para Mini HDMI Macho	01	15,60	3.00	15,60	3.00
Cabo UTP CAT. 6 (Gigabit Ethernet) de 2,5 metros	10	15,60	3.00	156,00	30.00
Cartão de memória SanDisk Ultra microSDXC UHS-I Card com Adaptador – 128GB – Speed up to 120 MB/s	09	104,00	20.00	936,00	180.00
Nobreak UPS Mini 600 Bivolt *Black*, com 06 tomadas (110/220V).	03	310,00	59,61	930,00	178,84
	Total			**6.431,60**	**1,236.84**

Fonte: Elaborada pelos Autores, 2025.

Cabe ressaltar, que os valores utilizados na Tabela 47 são referentes à cotação do dólar (*US$*) do dia em que os equipamentos foram adquiridos, ou seja, um dólar valia R$ 5,20 nesse dia. Portanto, podem variar para mais ou para menos, a depender da cotação do dólar (*US$*) em relação ao real (R$) atual.

A Tabela 49 mostra a capacidade nominal computacional do *cluster* implementado, de acordo com os equipamentos utilizados, que é de aproximadamente 36 *GBytes* de memória *RAM*, de 1.2 *TBytes* de espaço em disco e de 13,5 *GHz* de processamento.

Tabela 49 – Capacidade computacional nominal do *cluster*

Equipamentos	Quantidade	*RAM* (*GB*)	Processador (*GHz*)	Disco (*GB*)	*RAM* Total (*GB*)	Processador Total (*GHz*)	Disco Total (*TB*)
Raspberry Pi 4 Model B	09	4	1,5	128	36	13,5	1,2

Fonte: Elaborada pelos Autores, 2025.

As *Raspberry Pi 4 Model B*, utilizadas na implementação do *cluster*, foram todas de 4 *GB* de memória *RAM*, com as seguintes especificações técnicas (RASPBERRY PI FOUNDATION, 2024):

- Processador *Broadcom 2711 Quad Core Cortex-A72 64-bit SoC @ 1,5* GHz;
- 4 GB de memória *RAM - LPDDR4*;
- *WiFI 2,4* GHz */ 5,0* GHz *IEEE 802.11.b/g/n/ac*;
- *Bluetooth 5.0*;
- Portas *USB 2.0* (duas);
- Portas *USB 3.0* (duas);
- *True Gigabit Ethernet over USB 3.0*;
- *GPIO* com 40 pinos;
- 2 portas micro *HDMI*, vídeo de 4K;
- Interface para *display* (*DSI – Display Serial Interface*) de alta velocidade, chamada pela indústria de *MIPI DSI* (*mobile industry processor interface display serial interface*);
- Interface para câmera (*CSI - Camera Serial Interface*);
- Conector *P2* para saída de áudio e vídeo;
- *Slot* para cartão micro *SD*;
- Alimentação 5,1V / 3A via conector *USB* tipo C;
- Dimensões: 88 x 58 x 19,5 mm.

Para a interconexão dos equipamentos utilizados na implementação e monitoramento do *cluster* (*master* e *slaves*), bem como na interconexão com a Internet, foi utilizado um *Switch Gigabit Ethernet* de 16 Portas – Modelo *TL-SG1016D*, da marca *TP Link*, com as seguintes especificações técnicas (TPLINK, 2024):

- 16 portas 10/100/1000 *Mbps*;
- Tecnologia eficiente e inovadora com baixo consumo de energia;
- Suporte a auto-aprendizagem de *MAC address*;
- Auto *MDI* / *MDIX* ;
- Autonegociação;
- *Plug & Play*.

Os cabos de rede (*patch cords*) utilizados para a interconexão dos equipamentos do *cluster* com o *Switch* foram todos do modelo *Patch Cord U/UTP Gigalan CAT.6* (de 2,5 metros), da fabricante FURUKAWA.

O modelo das fontes de alimentação utilizadas nas *Raspberry Pi 4 Model B* do *cluster (master* e *slaves)* segue as seguintes especificações técnicas:

- Modelo *USB-C*;
- Potência de 15*Watts*
- 5,1V *DC* / 3,0*A*
- 110 / 220 *AC*;
- Faixa de entrada operacional de 96-264*Vac*;
- Proteção contra curto-circuito, corrente elevada e temperatura elevada;
- Cabo de 1,5 metros 18 *AWG*, com conector de saída *USB-C*;

Para o armazenamento dos *softwares* utilizados na instalação e configuração do *cluster*, bem como dos algoritmos de *benchmark* utilizados nos testes e dos resultados das métricas obtidas na avaliação de desempenho do *cluster*, foram utilizados cartões de memória no modelo *SanDisk Ultra microSDXC UHS-I Card* de 128 GB, Classe 10, com velocidade de 120 *MB/s*.

O modelo do adaptador de áudio e vídeo, utilizado para a conexão do cabo *HDMI* com a interface (porta) *mini HDMI*, nativa da *Raspberry Pi 4* foi o *HDMI* Fêmea para Mini *HDMI* Macho.

O cabo *HDMI* utilizado para a conexão de áudio e vídeo nas *Raspberry Pi* possui as seguintes características: conectores banhados a ouro e compatibilidade com *3D* (*4K*).

No caso do teclado, o modelo utilizado foi o teclado *USB* genérico, com fio, modelo ABNT2, da fabricante MULTILASER. O *mouse*, por sua vez, foi um modelo *USB* genérico, com fio, da fabricante MULTILASER.

Para alimentar todos os equipamentos do *cluster*, foram utilizados 03 (três) *Nobreaks UPS Mini* 600 Bivolt *Black* (110/220V), de seis tomadas cada.

Para a configuração dos equipamentos do *cluster* foi utilizado um *Notebook* Positivo *Pentium modelo B950* de 2.10 *GHz*, com 6 *GB* de memória *RAM*.

Na Tabela 50, observa-se, a título de comparação, o valor de um *cluster* montado com máquinas *desktop*, ou seja, um *cluster Beowulf*, com as mesmas configurações e quantidade de máquinas do *cluster* implementado neste trabalho, utilizando as *Raspberry Pi*. Nela é possível observar que o valor de um *cluster* formado por placas *SBC*, como a *Raspberry Pi* custa, praticamente, três vezes menos, ou seja, um economia de aproximadamente R$ 8.642,40 (134%), tornando economicamente viável a solução implementa neste trabalho. O valor do dólar utilizado na conversão dos valores apresentados na tabela, tem como referência o preço do dólar praticado no dia da compra dos equipamentos do *cluster* implementado por *Raspberry Pi*, de R$ 5,20.

Tabela 50 – Valores dos equipamentos de um *cluster beowulf*

Equipamentos	Quantidade	Unitário (R$)	Unitário (*US$*)	Total em (R$)	Total em (*US$*)
Placa Mãe Gigabyte H410m H	9	462,80	89.00	4.165,20	801.00
Processador Intel Core I3-10100 - 3.60GHz	9	504,40	97.00	4.539,60	873.00
Memória RAM G.SKILL - 4GB - 2666MHz - DDR4	9	88,40	17.00	795,60	153.00
SSD KingSpec - NVMe - 128GB	9	83,20	16.00	748,80	144.00
Fonte RAIDMAX XT Series RX-400XT - 400W	9	208,00	40.00	1.872,00	360.00
Gabinete Fortrek SC501BK	9	109,20	21.00	982,80	189.00
Switch Gigabit Ethernet de 16 Portas - TL-SG1016D - TP Link – 110/220V	01	806,00	155.00	806,00	155.00
Teclado genérico com fio USB modelo ABNT2	01	31,20	6.00	31,20	6.00
Mouse com fio USB 12 V	01	26,00	5.00	26,00	5.00
Cabo HDMI de 1,8 metros 4K	01	20,80	4.00	20,80	4.00
Cabo UTP CAT. 6 (Gigabit Ethernet) de 2,5 metros	10	15,60	3.00	156,00	30.00
Nobreak UPS Mini 600 Bivolt *Black*, com 06 tomadas (110/220V).	03	310,00	59.61	930,00	178.84
Total				**15.074,00**	**2,898.84**

Fonte: Elaborada pelos Autores, 2025.

1.2 *Software*

Neste projeto, as *Raspberry Pi* receberam a instalação do sistema operacional oficial da *Rapsberry Foundation*, o *Raspberry Pi OS*, antigo *Raspbian*, baseado na distribuição *Debian* do *Linux*, distribuição *Desktop* para *PC* e *Mac*, com interface gráfica (*GUI*), disponível para para *Windows*, *MacOs* e *Ubuntu*.

Além do sistema operacional, outros *softwares* utilizados nas *Raspberry Pi* deste projeto foram: o *Apache Hadoop*, versão *3.4.0* e o *JAVA* 11.0.

Para efetuar o gerenciamento e monitoramento do *cluster*, foram instalados, na *Rapsberry Pi master* (*server* do *cluster*) as seguintes ferramentas: *Zabbix Server*, versão 6.0-5; e o *Grafana*, versão 11.0.0, utilizado para visualização de *dashboards* customizados com informações das métricas do *cluster*. Nas *Raspberry Pi Slaves* do *cluster*, foram instaladas uma versão *client* do *Zabbix* (6.0-0), para a coleta de informações.

APÊNDICE B

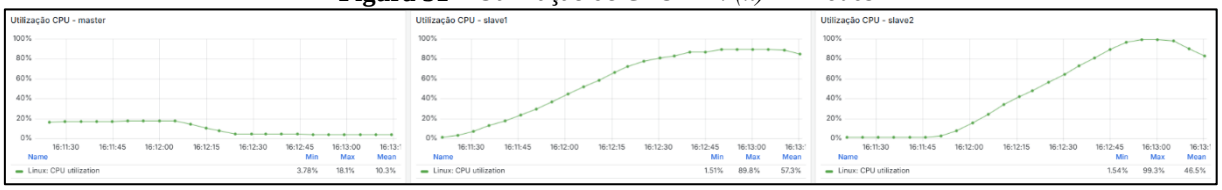
1. *DASHBOARDS* DE DESEMPENHO DO *CLUSTER*

Nesta seção serão apresentados os *dashboards* com as métricas de desempenho dos *cluster* coletadas durante a execução dos testes envolvendo os algoritmos de *benchmark*.

1.1 Algoritmo *Pi (π) - quasi-Monte Carlo (qMC)*

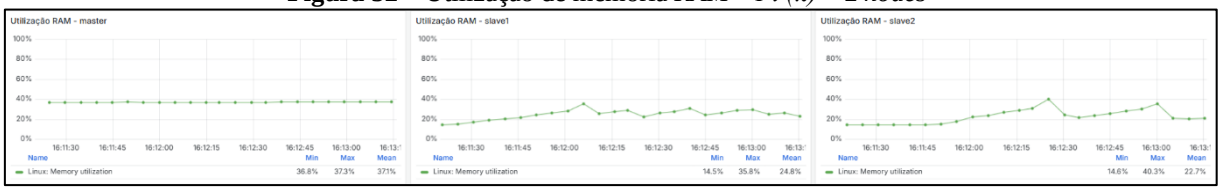
1.1.1 *Cluster* com 2 *nodes*

Figura 51 – Utilização de *CPU – Pi (π)* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 52 – Utilização de memória *RAM – Pi (π)* – 2 *nodes*

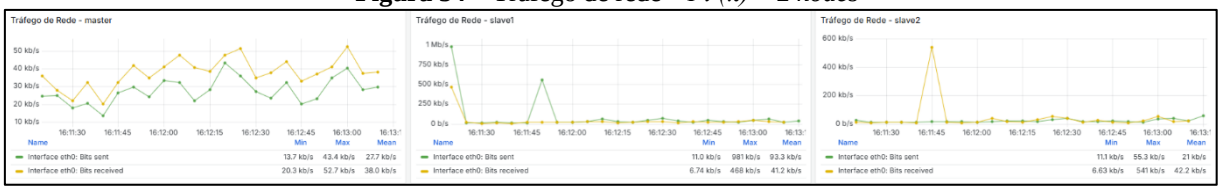


Fonte: Elaborada pelos Autores, 2025.

Figura 53 – Quantidade de processos – *Pi (π)* – 2 *nodes*

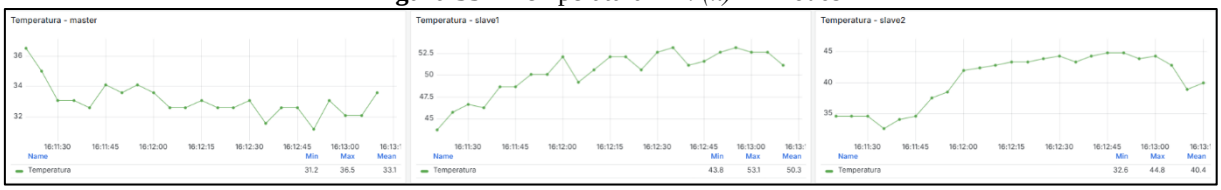
Fonte: Elaborada pelos Autores, 2025.

Figura 54 – Tráfego de rede – *Pi (π)* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 55 – Temperatura – *Pi (π)* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.1.2 *Cluster* com 4 *nodes*

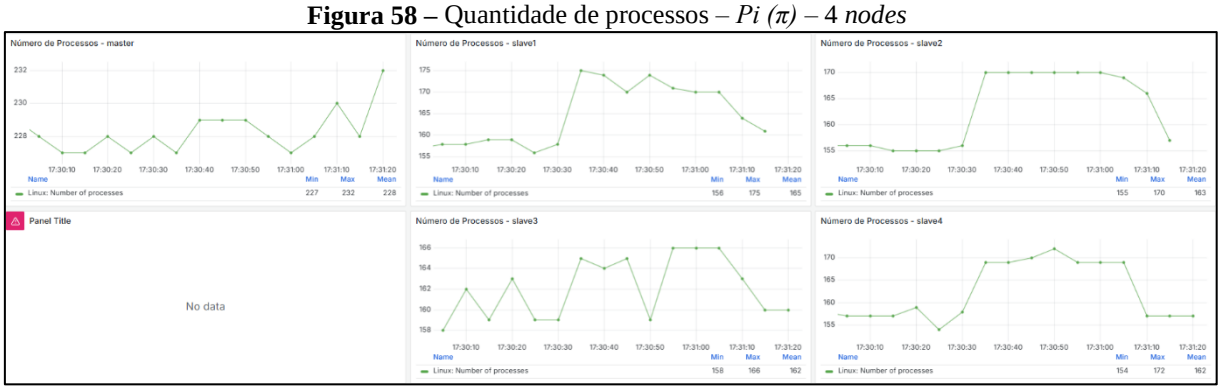
Figura 56 – Utilização de *CPU* – *Pi (π)* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 57 – Utilização de memória *RAM* – *Pi (π)* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 58 – Quantidade de processos – *Pi (π)* – 4 *nodes*

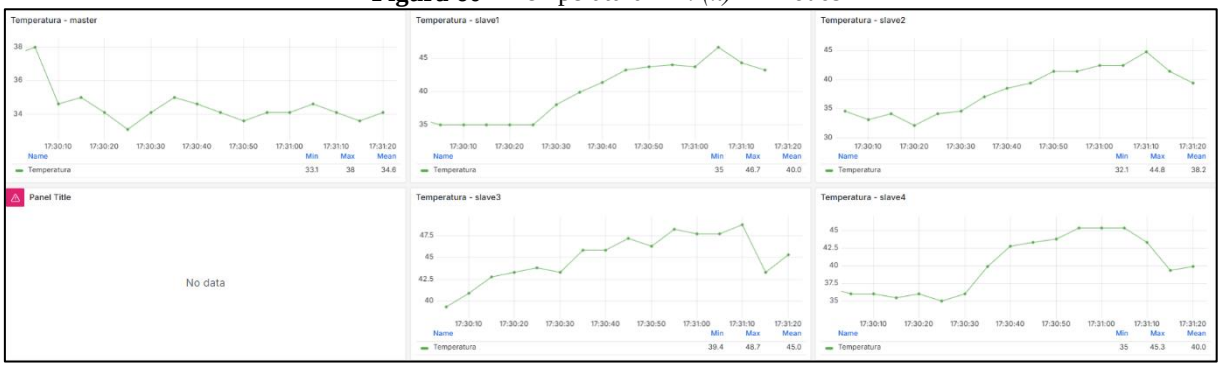


Fonte: Elaborada pelos Autores, 2025.

Figura 59 – Tráfego de rede – *Pi (π)* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

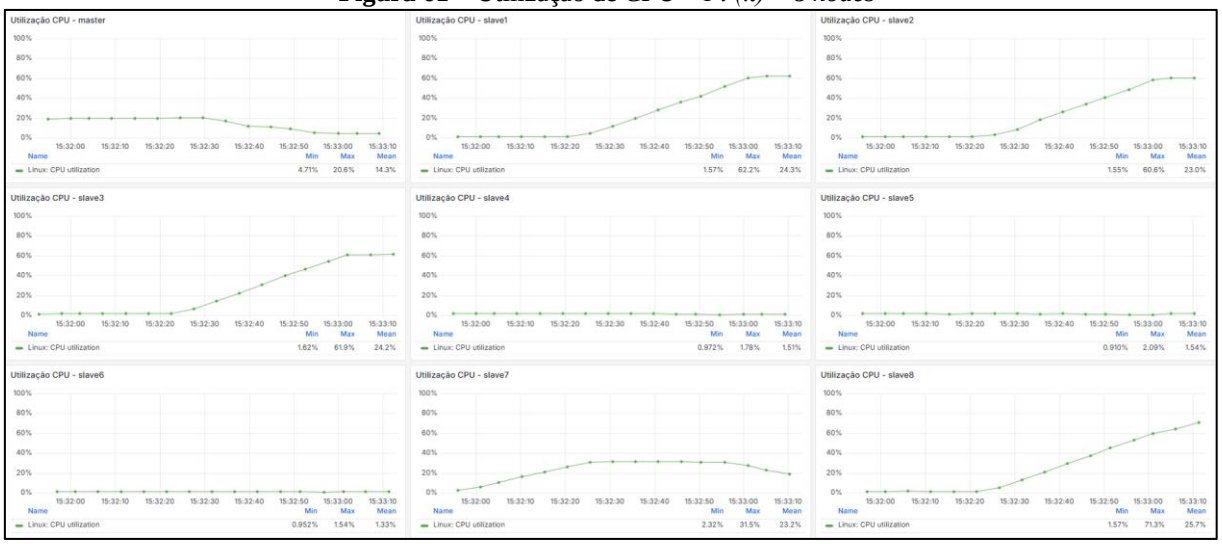
Figura 60 – Temperatura – *Pi (π)* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

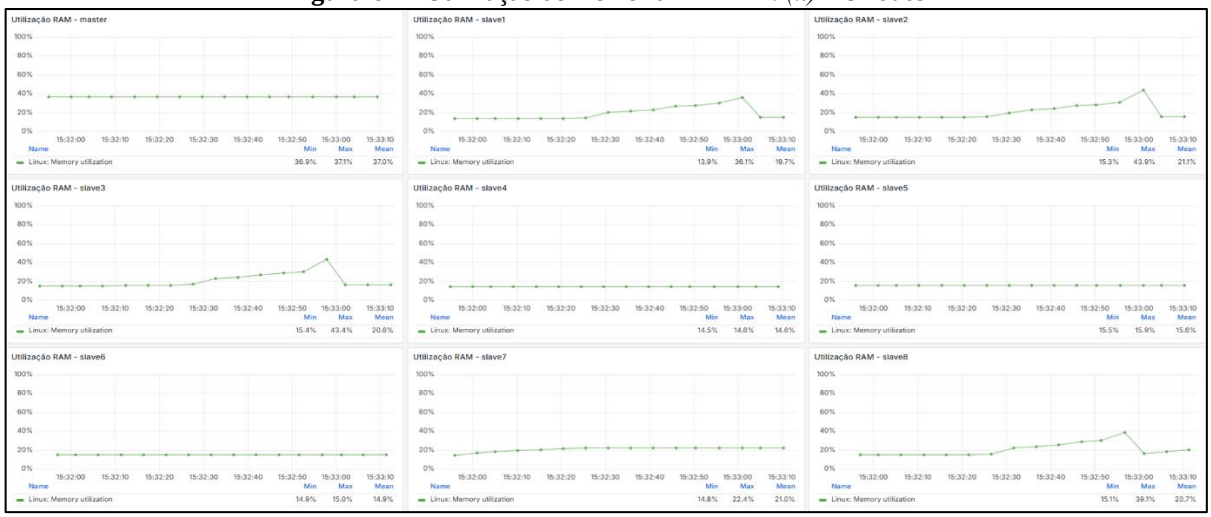
1.1.3 *Cluster* com 8 *nodes*

Figura 61 – Utilização de *CPU* – *Pi (π)* – 8 *nodes*



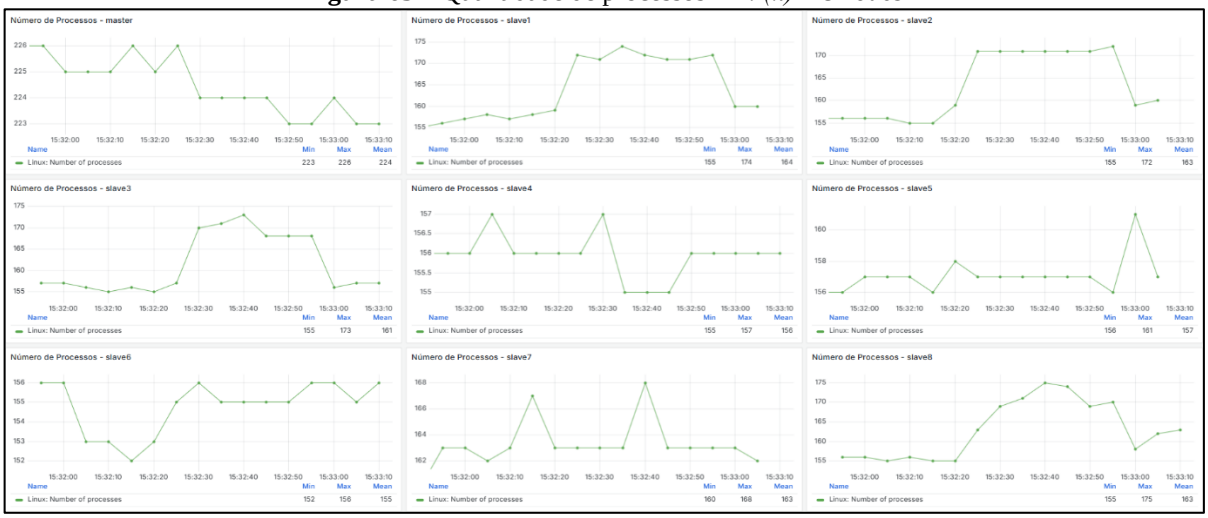
Fonte: Elaborada pelos Autores, 2025.

Figura 62 – Utilização de memória *RAM – Pi (π) –* 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 63 – Quantidade de processos *– Pi (π) –* 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 64 – Tráfego de rede *– Pi (π) –* 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

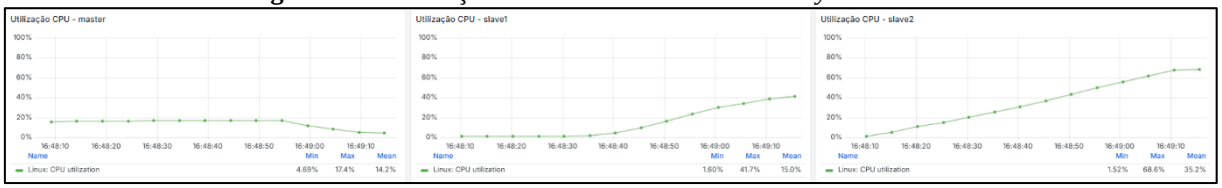
Figura 65 – Temperatura – *Pi (π)* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.2 Algoritmo *TeraGen*

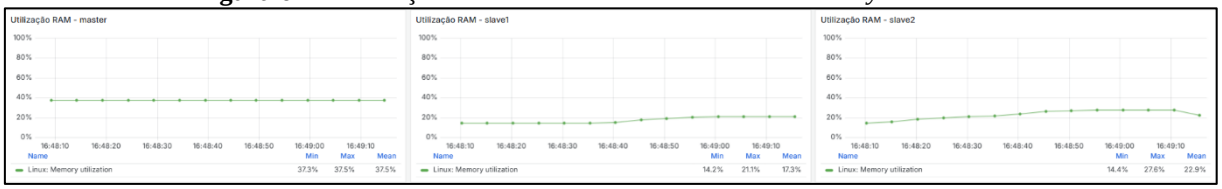
1.2.1 *Cluster* com 2 *nodes* – 500 *MBytes*

Figura 66 – Utilização de *CPU* – *TeraGen* – 500 *Mbytes* – 2 *nodes*



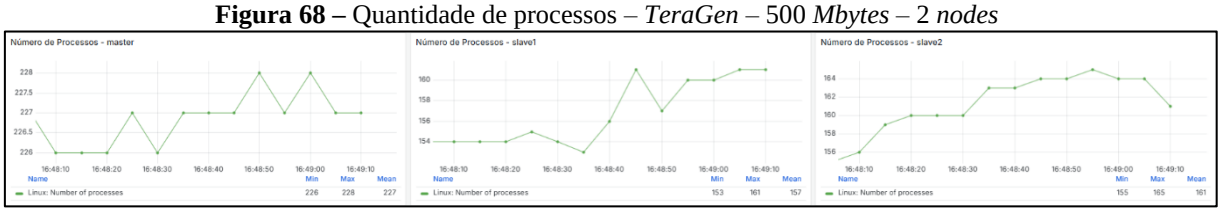
Fonte: Elaborada pelos Autores, 2025.

Figura 67 – Utilização de memória *RAM* – *TeraGen* – 500 *Mbytes* – 2 *nodes*



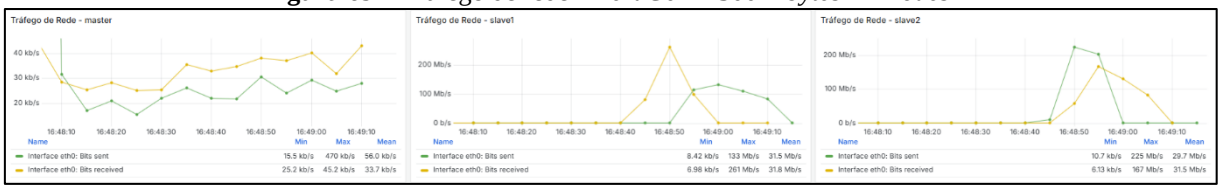
Fonte: Elaborada pelos Autores, 2025.

Figura 68 – Quantidade de processos – *TeraGen* – 500 *Mbytes* – 2 *nodes*



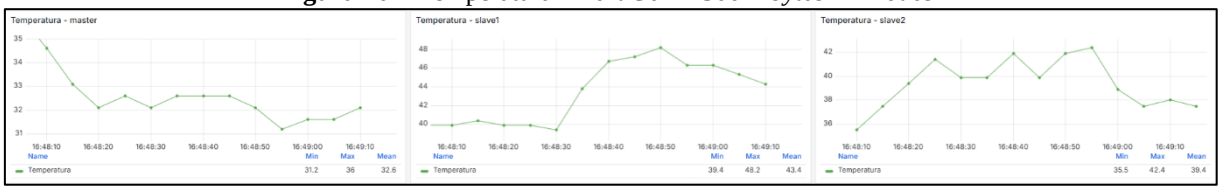
Fonte: Elaborada pelos Autores, 2025.

Figura 69 – Tráfego de rede – *TeraGen* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 70 – Temperatura – *TeraGen* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.2.2 *Cluster* com 4 *nodes* – 500 *MBytes*

Figura 71 – Utilização de *CPU* – *TeraGen* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 72 – Utilização de memória *RAM* – *TeraGen* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

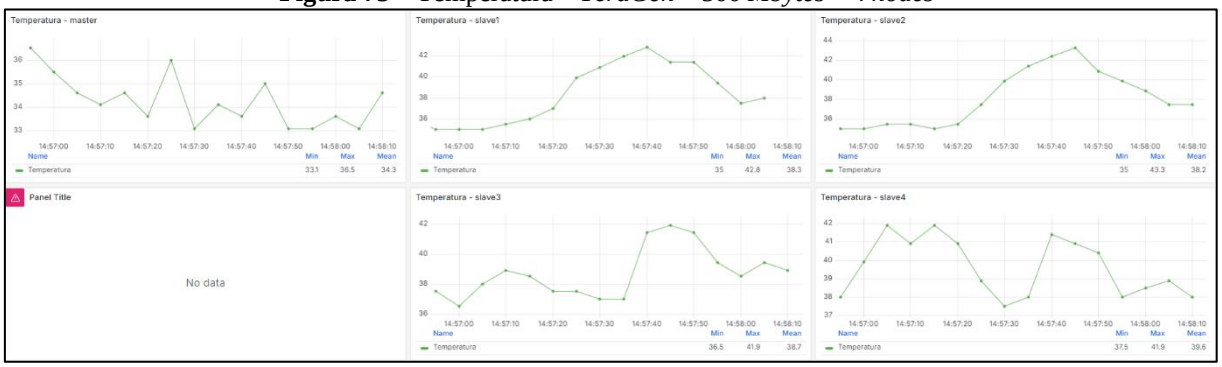
Figura 73 – Quantidade de processos – *TeraGen* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 74 – Tráfego de rede – *TeraGen* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 75 – Temperatura – *TeraGen* – 500 *Mbytes* – 4 *nodes*



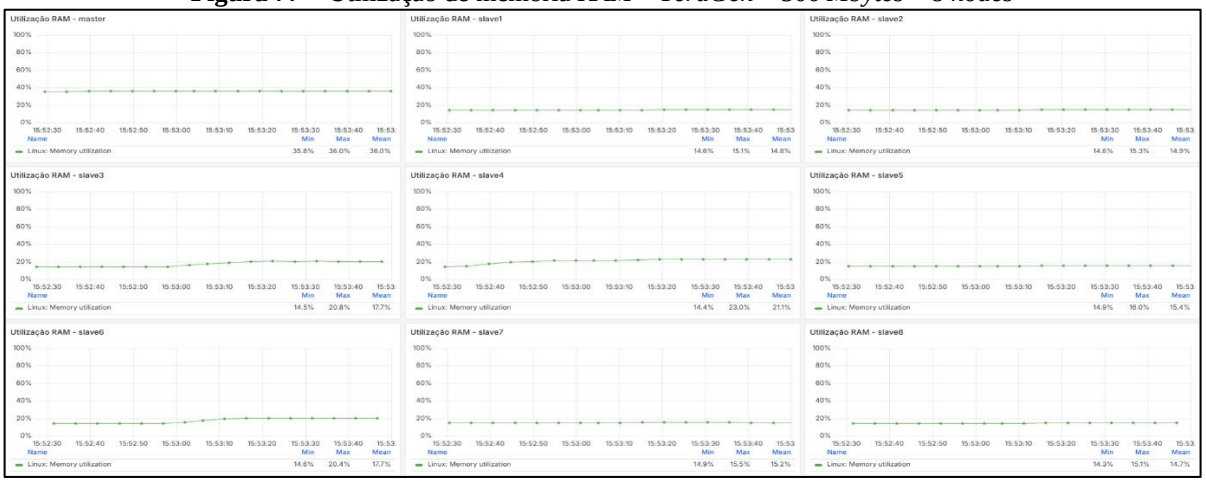
Fonte: Elaborada pelos Autores, 2025.

1.2.3 *Cluster* com 8 *nodes* – 500 *MBytes*

Figura 76 – Utilização de *CPU* – *TeraGen* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 77 – Utilização de memória *RAM* – *TeraGen* – 500 *Mbytes* – 8 *nodes*

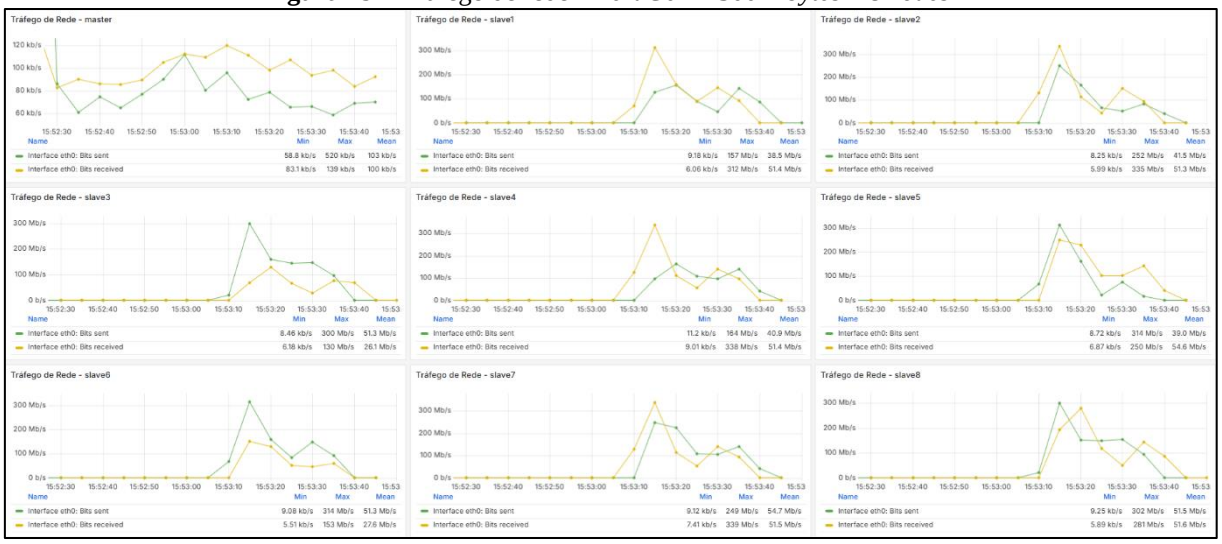


Fonte: Elaborada pelos Autores, 2025.

Figura 78 – Quantidade de processos – *TeraGen* – 500 *Mbytes* – 8 *nodes*

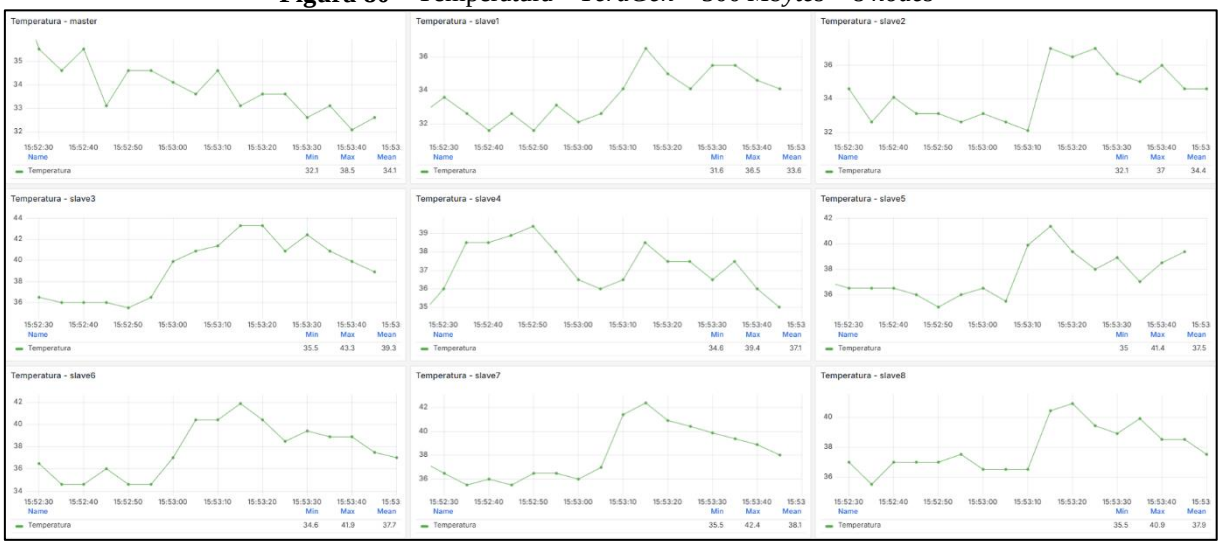
Fonte: Elaborada pelos Autores, 2025.

Figura 79 – Tráfego de rede – *TeraGen* – 500 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

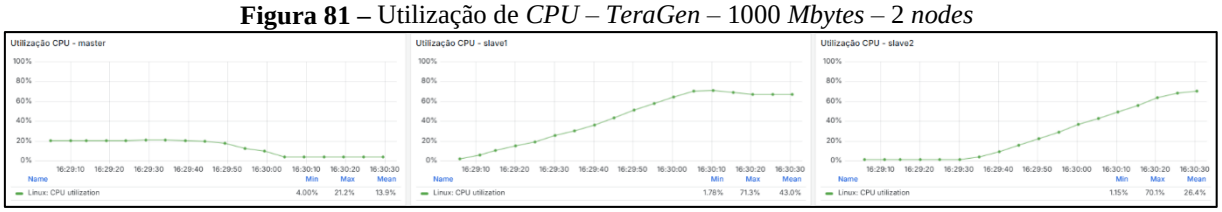
Figura 80 – Temperatura – *TeraGen* – 500 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

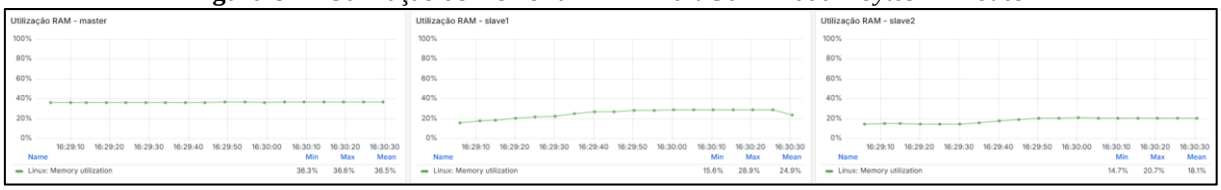
1.2.4 *Cluster* com 2 *nodes* – 1000 *MBytes*

Figura 81 – Utilização de *CPU* – *TeraGen* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 82 – Utilização de memória *RAM* – *TeraGen* – 1000 *Mbytes* – 2 *nodes*

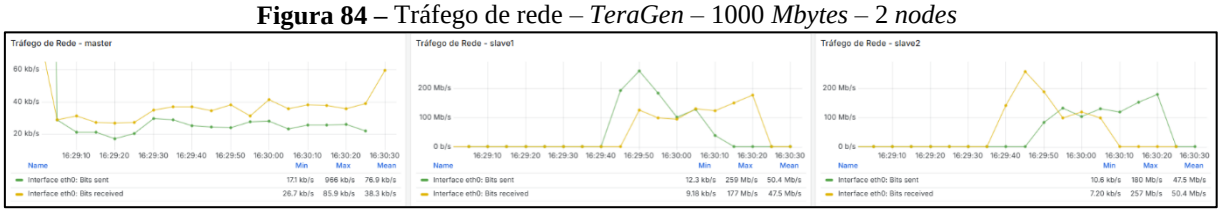


Fonte: Elaborada pelos Autores, 2025.

Figura 83 – Quantidade de processos – *TeraGen* – 1000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 84 – Tráfego de rede – *TeraGen* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 85 – Temperatura – *TeraGen* – 1000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.2.5 *Cluster* com 4 *nodes* – 1000 *MBytes*

Figura 86 – Utilização de *CPU* – *TeraGen* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 87 – Utilização de memória *RAM* – *TeraGen* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 88 – Quantidade de processos – *TeraGen* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 89 – Tráfego de rede – *TeraGen* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 90 – Temperatura – *TeraGen* – 1000 *Mbytes* – 4 *nodes*

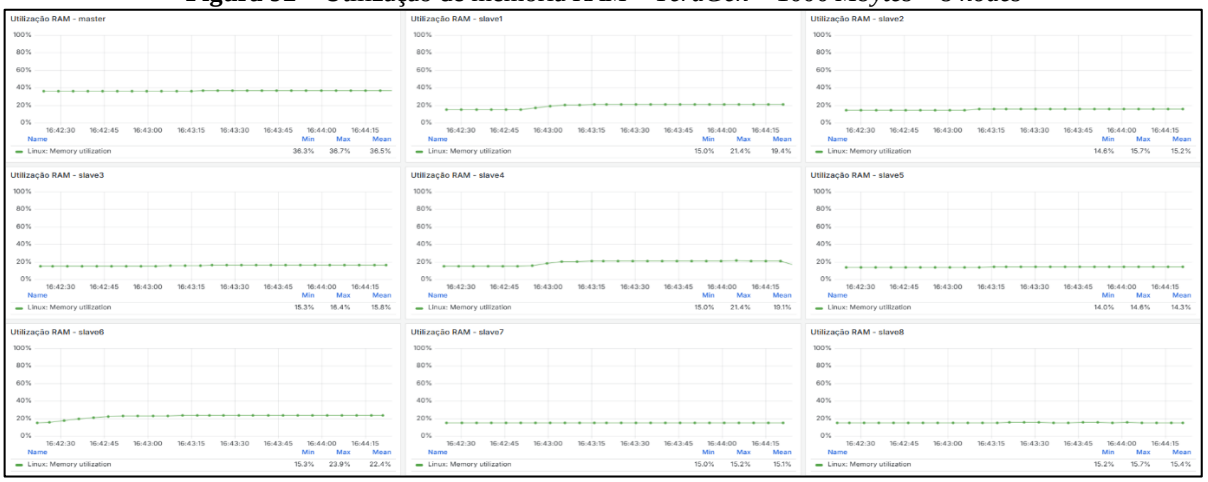
Fonte: Elaborada pelos Autores, 2025.

1.2.6 *Cluster* com 8 *nodes* – 1000 *MBytes*

Figura 91 – Utilização de *CPU* – *TeraGen* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 92 – Utilização de memória *RAM* – *TeraGen* – 1000 *Mbytes* – 8 *nodes*

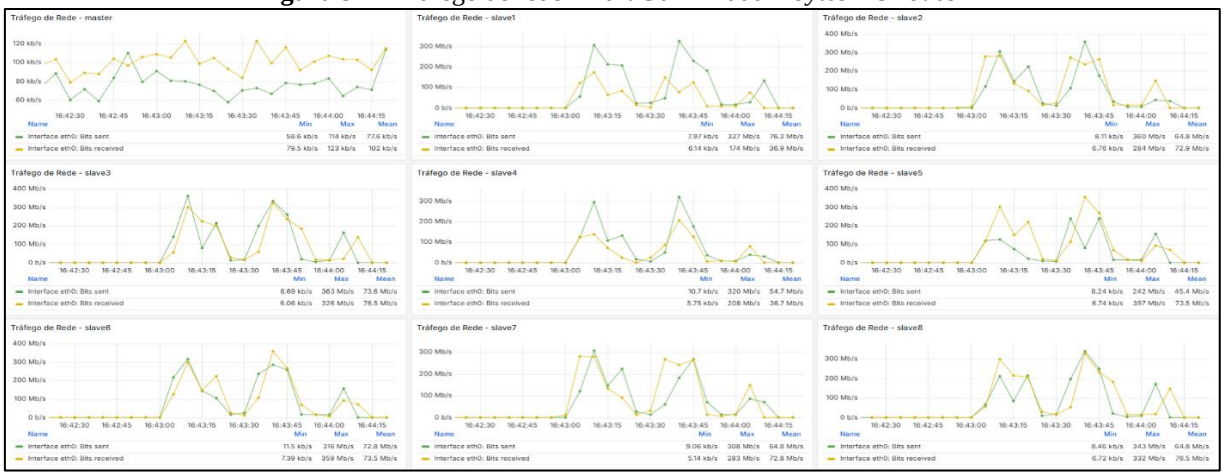


Fonte: Elaborada pelos Autores, 2025.

Figura 93 – Quantidade de processos – *TeraGen* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 94 – Tráfego de rede – *TeraGen* – 1000 *Mbytes* – 8 *nodes*



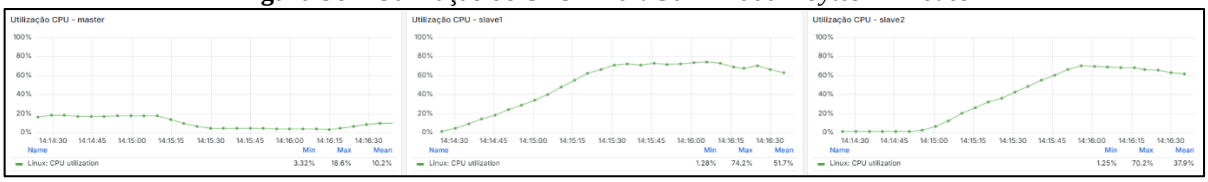
Fonte: Elaborada pelos Autores, 2025.

Figura 95 – Temperatura – *TeraGen* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

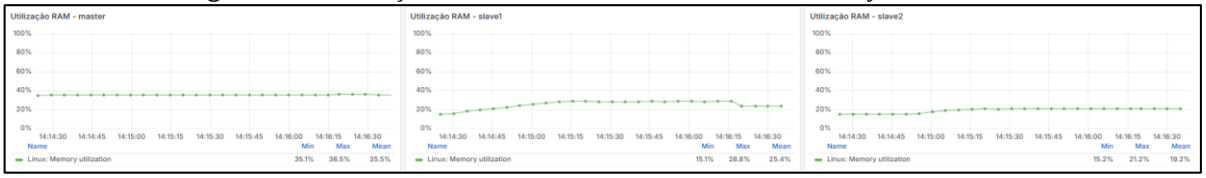
1.2.7 *Cluster* com 2 *nodes* – 2000 *MBytes*

Figura 96 – Utilização de *CPU* – *TeraGen* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 97 – Utilização de memória *RAM* – *TeraGen* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 98 – Quantidade de processos – *TeraGen* – 2000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 99 – Tráfego de rede – *TeraGen* – 2000 *Mbytes* – 2 *nodes*

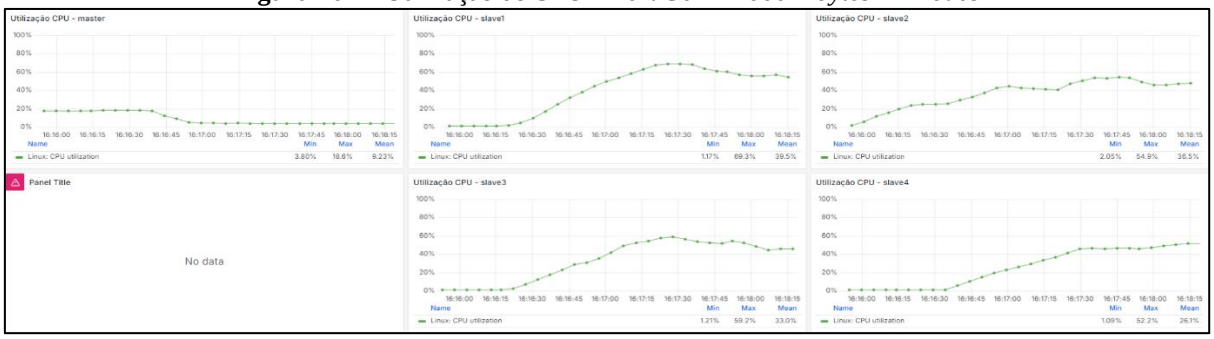
Fonte: Elaborada pelos Autores, 2025.

Figura 100 – Temperatura – *TeraGen* – 2000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

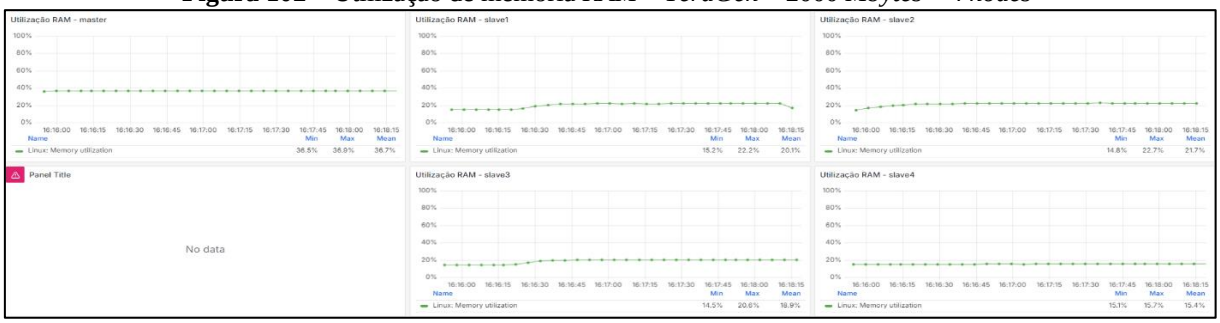
1.2.8 *Cluster* com 4 *nodes* – 2000 *MBytes*

Figura 101 – Utilização de *CPU* – *TeraGen* – 2000 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 102 – Utilização de memória *RAM* – *TeraGen* – 2000 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 103 – Quantidade de processos – *TeraGen* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 104 – Tráfego de rede – *TeraGen* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 105 – Temperatura – *TeraGen* – 2000 *Mbytes* – 4 *nodes*

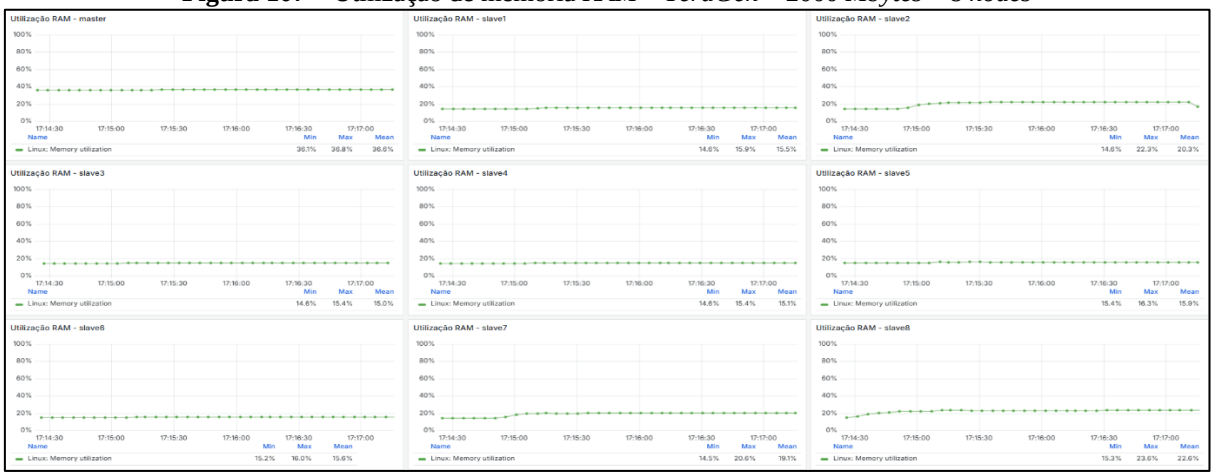
Fonte: Elaborada pelos Autores, 2025.

1.2.9 *Cluster* com 8 *nodes* – 2000 *MBytes*

Figura 106 – Utilização de *CPU* – *TeraGen* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 107 – Utilização de memória *RAM* – *TeraGen* – 2000 *Mbytes* – 8 *nodes*

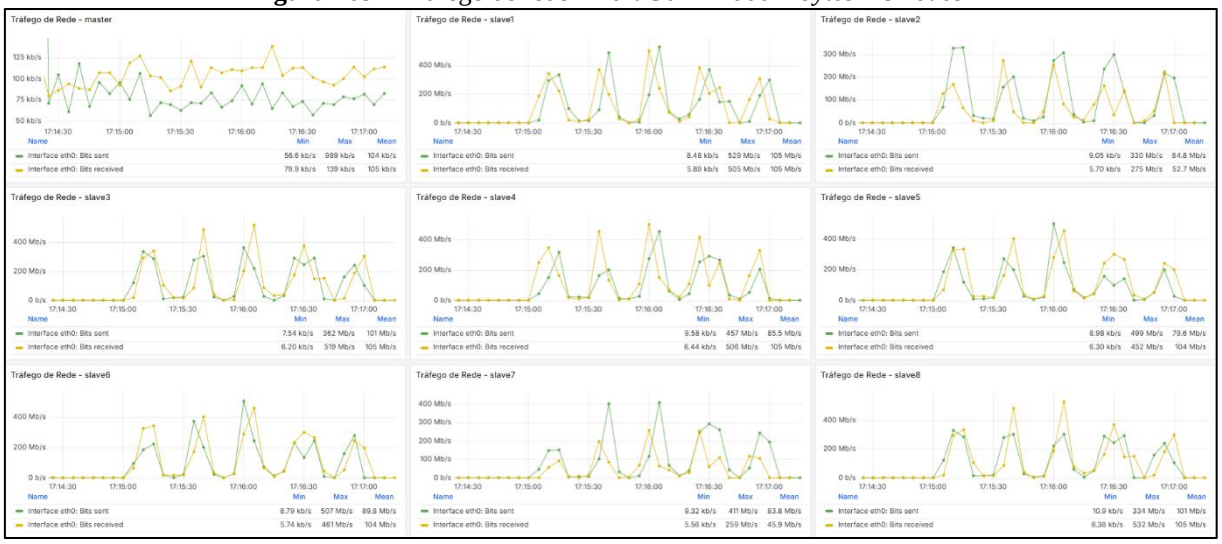


Fonte: Elaborada pelos Autores, 2025.

Figura 108 – Quantidade de processos – *TeraGen* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 109 – Tráfego de rede – *TeraGen* – 2000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

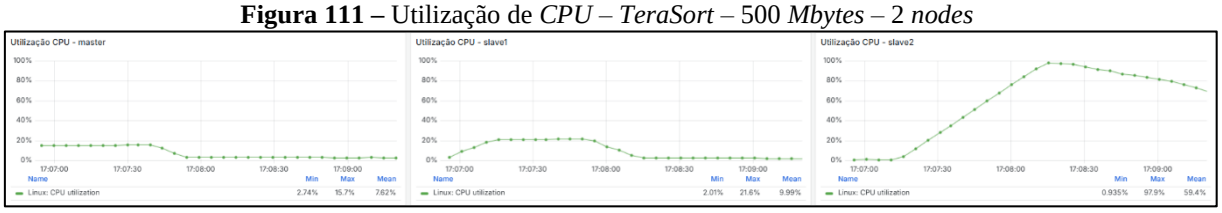
Figura 110 – Temperatura – *TeraGen* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.3 Algoritmo *TeraSort*

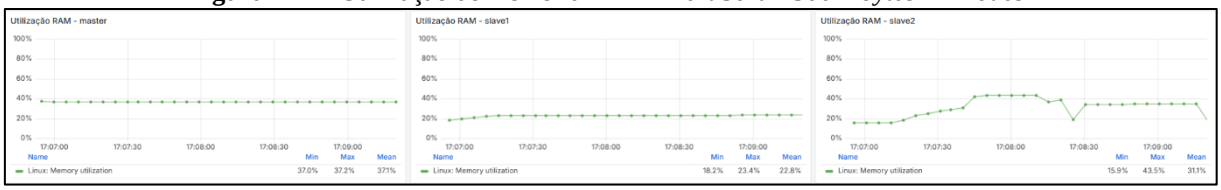
1.3.1 *Cluster* com 2 *nodes* – 500 *MBytes*

Figura 111 – Utilização de *CPU* – *TeraSort* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 112 – Utilização de memória *RAM* – *TeraSort* – 500 *Mbytes* – 2 *nodes*

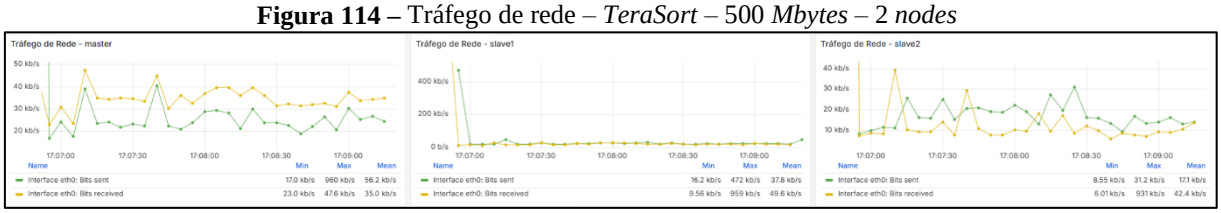


Fonte: Elaborada pelos Autores, 2025.

Figura 113 – Quantidade de processos – *TeraSort* – 500 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 114 – Tráfego de rede – *TeraSort* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 115 – Temperatura – *TeraSort* – 500 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.3.2 *Cluster* com 4 *nodes* – 500 *MBytes*

Figura 116 – Utilização de *CPU* – *TeraSort* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 117 – Utilização de memória *RAM* – *TeraSort* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 118 – Quantidade de processos – *TeraSort* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 119 – Tráfego de rede – *TeraSort* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 120 – Temperatura – *TeraSort* – 500 *Mbytes* – 4 *nodes*

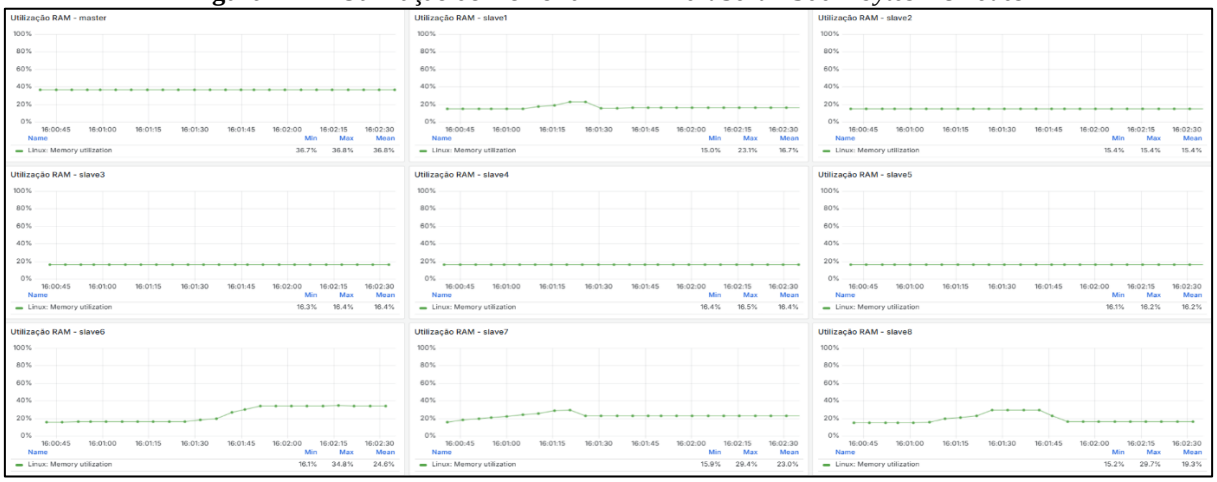
Fonte: Elaborada pelos Autores, 2025.

1.3.3 *Cluster* com 8 *nodes* – 500 *MBytes*

Figura 121 – Utilização de *CPU* – *TeraSort* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 122 – Utilização de memória *RAM* – *TeraSort* – 500 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 123 – Quantidade de processos – *TeraSort* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 124 – Tráfego de rede – *TeraSort* – 500 *Mbytes* – 8 *nodes*

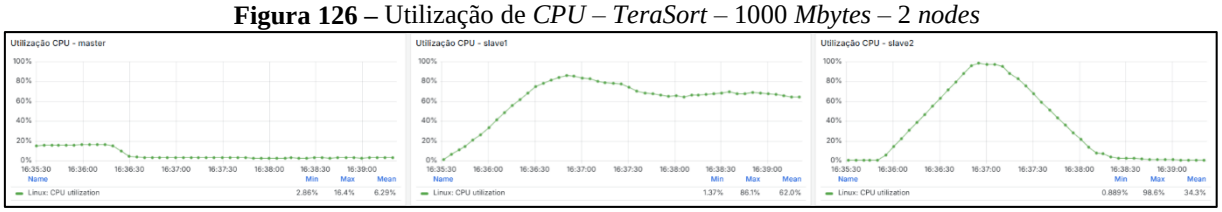
Fonte: Elaborada pelos Autores, 2025.

Figura 125 – Temperatura – *TeraSort* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.3.4 *Cluster* com 2 *nodes* – 1000 *MBytes*

Figura 126 – Utilização de *CPU* – *TeraSort* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

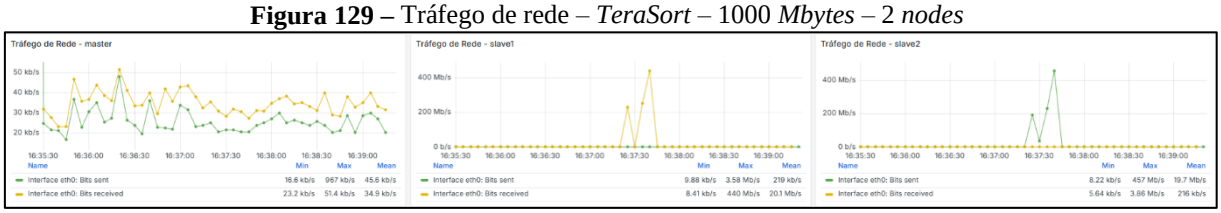
Figura 127 – Utilização de memória *RAM* – *TeraSort* – 1000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 128 – Quantidade de processos – *TeraSort* – 1000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 129 – Tráfego de rede – *TeraSort* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 130 – Temperatura – *TeraSort* – 1000 *Mbytes* – 2 *nodes*

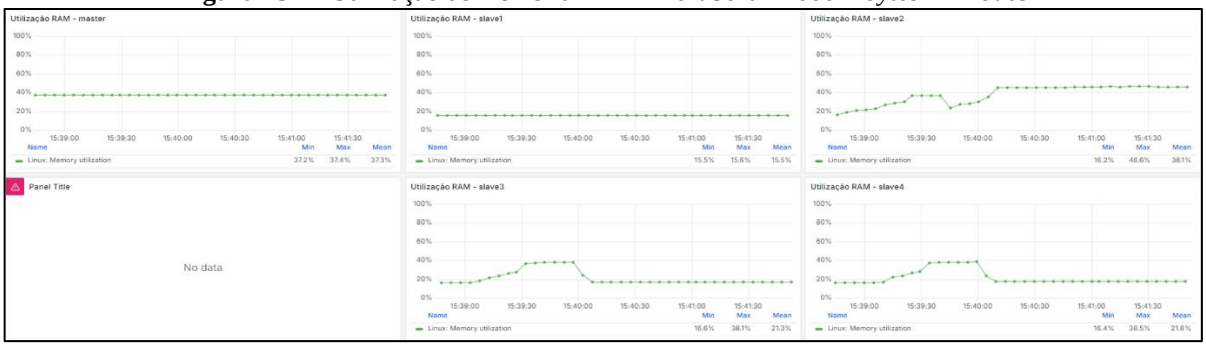
Fonte: Elaborada pelos Autores, 2025.

1.3.5 *Cluster* com 4 *nodes* – 1000 *MBytes*

Figura 131 – Utilização de *CPU* – *TeraSort* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 132 – Utilização de memória *RAM* – *TeraSort* – 1000 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 133 – Quantidade de processos – *TeraSort* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 134 – Tráfego de rede – *TeraSort* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 135 – Temperatura – *TeraSort* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.3.6 *Cluster* com 8 *nodes* – 1000 *MBytes*

Figura 136 – Utilização de *CPU* – *TeraSort* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 137 – Utilização de memória *RAM* – *TeraSort* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 138 – Quantidade de processos – *TeraSort* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 139 – Tráfego de rede – *TeraSort* – 1000 *Mbytes* – 8 *nodes*

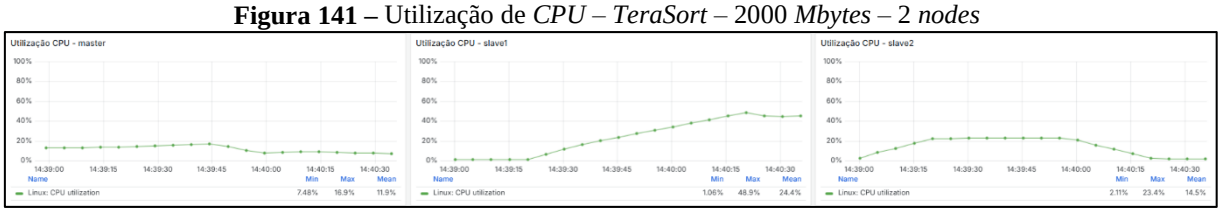
Fonte: Elaborada pelos Autores, 2025.

Figura 140 – Temperatura – *TeraSort* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

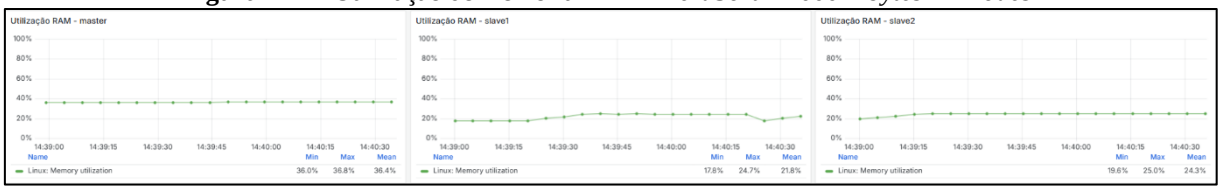
1.3.7 *Cluster* com 2 *nodes* – 2000 *MBytes*

Figura 141 – Utilização de *CPU* – *TeraSort* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 142 – Utilização de memória *RAM* – *TeraSort* – 2000 *Mbytes* – 2 *nodes*

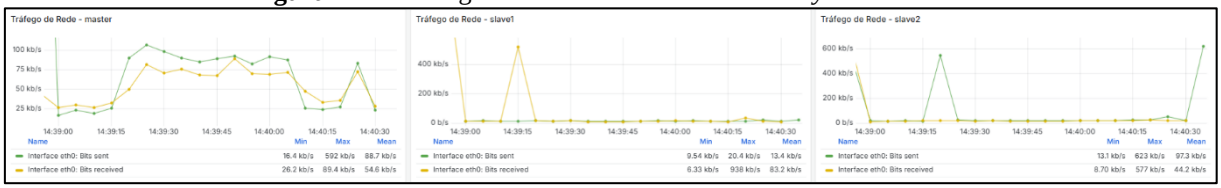


Fonte: Elaborada pelos Autores, 2025.

Figura 143 – Quantidade de processos – *TeraSort* – 2000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 144 – Tráfego de rede – *TeraSort* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 145 – Temperatura – *TeraSort* – 2000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.3.8 *Cluster* com 4 *nodes* – 2000 *MBytes*

Figura 146 – Utilização de *CPU* – *TeraSort* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

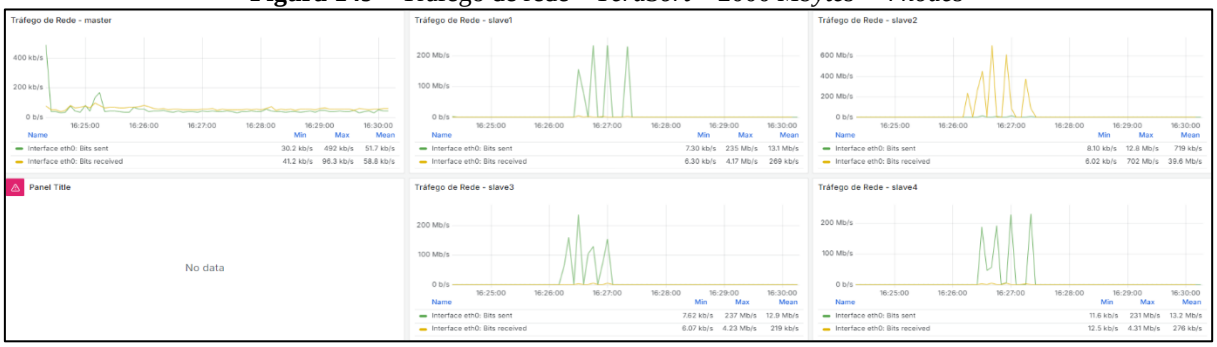
Figura 147 – Utilização de memória *RAM* – *TeraSort* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 148 – Quantidade de processos – *TeraSort* – 2000 *Mbytes* – 4 *nodes*

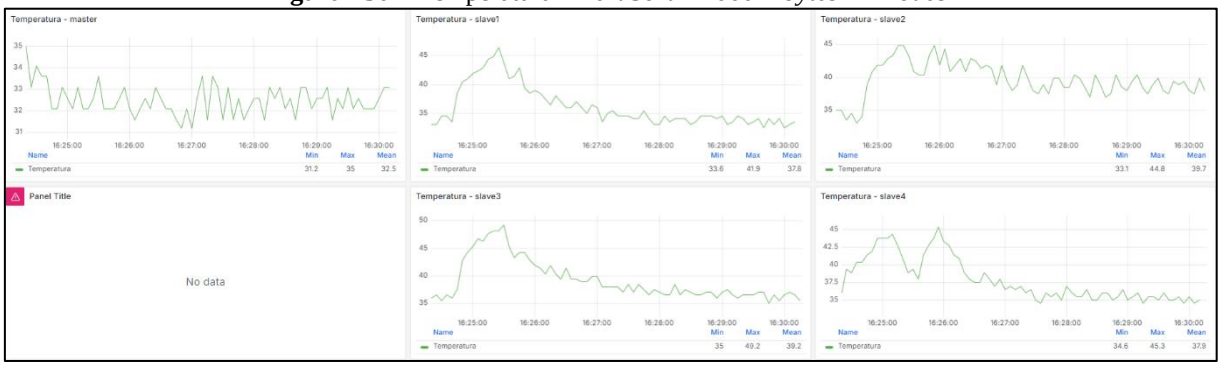
Fonte: Elaborada pelos Autores, 2025.

Figura 149 – Tráfego de rede – *TeraSort* – 2000 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 150 – Temperatura – *TeraSort* – 2000 *Mbytes* – 4 *nodes*



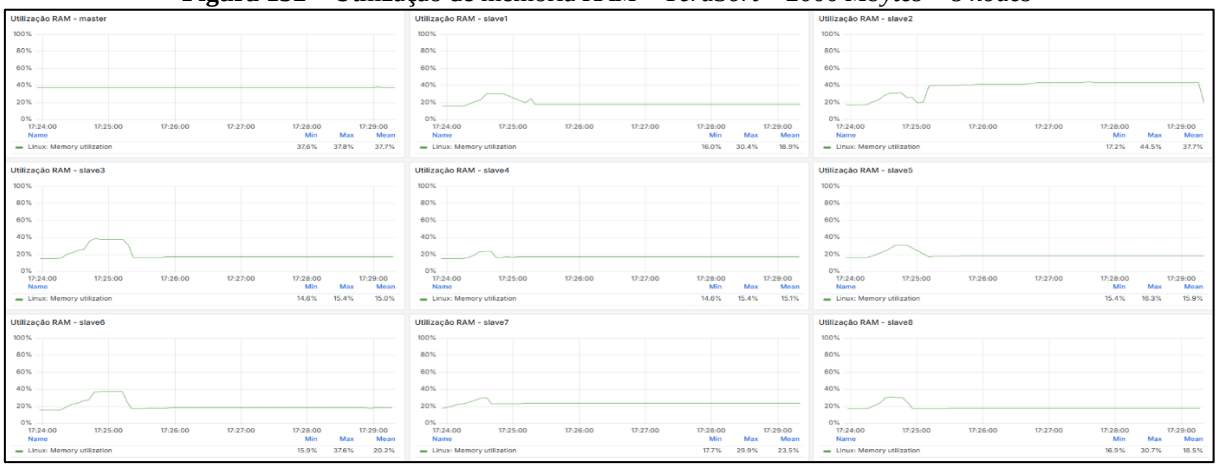
Fonte: Elaborada pelos Autores, 2025.

1.3.9 *Cluster* com 8 *nodes* – 2000 *MBytes*

Figura 151 – Utilização de *CPU* – *TeraSort* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 152 – Utilização de memória *RAM* – *TeraSort* – 2000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 153 – Quantidade de processos – *TeraSort* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 154 – Tráfego de rede – *TeraSort* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

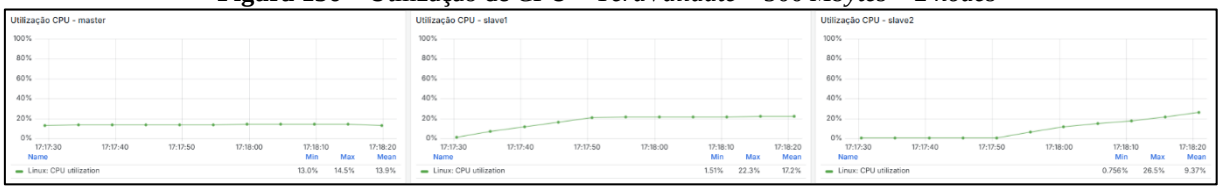
Figura 155 – Temperatura – *TeraSort* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.4 Algoritmo *TeraValidate*

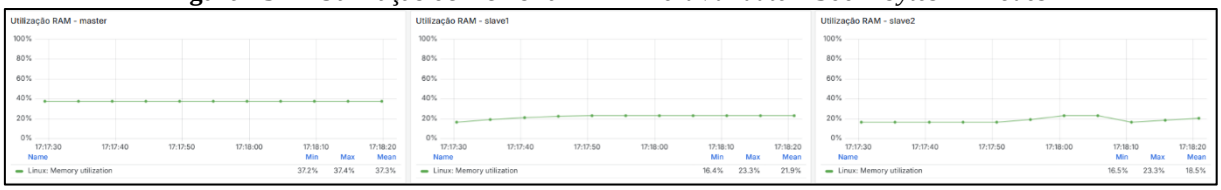
1.4.1 *Cluster* com 2 *nodes* – 500 *MBytes*

Figura 156 – Utilização de *CPU* – *TeraValidate* – 500 *Mbytes* – 2 *nodes*



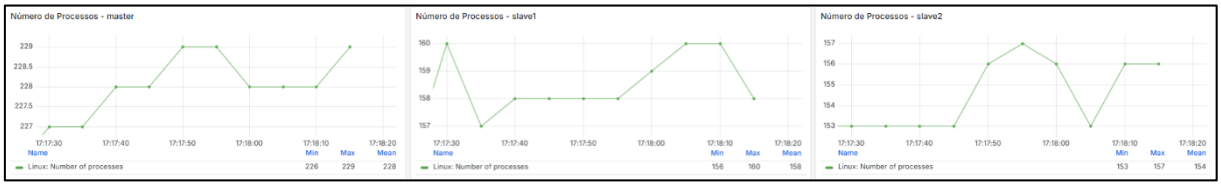
Fonte: Elaborada pelos Autores, 2025.

Figura 157 – Utilização de memória *RAM* – *TeraValidate* – 500 *Mbytes* – 2 *nodes*



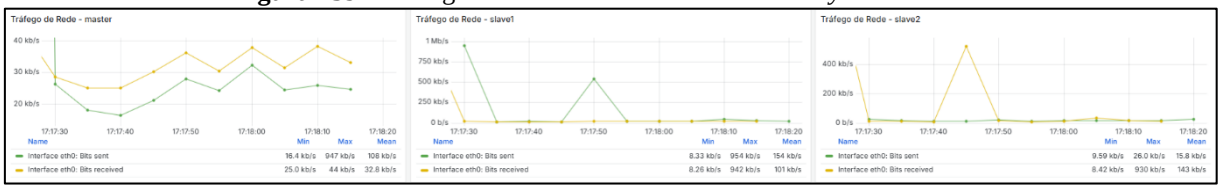
Fonte: Elaborada pelos Autores, 2025.

Figura 158 – Quantidade de processos – *TeraValidate* – 500 *Mbytes* – 2 *nodes*



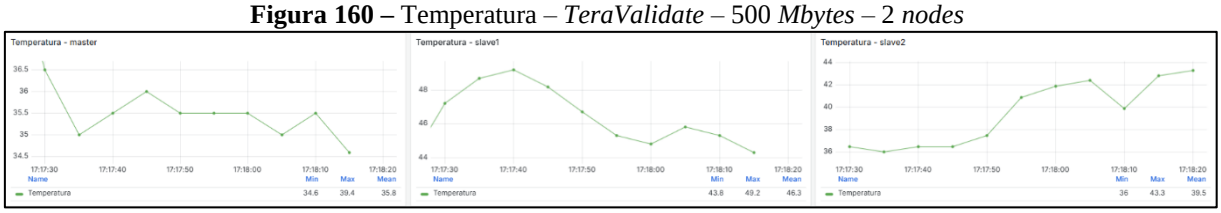
Fonte: Elaborada pelos Autores, 2025.

Figura 159 – Tráfego de rede – *TeraValidate* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 160 – Temperatura – *TeraValidate* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.4.2 *Cluster* com 4 *nodes* – 500 *MBytes*

Figura 161 – Utilização de *CPU* – *TeraValidate* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 162 – Utilização de memória *RAM* – *TeraValidate* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 163 – Quantidade de processos – *TeraValidate* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 164 – Tráfego de rede – *TeraValidate* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 165 – Temperatura – *TeraValidate* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.4.3 *Cluster* com 8 *nodes* – 500 *MBytes*

Figura 166 – Utilização de *CPU* – *TeraValidate* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 167 – Utilização de memória *RAM* – *TeraValidate* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

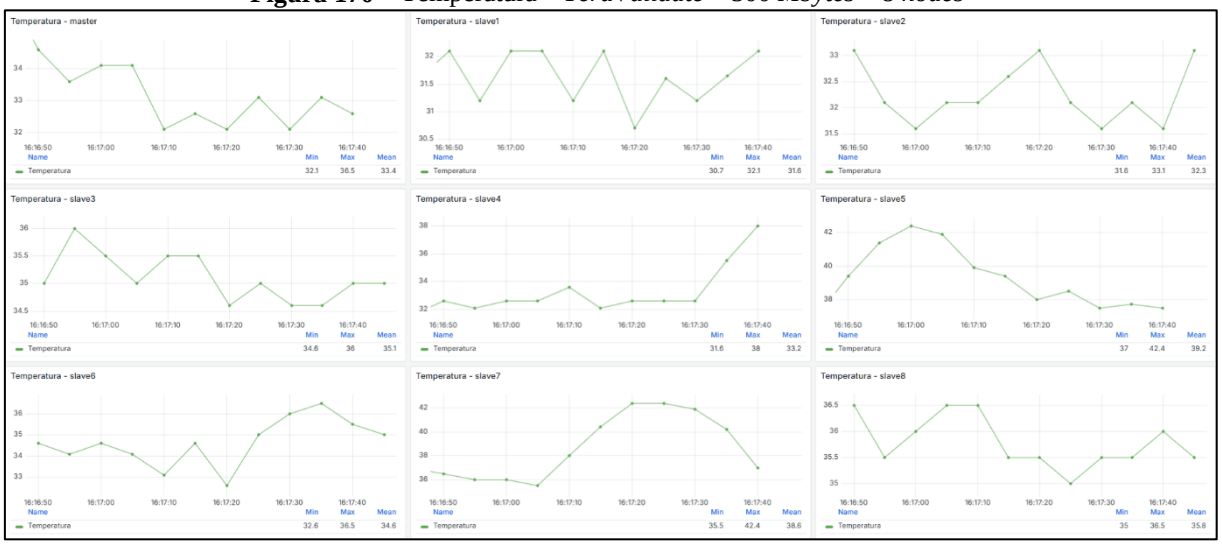
Figura 168 – Quantidade de processos – *TeraValidate* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 169 – Tráfego de rede – *TeraValidate* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

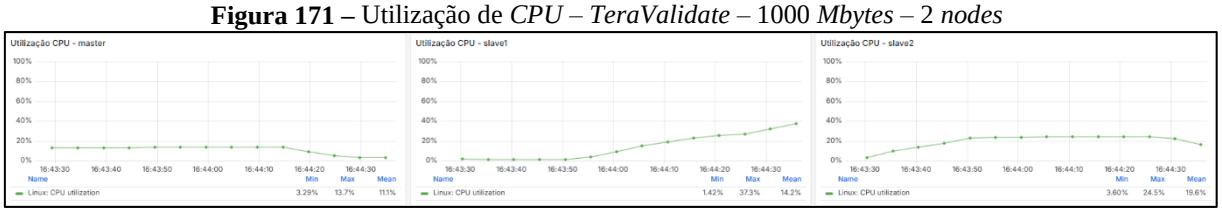
Figura 170 – Temperatura – *TeraValidate* – 500 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

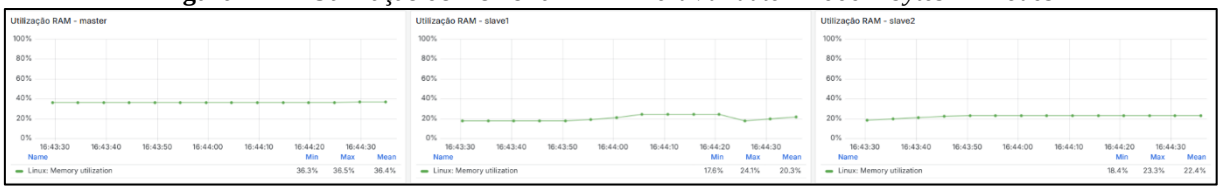
1.4.4 *Cluster* com 2 *nodes* – 1000 *MBytes*

Figura 171 – Utilização de *CPU* – *TeraValidate* – 1000 *Mbytes* – 2 *nodes*



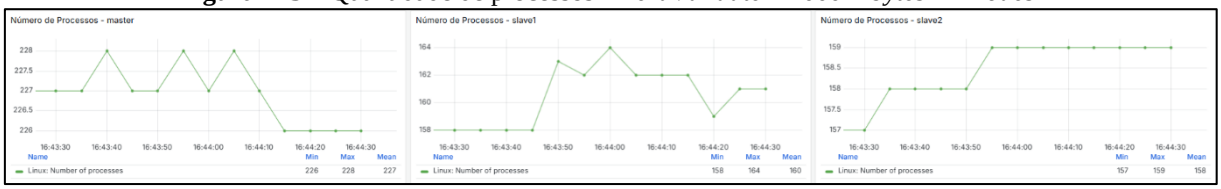
Fonte: Elaborada pelos Autores, 2025.

Figura 172 – Utilização de memória *RAM* – *TeraValidate* – 1000 *Mbytes* – 2 *nodes*



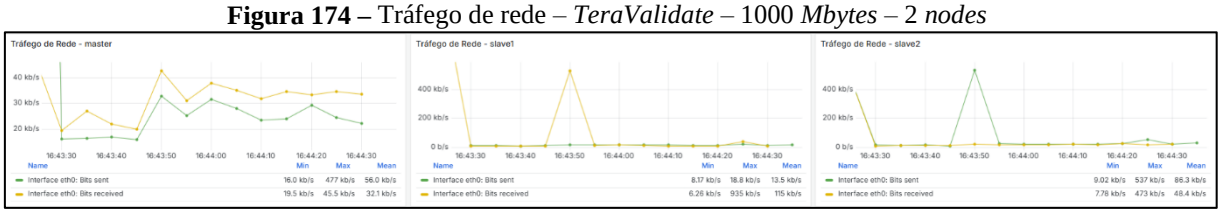
Fonte: Elaborada pelos Autores, 2025.

Figura 173 – Quantidade de processos – *TeraValidate* – 1000 *Mbytes* – 2 *nodes*



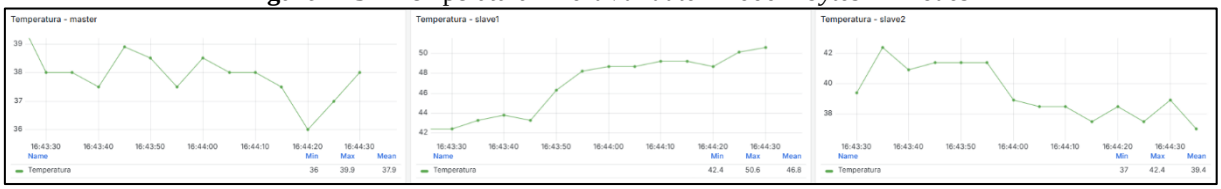
Fonte: Elaborada pelos Autores, 2025.

Figura 174 – Tráfego de rede – *TeraValidate* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 175 – Temperatura – *TeraValidate* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.4.5 *Cluster* com 4 *nodes* – 1000 *MBytes*

Figura 176 – Utilização de *CPU* – *TeraValidate* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 177 – Utilização de memória *RAM* – *TeraValidate* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 178 – Quantidade de processos – *TeraValidate* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 179 – Tráfego de rede – *TeraValidate* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 180 – Temperatura – *TeraValidate* – 1000 *Mbytes* – 4 *nodes*

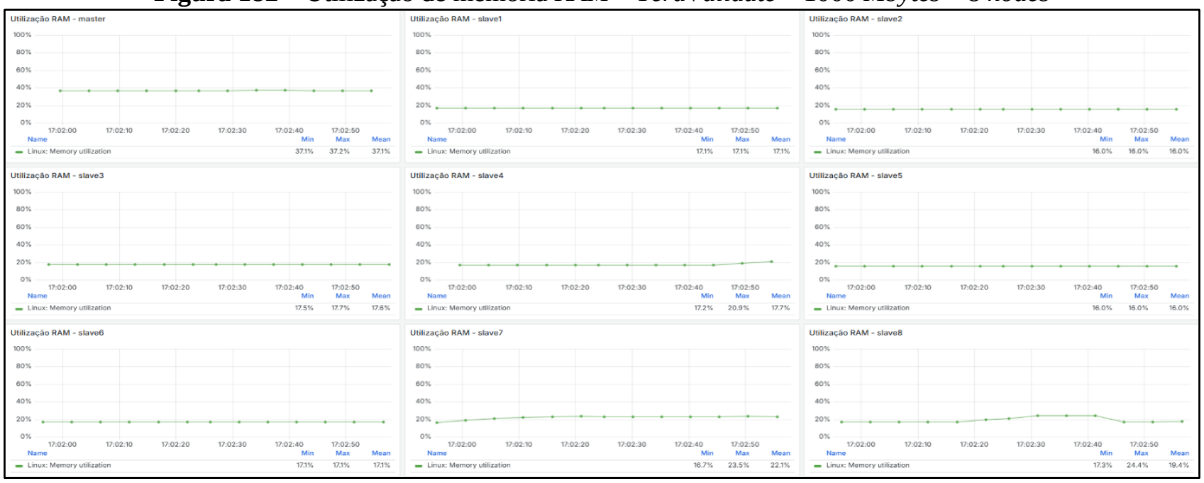
Fonte: Elaborada pelos Autores, 2025.

1.4.6 *Cluster* com 8 *nodes* – 1000 *MBytes*

Figura 181 – Utilização de *CPU* – *TeraValidate* – 1000 *Mbytes* – 8 *nodes*

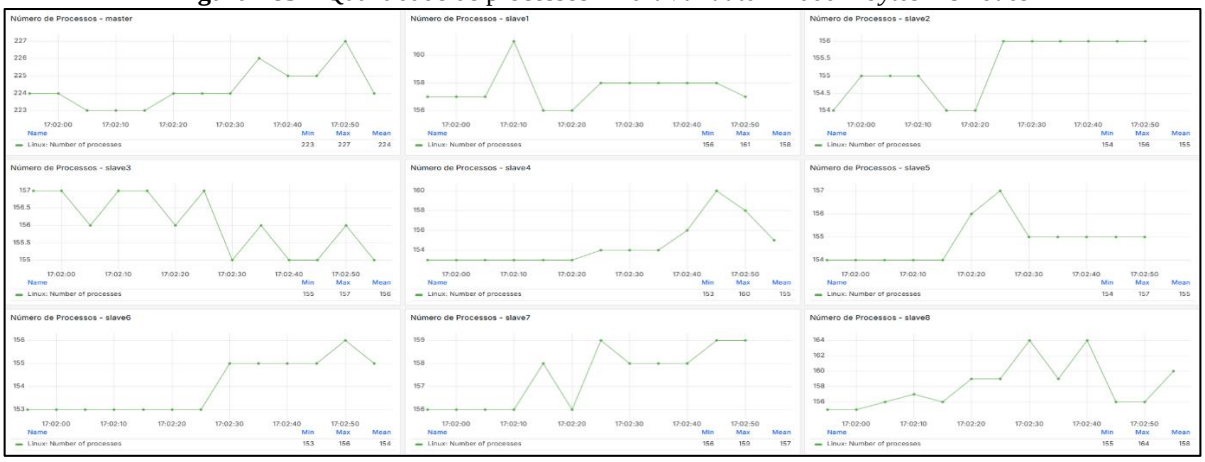
Fonte: Elaborada pelos Autores, 2025.

Figura 182 – Utilização de memória *RAM* – *TeraValidate* – 1000 *Mbytes* – 8 *nodes*



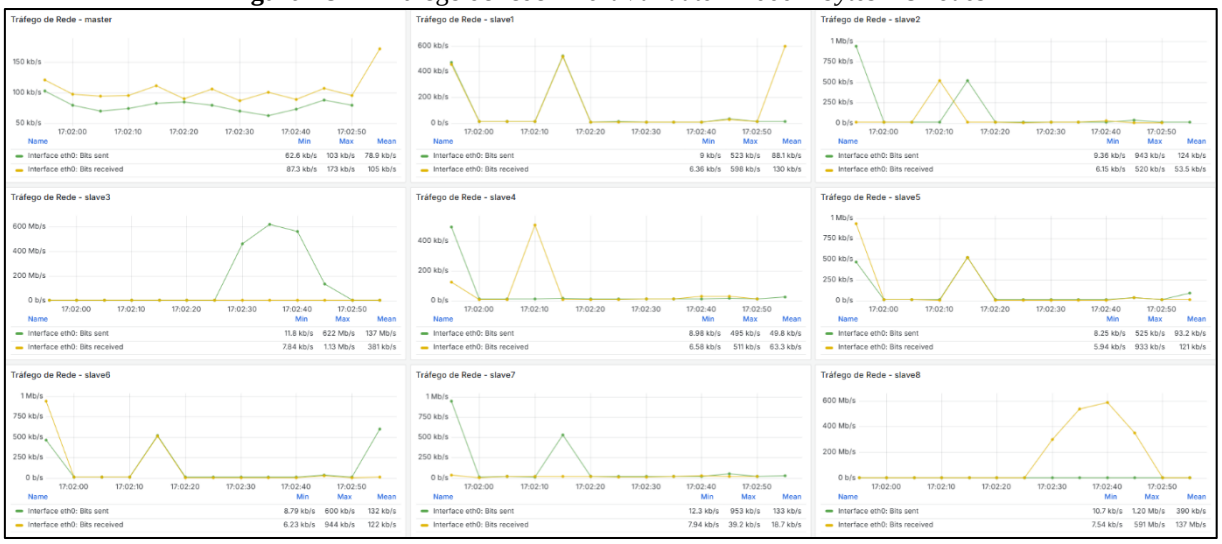
Fonte: Elaborada pelos Autores, 2025.

Figura 183 – Quantidade de processos – *TeraValidate* – 1000 *Mbytes* – 8 *nodes*



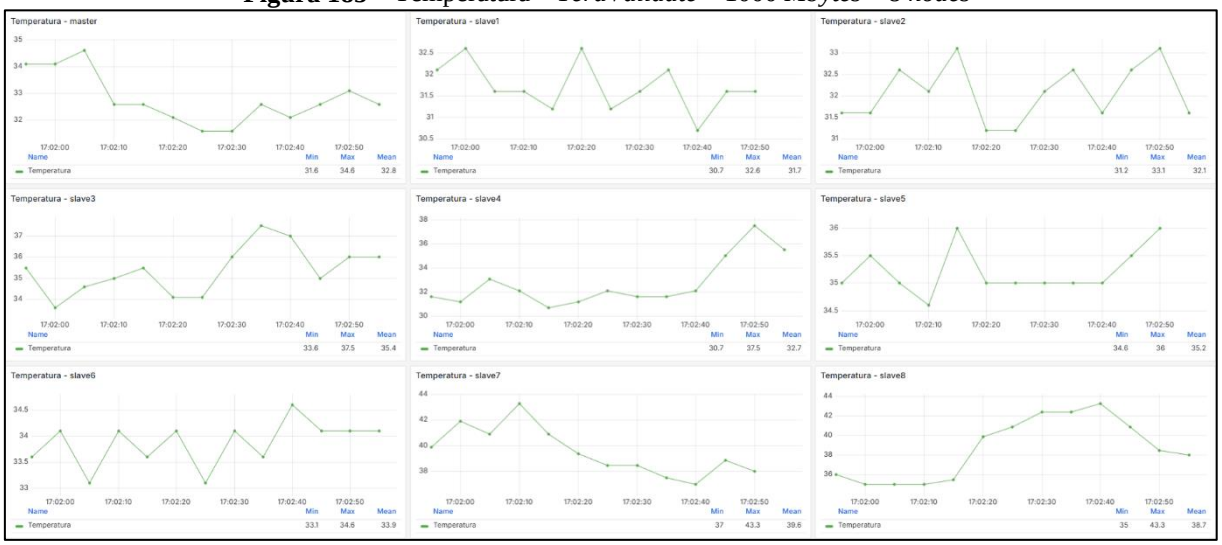
Fonte: Elaborada pelos Autores, 2025.

Figura 184 – Tráfego de rede – *TeraValidate* – 1000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

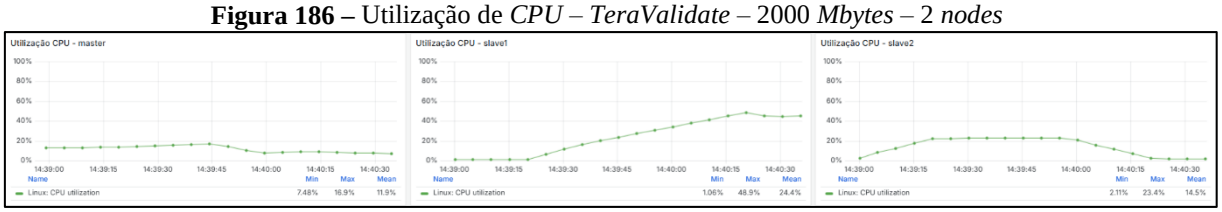
Figura 185 – Temperatura – *TeraValidate* – 1000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

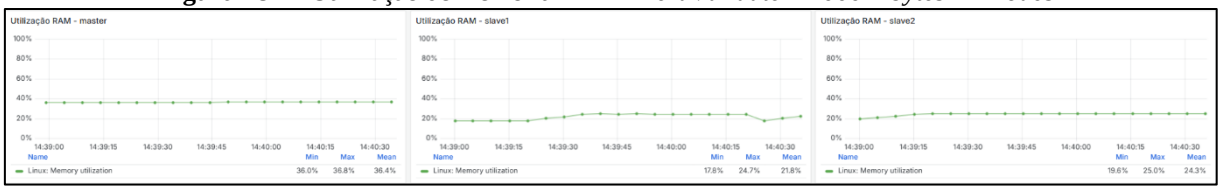
1.4.7 *Cluster* com 2 *nodes* – 2000 *MBytes*

Figura 186 – Utilização de *CPU* – *TeraValidate* – 2000 *Mbytes* – 2 *nodes*



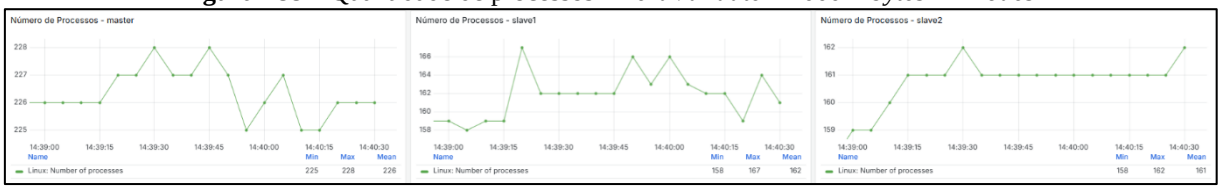
Fonte: Elaborada pelos Autores, 2025.

Figura 187 – Utilização de memória *RAM* – *TeraValidate* – 2000 *Mbytes* – 2 *nodes*



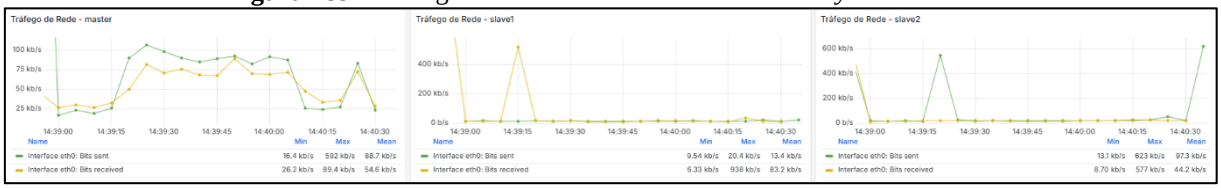
Fonte: Elaborada pelos Autores, 2025.

Figura 188 – Quantidade de processos – *TeraValidate* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 189 – Tráfego de rede – *TeraValidate* – 2000 *Mbytes* – 2 *nodes*



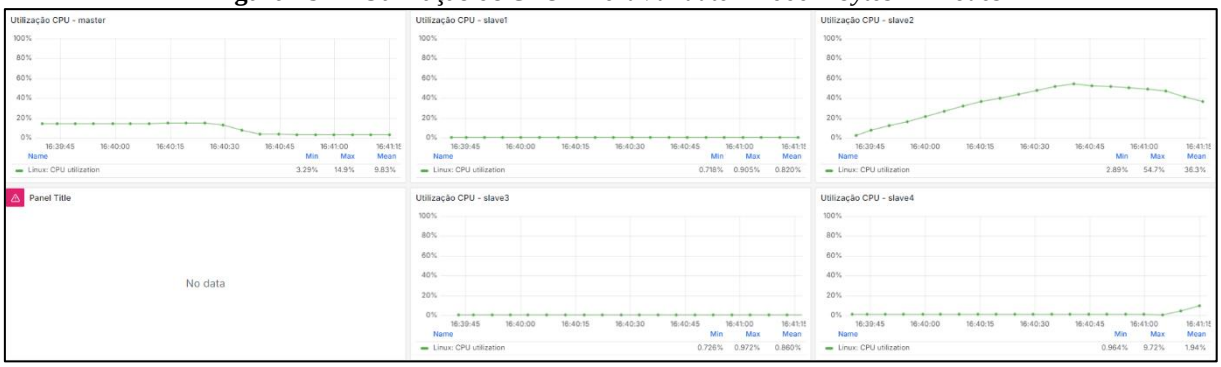
Fonte: Elaborada pelos Autores, 2025.

Figura 190 – Temperatura – *TeraValidate* – 2000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.4.8 *Cluster* com 4 *nodes* – 2000 *MBytes*

Figura 191 – Utilização de *CPU* – *TeraValidate* – 2000 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 192 – Utilização de memória *RAM* – *TeraValidate* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 193 – Quantidade de processos – *TeraValidate* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 194 – Tráfego de rede – *TeraValidate* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 195 – Temperatura – *TeraValidate* – 2000 *Mbytes* – 4 *nodes*

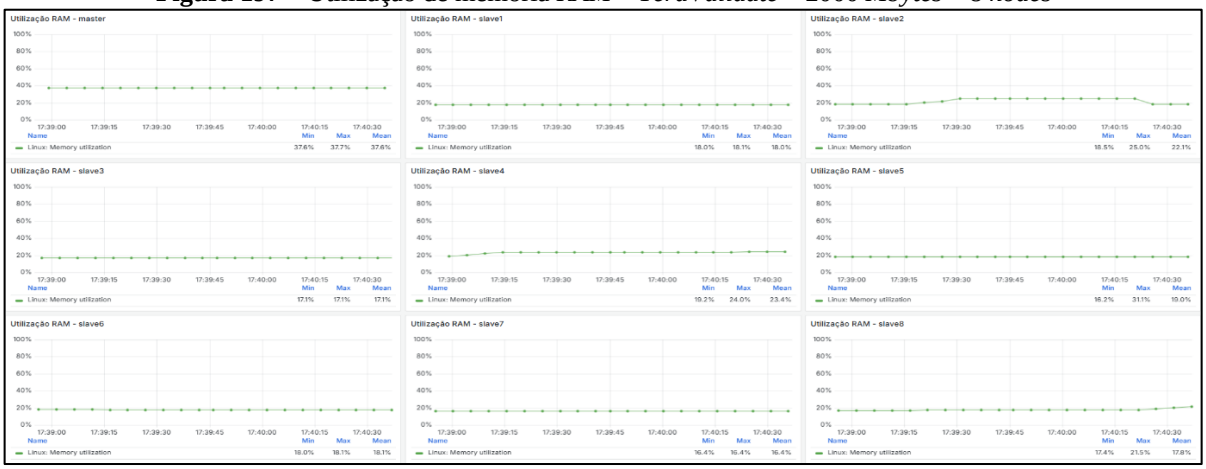
Fonte: Elaborada pelos Autores, 2025.

1.4.9 *Cluster* com 8 *nodes* – 2000 *MBytes*

Figura 196 – Utilização de *CPU* – *TeraValidate* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 197 – Utilização de memória *RAM* – *TeraValidate* – 2000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

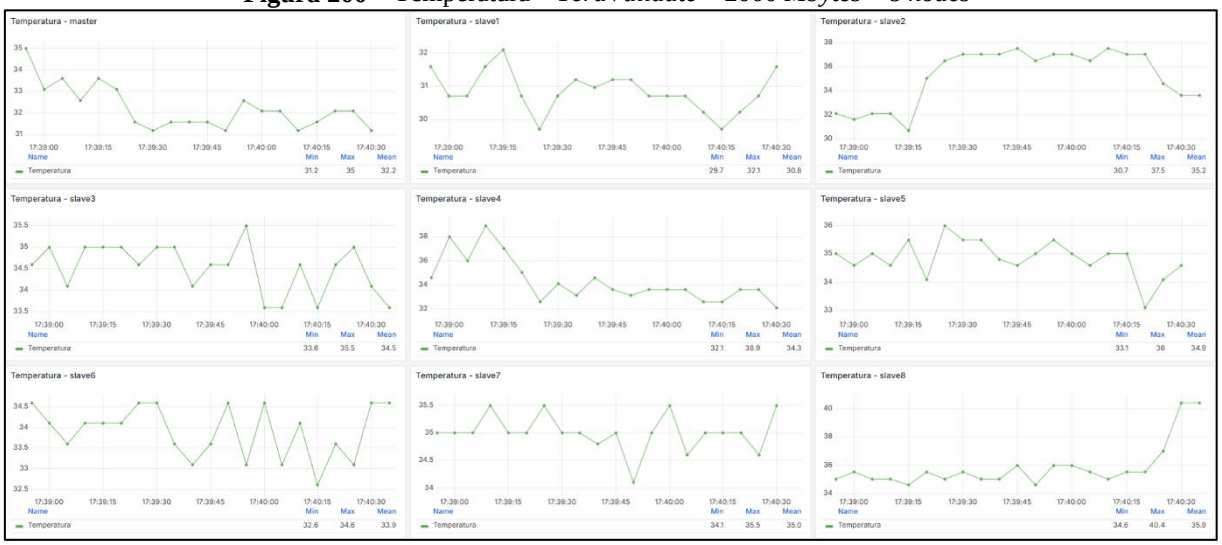
Figura 198 – Quantidade de processos – *TeraValidate* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 199 – Tráfego de rede – *TeraValidate* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 200 – Temperatura – *TeraValidate* – 2000 *Mbytes* – 8 *nodes*

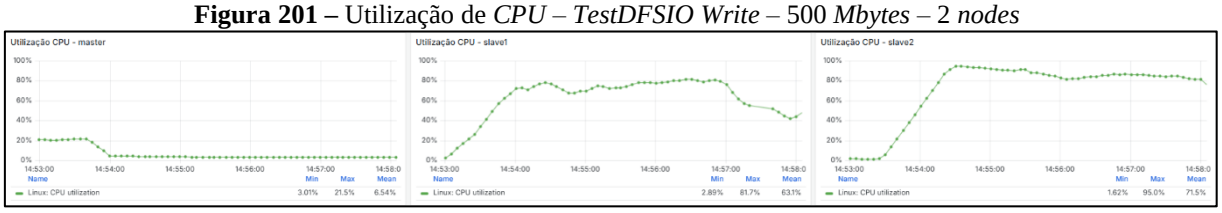


Fonte: Elaborada pelos Autores, 2025.

1.5 Algoritmo *TestDFSIO Write*

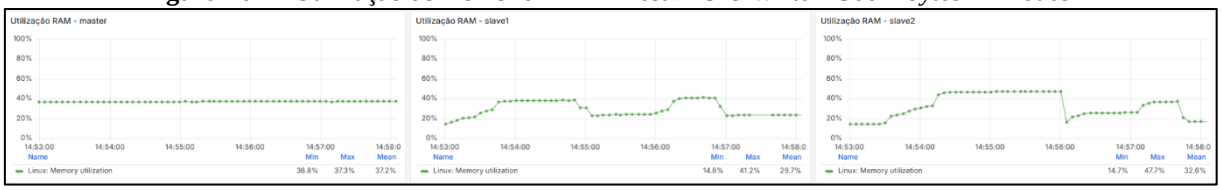
1.5.1 *Cluster* com 2 *nodes* – 500 *MBytes*

Figura 201 – Utilização de *CPU – TestDFSIO Write* – 500 *Mbytes* – 2 *nodes*



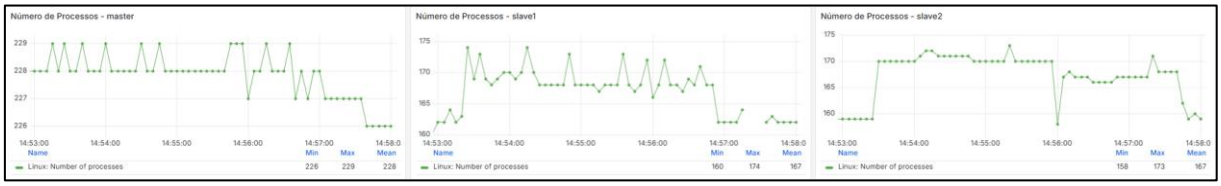
Fonte: Elaborada pelos Autores, 2025.

Figura 202 – Utilização de memória *RAM* – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes*



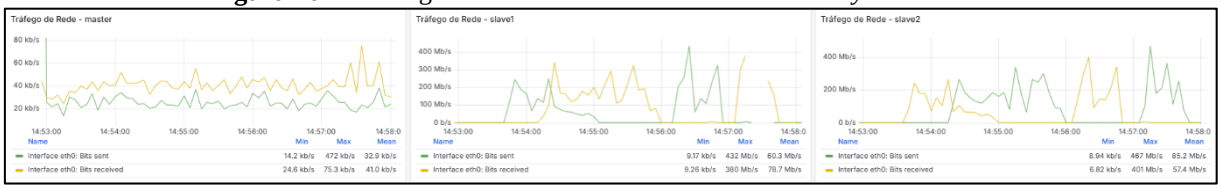
Fonte: Elaborada pelos Autores, 2025.

Figura 203 – Quantidade de processos – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes*



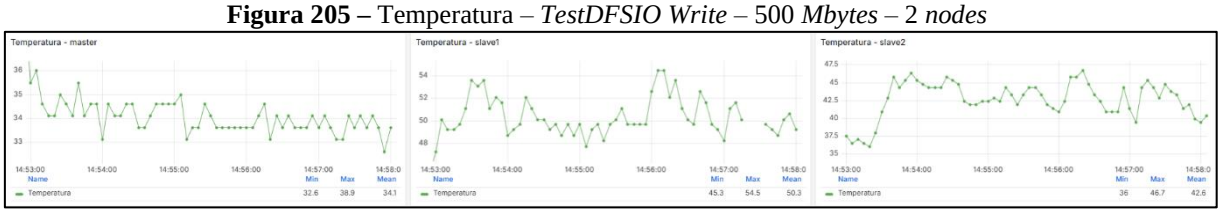
Fonte: Elaborada pelos Autores, 2025.

Figura 204 – Tráfego de rede – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 205 – Temperatura – *TestDFSIO Write* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.5.2 *Cluster* com 4 *nodes* – 500 *MBytes*

Figura 206 – Utilização de *CPU* – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 207 – Utilização de memória *RAM* – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

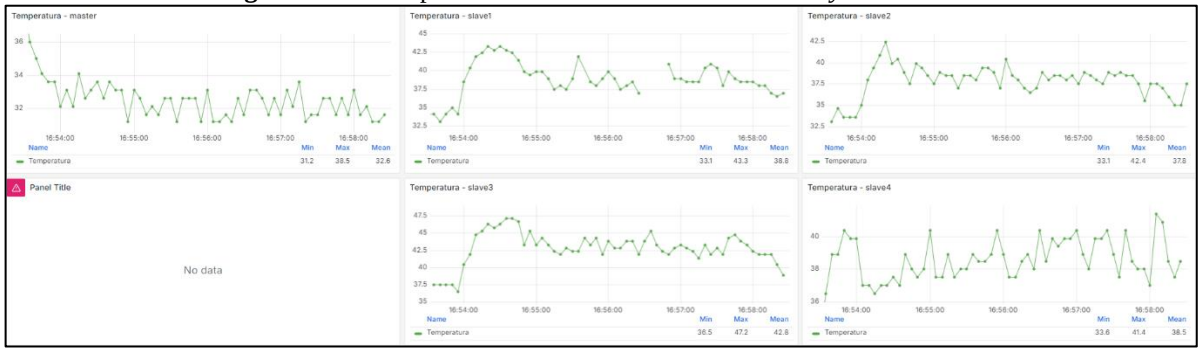
Figura 208 – Quantidade de processos – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 209 – Tráfego de rede – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 210 – Temperatura – *TestDFSIO Write* – 500 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.5.3 *Cluster* com 8 *nodes* – 500 *MBytes*

Figura 211 – Utilização de *CPU* – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 212 – Utilização de memória *RAM* – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 213 – Quantidade de processos – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 214 – Tráfego de rede – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes*

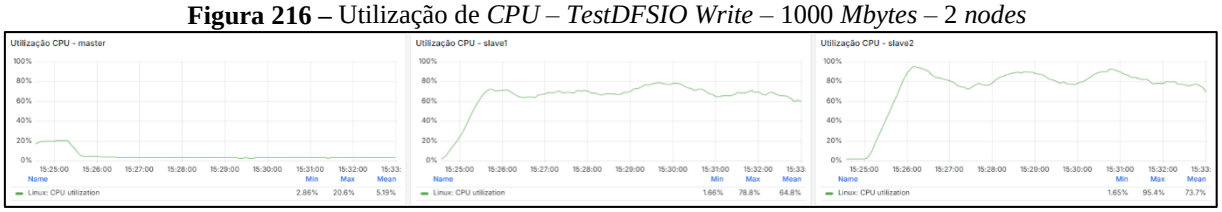
Fonte: Elaborada pelos Autores, 2025.

Figura 215 – Temperatura – *TestDFSIO Write* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

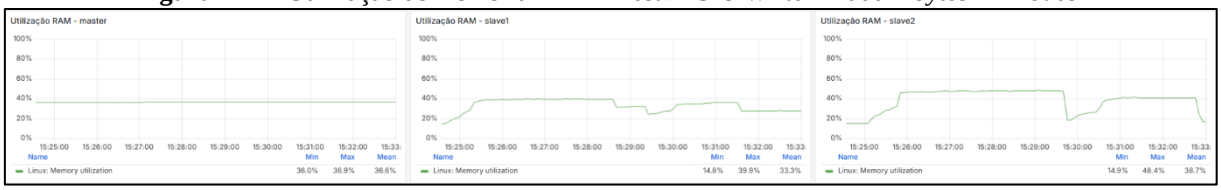
1.5.4 *Cluster* com 2 *nodes* – 1000 *MBytes*

Figura 216 – Utilização de *CPU* – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 217 – Utilização de memória *RAM* – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes*

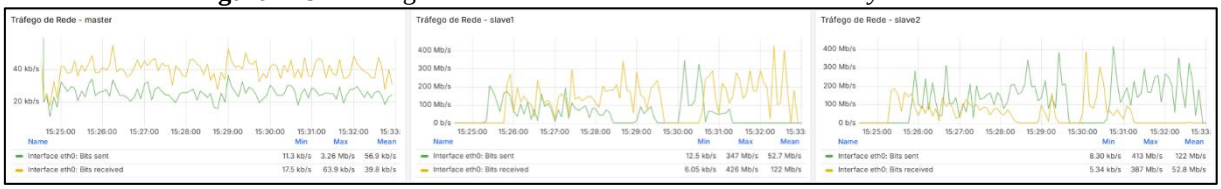


Fonte: Elaborada pelos Autores, 2025.

Figura 218 – Quantidade de processos – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes*

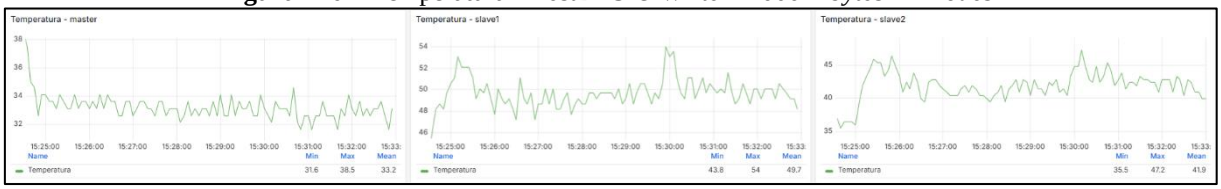
Fonte: Elaborada pelos Autores, 2025.

Figura 219 – Tráfego de rede – *TestDFSIO Write e* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 220 – Temperatura – *TestDFSIO Write* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.5.5 *Cluster* com 4 *nodes* – 1000 *MBytes*

Figura 221 – Utilização de *CPU* – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 222 – Utilização de memória *RAM* – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 223 – Quantidade de processos – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 224 – Tráfego de rede – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 225 – Temperatura – *TestDFSIO Write* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.5.6 *Cluster* com 8 *nodes* – 1000 *MBytes*

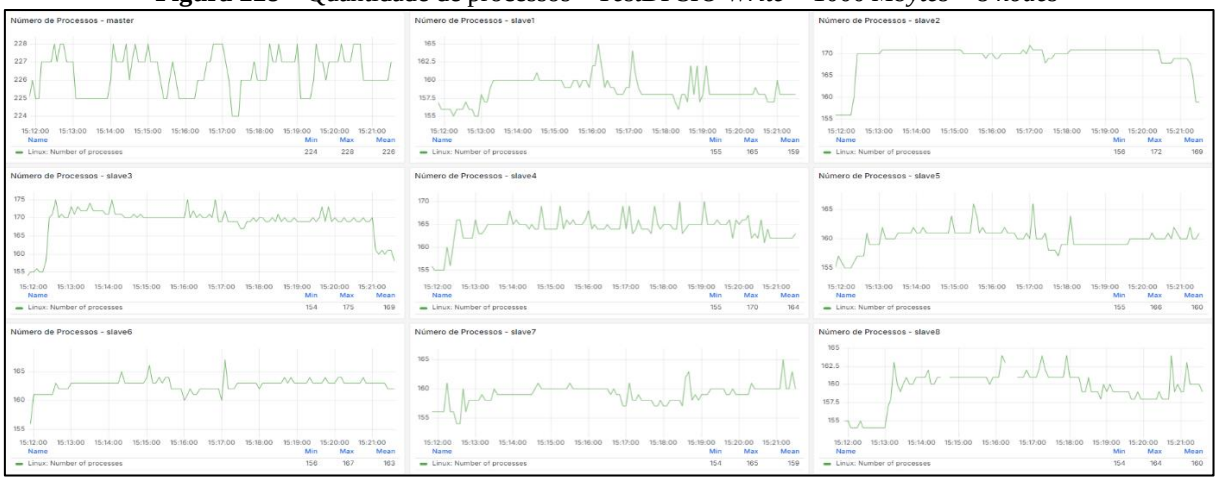
Figura 226 – Utilização de *CPU* – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 227 – Utilização de memória *RAM* – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 228 – Quantidade de processos – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes*

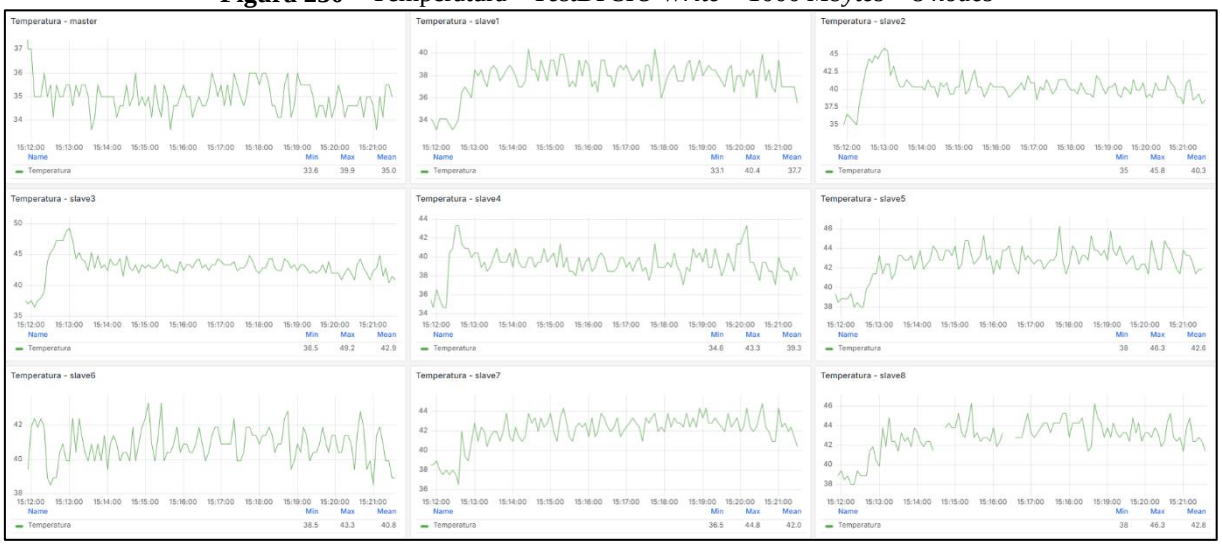


Fonte: Elaborada pelos Autores, 2025.

Figura 229 – Tráfego de rede – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

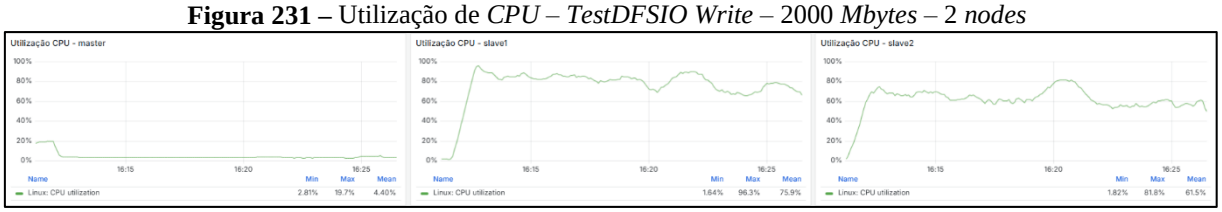
Figura 230 – Temperatura – *TestDFSIO Write* – 1000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

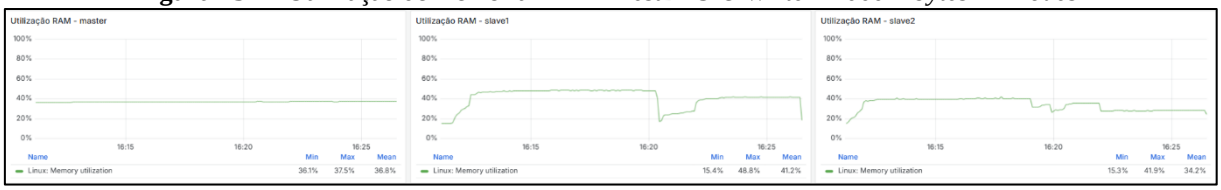
1.5.7 *Cluster* com 2 *nodes* – 2000 *MBytes*

Figura 231 – Utilização de *CPU* – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 232 – Utilização de memória *RAM* – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes*

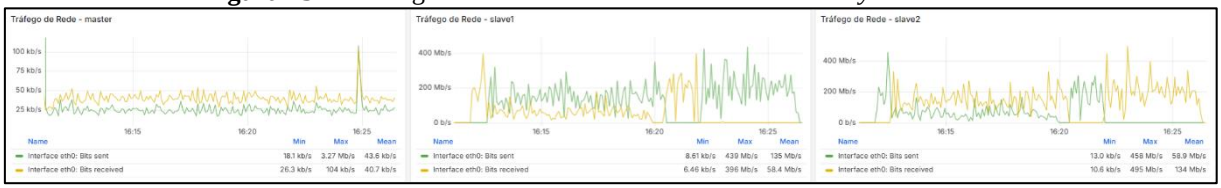


Fonte: Elaborada pelos Autores, 2025.

Figura 233 – Quantidade de processos – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes*

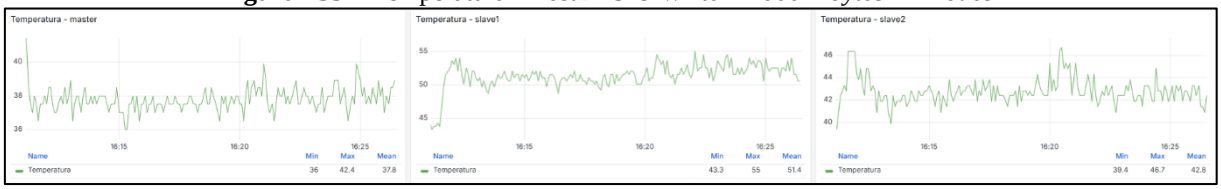
Fonte: Elaborada pelos Autores, 2025.

Figura 234 – Tráfego de rede – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 235 – Temperatura – *TestDFSIO Write* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.5.8 *Cluster* com 4 *nodes* – 2000 *MBytes*

Figura 236 – Utilização de *CPU* – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 237 – Utilização de memória *RAM* – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 238 – Quantidade de processos – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 239 – Tráfego de rede – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 240 – Temperatura – *TestDFSIO Write* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.5.9 *Cluster* com 8 *nodes* – 2000 *MBytes*

Figura 241 – Utilização de *CPU* – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 242 – Utilização de memória *RAM* – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 243 – Quantidade de processos – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 244 – Tráfego de rede – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

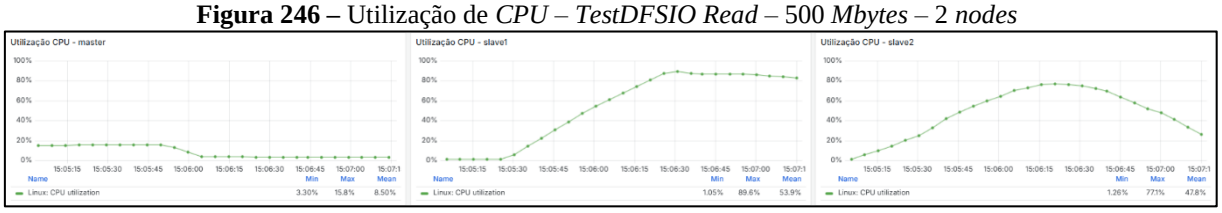
Figura 245 – Temperatura – *TestDFSIO Write* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.6 Algoritmo *TestDFSIO Read*

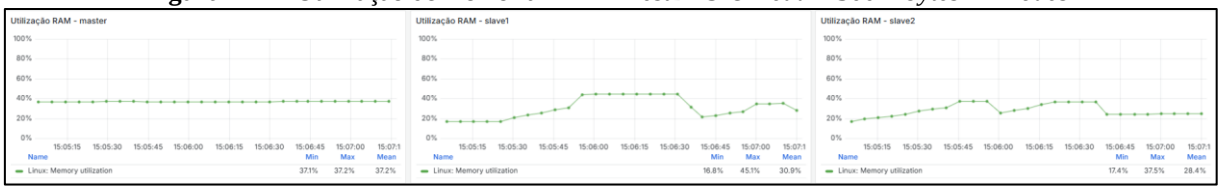
1.6.1 *Cluster* com 2 *nodes* – 500 *MBytes*

Figura 246 – Utilização de *CPU – TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*



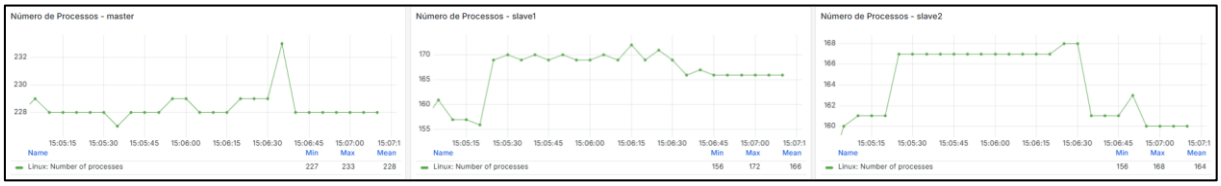
Fonte: Elaborada pelos Autores, 2025.

Figura 247 – Utilização de memória *RAM* – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*



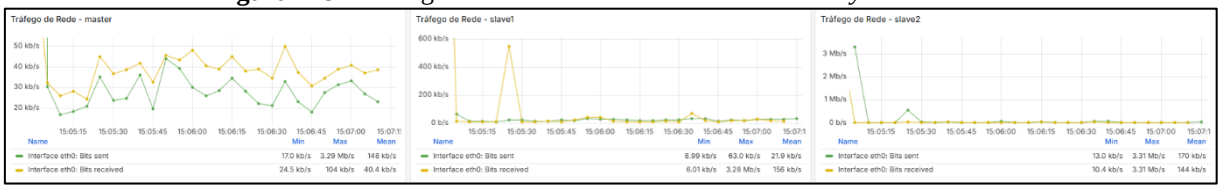
Fonte: Elaborada pelos Autores, 2025.

Figura 248 – Quantidade de processos – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*



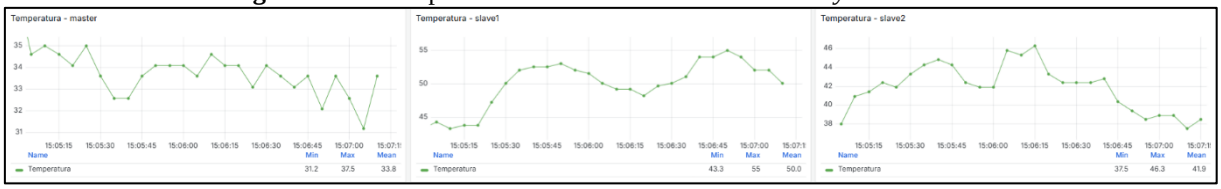
Fonte: Elaborada pelos Autores, 2025.

Figura 249 – Tráfego de rede – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 250 – Temperatura – *TestDFSIO Read* – 500 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.6.2 *Cluster* com 4 *nodes* – 500 *MBytes*

Figura 251 – Utilização de *CPU* – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

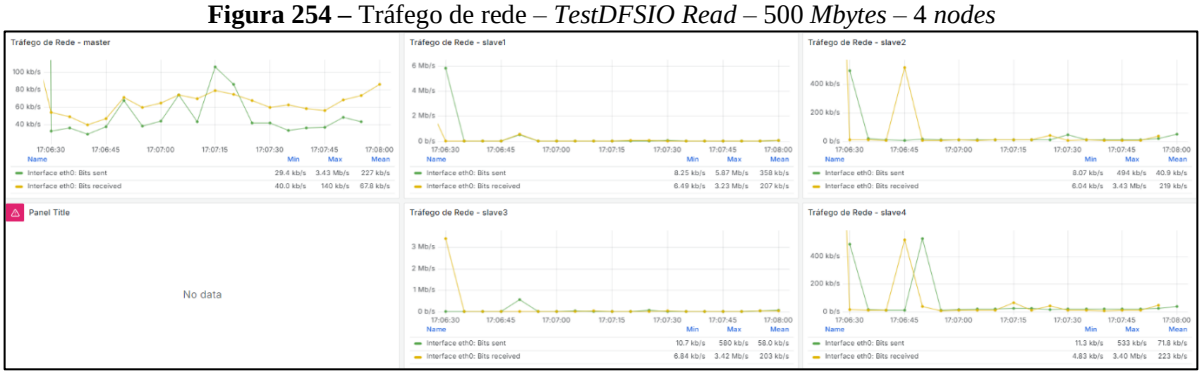
Figura 252 – Utilização de memória *RAM* – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 253 – Quantidade de processos – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 254 – Tráfego de rede – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 255 – Temperatura – *TestDFSIO Read* – 500 *Mbytes* – 4 *nodes*

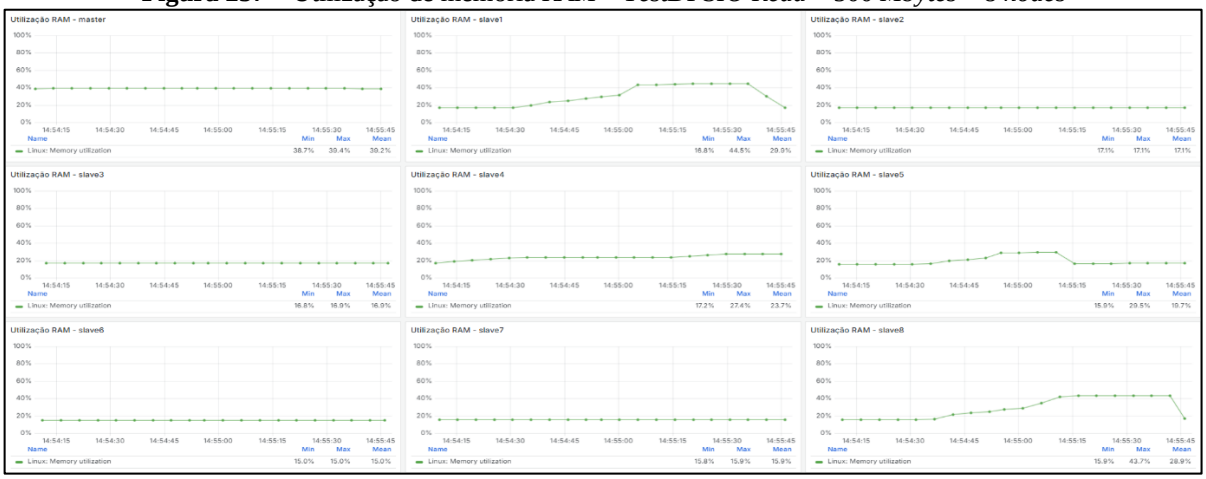
Fonte: Elaborada pelos Autores, 2025.

1.6.3 *Cluster* com 8 *nodes* – 500 *MBytes*

Figura 256 – Utilização de *CPU* – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 257 – Utilização de memória *RAM* – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 258 – Quantidade de processos – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 259 – Tráfego de rede – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*

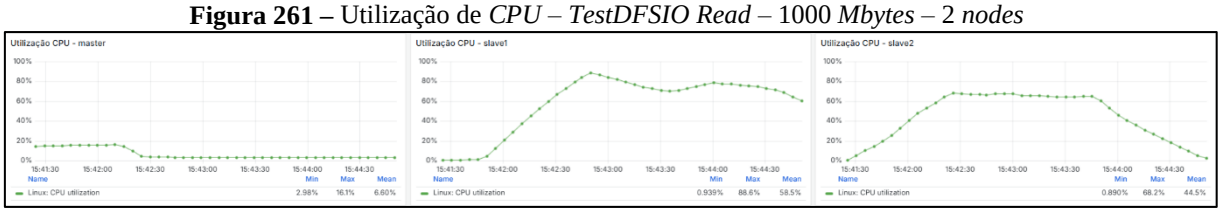
Fonte: Elaborada pelos Autores, 2025.

Figura 260 – Temperatura – *TestDFSIO Read* – 500 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

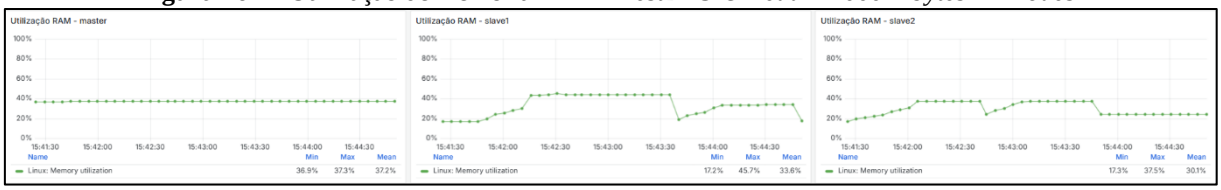
1.6.4 *Cluster* com 2 *nodes* – 1000 *MBytes*

Figura 261 – Utilização de *CPU* – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 262 – Utilização de memória *RAM* – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes*

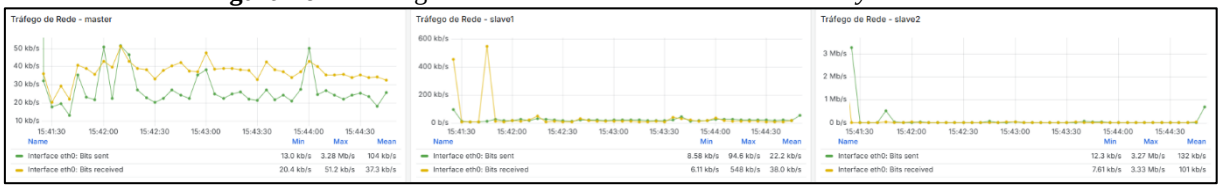


Fonte: Elaborada pelos Autores, 2025.

Figura 263 – Quantidade de processos – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes*

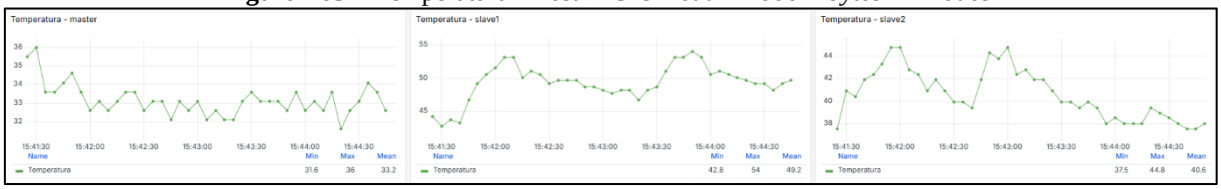
Fonte: Elaborada pelos Autores, 2025.

Figura 264 – Tráfego de rede – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 265 – Temperatura – *TestDFSIO Read* – 1000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

1.6.5 *Cluster* com 4 *nodes* – 1000 *MBytes*

Figura 266 – Utilização de *CPU* – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

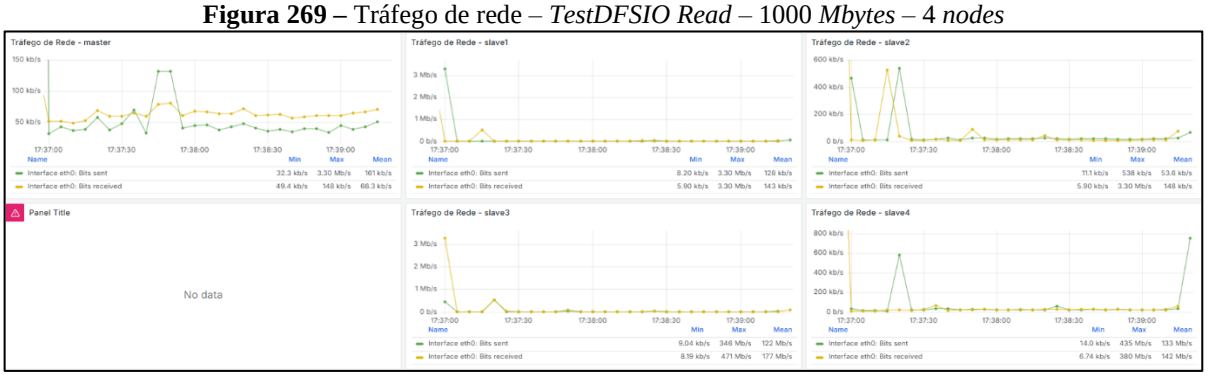
Figura 267 – Utilização de memória *RAM* – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 268 – Quantidade de processos – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 269 – Tráfego de rede – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 270 – Temperatura – *TestDFSIO Read* – 1000 *Mbytes* – 4 *nodes*

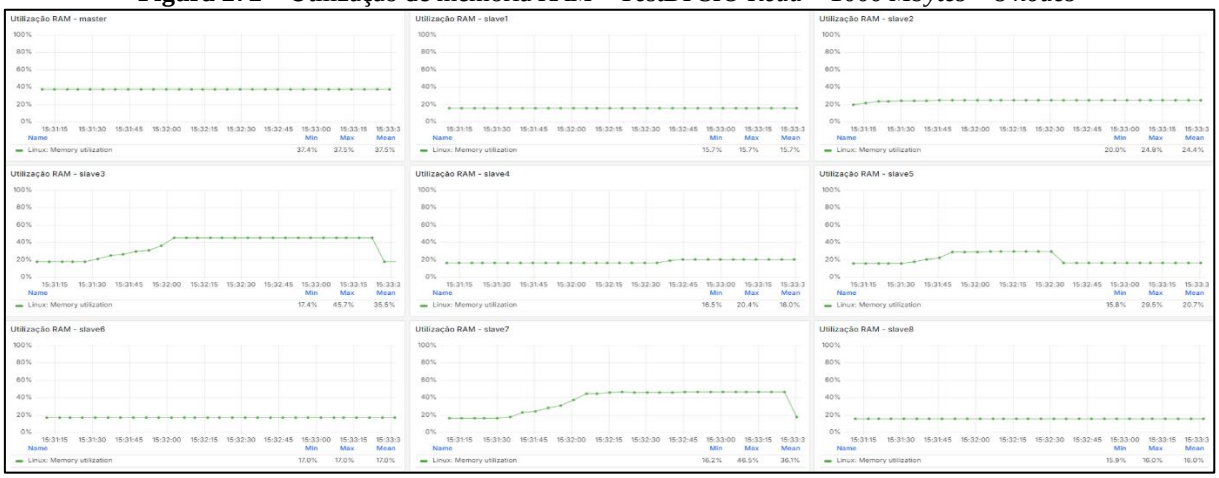
Fonte: Elaborada pelos Autores, 2025.

1.6.6 *Cluster* com 8 *nodes* – 1000 *MBytes*

Figura 271 – Utilização de *CPU* – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 272 – Utilização de memória *RAM* – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

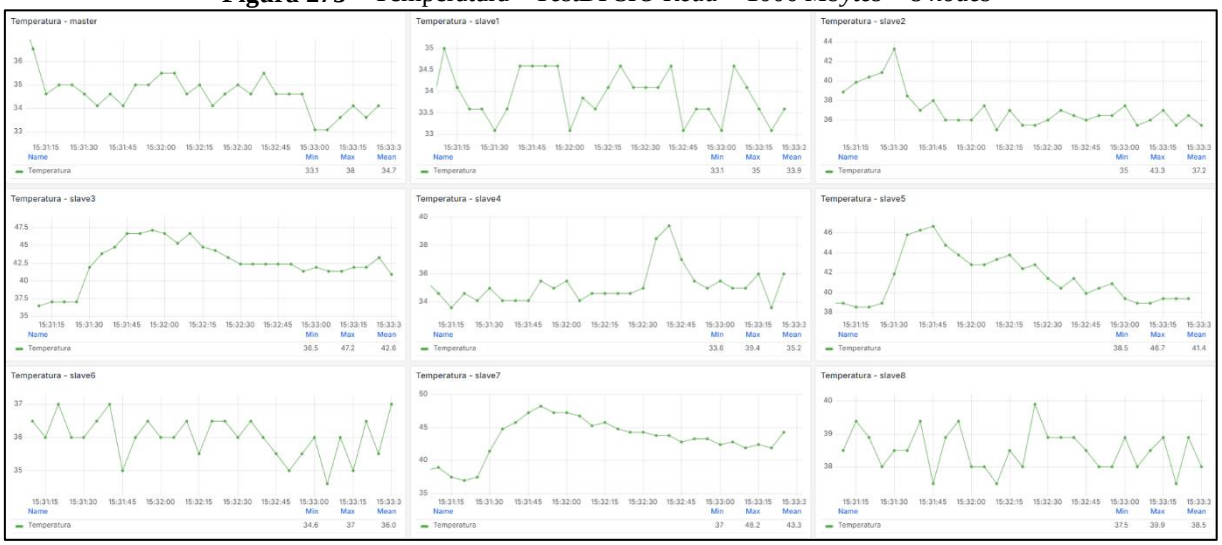
Figura 273 – Quantidade de processos – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 274 – Tráfego de rede – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

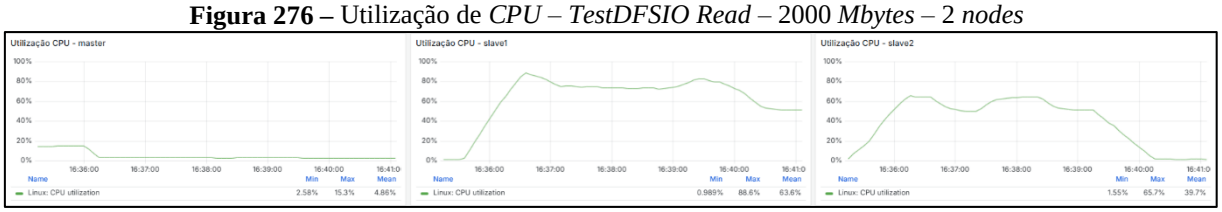
Figura 275 – Temperatura – *TestDFSIO Read* – 1000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

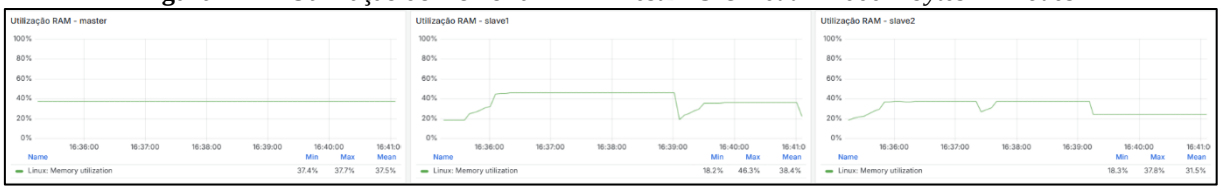
1.6.7 *Cluster* com 2 *nodes* – 2000 *MBytes*

Figura 276 – Utilização de *CPU* – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes*



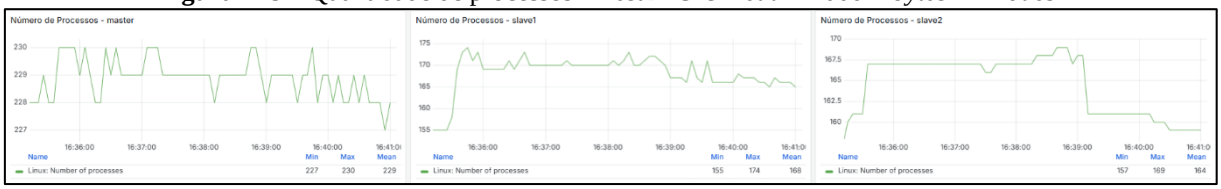
Fonte: Elaborada pelos Autores, 2025.

Figura 277 – Utilização de memória *RAM* – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes*



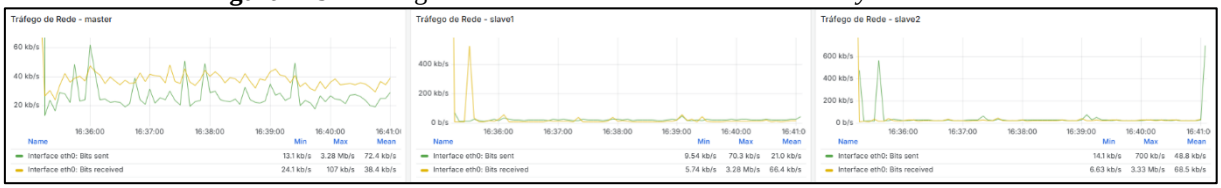
Fonte: Elaborada pelos Autores, 2025.

Figura 278 – Quantidade de processos – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 279 – Tráfego de rede – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 280 – Temperatura – *TestDFSIO Read* – 2000 *Mbytes* – 2 *nodes*

Fonte: Elaborada pelos Autores, 2025.

1.6.8 *Cluster* com 4 *nodes* – 2000 *MBytes*

Figura 281 – Utilização de *CPU* – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 282 – Utilização de memória *RAM* – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 283 – Quantidade de processos – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 284 – Tráfego de rede – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes*

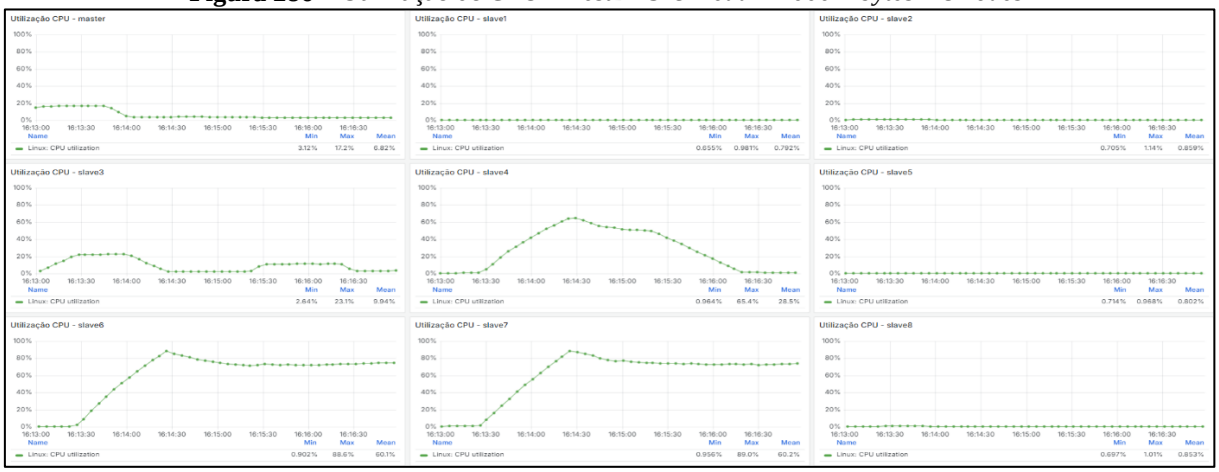
Fonte: Elaborada pelos Autores, 2025.

Figura 285 – Temperatura – *TestDFSIO Read* – 2000 *Mbytes* – 4 *nodes*

Fonte: Elaborada pelos Autores, 2025.

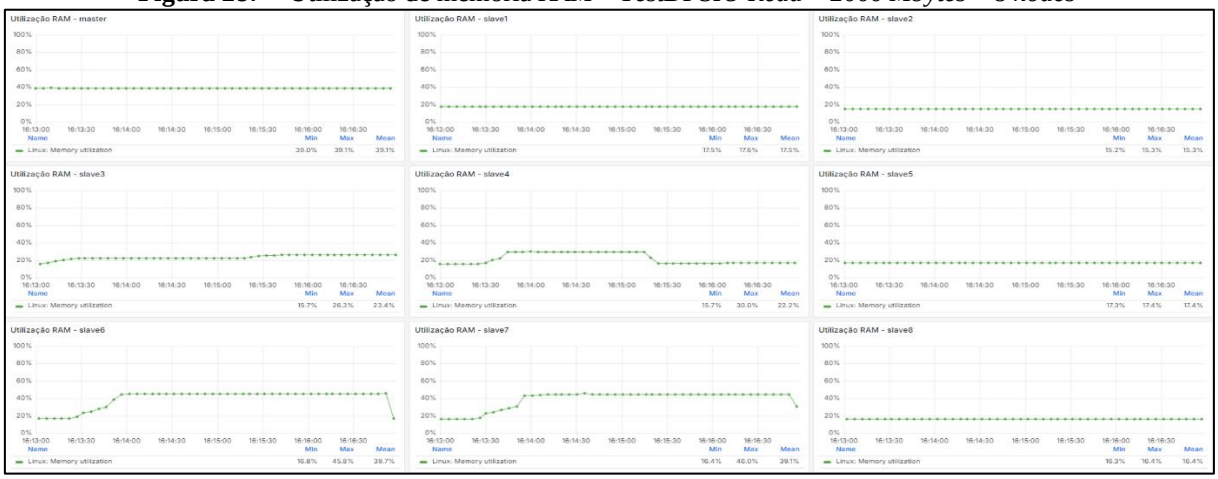
1.6.9 *Cluster* com 8 *nodes* – 2000 *MBytes*

Figura 286 – Utilização de *CPU* – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 287 – Utilização de memória *RAM* – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes*



Fonte: Elaborada pelos Autores, 2025.

Figura 288 – Quantidade de processos – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 289 – Tráfego de rede – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.

Figura 290 – Temperatura – *TestDFSIO Read* – 2000 *Mbytes* – 8 *nodes*

Fonte: Elaborada pelos Autores, 2025.